

UNICEF

HANDBOOK

CHIEF STATISTICIAN OFFICE

REPRODUCIBLE ANALYTICS

The CSO Handbook

One contract, three languages —
reproducible analytics in R,
Python, and Stata.

R

Python

Stata

v0.7.0

Table of contents

The CSO Handbook	x
Foreword	xi
Preface	xiii
What this handbook is	xiii
What this handbook is not	xiii
Who this handbook is for	xiv
How to read it	xiv
A note on authorship	xv
About the authors	xvi
Authors	xvi
How authorship is framed	xvii
Editors	xvii
Acknowledgement of the use of AI tools	xviii
Statement of responsibility	xviii
Affiliations and disclaimer	xviii
Cross-references	xix
Acknowledgements	xx
Authors and editors	xx
A note on collective authorship	xx
Consumer and adopter repositories	xxi
Methodological lineage	xxi
External reviewers	xxi
Funding and support	xxii
Acknowledgement of the use of AI tools	xxii
License	xxii
Tooling	xxii
Affiliation and disclaimer	xxii
How to cite this work	xxiii
Suggested citation (APA)	xxiii
BibTeX	xxiii
Chicago author-date	xxiv
Versioned citation	xxiv
License	xxiv
Acknowledgement of the use of AI tools	xxv

1	Introduction	1
1.1	What cso-toolkit is	1
1.2	One contract, three languages	2
1.3	The roles this toolkit serves	3
1.4	Who this handbook is for	4
1.5	How coding agents should use this handbook	5
1.6	How to navigate this book	6
I	Part I — Why & Architecture	7
2	Why a toolkit	8
2.1	The problem: analytics that cannot be re-run	8
2.2	The reproducibility floor	9
2.3	Vendoring, not installing	9
2.4	Scaling across sectors and projects	11
2.5	The producer / reviewer split, at a high level	12
2.6	What this buys you	13
3	Architecture: one contract, three languages, many consumers	14
3.1	The four layers	14
3.2	Layer one — the toolkit and its five capability families	15
3.3	Layer two — three language siblings of one contract	18
3.4	Layer three — consumer repositories vendor the helpers	19
3.5	Layer four — the canonical deposit	20
3.6	Reading the architecture diagram	20
3.7	The wider ecosystem: three tiers of shared code	21
II	Part II — The Contract	24
4	Roles and the mode contract	25
4.1	The three roles	25
4.2	What each role may read and write	26
4.3	The data flow	27
4.4	How <code>dw_mode</code> enforces the boundaries	28
4.5	The helper subset each role uses	32
4.6	A note on the PUBLISHER role	33
4.7	Why this matters	33
5	Wiring the contract into a repo	34
5.1	The single switch: <code>dw_mode</code> in <code>user_config.yml</code>	34
5.2	Deriving the producer/reviewer globals	36
5.3	The conductor shadow pattern	38
5.4	Branch-slug isolation	40
5.5	Defence in depth: the no-API guard	41
5.6	Verification checklist	42

5.7	Where this fits	43
6	Provenance sidecars and the error envelope	44
6.1	The provenance sidecar	44
6.2	The error envelope	49
6.3	How the two conventions reinforce each other	52
III	Part III — The API (cross-language)	53
7	The IO contract	54
7.1	Orientation: five terms this chapter leans on	54
7.2	Auto-dispatch by extension	55
7.3	Writing: <code>dw_save</code>	56
7.4	Reading: <code>dw_use</code>	61
7.5	The lower-level and resolver helpers	64
7.6	Comparing and joining	66
7.7	Putting it together	68
8	The external-API contract	70
8.1	Why a wrapper at all	70
8.2	The three entry points	71
8.3	The mode contract, applied to the network	72
8.4	The supported sources	73
8.5	The cache and its provenance	77
8.6	Auditing what is cached	78
8.7	The HTTP error envelope	79
8.8	Migrating a sector script	79
9	Aggregation	81
9.1	The core aggregator: <code>aggregate_data_v2</code>	81
9.2	Building the standard footnote: <code>generate_agg_footnote</code>	85
9.3	Picking the latest observation: <code>apply_time_window</code>	86
9.4	Redistributing weights over missing rows: <code>dw_nestweight</code>	88
9.5	Demographics helpers: <code>dw_pop</code> and <code>dw_regions</code>	89
9.6	Putting it together	92
10	Vintage sync	93
10.1	The pin: <code>.toolkit_manifest.yml</code>	93
10.2	<code>cso_toolkit_check</code> — am I behind?	95
10.3	<code>cso_toolkit_diff</code> — what changed?	97
10.4	<code>cso_toolkit_pull</code> — refresh to a tag	98
10.5	The refresh flow	100
10.6	Cross-language coverage, honestly	102
11	Scaffolding and the contract auditor	103
11.1	The scaffolding model	103

11.2	Generating a profile: <code>create_profile</code>	104
11.3	Generating a sector driver: <code>create_sector_script</code>	106
11.4	Auditing a profile: <code>review_profile</code>	109
11.5	The contract auditor: <code>test_scripts</code>	111
11.6	A note on naming	115
11.7	Summary	115
IV	Part IV — Adoption & Operations	116
12	The vendoring model	117
12.1	What “vendoring” means here	117
12.2	Why vendor instead of install	118
12.3	Vendoring is the production model — not the only path	119
12.4	What lives in <code>cso-toolkit</code> vs. the consumer	120
12.5	The refresh decision flow	121
12.6	Why not a package, in one sentence	124
13	Onboarding a repo: a worked example	125
13.1	Step 0: Decide what you are adopting	125
13.2	Step 1: Vendor the helpers	126
13.3	Step 2: Pin the vintage in <code>.toolkit_manifest.yml</code>	127
13.4	Step 3: Wire the mode contract into the profile	129
13.5	Step 4: First producer run — cache an API source and deposit an output	133
13.6	Step 5: Reviewer re-run and compare	136
13.7	Step 6: Add the contract auditor to CI	137
13.8	Recap: the adoption checklist	139
14	The contract auditor in continuous integration (CI)	141
14.1	What the auditor does	141
14.2	The rule set	143
14.3	Reporting and the build gate	146
14.4	Managing false positives with <code># cso-allow</code>	148
14.5	Wiring the auditor into a GitHub Actions job	149
14.6	How this fits the rest of the contract	151
15	Versioning and releases	152
15.1	Semantic versioning	152
15.2	The NEWS.md discipline	154
15.3	Tags and the release cut	156
15.4	The road to v1.0.0	158
V	Part V — The Microdata Archive (datalib)	160
16	The microdata archive: archive, catalogue, tool	161
16.1	Why a microdata archive	162

16.2	Two data layers, one office	162
16.3	Archive, catalogue, tool	163
16.4	Stock and flow	164
16.5	Deposit, gate, promote	165
16.6	Access and licensing	166
17	The IHSN convention and the folder grammar	168
17.1	Naming as structural provenance	169
17.2	The folder grammar	169
17.3	Vintages: masters and adaptations	171
17.4	Collections and modules	172
18	vendored from datalib-unicef config/collections.yml @ v0.7.0 — do not hand- edit; refresh via _manifest.yml	175
19	datalib collection registry — THE single source of truth for module lists,	177
20	module levels, and merge keys. R and Python consume synced byte-identical	179
21	copies (each package’s test suite asserts equality). The Stata leg mirrors	181
22	this registry by hand in stata/src/_/_dlw.ado (no YAML parser there); edits to	183
23	this file MUST be mirrored in _dlw.ado — the shared conformance cases in	185
24	tests/ are the drift guard.	187
26	contract_version: increment when semantics change; every implementation pins it.	191
27	Keys:	193
28	modules : the modules a collection ships (file suffix after the vintage stem)	195
29	level : hh (one row per household) or person (one row per person)	197
30	linevar : the module’s person-line variable when it is NOT already	199
31	called line_number; implementations normalize it on load	201
32	keys_hh : merge keys for hh-level modules	203
33	keys_person : merge keys for person-level modules (after linevar normalization)	205
35	Merge semantics (contract): person-level modules chain 1:1 on keys_person,	209
36	then hh-level modules attach m:1 on keys_hh. Keys are validated per module	210
37	(isid-equivalent); modules whose identifiers are missing/duplicated stop the	210

38 merge with an error. Loaders never mutate values.	210
38.1 The seven shared semantics	212
38.2 Deployment topologies	213
39 The datalib_* API and its conformance suite	215
39.1 The thirteen-function shared surface	215
39.2 Worked examples	217
39.3 Root and config resolution	220
39.4 Errors	221
39.5 The conformance suite	221
39.6 Access	222
 VI Part VI — The Ecosystem	 223
40 The UNICEF analytics ecosystem: three tiers	224
40.1 The three tiers	224
40.2 The tier-(ii) membership gate	228
40.3 Curated third-party functions	229
40.4 Configuration across the tiers	229
40.5 What the handbook asserts, tier by tier	230
41 unicefData: public child statistics in three languages	232
41.1 When to use what	233
41.2 Quick start	234
41.3 The surface	235
41.4 Configuration	236
41.5 A conformance exemplar	237
 Appendices	 238
A Cross-language coverage matrix	238
A.1 Legend	238
A.2 How parity is defined	239
A.3 Function coverage matrix	239
A.4 File-format support matrix	245
A.5 Content-integrity mechanism	246
A.6 Supporting machinery: documentation, tests, CI	247
A.7 Summary: how complete is each sibling?	248
A.8 Related contracts outside this matrix	249
 B Provenance sidecar schema	 250
B.1 Where the sidecar comes from	250
B.2 The core schema	251
B.3 A real example (R / Python)	255
B.4 The Stata sidecar: same shape, one difference that matters	256

B.5	A note on the API-fetch sidecar	257
B.6	Consuming the sidecar	258
B.7	Summary	259
C	Error taxonomy	260
C.1	The five categories at a glance	261
C.2	1. Mode-contract violation	261
C.3	2. Unsupported / mismatched format	264
C.4	3. API failure / blocked	265
C.5	4. Integrity / <code>isid</code> failure	267
C.6	5. Vendor drift	269
C.7	Reading any envelope: a debugging recipe	270
C.8	The datalib error taxonomy	270
D	Function reference	272
D.1	Where the per-function reference lives	272
D.2	Reading the family tables	273
D.3	IO contract	274
D.4	External-API contract	277
D.5	Aggregation and survey weights	277
D.6	Demographics	279
D.7	Vintage sync	279
D.8	Scaffolding and the contract auditor	280
D.9	Reporting and uniqueness utilities	281
D.10	Stata-only helpers	282
D.11	How to find a function fast	282
D.12	Authoritative source files	283
D.13	Microdata archive (<code>datalib_*</code>)	283
E	Glossary	287
E.1	Adaptation (vintage)	287
E.2	Aggregation contract	287
E.3	Cache key	288
E.4	Canonical deposit	288
E.5	Collection	288
E.6	Contract auditor	289
E.7	Coverage threshold	289
E.8	Cross-language parity	289
E.9	Data Steward	290
E.10	DBM / DBR / DBP	290
E.11	Domain	290
E.12	<code>dw_apis_allowed</code>	290
E.13	<code>dw_mode</code>	291
E.14	Error envelope	291
E.15	External-API contract	292
E.16	Frozen cache	292

E.17 HTTPS-URL freeze	292
E.18 IHSN convention	292
E.19 IO contract	293
E.20 isid	293
E.21 Manifest (<code>.toolkit_manifest.yml</code>)	294
E.22 Master vintage	294
E.23 Microdata archive	294
E.24 Microdata catalogue	295
E.25 Mode contract	295
E.26 Module	295
E.27 Producer	295
E.28 Provenance sidecar	295
E.29 Publisher	296
E.30 Review gate	296
E.31 Reviewer	296
E.32 Sandbox / <code>sandboxRoot</code>	297
E.33 Scaffolding	297
E.34 Sentinel object	297
E.35 Staging area	297
E.36 Stock and flow	298
E.37 Vendoring	298
E.38 Vintage	298
E.39 Vintage sync	298
E.40 Z: mirror	299
F Curated third-party functions	300
F.1 Inclusion rules	300
F.2 Staleness policy	300
F.3 The register	301

The CSO Handbook

One contract, three languages — reproducible analytics in R, Python, and Stata.

This handbook documents the `cso-toolkit` contract. Start with the [Introduction](#), then read the part that matches your role.

Foreword

Every number UNICEF publishes about children is, in the end, a claim that someone can check. A coverage rate, a stunting estimate, a count of children out of school — each is only as credible as our ability to say where it came from and to produce it again. In official statistics, “again” is often the hard word. The same indicator, recomputed by a second analyst in a second language, can come out a different number, and reconstructing why can take longer than the original analysis did. I have spent enough of my career re-deriving a number someone else had already published — once in R, once in Stata, arriving at two answers and no way to choose between them — to take this problem personally. That gap between *published* and *reproducible* is the problem this toolkit exists to close.

`cso-toolkit` is the Chief Statistician Office’s answer: **one contract, expressed identically in R, Python, and Stata**. It fixes one way to read, write, compare, and merge data; one way to call external sources; one way to aggregate, version-sync, and scaffold a project — so that an analytics product can be re-run by someone other than its author and yield the same numbers. The contract is not advice. It is enforced at every wrapped call site: a provenance sidecar — a small companion file, written beside every output, that records exactly where the numbers came from; an automatic check that the rows are uniquely identified by the key you declare; and a producer / reviewer split that makes the network *physically* unreachable to a reviewer, rather than trusting them not to reach for it. Those three guarantees turn “trust me” into “check me.”

One design choice deserves dwelling on — it *is* the institutional case for this book: we **vendor** the toolkit rather than install it. A consumer repository copies the helpers into its own tree and keeps them under version control alongside the analysis they serve. This is, on the surface, the less fashionable option — package managers exist precisely to spare us from copying code. But UNICEF’s statistical files are meant to outlive the environment that produced them. A figure citing `dw_<sector>.csv` in 2026 must reproduce identically when a reviewer re-runs it years later, working from the canonical deposit and the vendored helpers alone — no network, no upstream package drift, no ambient version skew silently changing a rounding rule. Vintage permanence is not a feature we add on top; it is the reason the toolkit is shaped the way it is. This matters more for us than for most: the numbers in these files set corporate-scorecard targets and inform what national budgets prioritise for children, and when a government acts on a UNICEF estimate, that estimate has to still mean the same thing the year someone audits the decision — long after the software that produced it has moved three versions on. Vending is how a statistical office that publishes for the long record protects that record from the churn of the software underneath it.

This handbook is deliberately scoped to the **contract itself**, not to any one warehouse that uses it. Its companion, the DW-Production handbook, documents the child-data warehouse

— the operational store of signed-off sector files. This book documents the reusable machinery beneath that and any future repository that publishes signed-off data: how the wrappers behave, what the sidecars promise, how the roles are enforced, and how a new repo adopts and stays in sync. The two are complementary by design. Read together, they describe both *what* UNICEF Data & Analytics produces and *how* it guarantees the producing.

The twelve named authors are the members of UNICEF Data & Analytics’ Database Managers Working Group — the colleagues whose shared, hard-won operating practice the toolkit encodes. This is genuinely collective work: the toolkit is the written-down form of how this group already agreed to handle data, and authorship is credited to the group as a whole rather than parcelled out by sector or function. The other eight editors are the unit chiefs of UNICEF Data & Analytics, who reviewed the handbook in that editorial capacity; I chair the editorial group as Chief Statistician. The Working Group did this on top of full workloads, and what they wrote down is not a wish-list — it is the practice they had already fought their way to, sector by sector. I am grateful to all of them, and a little in awe of the discipline it represents.

A note on how the book was made, in keeping with our own standard of provenance: parts of it were drafted with generative-AI assistance. The AI did not author the book. It produced drafts that the named authors and editors reviewed, edited, and checked against the `cso-toolkit` codebase, and we take full responsibility for every methodological claim, factual statement, and code example in it. The handbook text is released under CC BY 4.0; the toolkit code under `r/`, `python/`, and `stata/` is MIT-licensed — attribute each part under its own terms.

The toolkit is young, and so is this book. The coverage is honestly uneven across the three languages today, and the handbook says so wherever that is true rather than papering over it. What is already settled is the principle: one contract, enforced, reproducible, and built to last longer than the tools it runs on. The toolkit will get more even — full parity across R, Python, and Stata, and more deposits adopting the contract — because the teams adopting it will hold it to that. That is the expectation I would set: do not treat the contract as a constraint to satisfy, but as the floor you build above.

João Pedro Azevedo

Chief Statistician and Deputy Director, Data and Analytics, UNICEF

Chair of Editors

Preface

i Stub. The prose below is a working draft. Passages marked *[editor-to-finalize]* — chiefly the personal framing of how the toolkit came to be written and any institutional positioning relative to the wider data programme — await sign-off by the named editors before the first tagged edition. (*Wiring note for the editor: this page is not yet listed in `book/_quarto.yml`; add `front-matter/preface.qmd` to the `book.chapters`: list so it renders.*)

What this handbook is

This handbook documents `cso-toolkit`: the UNICEF Chief Statistician Office’s reusable, cross-language contract for reproducible analytics. The toolkit encodes **one way** to read, write, compare, and merge data; **one way** to call external APIs; and **one way** to aggregate, version-sync, and scaffold projects — and it does so identically in R, Python, and Stata. Every call routes through wrappers that enforce a provenance sidecar on each output, an **isid** uniqueness check on stated keys, and a producer / reviewer mode contract. The promise these mechanisms add up to is simple: an analytics product can be re-run by someone other than its original author and yield the same numbers.

The toolkit is **vendored**, not network-installed. Its source files are copied into each consumer’s own repository rather than pulled from a package index at run time. That choice is deliberate and it is the spine of the whole design: a publication that cites a deposited file today should reproduce identical numbers years later from the deposit and the vendored helpers alone — no live network, no upstream package drift, no ambient version skew. The handbook returns to that idea throughout.

What this handbook is not

This is not a tutorial on R, Python, or Stata, and it is not a statistics text. It assumes you can already write a script in at least one of the three languages and that you understand the analytical work your sector or team does. Its subject is narrower and more specific: the *contract* — the shared conventions and the wrappers that enforce them — and nothing beyond what the code in the repository actually implements. Where a capability exists in one language but not another, the book says so plainly rather than describing an aspiration; the cross-language coverage matrix in Appendix A is the source of truth when the prose and the code seem to disagree.

It is also not the warehouse handbook. The companion *DW-Production* handbook documents UNICEF’s child-data warehouse — the operational store of signed-off `dw_<sector>` files that the public data products draw from. This book is **complementary** to that one: DW-Production documents a particular warehouse and its sector pipelines; *this* book documents the reusable contract itself, for any consumer that adopts it. DW-Production is the toolkit’s first and largest adopter, but it is not the only one the toolkit is built to serve.

Who this handbook is for

The book is written for several audiences at once, and most readers will live in one or two parts of it rather than reading front to back:

- **Database managers (producers)** who run sector pipelines, pull upstream APIs, and deposit canonical files. The contract — the roles, the mode that governs network access, the provenance sidecars — is your daily working environment, and Part II describes it.
- **Reviewers** who re-run a pipeline from pre-deposited inputs and compare the result against the canonical deposit. The same mode contract governs your work, with the network deliberately unreachable; Part II explains why that constraint is enforced rather than merely advised.
- **Publishers** who move signed-off deposits into downstream products. The role boundaries in Part II tell you where the toolkit’s guarantees end and the publication infrastructure begins.
- **Peer-institution adopters** — maintainers of other deposit-bearing repositories who want to vendor the toolkit into their own codebase and keep it in sync. Part IV is written for you, and it assumes the role and mode concepts from Part II.

Analysts and sector teams who simply need to read, write, and aggregate data the CSO way are served by Part III, which is the reference for that daily work in all three languages.

How to read it

The handbook is organised into four parts and a set of appendices.

- **Part I — Why & Architecture** motivates the toolkit and lays out its shape: one contract, three language siblings, many consumer repositories, and a single canonical deposit. Start here for the big picture.
- **Part II — The Contract** is the conceptual core: the three roles, the session-level mode that governs network access, how that contract is wired into a repository, and the two cross-cutting mechanisms — provenance sidecars and the standard error envelope — that make the toolkit trustworthy.

- **Part III — The API (cross-language)** is the daily reference, one chapter per capability family: IO, the external-API contract, aggregation, vintage sync, and scaffolding with the contract auditor. Every chapter presents R, Python, and Stata side by side and is explicit about coverage gaps.
- **Part IV — Adoption & Operations** is for adopters and maintainers: the vendoring model, a worked onboarding example, running the contract auditor in CI, and the project’s versioning and release discipline.
- **Appendices** collect the dense reference material: the coverage matrix (A), the provenance sidecar schema (B), the error taxonomy (C), a full function reference (D), and a glossary (E).

If you are new, read the architecture chapter in Part I and the roles chapter in Part II, then turn to the IO chapter in Part III. Those three give you enough to use the toolkit safely. If you are here to adopt it into a new repository, begin with the vendoring and onboarding chapters in Part IV, keeping the roles chapter to hand.

A note on authorship

The twelve authors are the members of UNICEF Data & Analytics’ Database Managers Working Group — the staff whose shared operating practice the toolkit encodes. The toolkit did not arrive as a specification handed down to be implemented; it grew out of the conventions this group had already converged on, made explicit and then enforced in code. Authorship is therefore framed collectively: the contract is a shared body of practice, and crediting it as such is more honest than parcelling it out by file or function. The nine editors are the Chief Statistician, as chair, and the unit chiefs of UNICEF Data & Analytics, acknowledged in that editorial capacity rather than as authors. *[editor-to-finalize: any first-person account of the toolkit’s origins, and the precise institutional positioning of this work within the wider data programme, are for the editors to settle.]*

i Note

Parts of this handbook were drafted with generative-AI assistance. The AI did not author the book; it produced drafts that the named authors and editors are reviewing, editing, and fact-checking against the `cso-toolkit` codebase. On release, the named authors and editors take full responsibility for the methodological claims, factual statements, and code examples reproduced here.

The handbook text is released under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#); the toolkit source code under `r/`, `python/`, and `stata/` is released under the MIT License. Attribute each part under its own license.

About the authors

i Stub. Authors are the members of UNICEF Data & Analytics' Database Managers Working Group — the staff whose shared operating practice this toolkit encodes. Editors are the Chief Statistician (chair) and the unit chiefs of UNICEF Data & Analytics, acknowledged in that editorial capacity rather than as authors. ORCID IDs are to be collected before the first release. Per-contribution detail is not reproduced here by design — see § [How authorship is framed](#).

This page credits the people behind `cso-toolkit`. The toolkit is not a single author's library; it is the reusable, cross-language contract that the CSO's Database Managers already run in practice — one way to read, write, compare, and merge data, call external APIs, aggregate, vintage-sync, and scaffold, identical in R, Python, and Stata. The named authors below are the staff whose shared working practice the toolkit codifies. This handbook documents the contract; it is complementary to the [DW-Production handbook](#), which documents the child-data warehouse that is one of the toolkit's consumers.

Authors

The authors are the members of UNICEF Data & Analytics' Database Managers Working Group. The toolkit encodes their shared operating practice — the conventions they had converged on for reading, writing, aggregating, and re-running warehouse data — into one enforced contract. Authorship is therefore framed collectively: the helpers in `r/`, `python/`, and `stata/` are the written form of a practice these colleagues hold in common, not a set of individually parcelled modules.

Listed alphabetically by surname.

- **Garen Avanesian** — UNICEF Data & Analytics, Database Managers Working Group
- **Sukhdeep Brar** — UNICEF Data & Analytics, Database Managers Working Group
- **Joel Conkle** — UNICEF Data & Analytics, Database Managers Working Group
- **Yadigar Coskun** — UNICEF Data & Analytics, Database Managers Working Group
- **Ayca Donmez** — UNICEF Data & Analytics, Database Managers Working Group
- **Lauren Francis** — UNICEF Data & Analytics, Database Managers Working Group
- **Munkhbadar Jugder** — UNICEF Data & Analytics, Database Managers Working Group
- **Yang Liu** — UNICEF Data & Analytics, Database Managers Working Group
- **Vrinda Mehra** — UNICEF Data & Analytics, Database Managers Working Group

- **Gladys Babirye Bagandanswa Nacuka** — UNICEF Data & Analytics, Database Managers Working Group
- **Daniele Olivotti** — UNICEF Data & Analytics, Database Managers Working Group
- **Sahar Zandinia** — UNICEF Data & Analytics, Database Managers Working Group

How authorship is framed

The DW-Production handbook attributes work per sector and per file, because that book documents a warehouse whose pipelines split cleanly along sector lines. That model does **not** transfer to the toolkit. `cso-toolkit` is a single shared contract, not a collection of sector pipelines; its value is precisely that one helper behaves the same for every sector and every consumer. We therefore credit the authors collectively rather than carving the toolkit into per-person slices.

The collective credit is deliberate, not a placeholder for finer-grained attribution. The toolkit is the written form of a working practice the named authors hold in common: no single helper “belongs” to one of them, and the contract’s worth lies in its uniformity rather than in who wrote which line. Splitting it into per-person modules would misdescribe how the work was actually done.

i Editor to finalize. If, before release, the team decides to surface finer-grained credit (for example, by opening the repository’s commit history to multiple named authors, or by adding a CONTRIBUTORS file), this section should point readers there. As currently maintained, the repository does not carry a per-author commit ledger, so this page does not claim one.

Editors

The volume is edited by the Chief Statistician (chair) and the unit chiefs of UNICEF Data & Analytics. The editors review the handbook before release and sign off on the final manuscript. Unit chiefs are listed in this editorial capacity rather than as authors — the structure mirrors a conventional academic-volume model (contributing authors above, volume editors here).

Listed alphabetically by surname, with the chair first.

- **João Pedro Azevedo** — *Chair (Editor)*. Chief Statistician and Deputy Director, Data and Analytics, UNICEF
- **Claudia Cappa** — Chief, *Child Protection, Disability and Early Childhood Development*, UNICEF Data & Analytics
- **Enrique Delamónica** — Chief, *Child Poverty and Wellbeing*, UNICEF Data & Analytics
- **Chika Hayashi** — Chief, *Nutrition*, UNICEF Data & Analytics

- **Yves Jaques** — Chief, *Geospatial and Computational Analytics*, UNICEF Data & Analytics
- **Maria Muniz** — Chief, *Integrated Health Analytics*, UNICEF Data & Analytics
- **Andrea Rossi** — Chief, *Data Collection*, UNICEF Data & Analytics
- **Tom Slaymaker** — Chief, *Climate, Water, Sanitation and Hygiene* (JMP co-chair), UNICEF Data & Analytics
- **Danzhen You** — Chief, *Child Health and Mortality* (UN IGME co-chair), UNICEF Data & Analytics

Acknowledgement of the use of AI tools

Parts of this handbook were drafted with generative-AI assistance. AI tools did not author the book — they generated drafts that the named authors and editors are reviewing, editing, and fact-checking against the `cso-toolkit` codebase. On release, the named authors and editors take full responsibility for the methodological claims, factual statements, worked examples, and verbatim code snippets reproduced here. AI assistance applies to the handbook prose only; the toolkit source code it documents is the work of the named authors’ team, encoding their shared operating practice.

Statement of responsibility

The authors and the editors review the text of the handbook before release. They take responsibility for the descriptions of the contract, the cross-language coverage claims, the worked examples, and the verbatim code snippets reproduced in this book.

Where this book describes a capability as present in one language but not another, that reflects the real state of the code at the documented version, not an aspiration; the coverage matrix in [Appendix A](#) is the source of truth. Errors that survive into a released edition are the responsibility of the authors and the editors — not of the AI tools used during drafting, and not of UNICEF as an institution.

Readers who find errors are invited to open an issue on the [book’s repository](#). Errata are addressed in subsequent releases per the project’s versioning and release discipline (Part IV).

Affiliations and disclaimer

The authors and the editors are staff or consultants of UNICEF’s Data & Analytics section (Office of the Chief Statistician), within the Office of Strategy and Evidence (OSE). The views expressed in this book are those of the named authors and editors and do not necessarily reflect the official position of UNICEF, its Executive Board, or its Member States. References to specific commercial products, processes, or services do not constitute or imply endorsement, recommendation, or favouring by UNICEF.

The handbook text is released under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#); the toolkit source code under `r/`, `python/`, and `stata/` is released under the [MIT License](#). See [How to cite this work](#) for the dual-license detail and the suggested citation.

Cross-references

- [How to cite this work](#) — suggested citation, BibTeX, and dual-license terms
- [Appendix A](#) — the cross-language coverage matrix (source of truth for what exists where)
- github.com/unicef-drp/cso-toolkit — the toolkit source code and issue tracker

Acknowledgements

i Stub. Authorship is framed collectively: this handbook documents a shared operating practice, and the contributions to that practice do not decompose cleanly into per-person, per-function credit. The named authors and editors are the people responsible for the toolkit and this book; they are listed in full on the [About the authors](#) page. External reviewers and any funding acknowledgement are left as marked placeholders for the editor to finalize before the first tagged release. Personal passages flagged *editor-to-finalize* await confirmation from the people named.

Authors and editors

The toolkit and this handbook are the work of twelve named authors — the members of UNICEF Data & Analytics’ **Database Managers Working Group**, the staff whose shared operating practice the toolkit encodes — and nine named editors: the Chief Statistician, as chair, together with the unit chiefs of UNICEF Data & Analytics.

See [About the authors](#) for the full roster — the twelve authors (listed alphabetically by surname) and the nine editors with their unit-chief titles. The full citation, including the BibTeX entry, is on the [How to cite this work](#) page.

A note on collective authorship

The `cso-toolkit` is not a collection of independently owned features; it is a single shared contract — one way to read, write, compare, and merge data, call external APIs, aggregate, version-sync, and scaffold — that the Database Managers Working Group converged on through shared practice across many sectors and three languages. The credit for that convergence belongs to the group, not to individuals holding discrete pieces of it. For that reason we name the authors collectively rather than apportioning per-function or per-language contributions. The full commit-level history remains reproducible from the repository:

```
git shortlog --summary --numbered --email --all
```

Consumer and adopter repositories

This toolkit exists to be vendored into other repositories — copied into a consumer’s own codebase rather than installed over the network — so that each consumer’s pipelines run against a pinned, drift-free copy of the contract. We thank the teams maintaining the repositories that adopt and exercise it:

- **DW-Production** — the operational copy of the UNICEF child-data warehouse, and the toolkit’s first and primary consumer. Its sector pipelines are where the contract is stress-tested in daily use. The companion *DW-Production* handbook documents that warehouse; this handbook documents the reusable contract those pipelines depend on. The two are complementary.
- **Future deposit-bearing repositories** (*editor-to-finalize: confirm which adopters to name at release — e.g. UNPD-Population, datalib*) that vendor the toolkit to gain the same provenance, uniqueness, and producer/reviewer guarantees.

Adoption feedback from these consumers — bug reports, coverage-gap requests, and on-boarding friction — has shaped the toolkit’s API and is the reason its asymmetries between languages are documented honestly rather than papered over.

Methodological lineage

The toolkit does not invent a measurement methodology; it serves one. It is the reusable contract that makes the analytics behind UNICEF’s flagship statistical products reproducible, as those products are derived through the warehouse it serves. The practices it encodes build on:

- UNICEF’s MICS programme and its long history of country-led survey methodology
- WHO/UNICEF Joint Monitoring Programme (JMP) for WASH measurement
- WHO/UNICEF Estimates of National Immunization Coverage (WUENIC)
- The UN Inter-agency Group for Child Mortality Estimation (UN IGME)
- UNESCO Institute for Statistics (UIS) for education indicators
- World Bank’s [DIME Data Handbook](#) as the genre exemplar this handbook follows, and the DIME Wiki community for ongoing reproducibility reference

The toolkit’s contribution is upstream of these: it is the plumbing — provenance sidecars, the `isid` uniqueness check, mode-aware path routing, and the vendoring model — that lets the numbers feeding these products be re-derived years later from a canonical deposit alone.

External reviewers

(To be filled in after the pre-release external review pass. Do not name reviewers until they have confirmed.)

Funding and support

(To be filled in per UNICEF Communications guidance. No funder is named pending that guidance.)

Acknowledgement of the use of AI tools

Parts of this handbook were drafted with generative-AI assistance. AI tools did not author the book — they generated drafts that the named authors and editors are reviewing, editing, and fact-checking against the `cso-toolkit` codebase. On release, the named authors and editors take full responsibility for the methodological claims, factual statements, worked examples, and verbatim code snippets reproduced here.

License

This repository is **dual-licensed**. The handbook text is released under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#); the toolkit source code under `r/`, `python/`, and `stata/` is released under the [MIT License](#). Attribute each part under its own license. See [How to cite this work](#) for details.

Tooling

The handbook is built with [Quarto](#), an open-source scientific and technical publishing system. The toolkit itself is written in [R](#), [Python](#), and [Stata](#), and this book shows all three siblings side by side. Continuous deployment runs on GitHub Actions; the built handbook is published at <https://unicef-drp.github.io/cso-toolkit/handbook/>.

Affiliation and disclaimer

This handbook is a publication of the UNICEF Chief Statistician Office — Data & Analytics (OSE). The statements in it are those of the authors and editors and do not necessarily reflect the policies or views of UNICEF. The designations employed and the presentation of material do not imply the expression of any opinion on the part of UNICEF.

How to cite this work

i Stub. A Zenodo DOI will be reserved when this handbook is first archived to Zenodo on a tagged release; the placeholder 10.5281/zenodo.XXXXXXX below is replaced then. Zenodo mints two DOIs: a **concept DOI** that always resolves to the latest version, and a **version DOI** unique to each tagged release. Cite the concept DOI for “this work in general”; cite a version DOI to pin a specific edition.

Authors are listed alphabetically by surname. Editors are the Chief Statistician (chair) and the unit chiefs of UNICEF Data & Analytics; they are acknowledged in that editorial capacity rather than as authors.

Suggested citation (APA)

Avanesian, G., Brar, S., Conkle, J., Coskun, Y., Donmez, A., Francis, L., Jugder, M., Liu, Y., Mehra, V., Nacuka, G. B. B., Olivotti, D., & Zandinia, S. (2026). *The CSO Handbook: One contract, three languages — reproducible analytics in R, Python, and Stata* (J. P. Azevedo, C. Cappa, E. Delamónica, C. Hayashi, Y. Jaques, M. Muniz, A. Rossi, T. Slaymaker, & D. You, Eds.). UNICEF Chief Statistician Office. <https://doi.org/10.5281/zenodo.XXXXXXX>

BibTeX

```
@book{csohandbook2026,  
  title      = {The CSO Handbook: One contract, three languages ---  
                reproducible analytics in R, Python, and Stata},  
  author     = {Avanesian, Garen and Brar, Sukhdeep and Conkle, Joel and  
                Coskun, Yadigar and Donmez, Ayca and Francis, Lauren and  
                Jugder, Munkhbadar and Liu, Yang and Mehra, Vrinda and  
                Nacuka, Gladys Babirye Bagandanswa and  
                Olivotti, Daniele and Zandinia, Sahar},  
  editor     = {Azevedo, Jo{\~a}o Pedro and Cappa, Claudia and  
                Delam{\`o}nica, Enrique and Hayashi, Chika and  
                Jaques, Yves and Muniz, Maria and Rossi, Andrea and  
                Slaymaker, Tom and You, Danzhen},
```

```

year      = {2026},
publisher = {UNICEF Chief Statistician Office},
url       = {https://unicef-drp.github.io/cso-toolkit/handbook/},
doi       = {10.5281/zenodo.XXXXXXX},
edition   = {First},
note      = {The book documents the cso-toolkit codebase at
             github.com/unicef-drp/cso-toolkit.
             Edited by João Pedro Azevedo (Chief Statistician,
             UNICEF Data & Analytics) as chair, with the unit chiefs
             of UNICEF Data & Analytics as co-editors.}
}

```

Chicago author-date

Avanesian, Garen, Sukhdeep Brar, Joel Conkle, Yadigar Coskun, Ayca Donmez, Lauren Francis, Munkhbadar Jugder, Yang Liu, Vrinda Mehra, Gladys Babirye Bagandanswa Nacuka, Daniele Olivotti, and Sahar Zandinia. 2026. *The CSO Handbook: One contract, three languages — reproducible analytics in R, Python, and Stata*. Edited by João Pedro Azevedo, Claudia Cappa, Enrique Delamónica, Chika Hayashi, Yves Jaques, Maria Muniz, Andrea Rossi, Tom Slaymaker, and Danzhen You. UNICEF Chief Statistician Office. <https://doi.org/10.5281/zenodo.XXXXXXX>.

Versioned citation

To cite a specific edition, append the version tag (e.g., v0.6.0) and use that release’s **version DOI** rather than the always-latest **concept DOI**. Each tagged release has its own version DOI; the concept DOI always resolves to the most recent version.

License

The handbook text is released under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#). You are free to share and adapt the material with appropriate attribution.

This repository is **dual-licensed**. The handbook prose (the built book under `docs/handbook/`, and the documentation it is sourced from) is CC BY 4.0; the toolkit source code under `r/`, `python/`, and `stata/` is released under the [MIT License](#). A whole-repository Zenodo archive therefore contains both the CC BY 4.0 handbook and the MIT-licensed code — attribute each part under its own license.

Acknowledgement of the use of AI tools

Parts of this handbook were drafted with generative-AI assistance. AI tools did not author the book — they generated drafts that the named authors and editors are reviewing, editing, and fact-checking against the `cso-toolkit` codebase. On release, the named authors and editors take full responsibility for the methodological claims, factual statements, worked examples, and verbatim code snippets reproduced here.

1 Introduction

`cso-toolkit` is the shared, cross-language contract that the UNICEF Chief Statistician Office (CSO) uses to make its analytics reproducible. The CSO runs a **data warehouse**: a shared store of signed-off statistical files, named `dw_<sector>.<ext>` (one per sector — education, water and sanitation, immunisation, and so on), that UNICEF’s public data products draw from. The repository that holds the operational copy of that warehouse is **DW-Production**. The toolkit is what lets anyone, not just the file’s original author, re-run a sector’s pipeline and reproduce those files exactly.

It does this by encoding **one way to read, write, compare, and merge data; one way to call external APIs; one way to aggregate, version-sync, and scaffold projects** — and it does so identically in R, Python, and Stata. Every call routes through wrappers that enforce provenance sidecars (a `.provenance.json` file written next to every output, recording who made it, when, and from what), a uniqueness check (`isid` — that rows are unique on a stated key), and a producer / reviewer mode contract, so that any analytics product can be re-run by someone other than its original author and yield the same numbers. This handbook documents that contract.

1.1 What `cso-toolkit` is

`cso-toolkit` (the repository `unicef-drp/cso-toolkit`, currently v0.6.0) is a set of small helpers that enforce one way of doing each task, grouped into five capability families. Part III gives each family its own chapter.

- **IO** — `dw_save`, `dw_use`, `dw_compare`, `dw_merge`, `dw_isid`, `dw_verify_z`, `dw_stage`, `dw_root`: uniform file IO with auto-dispatch by extension, an `isid` uniqueness check, an automatic provenance sidecar on writes, a Z: drive mirror on canonical writes (a *canonical* write being the final, signed-off `dw_<sector>` file that downstream products cite, as opposed to an intermediate *working* file), and an integrity check on canonical reads.
- **External API** — `dw_api_fetch`, `dw_api_cached`, `dw_api_inventory`: a mode-aware wrapper around external data sources (UIS, SDMX, World Bank, ILO, UNSD-SDG, GitHub-raw, generic HTTP/JSON) with a deposit-side cache.
- **Aggregation** — `aggregate_data_v2`, `apply_time_window`, `generate_agg_footnote`, `dw_nestweight`, `dw_pop`, `dw_regions` (the earlier `aggregate_data` is kept for back-compat): weighted aggregation with population and country coverage, latest-observation time windowing, survey-weight redistribution, and population / region lookups.

- **Vintage sync** — `cso_toolkit_check`, `cso_toolkit_diff`, `cso_toolkit_pull`: consumer-side helpers that compare the locally **vendored** copy of the toolkit (its source files copied into the consumer’s own repository rather than installed over the network) against the latest upstream release and refresh it on demand.
- **Scaffolding and contract auditing** — `create_profile`, `create_sector_script`, `review_profile`, `test_scripts`: project scaffolds plus a static auditor that flags raw IO/API calls that should have gone through the toolkit.

These families look miscellaneous, but one idea ties them together: **vintage permanence**. A *vintage* is a dated snapshot of upstream data — the data as it stood when a release was cut. Vintage permanence is the guarantee that a publication citing `dw_<sector>.csv` in 2026 will, years later, produce identical numbers when re-run by a reviewer who has only the canonical deposit and the vendored helpers — no network access, no upstream package drift, no ambient `dplyr` version skew. Everything in the toolkit serves that goal.

1.2 One contract, three languages

The defining commitment of `cso-toolkit` is **cross-language parity**: the same command names and the same behaviour across R, Python, and Stata. A `dw_save` in R looks and behaves like a `dw_save` in Python, which looks and behaves like `dw_save` in Stata. The same provenance sidecar schema, the same mode-aware path routing, and the same three-part error envelope (`[cso_toolkit.<func>] WHAT / Why / Fix`) hold across all three siblings.

1.2.0.1 R

```
df <- tibble::tibble(REF_AREA = c("AGO", "BFA"), OBS_VALUE = c(0.5, 0.7))
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        isid = "REF_AREA",
        metadata = list(title = "Education indicators", vintage = "2026-05"))
```

1.2.0.2 Python

```
df = pd.DataFrame({"REF_AREA": ["AGO", "BFA"], "OBS_VALUE": [0.5, 0.7]})
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        isid=["REF_AREA"],
        metadata={"title": "Education indicators", "vintage": "2026-05"})
```

1.2.0.3 Stata

```

use "input.dta", clear
dw_save, filename("dw_ed_edu.dta") ///
    path("$teamsWrkData/ed")        ///
    idvars(REF_AREA INDICATOR)      ///
    title("Education indicators")    ///
    vintage("2026-05")

```

The `kind` argument in the examples names which tier of the warehouse a file belongs to: `kind = "wrk"` is a *working* (intermediate) output, written to scratch and freely re-runnable; a *canonical* write is the final deposit that downstream products cite, and the toolkit guards it with the Z: drive mirror and the reviewer-mode write lock described below. The examples deliberately use the working tier, which is the common case during a pipeline run.

That parity is the ideal, and most of this handbook is organised around it. But the parity is not yet complete. The current gaps:

- **Stata has no external-API wrapper** — the API contract (`dw_api_fetch` and friends) exists only in R and Python. Stata also has no Parquet support; pipelines that need Parquet route through R or Python on the producer side and deposit a `.dta` for Stata consumption.
- **`dw_merge` and `dw_verify_z` are R + Python only.** Stata folds the uniqueness check into `dw_save` (via its `idvars()` option) rather than exposing a standalone `dw_isid`.
- **The demographics family:** `dw_pop` ships in R and Python; `dw_regions` is still R-only. A Stata sibling for `dw_pop` and Python/Stata siblings for `dw_regions` remain on the roadmap.
- **Reference documentation is asymmetric.** R has Roxygen-complete reference pages and a pkgdown site; Python has hand-written references rather than autogenerated, pkgdown-style docs.

The cross-language coverage matrix in Appendix A is the authoritative, function-by-function record of what exists where. Wherever this book shows code, it shows all three siblings side by side; where a language is missing a capability, it is marked, not faked.

1.3 The roles this toolkit serves

`cso-toolkit` divides everyone who touches the data warehouse into three roles, each with a strict capability boundary the toolkit enforces at every wrapped call site:

Role	What they do	Network access
PRODUCER (Database Manager, DBM)	Runs the sector pipeline, pulls upstream APIs, deposits the final <code>dw_<sector>.<ext></code> into the canonical warehouse, and writes a submission template (a pre-deposit checklist documenting the deposit).	Yes — <code>dw_apis_allowed = TRUE</code> .
REVIEWER (Database Reviewer, DBR)	Re-runs the pipeline from pre-deposited inputs, compares against the canonical deposit, files issues. Writes only to a sandbox.	No — every API call site raises a mode-lock error.
PUBLISHER (Database Publisher, DBP)	Pulls signed-off deposits into <code>data.unicef.org</code> , SDMX, and downstream products.	N/A (infrastructure outside this repo).

The mode is not a per-call argument but a **session property**. It is read once at profile-load time (`dw_mode`) and then enforced automatically everywhere. A reviewer does not have to remember to avoid the network; the toolkit makes the network unreachable for them. This is why we say the contract is enforced, not merely conventional.

1.4 Who this handbook is for

This book is written for several audiences at once:

- **Analysts and sector teams** working in R, Python, or Stata who need to read, write, and aggregate warehouse data the CSO way. You will spend most of your time in Part III (the cross-language API).
- **Database managers** (producers) and **reviewers** who operate the producer / reviewer / publisher mode contract day to day. Part II explains the roles, how the mode is wired into a repo, and what the provenance sidecars and error envelopes guarantee.
- **Repo adopters** — maintainers of DW-Production and of future deposit-bearing repositories such as UNPD-Population and datalib — who want to vendor the toolkit into their own codebase and keep it in sync. Part IV is your section.
- **LLM and coding-agent developers** — building *assistants* that turn documentation, existing sector-specific helpers, and legacy code into the standard functions this toolkit supports; or *orchestrators* that compose the toolkit’s deterministic functions as

trusted building blocks. The handbook is written to be precise enough for a machine to follow — see § [How coding agents should use this handbook](#).

You do not need to read the book front to back. Use the part that matches your role and follow the cross-references to adjacent chapters.

1.5 How coding agents should use this handbook

A growing share of code — across software generally, and increasingly in official statistics — is written or reviewed with LLM assistance. Part of the toolkit’s value is that it imposes a **minimum common architecture** — a minimum set of guardrails — on that code: no matter who (or what) writes a script, routing it through the wrappers guarantees the same provenance sidecars, the same `isid` uniqueness check, the same mode-aware role enforcement, and the same error envelope. Those guardrails do not depend on the author’s discipline — which makes LLM-assisted code *safer to integrate*: they enforce provenance, uniqueness, and role boundaries regardless of who wrote the code, but they do not certify that the numbers are correct. That remains the reviewer’s job.

But that safety is structural; it is not a transfer of responsibility. **LLM assistance does not dilute accountability — it concentrates it.** Code now gets produced and checked at machine speed and scale. That means the people who *lead, review, and publish* the results must vouch for more output, generated faster by a process whose plausible-looking errors are easy to miss. The toolkit shrinks the surface they have to police; it does not sign off in their place. The named producer, reviewer, and publisher remain accountable for every number (see § [The roles this toolkit serves](#)) — and the higher the degree of automation, the higher the bar for the human scrutiny that accountability demands.

For an LLM or an agentic orchestrator, that implies:

- **Map onto the toolkit’s functions; don’t reinvent them.** When porting documentation or a sector’s bespoke helpers, translate them into `dw_save`, `dw_api_fetch`, `aggregate_data_v2`, and the rest — the verified, reproducible primitives — not equivalent ad-hoc logic.
- **Compose, don’t regenerate.** An orchestrator calls the deterministic functions as building blocks; the LLM handles orchestration and glue, not the contract’s enforced behaviour. The determinism is the point.
- **Operate within the human’s role and session.** An agent runs under the session’s `dw_mode` — in reviewer mode the network is unreachable — and must not flip the mode or reach for the `allow_canonical_write` / producer-bootstrap escape hatches on its own; those exist for explicit human action. The agent gets no role of its own; accountability stays with the named producer, reviewer, or publisher.
- **Never fabricate values, provenance, or coverage.** Read and cite the provenance sidecar; surface gaps from the coverage matrix ([Appendix A](#)) rather than faking parity.
- **Treat the contract as binding.** Code that bypasses the wrappers is flagged by the contract auditor ([Part IV](#)); generate code that passes it.

1.6 How to navigate this book

The handbook is organised into four parts and a set of appendices.

- **Part I — Why & Architecture** motivates the toolkit (why a shared repo at all) and lays out the architecture: one contract, three language siblings, many consumer repos, and a single canonical deposit. Start here if you want the big picture before the API details.
- **Part II — The Contract** is the conceptual core. It defines the three roles and the session-level mode contract, shows how to wire that contract into a repo's profile, and explains the two cross-cutting mechanisms that make the toolkit trustworthy: provenance sidecars and the standard error envelope.
- **Part III — The API (cross-language)** is the reference for daily work, one chapter per capability family: the IO contract; the external-API contract; aggregation; vintage sync; and scaffolding plus the contract auditor. Every chapter presents R, Python, and Stata together and is explicit about coverage gaps.
- **Part IV — Adoption & Operations** is for adopters and maintainers: the vendoring model and why it is preferred over network installation, a worked example of onboarding a repo, running the contract auditor in CI, and the project's versioning and release discipline.
- **Appendices** collect the dense reference material: the cross-language coverage matrix (A), the provenance sidecar schema (B), the error taxonomy (C), a full function reference (D), and a glossary (E).

If you are brand new, read the architecture chapter in Part I and the roles chapter in Part II, then jump to the IO contract chapter in Part III. Those three pages give you enough to use the toolkit safely.

If you are here to adopt the toolkit into a new repository, read the vendoring-model and onboarding chapters in Part IV first; they assume the role and mode concepts from Part II, so keep the roles chapter to hand.

i Note

This handbook documents the toolkit as it actually exists in the repository, including its current asymmetries between languages. Where this book describes a capability as present in one language but not another, that reflects the real state of the code, not an aspiration. Treat the coverage matrix in Appendix A as the source of truth when in doubt.

Part I

Part I — Why & Architecture

2 Why a toolkit

A 2026 analysis must re-run identically in 2028. That single requirement — call it the *reproducibility floor* — is the reason `cso-toolkit` exists. Everything else in this book follows from it.

The toolkit is the shared, cross-language contract that the UNICEF Chief Statistician Office (CSO), within the Office of Strategy and Evidence (OSE), uses to read, write, compare, and merge data; to call external APIs; to aggregate; and to keep copied-in (“vendored”) helper code in sync. Concretely, it is one input/output (IO) contract, one external-API contract, one *mode contract* — a session-level switch that governs whether a run may touch the network — three implementations (R, Python, Stata), and one vendoring model. This chapter sets out the problem the toolkit solves and motivates the design choices the rest of the book documents.

2.1 The problem: analytics that cannot be re-run

A statistical office publishes numbers, and those numbers carry a promise: that they are not an accident of the machine, the week, or the analyst who happened to produce them. A figure cited in a 2026 publication — say, an education indicator deposited as `dw_ed.csv` — must, two years later, produce the *same* number when a different person re-runs the pipeline.

That promise is surprisingly hard to keep. Three things routinely break it:

- **Upstream drift.** The external data source changes between the original run and the re-run. An SDMX (the statistical data-exchange standard) dataflow gets revised, a World Bank indicator is re-based, a UIS (UNESCO statistics) endpoint returns a slightly different shape. The re-run pulls “live” data and quietly produces different numbers.
- **Ambient version skew.** The original analyst had one version of `dplyr` (or `pandas`, or a Stata `.ado`); the reviewer has another. A default changes, a sort becomes unstable, a join behaves differently. Nothing errors — the answer just shifts.
- **Untracked provenance.** Nobody recorded what was read, from where, with what schema, by whom, and when. When the numbers disagree, there is no artefact to point at and no way to tell which run is wrong.

These are not hypothetical. For instance, an internal cross-sector audit (23 May 2026) found four regional-metadata CSV files being read independently across five sectors, each with no shared vintage record — four chances for the same kind of file to disagree, with nothing

recording which copy any run actually used. That is exactly the failure mode the toolkit is built to remove.

The toolkit’s answer is to make every data touch-point go through a wrapper that enforces the same discipline everywhere: a single way to read and write that records provenance automatically, a single way to call an external API that can be frozen and replayed, and a session-level contract that decides whether the network may be touched at all. The repository README states the goal directly: the toolkit exists “to facilitate the reproducibility and scalability of analytics developed by the UNICEF Chief Statistician Office.”

2.2 The reproducibility floor

A floor is a guarantee you inherit for free, not a feature you opt into. The toolkit’s ambition is that any sector codebase that vendors the helpers gets the floor automatically — the analyst does not have to remember to record provenance, does not have to wire up a cache, does not have to police whether a reviewer accidentally hit the network. The wrappers do it.

Concretely, the floor is built from a few primitives that recur throughout the book:

- **Provenance sidecars.** Every write through `dw_save()` — the toolkit’s write wrapper — emits a `.provenance.json` sidecar by default, recording a `sha256` of the artefact, its schema, the user, a timestamp, and caller-supplied metadata. (You can opt out with `provenance = FALSE`; `.RData` / `.rda` writes skip the sidecar.) When two runs disagree, the sidecars tell you which input changed. The provenance-sidecars-and-error-envelope chapter, and the sidecar-schema appendix, document the format.
- **A frozen deposit you can replay.** External API calls route through `dw_api_fetch()` and its companions, which cache responses under a deposit directory. A re-run reads the cache rather than the live source, so upstream drift cannot leak into a reviewer’s numbers. The external-API contract chapter covers this.
- **Uniqueness and integrity checks.** `dw_isid()` asserts that an identifier set is unique; canonical reads verify integrity. Silent duplication and silent corruption are caught at the call site, not three steps downstream.

The point of routing every call through a wrapper is not ceremony. It is that the contract is enforced “at every wrapped call site, not by convention” — the README is emphatic on this. Convention is what fails when someone is in a hurry; an enforced contract does not.

2.3 Vending, not installing

The second pillar follows from the first. If a 2026 analysis must re-run identically in 2028, then the *helper code itself* must be the same in 2028 as it was in 2026. A toolkit that is installed over the network — `devtools::install_github(...)`, `pip install`, `net install`

— does not give you that. At session start you get whatever version the network happens to serve, and that is precisely the upstream-drift problem applied to your own tools.

So consumers do not install the toolkit. They **vendor** it: they copy the helper files into their own `00_functions/` directory, commit those copies to their own git history, and pin the vendored vintage in a one-line manifest. The toolkit-strategy document gives four reasons for this choice:

1. **Reproducibility / vintage permanence.** Every consuming repo’s git history shows exactly which helper code was in effect at any commit. There is no gap between “the repo’s pinned version was X” and “the running session installed Y.” A 2026-05 release re-run uses the helper code as it stood in 2026-05.
2. **Producer-controlled refresh.** Toolkit updates are not pulled automatically. The producer reads the release notes, decides whether the new vintage is appropriate, and refreshes explicitly. Upgrades are a deliberate act, not a side effect of opening a session.
3. **Reviewer offline reproducibility.** A reviewer with no network access can still re-run the full pipeline — on a plane, behind a corporate firewall, at a customs desk — because the vendored helpers are right there in the repo.
4. **Cross-language symmetry.** Stata equivalents follow the same model (the `ado`-files are copied into the consumer’s repo), and a package-install model would not fit Stata well in the first place. Vendoring is the one model that works for all three languages.

There is a fifth, blunt reason specific to the operating environment: **AppLocker reality.** UNICEF laptops block many script-installable paths. Copying files into `00_functions/` always works where a network install may not.

The vendored vintage is recorded in a small manifest, `.toolkit_manifest.yml`, that names the upstream source, the pinned tag, when and by whom it was pulled, and which files were vendored:

```
source: "unicef-drp/cso-toolkit"
pulled_version: "v0.5.0"
pulled_date: "2026-06-21"
pulled_by: "<your-name>"
vendored:
  - dw_io.R
  - dw_api.R
  - cso_toolkit_sync.R
local_edits: []
```

For local development each language still offers a native install path (`devtools::install_local("r/")`, `pip install -e python/`, `adopath ++ "stata/src"`), but vendoring stays the production model. The vendoring-model chapter and the manifest material in Part IV cover the mechanics and the upgrade flow; here the point is only *why* the toolkit is copied rather than installed.

i Note

Vendoring trades a little convenience for a hard guarantee. You take on the small chore of an explicit refresh; in return, the helper code can never drift out from under a published result. For a statistical office, that trade is the whole point.

2.4 Scaling across sectors and projects

The third pillar is scale. The CSO is not one pipeline; it is many — education, water and sanitation, nutrition, health, poverty, immunisation, and more — and the toolkit’s consumers extend beyond DW-Production (the cross-sector deposit pipeline) to other repositories that will deposit canonical data, such as UNPD-Population and `datalib`. Without a shared contract, each of these would re-invent the same primitives, and each re-invention is a new chance to drift, as the five-sector regional-metadata duplication showed.

A single vendored toolkit changes the economics. The same helpers, the same templates, and the same audit functions are vendored into every codebase, so a new sector or a new project “inherits the reproducibility floor for free instead of re-inventing it.” Adding the next sector is a vendoring step, not a re-implementation. This idea explicitly gates the toolkit’s v1.0.0: the toolkit reserves that release for after the education pilot lands *and a second sector vendors the helpers without modification* — proof that the contract generalises rather than being bent to one team’s habits.

Scale is one reason the contract is cross-language. Sectors work in R, Python, and Stata, and the toolkit’s core principle is parity: the commands share the same names and the same behaviour across all three. The same write call looks like this in each:

2.4.0.1 R

```
dw_save(df, name = "dw_ed.csv", sector = "ed", kind = "wrk",
        isid = "REF_AREA",
        metadata = list(title = "Education indicators", vintage = "2026-05"))
```

2.4.0.2 Python

```
dw_save(df, name="dw_ed.csv", sector="ed", kind="wrk",
        isid=["REF_AREA"],
        metadata={"title": "Education indicators", "vintage": "2026-05"})
```

2.4.0.3 Stata

```
dw_save, filename("dw_ed") path("$teamsWrkData/ed") ///
      idvars(REF_AREA INDICATOR)                      ///
      title("Education indicators")                   ///
      vintage("2026-05")
```

Warning

Parity is the goal, not yet the whole reality, and the book is honest about the gaps. As of v0.5.0 the external-API wrapper exists only in R and Python — Stata has no API wrapper — and the Stata side has no parquet support (`.parquet` reads are explicitly out of scope; route them through R or Python and `dw_save()` to `.dta`). Several IO helpers — `dw_merge` and `dw_verify_z` — are R + Python only. Note too that the Stata commands take comma-separated options (`dw_save, filename(...) path(...)`), not the `using` form some R-flavoured examples might lead you to expect. The cross-language coverage matrix appendix records exactly which function exists in which language and in which shape; treat it as the source of truth rather than assuming every name works everywhere.

2.5 The producer / reviewer split, at a high level

The reproducibility floor needs one more idea to hold: someone has to be allowed to touch the live network, and someone has to be forbidden from it — and the toolkit has to *enforce* the difference rather than trust people to remember it.

The toolkit therefore distinguishes roles by what they are permitted to do, and it makes the distinction a session-level mode — `dw_mode`, read once when the profile loads from `~/.config/user_config.yml` — not a per-call argument an analyst can forget or override. At a high level:

- **Producer** (the Database Manager) runs the sector pipeline, pulls upstream APIs — with caching — and deposits the canonical artefacts. *Canonical* here means the official, signed-off version of record: the deposit other people read and trust. The producer is the one role with network access.
- **Reviewer** re-runs the pipeline from the frozen canonical deposit, compares against it, and files issues. The reviewer is *physically prevented* from calling the network: with `dw_apis_allowed = FALSE`, every API call site raises a mode-lock error rather than reaching out. A reviewer who is forced offline cannot accidentally “fix” a discrepancy by pulling fresh data — which is exactly what makes their re-run a real check.
- **Publisher** (a downstream role) takes signed-off deposits into `data.unicef.org`, SDMX, and other products. This role lives largely outside the toolkit’s own boundary.

This split is what closes the loop on the reproducibility floor. The producer’s run is the run of record; the reviewer’s run, executed against frozen inputs with the network sealed off, is the proof that the run of record reproduces. Because the boundary is enforced at every wrapped call site, the guarantee does not depend on anyone behaving well on a busy day.

The roles and the mode contract are the subject of the next part of the book — the roles-and-mode-contract chapter defines each role precisely, and the wiring chapter shows how `dw_mode` is set in a sector profile. Hold the shape in mind: *one contract, two enforced postures toward the network, one direction of trust from producer to reviewer.*

2.6 What this buys you

Pulling the three pillars together: the toolkit gives the CSO a reproducibility floor (provenance, frozen replay, integrity checks, enforced at the call site), a vendoring model that pins helper code to a vintage so published results never drift, and a single cross-language contract that lets new sectors and new repositories inherit all of it instead of rebuilding it. The producer / reviewer split is the mechanism that turns “we wrote it down” into “we can prove it re-runs.”

The architecture chapter that follows makes this concrete — one contract, three languages, many consumers — and the rest of the book documents each capability family in turn.

3 Architecture: one contract, three languages, many consumers

The previous chapter argued *why* a toolkit is worth building: a 2026 publication that cites `dw_<sector>.csv` must, two years later, produce identical numbers when re-run by someone who has only the canonical deposit and the vendored helpers — no network access, no upstream package drift, no ambient `dplyr` version skew. This chapter describes the *shape* that delivers on that promise. The shape is layered, and it can be read in one sentence: the toolkit encodes a contract once; three language siblings implement that contract identically; consumer repositories copy (“vendor”) the implementations into their own code; and everything they produce lands in a single canonical deposit that the contract keeps reproducible.

The README’s “Architecture at a glance” diagram (a Mermaid flowchart in the repository’s `README.md`) draws all four layers in one picture: external data sources on the left, the toolkit and its five capability families in the centre, the three language siblings below, the consumer repositories that vendor them, and the canonical deposit on the right. The text that follows walks through each layer in turn.

3.1 The four layers

The layers nest, so name them first:

Layer	What it is	Where it lives
The toolkit	One contract, expressed as five capability families	<code>unicef-drp/cso-toolkit</code> (this repo)
Language siblings	Three idiomatic implementations of that one contract	<code>r/R/</code> , <code>python/src/</code> , <code>stata/src/</code>
Consumer repos	Codebases that vendor the helpers and use the contract	DW-Production, sector pipelines, CCRI (the Children’s Climate Risk Index) / geospatial work
Canonical deposit	The frozen store every producer writes and every reviewer reads	Teams 060.DW-MASTER/, the Z: drive mirror, downstream Helix / SDMX

The layers have a strict direction of dependence. The toolkit knows nothing about any consumer. The language siblings know nothing about each other — they agree only on the contract, never on shared code. Consumers depend on a pinned version of one language sibling. The canonical deposit depends on nothing; it is the inert artefact at the end of the flow — and that inertness is exactly what we want: a reviewer reads it without re-running anything upstream of it.

A note on the names in that last table column. The **Z: drive** is a network-mapped mirror of the Teams deposit, used so that scripts can address the canonical store through a stable drive letter rather than a synced-folder path. **Helix** is UNICEF’s internal data platform, and **SDMX** (Statistical Data and Metadata eXchange) is the statistical-data exchange standard through which blessed artefacts reach `data.unicef.org`. These are downstream of the toolkit, not part of it; the architecture’s job ends when the deposit is written.

3.2 Layer one — the toolkit and its five capability families

The toolkit’s helpers group into five capability families, and each family is the *single* sanctioned way to do its job inside a CSO analytics product — not a grab-bag of utilities. The families are: IO, external API, vintage sync, aggregation, and scaffolding / audit. The rest of Part III treats each family in full; this section names each one and its public functions, so the architecture reads as a whole.

3.2.1 IO contract

One way to read, write, compare, and merge data, auto-dispatched by file extension: CSV / TSV / XLSX / RDS-or-PKL (R / Python native) / DTA (Stata) / Parquet / JSON / YAML. The family’s public functions are `dw_save`, `dw_use`, `dw_compare`, `dw_merge`, `dw_isid`, `dw_verify_z`, `dw_stage`, and `dw_root`: the first four read, write, diff, and join; `dw_isid` enforces row uniqueness; `dw_verify_z` checks the Z: mirror byte-for-byte; `dw_stage` copies a canonical input into the reviewer sandbox under a hash guard; and `dw_root` resolves a logical store (`wrk` / `raw` / `meta`) to its filesystem path. Writes emit a `.provenance.json` sidecar by default; canonical writes are carbon-copied to the Z: drive; canonical reads run an integrity check. See the IO contract chapter for the full treatment.

3.2.1.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        isid = "REF_AREA",
        metadata = list(title = "Education indicators", vintage = "2026-05"))
df <- dw_use(name = "dw_ed_edu.csv", sector = "ed", kind = "wrk")
```


3.2.1.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        isid=["REF_AREA"],
        metadata={"title": "Education indicators", "vintage": "2026-05"})
df = dw_use(name="dw_ed_edu.csv", sector="ed", kind="wrk")
```

3.2.1.3 Stata

```
dw_save, filename("dw_ed_edu.dta") ///
    path("$teamsWrkData/ed")          ///
    idvars(REF_AREA INDICATOR)        ///
    title("Education indicators")      ///
    vintage("2026-05")
dw_use, filename("dw_ed_edu.dta") path("$teamsWrkData/ed")
```

Here `isid` names the columns that uniquely identify a row; the IO family enforces that uniqueness on write. The Stata sibling now ships `dw_use` as well as `dw_save` (the read contract landed in v0.4.0, closing issue #5), so the read / write pair is available in all three languages.

3.2.2 External-API contract

One mode-aware wrapper around ten external data sources — UNESCO UIS, generic SDMX, the UNICEF SDMX codelist endpoint, World Bank WDI, ILO SDMX, the UNSD SDG API, pinned-commit GitHub-raw, and generic HTTP / JSON. A producer hits the live API and caches the response under `_apis/<api>/<cache_key>.<ext>`; a reviewer reads only from that cache and is physically prevented from reaching the network. Secrets in caller keyword arguments are redacted before they reach the provenance sidecar. See the external-API contract chapter.

3.2.2.1 R

```
df <- dw_api_fetch(api = "wb", indicator = "SE.PRM.NENR", refresh = TRUE)
df <- dw_api_cached(api = "wb", cache_key = "SE.PRM.NENR")
dw_api_inventory(api = "wb")
```

3.2.2.2 Python

```
df = dw_api_fetch(api="wb", indicator="SE.PRM.NENR", refresh=True)
df = dw_api_cached(api="wb", cache_key="SE.PRM.NENR")
dw_api_inventory(api="wb")
```

3.2.2.3 Stata

```
* No API *fetch* wrapper exists in the Stata sibling.
* Stata producers fetch upstream data through the R or Python sibling,
* then read the resulting cache or canonical artefact.
* The Stata sibling DOES enforce the reviewer-mode no-API guard:
dw_require_no_api , context("ed/06_pull_uis")
```

Warning

The external-API *fetch* family is an R-and-Python capability only. The Stata sibling has no `dw_api_fetch` / `dw_api_cached` / `dw_api_inventory`; a Stata-only producer cannot refresh an upstream cache, and that step runs in R or Python. What Stata does provide is the *enforcement* half of the contract — `dw_require_no_api` aborts a reviewer-mode session that is about to make a live network call. The cross-language coverage matrix appendix records the gap explicitly.

3.2.3 Vintage sync

Consumer-side helpers — `cso_toolkit_check`, `cso_toolkit_diff`, and `cso_toolkit_pull` — that read the local `.toolkit_manifest.yml`, query upstream for the latest tag, and warn or refresh when the consumer is behind the pinned version. The network call these make is itself gated by the mode contract: reviewer mode forbids it. See the vintage-sync chapter.

3.2.4 Aggregation and survey weights

`aggregate_data_v2()` computes `weighted_mean` / `mean` / `sum` / `proportion` with population and country coverage and a coverage threshold (the original `aggregate_data()` is retained for back-compatibility); `apply_time_window()` filters to the latest observation per country within an inclusive year window; `generate_agg_footnote()` produces the matching coverage footnote; `dw_nestweight()` is an R port of the World Bank EduAnalyticsToolkit `edukit_nestweight` (Diana Goldemberg), which redistributes survey weights from missing nested observations so per-stratum totals are preserved; and `dw_pop()` and `dw_regions()` supply the population denominators and regional groupings the aggregators consume. The two demographics helpers are R-only for now (no Python or Stata sibling yet). See the aggregation chapter.

3.2.5 Scaffolding and contract audit

`create_profile()` scaffolds a `profile_<repo>.R` (or `.py`) with the standard CSO building blocks — cross-platform user identification, YAML config load, producer / reviewer `dw_mode` resolution, an optional Z: drive advisory, a packages block, and a sentinel object. `review_profile()` audits an existing profile against the same building blocks and reports `pass` / `warn` / `fail` per check. `create_sector_script()` scaffolds a sector run-script template with profile verification, logging, runtime tracking, and try/catch. `test_scripts()` is the contract auditor: it recursively scans a directory of scripts and flags any direct call to a raw file-IO or external-API command that the toolkit wraps — `read_csv`, `pd.read_csv`, `http::GET`, `requests.get`, `rsdmx::readSDMX`, `sdmx.Client`, and so on — with a per-line escape hatch (`# cso-allow: <rule-id>`) and a CI mode (`error_on_violation = TRUE`) that fails the build on any violation. See the scaffolding and contract-auditor chapter.

3.2.6 The error envelope is not a sixth family

One behaviour cuts across all five families rather than forming its own: the graceful error envelope. Because a consumer needs to grep for and act on library errors, every `stop()` in R, every `raise` in Python, and every `abort` in Stata follows a three-part shape:

```
[cso_toolkit.dw_save] Reviewer mode forbids writes under canonical: /path
Reviewer sessions must keep canonical deposits read-only to preserve
vintage permanence; writes go to the sandbox.
Fix:
  1. Resolve a sandbox path instead, OR
  2. If this is a deliberate DBM bootstrap, pass `allow_canonical_write = TRUE`.
```

The leading `[cso_toolkit.<func>]` prefix lets a consumer project grep for every site that hits a given error class. The shape is `[cso_toolkit.<func>] WHAT / Why / Fix`. The roles chapter and the error taxonomy appendix return to this.

3.3 Layer two — three language siblings of one contract

The toolkit ships three implementations: R under `r/R/`, Python under `python/src/`, and Stata under `stata/src/`. The word that matters is *siblings*, not *ports* and not *bindings*. They are not generated from a common source, and they do not call into one another at runtime. They are independent code that agrees on a contract. The README states the agreement precisely: same function names, same `.provenance.json` schema, same mode-aware path routing, same error-envelope shape.

This works only because the contract is defined behaviourally, not as shared code. A `dw_save` in any sibling dispatches on file extension, enforces an `isid` uniqueness check, writes a sidecar with the same JSON schema, mirrors canonical writes to Z:, and honours the session-level producer / reviewer mode — and it produces the *same numbers* given the same inputs. The

compatibility is in the observable behaviour, which is exactly what reproducibility cares about.

i Note

Cross-language parity is the design goal, not a finished fact. The gaps worth naming up front, all recorded in the coverage matrix appendix, are:

- **No Stata API *fetch* wrapper.** `dw_api_fetch` / `dw_api_cached` / `dw_api_inventory` are R and Python only. Stata ships the enforcement half (`dw_require_no_api`) but not the fetch half.
- **No Stata *parquet* or *RDS*, and `dw_merge` / `dw_verify_z` are not in the Stata sibling.** `dw_isid` in Stata is embedded inside `dw_save` (the `idvars()` option) rather than exposed as its own command. (The read contract, `dw_use`, is now in Stata as of v0.4.0; issue #5 is closed.)
- **The demographics helpers are partially ported.** `dw_pop` ships in R and Python; `dw_regions` is still R-only. Neither has a Stata sibling yet, so a Stata consumer that needs population denominators or regional groupings routes through a sibling or a pre-computed artefact.
- **No Python *autodoc*.** R has Roxygen-complete reference documentation (34 `.Rd` files plus a pkgdown site configured in `r/_pkgdown.yml`); Python has no equivalent pkgdown-style generated reference, only hand-written references and module READMEs.

Treat the parity claim as “same names, same behaviour where a function exists in both,” not “every function exists everywhere.”

Maturity differs by sibling as well as by function. The R sibling was feature-complete first and leads the release line; the Python sibling followed as a full port (nine functional modules and twenty-eight public entries) but has not yet reached full feature parity with it; the Stata sibling shipped a focused subset and is still filling in. As of v0.5.0 the Stata sibling exposes six commands — `dw_save`, `dw_use`, `dw_compare`, `dw_mkdir`, `dw_load_config`, and `dw_require_no_api`. The versioning-and-releases chapter traces this timeline; the point for the architecture is that a consumer chooses *one* sibling and gets whatever that sibling implements, so the choice of language is also a choice of current coverage.

3.4 Layer three — consumer repositories vendor the helpers

A consumer repository does not install the toolkit over the network. It *vendors* it: it copies the helpers for its chosen language into its own `00_functions/` directory and pins a version in a one-line manifest, `.toolkit_manifest.yml`. DW-Production is the first consumer; the sector pipelines are consumers through it — education, water and sanitation, nutrition, HIV/AIDS, child poverty, immunisation, and the rest; the CCRI / geospatial work consumes the Python sibling directly. Future repositories — UNPD-Population, datalib — are expected to vendor the same way.

Vendoring rather than network-installing is deliberate. The reasons:

- **Vintage permanence.** A 2026-05 release re-run must use the helper code as it stood in 2026-05. Network `source()` / `pip install` / `net install` would silently pull whatever the latest code is, breaking the freeze.
- **AppLocker reality.** UNICEF laptops block many script-installable paths; copying files into `00_functions/` always works where a network install may not.
- **Offline reproducibility.** Reviewers on planes, behind customs firewalls, or on locked-down corporate networks need the helpers present locally.

Each language does still offer a native install path for local development — `devtools::install_local("r/")` `pip install -e python/`, `adopath ++ "stata/src"` — but vendoring stays the production model. The vendoring chapter and the onboarding worked example in Part IV cover the mechanics; the architectural fact is simply that the dependency from consumer to toolkit is a *copy plus a pin*, not a live link. That is what lets a consumer be re-run identically years later, and it is what lets the vintage-sync family detect, without ever forcing, when a consumer has drifted behind the upstream tag.

3.5 Layer four — the canonical deposit

Everything the consumers produce in producer mode flows into one place: the canonical deposit. Physically it is the Teams folder `060.DW-MASTER/`, carbon-copied to a Z: drive mirror, and from there published downstream to Helix, the SDMX endpoints, and `data.unicef.org`. Architecturally it is the immutable record that closes the reproducibility loop. The producer writes it; the reviewer reads it, frozen, and never re-runs anything upstream of it; the publisher pushes blessed copies of it to the outside world.

The deposit is where the contract’s two halves meet. The IO family writes artefacts here with provenance sidecars and a Z: mirror; the external-API family caches upstream pulls here so they too become part of the frozen record; the mode contract draws its hard line right at this boundary, allowing a producer to write canonical paths and forcing a reviewer’s writes into a per-user sandbox instead. The roles-and-mode-contract chapter develops the producer / reviewer / publisher split in full; for the architecture, the deposit is best understood as the inert artefact that makes the whole layered design pay off — because it can be read without being recomputed, the numbers it holds stay reproducible.

3.6 Reading the architecture diagram

The README’s “Architecture at a glance” Mermaid flowchart encodes the same four layers and the direction of flow between them. The diagram uses exactly two line styles — solid arrows (`-->`) and dashed arrows (`-.>`) — and three reading aids unlock it:

- **Solid versus dashed arrows.** Solid arrows are normal producing flow: external sources into the API family, the toolkit into the consumers (via the siblings), and consumers depositing into the canonical store. Dashed arrows are the two reproducibility-and-parity relationships, and they are the *only* non-solid lines in the picture.
- **The dashed line into the language siblings** (TK $-. \rightarrow$ LANG, labelled “language-parallel implementations”) marks the “same contract, three idiomatic implementations” relationship — independent code that agrees behaviourally, not generated bindings.
- **The dashed line from the deposit back to the consumers** (CANON $-. \rightarrow$ CONS, labelled “reviewer reads (frozen)”) is the reviewer path: reading the frozen deposit without recomputation.

The one edge that carries a version number is the **vendorship edge** (LANG $-->$ CONS), a *solid* arrow labelled `.toolkit_manifest.yml` — the file that records the pinned version. It is the copy-plus-pin dependency from each consumer back to a sibling, and the vintage-sync family is the machinery that watches it.

i Note

If you are matching the prose to the rendered picture: there is no dotted style in this diagram. Every non-solid arrow is the same Mermaid dashed style ($-. \rightarrow$), and there are exactly two of them — the language-parallel edge and the reviewer-reads edge.

3.7 The wider ecosystem: three tiers of shared code

The toolkit is not the only shared code a CSO analytics product touches, and this handbook’s authority is not uniform across everything a pipeline imports. As of 2026-07, the shared code around a CSO pipeline falls into three tiers, and the tier determines both who maintains the code and what this book asserts about it:

Tier	What it is	Members	Canonical home	What this book asserts
(i) The toolkit	The <code>dw_*</code> contract and its three language siblings — everything the preceding sections describe	<code>csotoolkit</code> R / Python / Stata	<code>unicef-drp/cso-toolkit</code>	Normative. This book <i>is</i> the documentation; the coverage-matrix appendix is its honesty device

Tier	What it is	Members	Canonical home	What this book asserts
(ii) UNICEF-supported packages	Full packages with their own release lines, maintained by CSO staff in their own repositories	<code>unicefData</code> (public); <code>datalib</code> (provisional member — public release pending); a mapping/boundaries capability (under evaluation)	Each package’s own <code>unicef-drp</code> repository	Positioning and the cross-cutting contracts (configuration, conformance conventions). Per-function documentation stays in the package’s own repository — this book points, it never re-documents
(iii) Third-party functions	External packages used inside production pipelines	e.g. community Stata commands, CRAN/PyPI utilities	Upstream registries (CRAN, PyPI, SSC)	Only that a named version behaved as described on a stated date in DW-Production. No maintenance promise, no conformance claim

Membership in tier (ii) is criteria-based, not honorary: a public `unicef-drp` home, tagged releases, CI-executed tests, and changelog discipline. `datalib` — the survey-microdata archive tool — meets all but the public-release criterion today and is listed as *provisional*; a supported **mapping/boundaries package is under evaluation, with two internal candidates**, and is not documented here until the evaluation resolves and the winner passes the same gate. The inclusion rules and the verified list for tier (iii) live in the curated-third-party appendix. This section is the orientation; the ecosystem chapter in Part VI develops the tiers, the membership gate, the per-tier authority claims, and the three configuration contracts in full, and documents `unicefData`, the first full tier-(ii) member.

The parity pledge. Within tier (i), the steady-state commitment is *equal* R, Python, and Stata support: every cell of the coverage-matrix appendix must eventually read as fully supported, partially supported with a documented scope, or not-applicable with a stated design reason — never a bare gap. The matrix is the honesty device that tracks the distance between that pledge and today’s reality; the cross-language conformance harness (`conformance/`) is the machinery that keeps the “supported” cells honest by executing the parity claim in CI.

i Configuration across the tiers

The three tiers do not yet share a configuration contract, and the divergence is deliberate until a common design is agreed. The toolkit reads flat keys from `~/.config/user_config.yml` through generated profile blocks and `dw_load_config`; `datalib` reads a `datalib:` block from its **own package-specific config file** (a shared-file arrangement is planned but waits on a parser guard in `dw_load_config`, so do not add a `datalib:` block to the toolkit's `user_config.yml` yet); `unicefData` is configured through environment variables (`UNICEF_DATA_HOME` and friends) and reads no user config file at all. If you operate across tiers, treat the three mechanisms as separate namespaces.

The four layers — toolkit, siblings, consumers, deposit — are the frame to hold for the rest of the book. From here, Part II elaborates the contract those layers carry; Part III gives each capability family its own chapter; and Part IV covers how a repository joins the architecture as a new consumer and stays in sync over time.

Part II

Part II — The Contract

4 Roles and the mode contract

DW-Production is not one job done by one person. It is a relay. Upstream agencies publish their statistics; a Database Manager refreshes caches and produces deposited indicators; an auditor reproduces those outputs and files findings; and a publisher pushes approved data packages out to UNICEF’s data warehouse and the public-facing pages that draw on it. Each runner holds a different baton, touches a different part of the filesystem, and carries a different obligation. The toolkit names these three runners explicitly — **PRODUCER**, **REVIEWER**, and **PUBLISHER** — and it does not rely on discipline alone to keep them apart. The toolkit enforces two of the three in code: a session set to reviewer mode is physically prevented from writing canonical artefacts (the single authoritative copy of the data everyone downstream trusts) or from calling live APIs.

This chapter sets out the three roles, what each may read and write, and how the `dw_mode` session setting turns those policies into mechanical guarantees.

Two terms recur throughout and are worth fixing now. A **deposit** is a finished data package a producer places in the canonical store. A deposit is **blessed** once it has passed the producer’s pre-deposit checklist and the reviewer’s compare report — only blessed deposits may be published.

4.1 The three roles

The roles map onto a real organisational lifecycle. Each carries a short acronym used in the repo’s internal documentation; we use the plain role names — PRODUCER, REVIEWER, PUBLISHER — for the rest of this chapter.

Role	Acronym	Plain meaning
PRODUCER	DBM (Database Manager)	Owns the deposit. Refreshes upstream caches and produces the deposited indicator outputs.
REVIEWER	DBR (Database Reviewer)	Reproduces and validates. Audits, finds bugs, gathers sector-lead sign-off.
PUBLISHER	DBP (Database Publisher)	Pushes blessed deposits into the canonical UNICEF data warehouse and downstream surfaces.

All three share the same helper functions — `dw_save`, `dw_use`, `dw_api_fetch`, `dw_compare`, and the rest — and the same path globals derived from the user profile. What differs is the slice of the filesystem each one may read and write, and whether it may reach the network at all.

4.2 What each role may read and write

The read/write/API matrix is the core of this chapter; everything else elaborates it.

Role	Reads from	Writes to	API calls	Owns
PRODUCER	upstream APIs + the canonical deposit (060.DW-MASTER)	the canonical deposit (Teams + the Z: mirror, automatically)	YES	refreshing cached upstream data; producing the deposited outputs
REVIEWER	the canonical deposit (read-only) + own sandbox	the sandbox only (<code>sandboxRoot</code> , per user)	NO — forbidden by the reviewer-mode guard	reproducibility validation; sector-lead sign-off; finding bugs
PUBLISHER	the blessed work-data deposit (013_wrkdata)	the data warehouse: Helix, SDMX endpoints, data.unicef.org, SoWC tables	reads warehouse endpoints only	publication into the canonical data warehouse and downstream surfaces

The paths are numbered by lifecycle stage. Everything lives under a single root, 060.DW-MASTER: 011_rawdata holds upstream pulls (including the `_apis/` cache), and 013_wrkdata holds finished, promoted deposits. The numbering implies a sequence — raw input becomes work data — not a directory count.

Three things follow from the matrix.

- **Only the PRODUCER touches upstream.** Reviewer sessions never call live APIs; they read frozen, producer-deposited caches instead. This is not a convention you are trusted to honour — the reviewer-mode guard asserts it at every network call site.
- **The REVIEWER reads canonical but writes sandbox.** Reviewer-mode reads fall back to the canonical deposit (with a Z:-drive integrity check), but reviewer-mode writes are routed to a per-user sandbox; an attempted canonical write hard-fails.
- **The PUBLISHER consumes only blessed outputs.** It reads from 013_wrkdata, the work-data deposit a producer has explicitly promoted, never from raw or staging paths.

4.2.1 What each role MUST NOT do

The matrix above says what each role *may* do; the prohibitions below say what each role *must not* — the same boundaries seen from the other side. A relay fails when a runner reaches outside their leg.

Role	Forbidden
PRODUCER	skip the pre-deposit checklist; deposit without a compare-vs-prior-vintage report; deposit without sector-lead sign-off on non-trivial deltas
REVIEWER	write to canonical paths; call external APIs; modify caches in the deposit
PUBLISHER	promote outputs that haven't passed the producer's checklist and the reviewer's compare; publish without a provenance chain back to original API fetches; roll back without amending the source

The producer's prohibitions are *procedural* — enforced by checklist and review, not by code. The reviewer's prohibitions are *mechanical* — enforced by `dw_mode`, as the next section shows. The publisher's prohibitions are mostly procedural today, with one mechanical exception: the R `dw_publish` helper already refuses reviewer-mode submissions (see the publisher note below).

4.3 The data flow

A single indicator moves through the relay like this:

```

External APIs          (UIS · SDMX · WB · ILO · UNSD)
  PRODUCER: dw_api_fetch(refresh = TRUE)

API cache      011_rawdata/.../_apis
                                                    REVIEWER: dw_api_cached
                                                    (cache reads only -
no live calls)
Sector pipeline
  PRODUCER: dw_save(isid = ...)          REVIEWER: writes routed to sandbox

Canonical deposit  013_wrkdata  carbon copy  Z: mirror (integrity check)
  REVIEWER: dw_use (read-only)
```

Compare report Issues filed to PRODUCER / sector lead

PUBLISHER Helix · SDMX endpoints · data.unicef.org · SoWC / SI tables

Read the flow as three lanes. Solid arrows are normal forward flow. The reviewer's arrows into the pipeline and deposit are read-only. The publisher's arrows fan out from the blessed 013_wrkdata deposit to downstream consumers. The diagram makes plain why mode enforcement matters: the reviewer's pipeline runs the *same scripts* as the producer's, so the only thing standing between an audit run and a corrupted deposit is the mode contract redirecting every write.

4.4 How dw_mode enforces the boundaries

The PRODUCER / REVIEWER split is a single session-level setting. Each user keeps a personal block in ~/.config/user_config.yml:

```
jpazevedo:
  githubFolder: "C:/GitHub/mytasks"
  teamsRoot: "C:/Users/jpazevedo/UNICEF/Chief Statistician Office - Documents"
  dw_mode: "reviewer" # producer | reviewer
  sandboxRoot: "C:/Users/jpazevedo/dw-repro" # only used when dw_mode = reviewer
```

dw_mode is **required**. Profile loading hard-fails if it is absent or set to anything other than **producer** or **reviewer**, because defaulting to either side is unsafe: defaulting to producer would let an unconfigured session corrupt the deposit, and defaulting to reviewer would surprise a Database Manager mid-deposit.

From **dw_mode** the profile derives the path globals — **teamsWrkData**, the ***Canonical** read-only aliases, and a **dw_apis_allowed** flag. That derivation lets the same conductor scripts route to the deposit in producer mode and to the sandbox in reviewer mode without per-script edits. The wiring chapter covers the derivation in full; here we focus on the two enforcement points the contract relies on.

The contract is enforced by **defence in depth**. The mode setting routes paths at the profile, and then the individual write and fetch helpers check *again* at the call site. Two guards do the work.

4.4.1 Guard 1 — reviewer mode forbids canonical and Z: writes (dw_save)

When **dw_save** is asked to write a path that is canonical (under the Teams deposit) or under the Z: drive mirror, and the session is in reviewer mode, it refuses. The guard broadened in v0.4.0 to cover the Z: mirror as well as the Teams deposit; v0.3.0 refused canonical Teams writes only. The single escape hatch is **allow_canonical_write = TRUE**, intended

for Database Manager bootstraps, never for routine reviewer work. The `isid` argument shown below names the columns that uniquely identify a row; `dw_save` checks them before writing so a duplicate key can never reach the deposit.

4.4.1.1 R

```
# Reviewer-mode session. This canonical path raises an error.
# isid = the columns that uniquely identify a row.
dw_save(df, file.path(teamsWrkDataCanonical, "dw_im.csv"), isid = c("iso3", "year"))
#> Error: [cso_toolkit.dw_save] Reviewer mode forbids writes to
#> canonical (Teams) deposit: ../dw_im.csv
#> Reviewer sessions must keep canonical / Z: deposits read-only to
#> preserve vintage permanence; writes go to the local repo sandbox.
#> Fix:
#> 1. Resolve a sandbox path instead (the profile's `teamsWrkData`
#> should point there in reviewer mode), OR
#> 2. If this is a deliberate Database Manager bootstrap,
#> pass `allow_canonical_write = TRUE`.

# Producer bootstrap (rare): explicit opt-in bypasses the guard.
dw_save(df, canonical_path, isid = c("iso3", "year"), allow_canonical_write = TRUE)
```

4.4.1.2 Python

```
# Reviewer-mode session. This canonical path raises PermissionError.
# isid = the columns that uniquely identify a row.
dw_save(df, os.path.join(teams_wrk_data_canonical, "dw_im.csv"), isid=["iso3", "year"])
# PermissionError: [cso_toolkit.dw_save] Reviewer mode forbids writes to
# canonical (Teams) deposit: ../dw_im.csv
# Reviewer sessions must keep canonical + Z: deposits read-only to
# preserve vintage permanence; writes go to the sandbox.
# Fix:
# 1. Resolve a sandbox path instead (the profile's teamsWrkData
# usually points there in reviewer mode), OR
# 2. If this is a deliberate Database Manager bootstrap,
# pass `allow_canonical_write=True` to bypass the guard.

dw_save(df, canonical_path, isid=["iso3", "year"], allow_canonical_write=True)
```

4.4.1.3 Stata

```
* The Stata dw_save mirrors the same path routing via $teamsWrkData,
* which the profile points at the sandbox in reviewer mode, so writes
* land in the sandbox without a per-call flag. The uniqueness check is
* folded in via idvars() (the Stata equivalent of the R isid= argument).
dw_save , filename("dw_im.csv") path("$teamsWrkData") idvars(iso3 year)
```

i Note

The same guard also runs a *producer-side* pre-flight: in producer mode a non-canonical (sandbox) write requires at least one of the Teams or Z: mirrors to be reachable, so a deposit never ends up living only on the producer's laptop. Producer outputs are redundant by design — every canonical write fans out to both mirrors when available, and the Z: carbon copy is what later lets a reviewer verify Teams equals Z: byte-for-byte.

4.4.2 Guard 2 — reviewer mode forbids live API calls (dw_require_no_api)

The second guard sits at every entry point that would hit the network. `dw_api_fetch` checks the `dw_apis_allowed` flag at the very top, before any HTTP request, and any sector-internal script that talks to a remote is expected to call `dw_require_no_api` too. In reviewer mode the call aborts; in producer mode it returns silently. The frozen cache is read with `dw_api_cached` instead of fetching live.

4.4.2.1 R

```
# At the top of a remote-touching analysis script:
dw_require_no_api(context = "ed/06_pull_uis")
# Reviewer mode:
#> Error: [X] External API access is forbidden in reviewer mode. Context: ed/06_pull_uis

# dw_api_fetch performs the same check internally before any http::GET;
# in reviewer mode it raises the cso_toolkit.dw_api envelope and stops.

# To read the frozen cache instead of fetching live:
dw_api_cached("uis", cache_key = "uis_entrance_age_primary")
```

4.4.2.2 Python

```
# Python analogue, raised by dw_api_fetch before any request:
_dw_require_no_api("dw_api_fetch('uis', cache_key='...')",
                  reason="cache missing at ../uis/...")
```

```
# PermissionError: [cso_toolkit.dw_api] Reviewer mode forbids live API calls.
#   Call:   dw_api_fetch('uis', cache_key='...')
#   Reason: cache missing at .../uis/...
#   Fix:
#       1. The expected workflow is that a Database Manager has already
#          populated the cache under teamsRawDataCanonical/_apis/.
#          Verify the cache file exists at the path above.
#       2. If you ARE the producer, switch modes:
#          from cso_toolkit import _state
#          _state.configure(dw_mode="producer", dw_apis_allowed=True)

dw_api_cached("uis", cache_key="uis_entrance_age_primary")
```

4.4.2.3 Stata

```
* Assert the contract just before a live network call:
dw_require_no_api , context("ed/06_pull_uis")
* Reviewer mode aborts with Stata error 459 and the envelope message:
*   [cso_toolkit.dw_require_no_api] Reviewer mode forbids live API calls
*   (context: ed/06_pull_uis).
*   Why: reviewer sessions must read frozen producer caches to preserve
*        vintage permanence; any live API call breaks provenance.
*   Fix: switch the call to dw_use() against the frozen cache, or
*        re-run in producer mode.
```

The three guards are the same shape — same intent, same reviewer-blocks/producer-passes logic, same envelope-style message with a *Why* and a *Fix*. They differ in their language’s native failure mechanism (an R `stop()`, a Python `PermissionError`, a Stata `error 459`) and in their plumbing. In R the gate is split: `dw_api_fetch` tests the `dw_apis_allowed` flag itself and raises the `cso_toolkit.dw_api` envelope, while the standalone `dw_require_no_api(context = ...)` helper — defined by the consumer profile, not exported by the package — is what analysis scripts call directly. The Stata side, by contrast, has no API wrapper at all, so `dw_require_no_api` is the *only* mechanism a Stata script has; you must call it explicitly before every network operation.

Warning

The Stata helper carries a subtlety: an **unset** mode passes through. It blocks only when `$dw_mode == "reviewer"`; producer and any empty or unknown value pass silently. This is deliberate — it means a stray script never falsely blocks a producer — but it also means the *profile* must set `dw_mode` correctly. The required-field rule in `user_config.yml` is what closes that gap: an unconfigured session fails at profile load, before it ever reaches a guard.

4.5 The helper subset each role uses

The same functions serve all three roles, but each role exercises a different subset and gets different behaviour from the shared helpers.

Helper	PRODUCER	REVIEWER	PUBLISHER
<code>dw_save</code>	writes to the deposit (Z: mirror automatic)	sandbox writes only; canonical/Z: writes hard-fail	reads only
<code>dw_use</code>	reads anywhere	reads sandbox with canonical fallback; network-first with Z: integrity check	reads <code>013_wrkdata</code>
<code>dw_api_fetch</code>	fetches live and caches in the deposit	blocked in reviewer mode; reads cache instead	n/a
<code>dw_api_cached</code>	reads cache	reads cache	n/a
<code>dw_api_inventory</code>	audits cache freshness	audits what is available	audits provenance pre-publication
<code>dw_compare</code>	runs on every production run	runs on every reviewer run	reads compare reports as a publication gate
<code>dw_verify_z</code>	runs after a canonical write to confirm the mirror landed	runs on canonical reads to verify integrity	runs as a final pre-publication check
<code>dw_isid</code>	enforces row uniqueness pre-write	spot-checks reproduced data	optional spot-check
<code>dw_publish</code>	n/a	refused in reviewer mode	submits to Helix (R only; dry-run stub today)

The cross-language coverage of these helpers is uneven, and the chapters on the IO and external-API contracts cover the gaps honestly. Two are worth flagging here because they bear directly on the role split. First, Stata has **no API wrapper** — `dw_api_fetch` and `dw_api_cached` exist only in R and Python — so the producer’s cache-refresh leg is an R/Python responsibility, and Stata’s contribution to the reviewer guard is the standalone `dw_require_no_api` assertion alone. Second, `dw_publish` exists only in R. The coverage matrix appendix gives the full per-function picture.

4.6 A note on the PUBLISHER role

Of the three roles, PRODUCER and REVIEWER are fully wired into the mode contract; PUBLISHER is only partly so. The role is documented in full — its inputs, its pre-publication gates, its targets (Helix, SDMX endpoints, data.unicef.org, the State of the World's Children tables), and its rollback procedure — and one piece of it is now code. The R package ships `dw_publish(path, indicator, vintage, sector, endpoint = "helix", dry_run = TRUE)`, parallel to `dw_save` and `dw_api_fetch`. It already enforces the call-site contract a publisher needs: a reviewer-mode call raises before any I/O, required arguments are validated, the artifact must exist on disk, and the endpoint allowlist currently admits only "helix".

What `dw_publish` does *not* yet do is submit. As of v0.5.0 it is a dry-run-only stub: `dry_run = TRUE` validates the call site and assembles the exact payload the live branch will POST, while `dry_run = FALSE` raises an envelope-shaped error pointing at the issue where the Helix endpoint contract is being negotiated with sector leads. There is no Python or Stata equivalent. So the publisher's *transport* — the actual push to Helix, SDMX, the data portal, and the SoWC tables — and its remaining boundaries (read only blessed `013_wrkdata`, require a provenance chain back to original API fetches, never roll back without amending the source) are still enforced by procedure and review rather than by code. Treat the PUBLISHER role as a contract honoured mostly by hand, with the mechanical version arriving one helper at a time.

4.7 Why this matters

The whole point of the relay is *reproducibility with permanence*. A reviewer must be able to run the producer's exact scripts and get the producer's exact numbers — which means the reviewer must read the same frozen caches the producer deposited, never a fresher pull from a live API that has moved on. Each cache is a *vintage*: the exact snapshot of the data as it stood when the producer pulled it. And a reviewer must be able to reproduce those numbers without any risk of overwriting the very deposit they are auditing. The two guards deliver exactly this: the API guard keeps the reviewer reading frozen vintages, and the `dw_save` reviewer guard keeps the reviewer's output quarantined in a sandbox. Set `dw_mode` correctly once, and the same code base behaves as a producer or as a reviewer — safely, and without editing a single sector script.

5 Wiring the contract into a repo

The roles chapter described *who* the producer and reviewer are and *what* obligations the `dw_mode` contract places on each. This chapter is the operational counterpart: how a repository makes its own code enforce that contract.

A word of orientation first, so this chapter stands on its own. The CSO toolkit is the shared R, Python, and Stata library — vendored into DW-Production and other Chief Statistician Office (CSO) pipelines — that helper functions such as `dw_save`, `dw_use`, and `dw_api_fetch` are built on. `dw_mode` is the toolkit's master switch: a session property, set once per user, that takes the value "producer" or "reviewer". A *producer* (the sector's database manager) is entitled to write the **canonical deposit** — the authoritative directory of frozen data artefacts synced through Teams. A *reviewer* may reproduce and check those artefacts but must never overwrite them, and must never reach a live external API, because every input a reviewer consumes has to come from a snapshot the producer already froze. Wiring the contract means making the repository's own code honour that division automatically.

The work is concrete and mostly mechanical: set one value in a configuration file, derive a handful of path globals from it, shadow those globals at the top of each pipeline run, and verify the result with a short checklist. Each step has a reason, and getting any one wrong silently re-introduces the failure the contract exists to prevent — a reviewer overwriting the canonical deposit, or a reviewer-mode run quietly hitting a live API and breaking provenance, the guarantee that every input traces back to a specific frozen snapshot.

Throughout, the load-bearing idea is *single-switch routing*. A user sets `dw_mode` once, in one place. Everything downstream — where writes land, whether external API calls are permitted, which directory reads fall back to — is *derived* from that one value. No per-script edits, no per-call flags. The contract is wired once and inherited everywhere.

5.1 The single switch: `dw_mode` in `user_config.yml`

Every user of a CSO pipeline keeps a personal configuration file at `~/.config/user_config.yml` (on Windows, `%USERPROFILE%/.config/user_config.yml`). This file is per-machine and per-user; it is never committed to the repository. The keys sit at the top level — the profile that `create_profile` scaffolds reads `user_config$dw_mode` (R) or `user_config.get("dw_mode")` (Python) directly, with no per-user wrapping mapping. A minimal file looks like this:

```
githubFolder: "C:/GitHub/mytasks"
teamsRoot: "C:/Users/jpazevedo/UNICEF/Chief Statistician Office - Documents"
dw_mode: "reviewer" # producer | reviewer
sandboxRoot: "C:/Users/jpazevedo/dw-repro" # used only when dw_mode = reviewer
```

The one required key is `dw_mode`. It takes exactly one of two values, "producer" or "reviewer". Loading the profile must hard-fail if `dw_mode` is missing or set to anything else. Defaulting either way is unsafe. Defaulting to `producer` would let a reviewer corrupt the canonical deposit the first time they forget to set the key; defaulting to `reviewer` would surprise a database manager in the middle of a legitimate deposit by silently diverting their writes into a sandbox. There is no safe default, so the loader refuses to guess.

All three language ports enforce this identically. The `create_profile` scaffolder (covered in the scaffolding chapter) emits a `dw_mode` block that validates the value at load time and stops the session otherwise:

5.1.0.1 R

```
dw_mode <- user_config$dw_mode
if (is.null(dw_mode) || !dw_mode %in% c("producer", "reviewer")) {
  stop("[X] user_config.yml must set dw_mode to 'producer' or 'reviewer'.")
}
```

5.1.0.2 Python

```
dw_mode = user_config.get("dw_mode")
if dw_mode not in ("producer", "reviewer"):
    raise ValueError(
        "[X] user_config.yml must set dw_mode to 'producer' or 'reviewer'."
    )
```

5.1.0.3 Stata

```
* dw_load_config validates dw_mode while parsing the YAML and
* hard-stops with error 459 if it is missing or out of range.
dw_load_config
* -> sets the global $dw_mode
```

A note on the Stata path. Stata cannot link an external YAML library on AppLocker-locked corporate Windows installs, so `dw_load_config.ado` ships a minimal hand-rolled `key: value` parser. It handles the documented subset of the config schema — one pair

per line, optional quoting, inline `#` comments stripped — but not nested mappings, lists, anchors, or multi-line strings. (The R and Python profiles lean on a real YAML library and so could parse a richer file, but in practice all three ports read the same flat top-level keys.) The Stata loader reads exactly the keys the YAML provides into `$`-globals and validates `dw_mode` (hard-stopping with error 459 if it is missing or out of range). Unknown keys are silently ignored, since the file may hold entries other helpers consume.

i Note

One key the loaders do *not* validate is `sandboxRoot`, even though it is mandatory in reviewer mode (it is the base the reviewer’s write globals are derived from). If `dw_mode: reviewer` is set but `sandboxRoot` is absent, the derivation below produces empty or malformed paths rather than a clean error at load time. Until the loaders gain an explicit check, a profile author should validate `sandboxRoot` themselves alongside `dw_mode`.

5.2 Deriving the producer/reviewer globals

Once `dw_mode` is known, the profile derives a small set of path globals from it. Deriving these globals is the heart of the wiring. In producer mode the globals point at the real Teams deposit; in reviewer mode they point at the user’s private sandbox, while a parallel set of `*Canonical` aliases preserve a *read-only* view of the real deposit so that reviewer code can still compare its outputs against canonical.

The reference derivation (patterned on `profile_DW-Production.R`) computes the following from `dw_mode`:

Global	Producer mode	Reviewer mode
<code>teamsFolder</code>	user’s actual Teams sync root	<code>sandboxRoot</code>
<code>teamsRawData</code>	<code>teamsFolder/01_dw_prep/011_rawdata</code>	<code>sandboxRoot/01_dw_prep/011_rawdata</code>
<code>teamsWrkData</code>	<code>teamsFolder/01_dw_prep/013_wrkdata</code>	<code>sandboxRoot/01_dw_prep/013_wrkdata</code>
<code>teamsFolderCanonical</code>	identical to <code>teamsFolder</code>	user’s actual Teams sync root (read-only)
<code>teamsRawDataCanonical</code>	identical to <code>teamsRawData</code>	<code>teamsFolderCanonical/01_dw_prep/011_rawdata</code>
<code>teamsWrkDataCanonical</code>	identical to <code>teamsWrkData</code>	<code>teamsFolderCanonical/01_dw_prep/013_wrkdata</code>
<code>dw_apis_allowed</code>	TRUE	FALSE

Two patterns deserve attention. The `*Canonical` aliases come first. In reviewer mode the working globals (`teamsRawData`, `teamsWrkData`) are diverted into the sandbox, but the `*Canonical` aliases continue to point at the real deposit. This split — diverted write globals, preserved read aliases — lets reviewer-mode code *read* the canonical inputs and outputs for a compare-against-warehouse step (checking reviewer outputs against the canonical deposit)

without ever being able to *write* over them; the write globals simply do not reach the deposit. In producer mode the aliases collapse onto the same paths as the working globals, because the producer reads and writes the same canonical location.

Second is `dw_apis_allowed`. This single boolean is `TRUE` only in producer mode. It is the value the no-API guard consults (described below), and it is the one place the “reviewers never touch live APIs” rule is encoded.

How each language holds this derived state differs:

5.2.0.1 R

```
# The R profile sets the derived globals directly in .GlobalEnv.
# Downstream helpers read them from there (via an internal .try_get).
teamsRawData      <- file.path(teamsFolder, "01_dw_prep", "011_rawdata")
teamsRawDataCanonical <- file.path(teamsFolderCanonical, "01_dw_prep", "011_rawdata")
dw_apis_allowed    <- identical(dw_mode, "producer")
```

5.2.0.2 Python

```
# The Python profile pushes the derived state into the shared
# cso_toolkit._state module via configure(); helpers read it back
# through _state._get(...).
from cso_toolkit import _state as _cso_state

_cso_state.configure(
    dw_mode=dw_mode,
    dw_apis_allowed=(dw_mode == "producer"),
    teamsRawData="/path/to/raw",
    teamsRawDataCanonical="/path/to/raw-canonical",
)
```

5.2.0.3 Stata

```
* dw_load_config sets $dw_mode and any team* / canonical / sandboxRoot
* keys it finds in the YAML, but it does NOT derive the mode-aware
* paths for you. The profile .do must do the producer/reviewer swap
* itself after the loader returns -- e.g. point $teamsWrkData at the
* canonical root in producer mode, the sandbox root in reviewer mode.
```

The Python `_state` module is the explicit equivalent of R’s `.GlobalEnv`: where the R helpers read mode-aware globals straight out of the global environment, the Python

helpers read them from a single shared module configured by `_state.configure(...)`. Its `configure()` accepts only a fixed allow-list of keys and raises `AttributeError` on any unknown key, so a typo such as a lowercase `teamswrkdata` surfaces immediately rather than silently creating a dead attribute. The accessor `_state._get()` returns `None` only for genuinely unset names — it deliberately returns falsy-but-set values like `False` or `""` as-is, so that `dw_apis_allowed = False` is observable as `False` and not silently coerced to a default.

The Stata side leaves the producer/reviewer path derivation to the profile author. `dw_load_config` populates exactly the globals the YAML names and validates `dw_mode`, but there is no Stata equivalent of the R/Python derivation table baked into the loader. The profile `.do` must perform the swap itself after the loader returns.

5.2.1 The mode banner

A profile should announce its mode prominently at load time, so that a user can never be unsure which side of the contract they are on. The reference profiles emit one of two banners:

PRODUCER mode - writes will land in the Teams deposit.

Use this mode only when you intend to promote outputs to canonical.

REVIEWER mode - writes isolated to `<sandboxRoot>`.

Canonical deposit at `<teamsFolderCanonical>` is read by code but never written.

The banner is cheap and high-value. Most accidental cross-mode writes are caught simply by a user noticing the wrong banner scroll past at session start.

Note

The reference profiles prefix these banners with warning and check-mark emoji (a yellow triangle for producer, a green check for reviewer), whereas the toolkit's error strings use a plain-text `[X]` sentinel. The signalling vocabulary is therefore inconsistent between banners and errors. A consumer that wants uniform output across terminals that render emoji poorly may prefer to standardise on the `[X]` / `[OK]` / `[!]` plain-text markers the helpers already use elsewhere.

5.3 The conductor shadow pattern

The conductor shadow pattern re-binds the path globals once, at the top of the run driver, so that no sector script needs editing. It is the most error-prone step in the whole wiring, and the place where adoption most often succeeds or fails.

The problem it solves is one of scale. A sector codebase typically contains ten to forty scripts, each of which references the path globals — `teamsRawData`, `teamsWrkData`, and the other working roots — directly. The naive way to make those scripts mode-aware would be to edit every one of them. Editing every script is exactly wrong: it multiplies the change surface, and it breaks the diff audit reviewers rely on, since every script would show churn unrelated to the analysis. And it guarantees that someone, someday, edits thirty-nine scripts and forgets the fortieth.

The shadow pattern solves this once. At the top of each sector's run driver (`0_execute_conductor.R`), *after* sourcing the profile, the conductor re-binds the working globals to mode-correct values. Because R, Python, and Stata all resolve those names dynamically when each downstream script runs, every script keeps its existing code unchanged and still gets the right paths:

5.3.0.1 R

```
# After computing outputdir for this run (with branch-slug isolation):
finaldir <- file.path(outputdir, "final")

# Conductor-level shadow: redirect downstream reads/writes without
# editing any sector script.
teamsRawData <- teamsRawDataCanonical    # reviewer-mode: read canonical inputs
teamsWrkData <- finaldir                  # reviewer-mode: writes go to sandbox final
dwWrkData    <- finaldir                  # same
```

5.3.0.2 Python

```
# After computing outputdir for this run (with branch-slug isolation):
finaldir = os.path.join(outputdir, "final")

# Conductor-level shadow: re-point the shared state so every helper
# call downstream resolves the mode-correct paths.
from cso_toolkit import _state
_state.configure(
    teamsRawData=_state.teamsRawDataCanonical, # read canonical inputs
    teamsWrkData=finaldir,                    # writes go to sandbox final
)
```

5.3.0.3 Stata


```

* After computing $outputdir for this run:
global finaldir "$outputdir/final"

* Conductor-level shadow: re-point the globals once; every later
* `save'/'dw_save' against $teamsWrkData lands in the sandbox
* without per-call edits -- the same trick as R/Python.
global teamsRawData "$teamsRawDataCanonical"
global teamsWrkData "$finaldir"

```

The shadow reads *from* the canonical alias and writes *to* the sandbox-derived `finaldir`. In reviewer mode this means inputs come from the read-only canonical deposit while every output is confined to the sandbox. In producer mode the canonical alias is identical to the working global, so the same line is a harmless identity and outputs flow to the real deposit as before. One pattern, both modes, no branching in the sector scripts themselves.

i Note

The shadow must run *before any other script is sourced* by the conductor. A script that is sourced before the shadow re-binds the globals will capture the pre-shadow values. Order matters: source profile, compute `outputdir` and `finaldir`, shadow the globals, *then* source the pipeline.

5.4 Branch-slug isolation

Branch-slug isolation keys output directories to the current Git branch, so that two feature branches — or a feature branch and the canonical line — never write to the same output directory, even within a single user's sandbox. The shadow pattern composes with this second, optional but recommended, layer. The documented pattern reads the current branch, slugifies it into a filesystem-safe token, and routes non-canonical branches into a `branches/<slug>/` subdirectory:

```

branch <- tryCatch(
  system("git rev-parse --abbrev-ref HEAD", intern = TRUE),
  error = function(e) "main"
)
branch_slug <- gsub("[^A-Za-z0-9._-]", "-", branch)

canonical_outputdir <- file.path(teamsRawData, sector, "output")
outputdir <- if (branch %in% c("main", "dev", "master")) {
  canonical_outputdir
} else {
  file.path(canonical_outputdir, "branches", branch_slug)
}

```

This is a conductor-side convention demonstrated in the integration documentation, not a function shipped by the toolkit; each sector's conductor implements it (or its Python/Stata equivalent) inline. It layers cleanly on top of the mode contract. In reviewer mode `teamsRawData` is already rooted under `sandboxRoot`, so a feature-branch run lands at `<sandboxRoot>/.../<sector>/output/branches/<slug>/` and the canonical deposit is untouched even when the reviewer is working on a branch. Only the canonical branches (`main`, `dev`, `master`) write to the un-suffixed output directory — and in reviewer mode even those are inside the sandbox.

5.5 Defence in depth: the no-API guard

A guard enforces the other half of the contract: every network entry point calls it before touching a remote. Reviewer-mode sessions never make live external API calls, because every upstream input must already be frozen in a producer-deposited cache. The profile enforces the mode at load time, but each API helper checks again at the call site — deliberate redundancy, because a single guard at the profile could be bypassed by a sector script that reaches for `http::GET` directly. So the guard is invoked at every place a live call could originate.

5.5.0.1 R

```
dw_require_no_api <- function(context = NULL) {  
  if (identical(dw_mode, "reviewer")) {  
    msg <- "[X] External API access is forbidden in reviewer mode."  
    if (!is.null(context)) msg <- paste0(msg, " Context: ", context)  
    stop(msg, call. = FALSE)  
  }  
}
```

5.5.0.2 Python

```
def dw_require_no_api(context: str | None = None) -> None:  
    if dw_mode == "reviewer":  
        msg = "[X] External API access is forbidden in reviewer mode."  
        if context:  
            msg += f" Context: {context}"  
        raise PermissionError(msg)
```

5.5.0.3 Stata

```
dw_require_no_api , context("ed/06_pull_uis")
* On $dw_mode == "reviewer" this aborts with error 459 and the
* standard guard message; producer and any non-reviewer value pass
* through silently.
```

All three share the same shape: the guard blocks only when the mode is **reviewer**, and passes silently for producer (and, on the Stata side, for any value other than **reviewer**). The reviewer-mode message points the user at the right remedy — read the frozen cache through **dw_use**, or escalate a missing input to the sector database manager — rather than at the API.

The block above shows the lightweight guard the profile *emits* for sector scripts to call. The toolkit’s own **dw_api_fetch** (in R and Python; see the external-API contract chapter) carries a stricter internal variant of the same guard, which it consults against **dw_apis_allowed** at the very top of every fetch, before any HTTP request. Note one further parity gap, surfaced in the cross-language coverage appendix: Stata ships the guard but has *no* API-fetch wrapper of its own, so on the Stata side the guard is the whole of the API contract.

5.6 Verification checklist

A sector is not “wired” until it passes a short, mechanical checklist. Steps four through six are the operational definition of adoption — they are what actually proves the contract holds, as opposed to merely appearing in the code.

1. **Profile loads green.** Sourcing the profile with **dw_mode: "reviewer"** produces the green reviewer banner naming the sandbox.
2. **Conductor shadows the globals.** The sector’s run driver writes **finaldir** and shadows the working globals before any other script is sourced.
3. **No back-door network calls.** No script inside the sector calls a remote without going through the API fetch helper. Grep for the obvious culprits — **httr::GET**, **httr::POST**, **httr::request**, **RETRY**, and **read.csv("http** in R, and the corresponding patterns in Python/Stata.
4. **Reviewer run is sandboxed.** A reviewer-mode end-to-end run produces every artifact under **sandboxRoot**, and the modification time of the canonical deposit is unchanged before and after the run.
5. **Producer run reaches canonical.** A producer-mode end-to-end run produces every artifact under the canonical deposit, exactly as before adoption.
6. **Compare step works in reviewer mode.** The sector’s compare-against-warehouse script runs in reviewer mode and writes a per-segment report under the sandbox’s **output/temp/** directory.

The contract auditor (described in the scaffolding chapter, and its CI use in a later chapter) automates much of steps one through three. The **review_profile** helper audits a profile for exactly these blocks — the profile sentinel object (the **profile_<repo> <- TRUE** marker

that sector scripts assert was sourced), cross-platform user identification, the YAML config load, the `dw_mode` resolution, the `dw_require_no_api` guard, a reproducibility seed, an optional Z:-drive advisory, and a packages block — and reports each as `pass`, `warn`, or `fail`. Steps four through six remain manual, because only an actual end-to-end run can prove that writes land where the contract says they should.

 Warning

Step four's mtime check is not optional ceremony. It is the only test that catches a script which *reads* a canonical global correctly but *writes* through a stale, un-shadowed handle it captured before the conductor shadow ran. The contract auditor cannot see this; only an end-to-end run with a before/after mtime comparison can.

5.7 Where this fits

Wiring the contract is the bridge between policy and enforcement. The roles chapter sets the obligations; the IO, external-API, aggregation, and vintage-sync chapters describe the helpers whose behaviour this wiring routes; the scaffolding chapter provides `create_profile` and `review_profile`, which generate a profile that passes the checklist by construction and audit one that was hand-written before the helpers existed. Once a repository has done the work in this chapter — one switch in `user_config.yml`, the derived globals, the conductor shadow, optional branch-slug isolation, and a passing checklist — every helper call it makes is mode-correct without any further per-call thought. That is the whole point: wire it once, inherit it everywhere.

6 Provenance sidecars and the error envelope

Two small conventions make `cso-toolkit` outputs trustworthy and its failures readable. The first is the **provenance sidecar**: a tiny JSON file written next to every data artefact, recording who wrote it, when, in what mode, and with what content hash. The second is the **error envelope**: a fixed three-part shape that every deliberate error and warning in the toolkit follows, so that library failures read like instructions rather than stack traces. Both conventions are reproduced across the R, Python, and Stata siblings. This chapter documents what each contains, why it exists, and where the cross-language parity is honest versus aspirational.

6.1 The provenance sidecar

Whenever you write a data file through the IO contract — the toolkit’s single sanctioned read/write path (see the IO contract chapter) — the toolkit emits a companion file named `<path>.provenance.json` in the same directory. If you write `dw_ed_edu.csv`, you also get `dw_ed_edu.csv.provenance.json`. The sidecar is not the data; it is a record *about* the data — a small, human-readable, machine-parseable receipt that travels with the artefact through the canonical Teams folder and the Z: drive mirror.

6.1.1 Why a sidecar at all

The toolkit exists to make analytics reproducible by someone other than the original author (see the “Why a toolkit” chapter). Reproducibility is not only about being able to *re-run* a pipeline; it is also about being able to *trust* a frozen artefact two years after it was deposited. A reviewer who opens `dw_ed_edu.csv` from the canonical deposit needs to answer questions the file itself cannot: Who produced this? Under which session mode — the toolkit runs as either a *producer*, who writes authoritative data, or a *reviewer*, who works read-only (see the roles and mode-contract chapter), and a deposit means something different in each. Has the file been altered since it was written? What were its dimensions, and what upstream sources fed it? The sidecar answers exactly these questions, and it does so without changing the data file’s own schema.

The toolkit uses a sidecar rather than embedding metadata inside the file (the way Stata’s native `char _dta[]` mechanism or a Parquet key-value footer would). A sidecar wins on three counts. It is format-agnostic — the same scheme works for CSV, Parquet, DTA, XLSX, JSON, and YAML; it never changes the bytes of the artefact, so a content hash of

the data file stays stable regardless of how rich the metadata grows; and it is easy to diff in review and easy to drop when unwanted.

6.1.2 What the sidecar contains

The R and Python siblings write an identical schema. For a data frame written through `dw_save`, the sidecar carries:

Field	Meaning
<code>path</code>	Absolute path of the primary file the sidecar describes.
<code>format</code>	The dispatched format (e.g. <code>csv</code> , <code>parquet</code> , <code>dta</code>).
<code>written_at</code>	UTC timestamp in ISO-8601 form (<code>%Y-%m-%dT%H:%M:%S%Z</code>).
<code>user</code>	OS username (<code>USERNAME</code> env var, with a <code>getpass</code> fallback in Python).
<code>dw_mode</code>	The session mode at write time — <code>producer</code> , <code>reviewer</code> , or <code>unset</code> .
<code>vintage</code>	Optional release tag passed by the caller (e.g. <code>2026-05</code>).
<code>sha256</code>	SHA-256 hash of the <i>written file's</i> bytes — the integrity anchor.
<code>isid</code>	The columns asserted to uniquely identify each row at write time, if any (the “is-ID” check).
<code>schema</code>	For data frames: { <code>rows</code> , <code>cols</code> , <code>columns</code> }.
<code>metadata</code>	Optional caller-supplied block, present only when supplied.

The `metadata` block is free-form and is where domain context lives — title, producer script, upstream sources, a contact handle, the data vintage. The toolkit does not prescribe its keys; it merges whatever the caller passes.

Here is how you populate the sidecar at the call site. You write the `metadata` argument by hand; the toolkit captures every other field automatically.

6.1.2.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        isid = c("REF_AREA", "INDICATOR", "TIME_PERIOD"),
        vintage = "2026-05",
```

```

metadata = list(
  title      = "Education indicators - UNICEF DW format",
  producer   = "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.R",
  sources    = c("UIS bulk SDG_092025", "WPP 2024"),
  contact    = "@karavan88",
  vintage    = "2026-05"
))

```

6.1.2.2 Python

```

dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
  isid=["REF_AREA", "INDICATOR", "TIME_PERIOD"],
  vintage="2026-05",
  metadata={
    "title":      "Education indicators - UNICEF DW format",
    "producer":   "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.py",
    "sources":    ["UIS bulk SDG_092025", "WPP 2024"],
    "contact":    "@karavan88",
    "vintage":    "2026-05",
  })

```

6.1.2.3 Stata

```

dw_save, filename("dw_ed_edu.dta")           ///
  path("$teamsWrkData/ed")                   ///
  idvars(REF_AREA INDICATOR TIME_PERIOD)     ///
  title("Education indicators - UNICEF DW format") ///
  producer("01_dw_prep/.../02_aggregate_uis_sdg.do") ///
  sources("UIS bulk SDG_092025; WPP 2024")   ///
  contact("@karavan88")                     ///
  vintage("2026-05")

```

A producer-mode write of a CSV produces a sidecar of roughly this shape:

```

{
  "path": "C:/.../060.DW-MASTER/013_wrkdata/ed/dw_ed_edu.csv",
  "format": "csv",
  "written_at": "2026-05-30T14:22:07+0000",
  "user": "jpazevedo",
  "dw_mode": "producer",
  "vintage": "2026-05",
  "sha256": "9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08",

```

```

"isd": ["REF_AREA", "INDICATOR", "TIME_PERIOD"],
"schema": { "rows": 1840, "cols": 7,
            "columns": ["DATAFLOW", "REF_AREA", "INDICATOR",
                        "SEX", "WEALTH_QUINTILE", "TIME_PERIOD",
                        "OBS_VALUE"] },
"metadata": {
  "title": "Education indicators - UNICEF DW format",
  "producer": "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.R",
  "sources": ["UIS bulk SDG_092025", "WPP 2024"],
  "contact": "@karavan88",
  "vintage": "2026-05"
}
}

```

The exhaustive field-by-field schema, including the optional and language-specific keys, is tabulated in the provenance sidecar schema appendix.

6.1.3 How and when it is written

The sidecar is written *after* the primary file lands, by an internal helper (`.write_provenance` in R, `_write_provenance` in Python) invoked by `dw_save`. The SHA-256 is computed by reading back the file that was just written, so the hash describes the bytes actually on disk, not the in-memory object. Two switches govern whether a sidecar appears at all:

- **provenance = FALSE** opts out entirely for a single call.
- **.RData / .rda writes skip the sidecar unconditionally**, because those formats can carry arbitrary R objects rather than a single tabular artefact, and a row/column schema would be meaningless.

The redundant-mirror logic in producer mode copies the sidecar alongside the primary file to both the Teams canonical equivalent and the Z: drive equivalent, so each copy of the artefact carries its own receipt (see the IO contract chapter for the mirroring rules).

i Note

The sidecar write is **non-blocking**. If the metadata contains a value that cannot be serialised to JSON — an R `Date` object, a NumPy scalar, a custom class — the write is wrapped so that it emits an envelope-shaped warning and continues, rather than rolling back the primary file. The reasoning is deliberate: the sidecar is metadata; the asset is what matters. You never lose a successfully written data file because its receipt could not be serialised. This behaviour was backported from a DW-Production hardening (the “B3” alignment item).

6.1.4 Verifying the hash

The `sha256` field is not just a label; it is the verb behind the sidecar's central promise. To check whether a canonical artefact has drifted from its mirror, `dw_use` runs a non-blocking Z: integrity check on every canonical read, and the dedicated `dw_verify_z` function performs the same comparison on demand. With `compare = "size"` (the default) it does a cheap size check; with `compare = "sha256"` it re-hashes both the Teams copy and the Z: copy and compares the digests, returning a status such as `"match_sha256"` or `"sha256_mismatch"`. To detect tampering against the recorded provenance, re-hash the file and compare the result with the sidecar's `sha256`; this is exactly what the staging path inside `dw_use` does when it re-reads a frozen artefact (see the IO contract and external-API contract chapters). The sidecar records the anchor; `dw_verify_z` is the tool that checks it.

6.1.5 A second sidecar shape: frozen remote files

When `dw_use` downloads an HTTPS URL and freezes it to the local cache (a producer-only operation; see the external-API contract chapter), it writes a sidecar with a *different* schema, tuned to a downloaded resource rather than a saved data frame:

```
{
  "url": "https://raw.githubusercontent.com/.../regions.csv",
  "sha256": "...",
  "bytes": 48213,
  "fetched_at": "2026-05-30T14:25:11Z",
  "fetched_by": "jpazevedo",
  "dw_mode": "producer"
}
```

This is intentionally distinct: `url/fetched_at/fetched_by/bytes` describe a fetch event, whereas the `dw_save` sidecar describes a write event with a tabular schema. A reviewer reading a frozen remote file is reading a provenance record of *where the bytes came from*, not of a local computation.

i Note

The two sidecar shapes spell the UTC timestamp slightly differently, and a careful reader will notice. The `dw_save` sidecar emits the numeric offset (+0000) in both R and Python. The freeze path is not fully aligned: R's `.write_remote_provenance` emits the Z suffix (...14:25:11Z), while Python's reuses the shared +0000 formatter. Both denote UTC; only the spelling differs. This is a minor parity wrinkle rather than a contract, and is noted here so it is not mistaken for a bug.

6.1.6 Cross-language parity — and the honest gaps

R and Python produce equivalent sidecars: same field names, same ISO-8601 UTC timestamp, same SHA-256 file hash, same `{rows, cols, columns}` schema, same optional metadata merge. Stata reaches the same goal but cannot match every detail, and the differences are worth stating plainly:

Field	R	Python	Stata
<code>format</code> tag	dispatched format	dispatched format	literal string <code>"dw_save.provenance/1"</code>
<code>written_at</code>	ISO-8601 UTC (+0000)	ISO-8601 UTC (+0000)	Stata <code>c(current_date)/c(current_time)</code> string (not ISO, not UTC)
content hash	<code>sha256</code> (file bytes)	<code>sha256</code> (file bytes)	<code>datasignature</code> — Stata's SHA-1 of variable values, not a <code>sha256</code> of the file
<code>schema.columns</code>	column names	column names	absent — only <code>{rows, cols}</code>
<code>isid</code> / <code>idvars</code>	<code>isid</code> array	<code>isid</code> array	<code>idvars</code> string
metadata	nested list	nested dict	nested block built from named options

Two gaps matter. First, **Stata records a `datasignature`, not a `sha256`**: a content checksum (SHA-1 over the data values) rather than a hash of the file on disk. The upside is that two artefacts with identical data but different on-disk bytes — say, different compression — still match; the cost is that a Stata sidecar can never be cross-checked byte-for-byte against an R/Python `sha256`, so `dw_verify_z`'s deep comparison cannot bridge a Stata-written file and an R/Python-written one. Second, **Stata omits the `columns` list**, recording only row and column counts.

Stata's sidecar also carries an explicit version tag, `"format": "dw_save.provenance/1"`, and an `_end` sentinel in its metadata block, reflecting that the JSON is hand-assembled by a `.ado` file rather than emitted by a JSON library. The `.ado` escapes every user-supplied value before writing it — backslashes first, then double-quotes, then control characters — so Windows paths and quotes or newlines in metadata do not produce malformed JSON that an R reader would later reject.

6.2 The error envelope

The toolkit's second convention governs failure. Every deliberate `stop()` in R, every `raise` in Python, and every error or advisory message in Stata follows the same three-part shape:

```
[cso_toolkit.<fn>] WHAT happened
  Why it happened / why the rule exists.
  Fix:
    1. The first concrete remedy, OR
    2. An alternative remedy / escape hatch.
```

The leading bracketed prefix names the function that raised — `[cso_toolkit.dw_save]`, `[cso_toolkit.dw_use]`, `[cso_toolkit.dw_api_fetch]`. The body explains *why* the rule exists rather than merely restating the symptom. The **Fix:** block lists the actionable next steps, often including the deliberate escape hatch for the case where the user genuinely means to do the forbidden thing.

6.2.1 Why a fixed shape

One shape on every error buys three things:

1. **Searchability.** The `[cso_toolkit.<fn>]` prefix is a stable, greppable token. To find every site in a consumer project that could trip the reviewer-mode write guard, you grep for `[cso_toolkit.dw_save] Reviewer mode forbids`. Library errors become an index.
2. **Actionability.** Because the **Fix:** block is mandatory, an error never leaves the user staring at *what* broke without *how to proceed*. The escape hatch — `allow_canonical_write = TRUE`, `overwrite = TRUE`, switching `dw_mode` — is surfaced at the point of failure, not buried in documentation.
3. **Distinguishing toolkit errors from upstream noise.** A failure prefixed `[cso_toolkit.*]` is a contract decision the toolkit made on purpose; an unprefixed error is something leaking from `arrow`, `haven`, `pandas`, or the OS. The prefix lets a reader triage at a glance.

6.2.2 A worked example

The canonical example is the reviewer-mode write guard. A reviewer session is physically prevented from writing under the canonical Teams deposit or the Z: drive root, to preserve vintage permanence (see the roles and mode-contract chapter). The error it raises is the envelope in its fullest form:

```
[cso_toolkit.dw_save] Reviewer mode forbids writes to canonical (Teams) deposit: /path/to/
  Reviewer sessions must keep canonical / Z: deposits read-only to preserve
  vintage permanence; writes go to the local repo sandbox.
  Fix:
    1. Resolve a sandbox path instead (the profile's `teamsWrkData` should
      point there in reviewer mode), OR
    2. If this is a deliberate Database Manager bootstrap, pass
      `allow_canonical_write = TRUE`.
```

Every clause of the envelope is doing work. The prefix identifies `dw_save`. The WHAT names both the rule and the offending path. The Why explains *vintage permanence* — the reason the rule exists. The Fix gives the ordinary remedy first and the deliberate-override escape hatch second.

The same shape recurs across unrelated failure classes — a missing dependency, an unsupported extension, a refused overwrite, an unknown `kind`:

```
[cso_toolkit.dw_save] Producer-mode write requires at least one of Teams
(preferred) or Z: drive to be mounted and writable; currently neither is
available on this machine.
  Primary path: /path/to/file.csv
  Fix: map at least one -- preferably both. See the profile's path block.
```

The complete catalogue of error classes, grouped by the rule each enforces, lives in the error taxonomy appendix.

6.2.3 Asserting the shape in tests

The envelope is not merely a style guideline — it is a tested invariant. The R `testthat` suite ships an `expect_envelope()` helper that asserts the `[cso_toolkit.<fn>]` prefix, a `Fix:` block, and (optionally) the specific raising function on every checked raise, and the Python siblings carry a dedicated error-envelope test covering roughly thirty raise paths. A new `stop()` or `raise` that omits the prefix or the `Fix:` block fails the suite. This is what keeps parity from rotting: the contract is enforced by the build, not by manual code review (see the contract-auditor and CI chapters).

6.2.4 Warnings wear the envelope too

The convention is not limited to fatal errors. The same shape decorates *non-fatal* warnings — most importantly the ones the sidecar machinery itself emits. When a sidecar write fails because of non-serialisable metadata, the warning is:

```
[cso_toolkit.dw_save] Provenance sidecar write failed for <path>: <reason>
  Primary file unaffected; metadata may contain non-serialisable objects.
  Fix: ensure all metadata values are JSON-serialisable (atomic types,
  lists of atomics, or named lists thereof).
```

The same shape covers another soft failure: when a reviewer falls back to a repo-local copy because the canonical and Z: locations are unreachable, the provenance-gap warning carries the `[cso_toolkit.dw_use]` prefix and explains that the fallback *breaks provenance* and that Teams/Z: should be re-mounted before the output is relied upon. The envelope makes even a soft degradation legible.

6.2.5 Cross-language parity for the envelope

The envelope is the most fully realised piece of cross-language parity in the toolkit, but the realisation is uneven and worth stating exactly. R and Python use the identical prefix-and-Fix shape, asserted in both test suites. Stata carries the same `[cso_toolkit.<fn>]` prefix throughout, but the *fullness* of the body varies by command: `dw_require_no_api` renders the complete three-part WHAT / Why / Fix envelope (across separate {phang} lines), whereas the `dw_save` reviewer-mode refusal compresses to a single line that names the rule and the `-allow_canonical_write-` override without a separate Why/Fix block. The shape and the diagnostic intent are the same across all three siblings; the test harness that polices it is not. The automated `expect_envelope`-style assertion exists for R and Python, while Stata's adherence is maintained by convention and review rather than by a packaged matcher.

6.3 How the two conventions reinforce each other

The sidecar and the envelope are two halves of the same reproducibility contract. The sidecar makes a *successful* write self-describing and verifiable after the fact: who, when, what mode, what hash, what schema. The envelope makes an *unsuccessful* operation self-explaining at the moment it happens: which function, which rule, why, and how to proceed. Together they ensure that a `cso-toolkit` artefact is never a bare file of unknown origin, and a `cso-toolkit` failure is never a bare exception of unknown cause. Both travel with the toolkit wherever it is vendored, in whichever of the three languages a consumer repo uses, with the gaps above stated openly.

Part III

Part III — The API (cross-language)

7 The IO contract

Every analytics pipeline begins and ends with reading and writing files. In the DW-Production world, however, an unguarded `read_csv` or `to_csv` is a liability: it can silently overwrite a published deposit, leave a producer's only copy on a laptop, read a stale cached file with no provenance, or let a duplicate-keyed table into the warehouse. The IO contract is `cso-toolkit`'s answer to all of these. It is a small family of paired read and write helpers — chiefly `dw_save` and `dw_use` — that wrap ordinary language-native IO with three layers of discipline: auto-dispatch on the file extension, mode-aware path routing, and a uniqueness and integrity contract enforced at the call site.

The toolkit promises parity — but not *full* parity. The same function names, the same arguments, and the same behaviour hold across R, Python, and Stata, so a sector that runs partly in R and partly in Stata uses one mental model on both sides. Where the languages must diverge, this chapter says where. Stata reads and writes only `.dta`, `.csv`, and `.xlsx`; it has no parquet, no RDS, no remote-URL fetch, and a few of the helpers below have no Stata sibling at all.

7.1 Orientation: five terms this chapter leans on

Before the mechanics, five load-bearing terms. The whole argument of the chapter — every guard, every mirror, every warning — rests on them, so they are worth fixing in place first.

- A **canonical deposit** is the permanent, published copy of a dataset. It is the version downstream work and external consumers depend on; once written for a given vintage it must never change.
- Deposits live in two shared network stores. **Teams** is the SharePoint-synced document library that holds the canonical area; **Z:** is a network drive that mirrors it byte-for-byte. The toolkit treats a path under either store as canonical.
- The **producer/reviewer mode** is a session-wide switch. In *producer* mode you write new deposits; in *reviewer* mode you may only read them, never overwrite canonical. Mode is a **session property**, never a per-call argument: it is set once by `dw_mode` in `~/.config/user_config.yml` and read by the consumer's profile at startup (see the chapter on wiring the contract into a repo). The profile makes the path globals (`teamsWrkData`, `teamsRawData`, and the `*Canonical` roots) mode-aware, and the helpers below resolve through them.
- An **error envelope** is the toolkit's structured failure message: it always states WHAT failed, WHY, and HOW to fix it. The same shape is used for hard stops and for non-blocking warnings (see the error-taxonomy appendix).

- **AppLocker** is the Windows policy on locked-down UNICEF laptops that blocks unsigned executables. It is why the Stata side never shells out to an external binary — a constraint that shapes two of the design boundaries below.

The `kind` argument that appears throughout selects which of three data roots a path resolves against: `raw` (inputs), `wrk` (working outputs), or `meta` (metadata). The functions covered here are `dw_save`, `dw_use`, `dw_compare`, `dw_merge`, `dw_isid`, `dw_verify_z`, `dw_stage`, and `dw_root`. They live in `r/R/dw_io.R`, `python/src/dw_io.py`, and the `.ado` files under `stata/src/`.

7.2 Auto-dispatch by extension

Both `dw_save` and `dw_use` choose their underlying reader or writer from the file extension. The caller never names a format; the path’s suffix decides. This keeps call sites uniform: switching an output from CSV to parquet is a one-character edit to the filename, not a rewrite.

Read the dispatch table below as three columns of parity — R and Python mirror each other almost exactly; Stata is the deliberate exception, and the first coverage gap.

Extension	R writer / reader	Python writer / reader	Stata
<code>.csv</code> / <code>.tsv</code> / <code>.txt</code>	<code>data.table::fwrite</code> / <code>fread</code>	<code>pandas.to_csv</code> / <code>read_csv</code>	yes (<code>.csv</code> only)
<code>.csv.gz</code> / <code>.tsv.gz</code>	<code>fwrite(compress="gzip")</code>	<code>to_csv(compression="gzip")</code>	—
<code>.xlsx</code>	<code>writexl</code> / <code>openxlsx</code> ; <code>readxl</code>	<code>openpyxl</code> via <code>pandas</code>	yes
<code>.dta</code>	<code>haven::write_dta</code> / <code>read_dta</code>	<code>pandas.to_stata</code> / <code>read_stata</code>	yes (native)
<code>.rds</code> (R) / <code>.pkl</code> (Py)	<code>saveRDS</code> / <code>readRDS</code>	<code>pickle.dump</code> / <code>load</code>	—
<code>.RData</code> / <code>.rda</code>	<code>save()</code> / <code>load()</code>	—	—
<code>.parquet</code>	<code>arrow::write_parquet</code> / <code>read_parquet</code>	<code>pandas.to_parquet</code> / <code>read_parquet</code>	—
<code>.json</code>	<code>jsonlite</code>	<code>json</code> / <code>to_json</code>	—
<code>.yaml</code> / <code>.yml</code>	<code>yaml</code>	<code>PyYAML</code>	—

The R and Python rows are near-complete mirrors. The only deliberate split is the in-language binary format: R’s `.rds` versus Python’s `.pkl`, both serving the same “fast native object dump” role. Stata covers `.dta`, `.csv`, and `.xlsx`, and nothing else.

Stata format limits

Parquet, RDS, and JSON reads and writes will not dispatch on the Stata side. A pipeline that needs a binary or columnar format in Stata routes the heavy step through R or Python and hands back a `.dta` (or `.csv`) that Stata can use. This is a permanent design boundary, not a missing feature: keeping Stata free of shell-outs and external binaries is what keeps the `.ado` helpers AppLocker-safe.

7.3 Writing: `dw_save`

`dw_save` is the uniform writer. It resolves a path, optionally asserts row uniqueness, writes atomically, emits a provenance sidecar — a companion JSON recording who wrote the file, when, and how — and, in producer mode, fans the write out to redundant mirrors. In its simplest form it takes an object and a path:

7.3.0.1 R

```
dw_save(df, path = "out.csv")
```

7.3.0.2 Python

```
dw_save(df, path="out.csv")
```

7.3.0.3 Stata

```
dw_save , filename("out") path("out") idvars(REF_AREA)
```

In practice you rarely pass a literal path. The structured call style lets the helper resolve the path through the mode-aware roots, so the same script lands in the canonical deposit in producer mode and in the local sandbox in reviewer mode with no change to the call. Three arguments do the work: `name` is the file basename, `sector` is the sector folder (`"ed"`, `"nt"`, ...), and `kind` selects the data root (`raw`, `wrk`, or `meta`, defaulting to `wrk`).

7.3.0.4 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk")
```

7.3.0.5 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk")
```

7.3.0.6 Stata

```
dw_save , filename("dw_ed_edu") path("$teamsWrkData/ed") idvars(REF_AREA INDICATOR)
```

Note the Stata divergence in calling convention. R and Python accept the `name / sector / kind` triple, which resolves against the profile roots via `dw_resolve_path`; Stata's `dw_save` takes an explicit `path()` and `filename()`, with `idvars()` mandatory (the uniqueness check is not optional in Stata). The structured resolver — `name / sector / kind` plus the `dw_resolve_path / dw_root` machinery behind it — is an R and Python feature.

7.3.1 The uniqueness contract: `isid`

Pass a key tuple and `dw_save` runs an `isid` check — asserting that the declared key columns uniquely identify every row — *before* writing a single byte. If any rows duplicate on the declared keys, the call stops with the duplicate count and a sample of the offending rows; the table never reaches disk. This is the toolkit's main guard against the classic warehouse defect of a silently double-counted series.

7.3.1.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",  
        isid = c("DATAFLOW", "REF_AREA", "INDICATOR", "SEX",  
                  "WEALTH_QUINTILE", "RESIDENCE", "TIME_PERIOD"))
```

7.3.1.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",  
        isid=["DATAFLOW", "REF_AREA", "INDICATOR", "SEX",  
              "WEALTH_QUINTILE", "RESIDENCE", "TIME_PERIOD"])
```

7.3.1.3 Stata

```
dw_save , filename("dw_ed_edu") path("$teamsWrkData/ed") ///  
        idvars(DATAFLOW REF_AREA INDICATOR SEX WEALTH_QUINTILE RESIDENCE TIME_PERIOD)
```

In R and Python `isid` is an optional argument; in Stata `idvars()` is required. The same check is also exposed standalone as `dw_isid` (see below).

7.3.2 The provenance sidecar

Every successful write (except R's `.RData` / `.rda`) emits a companion `<path>.provenance.json` next to the file. It records when the file was written, by whom, in which mode, the content hash, the schema (rows, columns, column names), the `isid` keys used, and any free-form metadata you supply. Pass `metadata` to make the sidecar self-describing:

7.3.2.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        metadata = list(
          title = "Education indicators - UNICEF DW format",
          producer = "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.R",
          sources = c("UIS bulk SDG_092025", "WPP 2024"),
          contact = "@karavan88",
          vintage = "2026-05"
        ))
```

7.3.2.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        metadata={
          "title": "Education indicators - UNICEF DW format",
          "producer": "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.py",
          "sources": ["UIS bulk SDG_092025", "WPP 2024"],
          "vintage": "2026-05",
        })
```

7.3.2.3 Stata

```
dw_save , filename("dw_ed_edu") path("$teamsWrkData/ed") ///
      idvars(REF_AREA INDICATOR)                        ///
      title("Education indicators")                      ///
      producer("01_dw_prep/012_codes/ed/example.do")    ///
      sources("UIS bulk SDG_092025")                    ///
      contact("@karavan88")                             ///
      vintage("2026-05")
```

On the Stata side, `title()`, `producer()`, `sources()`, `contact()`, and `vintage()` are dedicated options; arbitrary extra fields go through a semicolon-separated `metadata("key1 value1; key2 value2")` option. Stata also offers `nosidecar` to skip the sidecar entirely and `nocompress` to skip the `compress` step.

The sidecar schema is shared across all three languages, with one deliberate substitution: R and Python record a file-level **SHA-256**, while Stata records its native `datasignature` (a content hash of the dataset's values, not a hash of the file's bytes). The reason is the same AppLocker constraint that excludes the binary formats — computing SHA-256 in Stata would mean shelling out to an external binary. The practical consequence is that cross-tool integrity comparisons must compare *content* hashes, not *file* hashes. The full schema is documented in the provenance-sidecar appendix.

In R and Python, the sidecar write is wrapped so that a non-serialisable metadata value (a `Date` object or a numpy scalar, say) emits a warning rather than rolling back the primary write — the asset is what matters; the sidecar is metadata about it.

7.3.3 Mode-aware write routing

Mode-aware routing is the contract's central feature, and it does two opposite jobs depending on the session mode: it *blocks* unsafe writes in reviewer mode, and it *duplicates* safe writes in producer mode.

Reviewer mode refuses to write to a canonical deposit. Any write whose resolved path lands under a canonical Teams root or under the configured Z: drive root stops with the error envelope. Reviewer sessions must keep canonical deposits read-only so that a published vintage stays permanent. The Z: branch is the v0.4.0 broadening of the guard; earlier versions refused only canonical writes. For a deliberate Database Manager bootstrap, pass the bypass flag:

7.3.3.1 R

```
dw_save(df, path = canonical_path, allow_canonical_write = TRUE)
```

7.3.3.2 Python

```
dw_save(df, path=canonical_path, allow_canonical_write=True)
```

7.3.3.3 Stata

```
dw_save , filename("dw_ed_edu") path("$teamsWrkDataCanonical/ed") ///  
    idvars(REF_AREA) allow_canonical_write
```

In R the guard raises `stop()`; in Python it raises `PermissionError`; in Stata it raises error 459. All three carry the same WHAT / WHY / HOW envelope (see the error-taxonomy appendix).

Producer mode mirrors redundantly. A producer’s output must never live only on the producer’s laptop. So in producer mode each successful primary write is carbon-copied — along with its sidecar — to both its Teams canonical equivalent and its Z: drive equivalent, whichever are configured. The helper performs a pre-flight check: if *neither* mirror is reachable, `dw_save` hard-stops before writing, because the deposit would not be recoverable. Each mirror copy is non-blocking: a failure to reach Teams or Z: after the primary write emits an envelope-shaped warning (tagged “Teams mirror” or “Z: mirror”) but does not roll back the primary.

i `mirror_to_z` is gone

In v0.3.0 you chose mirroring per call with a `mirror_to_z` argument. From v0.4.0 mirroring is automatic and paired, driven entirely by which remote roots the profile makes available. `dw_save` still accepts the old argument but ignores it, with a deprecation warning.

The Z: drive matters here because it is a carbon copy of the Teams canonical deposit. A path under Z: is therefore *also* a canonical artifact: reviewer writes to it are barred, and reviewer reads of an explicit Z: path run the integrity check described below.

7.3.4 The overwrite gate

`dw_save` will not silently clobber an existing deposit, and the default behaviour is mode-aware. The `overwrite` argument is a sentinel: when left at its default it resolves to *permissive* in reviewer mode (the local sandbox is gitignored and safe to re-run) and *strict* in producer mode (re-running against the canonical deposit demands a deliberate decision). When the check runs strict, it examines **every** destination that will actually be written — primary, Teams mirror, and Z: mirror — and stops if any of them already exists.

7.3.4.1 R

```
# overwrite defaults to a mode-aware sentinel (NULL); pass TRUE to replace.
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        overwrite = TRUE)
```

7.3.4.2 Python

```
# In Python the default is the strict value (overwrite=False); pass True to replace.
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        overwrite=True)
```

7.3.4.3 Stata

```
* Stata always writes with `save, replace`; the canonical / Z: write barrier
* is enforced by the reviewer-mode guard above, not by an overwrite flag.
dw_save , filename("dw_ed_edu") path("$teamsWrkData/ed") idvars(REF_AREA)
```

The mode-aware sentinel is an R refinement. In R, `overwrite = NULL` resolves to `TRUE` in reviewer mode and `FALSE` in producer mode; Python's `overwrite` defaults to a plain `False` in every mode. Stata has no `overwrite` option at all: its `save, replace` always replaces the primary, and the canonical and Z: deposits are protected by the reviewer-mode guard rather than by an existence check.

In R there is one further safety stage: overwriting an *existing producer deposit* is treated as destructive even when `overwrite` is true. Interactively `dw_save` prompts [y/N]; non-interactively it stops unless you also pass `force = TRUE` for a deliberate automated re-deposit.

7.3.5 Other writing behaviours

- **Atomic writes.** R and Python both write to a `.tmp` sibling and rename into place, so a crash mid-write never leaves a half-written deposit at the canonical name. Stata writes via `save, replace`.
- **Compression.** For CSV / TSV / TXT, `compress = TRUE` appends `.gz` and gzips. If the path already ends in `.gz`, compression auto-enables — there is no foot-gun where you ask for `.gz` but get an uncompressed file.
- **CSV dialect (R only).** A `dialect` argument selects between `data.table::fwrite` (the default) and a `utils::write.table` path that preserves byte-parity with legacy `utils::write.csv()` output, for sector scripts that depend on the old bytes. The `base` dialect is incompatible with `compress = TRUE` and raises an explanatory error rather than failing silently.

7.4 Reading: `dw_use`

`dw_use` is the read counterpart: same path resolution, same extension dispatch. By default R returns a tibble; pass `as = "data.frame"` or `as = "data.table"` to change the shape. Python returns a pandas `DataFrame` for tabular formats. Stata loads the dataset into memory.

7.4.0.1 R

```
edu <- dw_use(name = "dw_ed_edu.csv", sector = "ed", kind = "wrk")
edu <- dw_use(path = "C:/path/to/file.parquet")
```

7.4.0.2 Python

```
edu = dw_use(name="dw_ed_edu.csv", sector="ed", kind="wrk")
edu = dw_use(path="/path/to/file.parquet")
```

7.4.0.3 Stata

```
dw_use , filename("dw_ed_edu.dta") path("$teamsWrkData/ed")
```

7.4.1 Mode-aware read routing

Reads, like writes, branch on the session mode — but in the *opposite* direction, because the goal here is provenance, not protection.

- **Producer / unknown mode (local-first).** Try the literal path; if it is missing and `fallback_canonical` is on (the default), walk the `teams*Data` → `teams*DataCanonical` prefix map and read the canonical copy.
- **Reviewer mode (network-first).** Try the Teams canonical equivalent first, then the Z: mirror, then fall back to the repo-local copy — but that last step emits an envelope-shaped warning, because a reviewer reading a local copy is reading bytes of unverified provenance. If the file is absent in all three places, `dw_use` raises an error pointing the reviewer at the sector producer. Disable the local fallback with `fallback_canonical = FALSE` (in Stata, the `nofallback_canonical` option) to fail fast instead.

The reviewer warning is the auditable signal of a provenance gap; it is meant to be noisy.

7.4.2 Column subsetting: strict and lenient

`dw_use` accepts a `cols` vector to restrict the read. The default is **strict**: every requested column must exist in the file’s schema, matching the contract of the underlying readers (`fread(select=)`, `read_parquet(col_select=)`, `read_dta(col_select=)`, `pandas usecols`). On the Stata side, `cols()` is a column subset on the dispatched `.dta` / `.csv` / `.xlsx`.

R adds a `cols_lenient = TRUE` flag that flips to “select if present” semantics: it introspects the file’s schema cheaply (parquet metadata, or a header-only read for dta / csv / xlsx), intersects the request with the actual columns, silently drops the missing ones, and — on an empty intersection — warns and reads all columns. The point is to give sector scripts the `dplyr::any_of()` intent *inside* `dw_use`, because calling `any_of()` at the top level errors fatally under `tidyselect` 1.2.0.

7.4.2.1 R

```
# Strict (default): errors if a requested col is missing from the schema
df <- dw_use(path = parquet_path, cols = c("REF_AREA", "OBS_VALUE"))

# Lenient: intersect with the file's actual schema; missing cols dropped
df <- dw_use(path = parquet_path,
             cols = c("REF_AREA", "OBS_VALUE", "MAYBE_PRESENT_COL"),
             cols_lenient = TRUE)
```

7.4.2.2 Python

```
# Strict column subset (pandas usecols / columns= semantics)
df = dw_use(path=parquet_path, cols=["REF_AREA", "OBS_VALUE"])
```

7.4.2.3 Stata

```
* Stata cols() subsets the dispatched .dta / .csv / .xlsx
dw_use , filename("dw_ed_edu.dta") path("$teamsWrkData/ed") ///
      cols(REF_AREA OBS_VALUE)
```

The `cols_lenient` flag and its metadata-only schema introspection are an R-side feature; Python and Stata offer the strict subset only.

i A read bug worth remembering

Before v0.4.3, R's `dw_use(parquet_path)` with no `cols` passed `col_select = NULL` to Arrow, which Arrow read as “select zero columns” — returning an empty-column tibble. The dta branch had the same defect. From v0.4.3 the helper passes `col_select` only when `cols` is non-NULL, so a bare `dw_use(path)` reliably returns all columns. If you see a zero-column read from an old vendored copy, this is why; bump the vendored version.

7.4.3 The Z: integrity check

When a read resolves to a *canonical* file and Z: is available, `dw_use` automatically compares the Teams copy against its Z: mirror. By default this is a fast file-size check; a mismatch emits a warning but the read still completes (with the Teams data). Pass `verify_z = "sha256"` for a deep hash comparison, or `verify_z = FALSE` (in Stata, `verify_z(off)`) to skip it. On the Stata side the deep comparison uses `datasignature` and only applies to the `.dta` path. The check is exposed standalone as `dw_verify_z`.

7.5 The lower-level and resolver helpers

7.5.1 `dw_isid` — standalone uniqueness check

The uniqueness assertion `dw_save` runs internally is also callable on its own, for validating an in-memory frame mid-pipeline:

7.5.1.1 R

```
dw_isid(df, keys = c("REF_AREA", "TIME_PERIOD"))
```

7.5.1.2 Python

```
dw_isid(df, keys=["REF_AREA", "TIME_PERIOD"])
```

7.5.1.3 Stata

```
* Stata uses the built-in isid; dw_save enforces it via idvars()  
isid REF_AREA TIME_PERIOD
```

It stops with the duplicate count and a sample of offending rows on failure, and it also catches the easy mistake of naming a key column that does not exist in the data.

7.5.2 `dw_verify_z` — Z: mirror integrity

`dw_verify_z` returns a status list (`match_size`, `size_mismatch`, `match_sha256`, `sha256_mismatch`, `z_missing`, `no_z_mirror`, ...) rather than stopping, so callers decide what to do with a mismatch. It is the building block behind `dw_use`'s automatic check. This is an R and Python helper; on the Stata side the equivalent check is folded into `dw_use` (size by default, `datasignature` deep check on request) and is not separately exposed.

7.5.3 `dw_root` — mode-aware data root

`dw_root("wrk" | "raw" | "meta")` returns the absolute path to one of the three vendored data roots, already resolved for the current mode. Sector scripts use it to build paths without knowing whether they are in producer or reviewer mode:

```
final_path <- file.path(dw_root("wrk"), "nt", "2025", "dw_nut.csv")
```

This is an R helper. The richer structured resolver, `dw_resolve_path(name=, sector=, kind=, vintage=)`, is the full path-builder that `dw_save` and `dw_use` call internally; it is available in both R and Python.

7.5.4 `dw_stage` — reviewer-mode sandbox staging

`dw_stage` (R only) is an opt-in reviewer helper that copies a canonical input from Z: (preferred) or Teams into the local sandbox the first time a script needs it, verifies the copy's hash against the source, and archives that hash in a `<path>.staged.json` sidecar. On every later run it re-checks the sandbox copy against the archived hash before letting the read proceed. It is a no-op outside reviewer mode.

The lifecycle has five states:

- **First stage** — the sandbox copy is missing, so copy from the first available source (Z: before Teams), re-hash the copy against the source, and archive the hash.
- **Clean later run** — the sandbox copy still matches its archived hash; skip the copy and return the sandbox path.
- **Sandbox drift** — the staged copy was modified out of band; stop with the SANDBOX DRIFT envelope.
- **Forced re-stage** — `overwrite = TRUE` unconditionally re-copies, re-verifies, and re-archives.
- **Upstream changed** — the canonical source moved on; warn, but do not auto-refresh, because rolling canonical inputs forward is the reviewer's explicit decision.

When both Z: and Teams hold the file, the two are hash-compared before either is used; disagreement is a canonical-integrity stop.

You rarely call `dw_stage` directly. The usual path is to let `dw_use` invoke it, per call or globally:

7.5.4.1 R

```
# Per-call opt-in: stage the canonical input into the sandbox, then read it
df <- dw_use(name = "dw_nut.csv", sector = "nt", kind = "wrk", stage = TRUE)

# Or globally, in the profile:
# dw_autostage <- TRUE
```

7.5.4.2 Python

```
# Not available - dw_stage is an R-only helper.
df = dw_use(name="dw_nut.csv", sector="nt", kind="wrk")
```

7.5.4.3 Stata

```
* Not available - dw_stage is an R-only helper.
dw_use , filename("dw_nut.dta") path("$teamsWrkData/nt")
```

`dw_stage` and `dw_use`'s `stage = integration` are R-only. The `.staged.json` sidecar is deliberately distinct from `dw_save`'s `.provenance.json`, so a file that is both written by `dw_save` and staged by `dw_stage` carries both metadata blocks side by side; each reader refuses to misread the other's schema by checking a `type` discriminator.

7.6 Comparing and joining

7.6.1 `dw_compare` — three-way diff against a reference

`dw_compare` answers “what changed since the last deposit?” It keys two data frames on `by`, then classifies rows as **added** (present in current, absent in reference), **removed** (the reverse), and **changed** (present in both, with a differing value column). Numeric columns can compare within a tolerance; string columns normalise missing-equivalents (`"`, `"NA"`, `"N/A"`, `"NULL"`, `"."`) to equal. In R and Python it returns a list with a one-row **summary** plus the **added**, **removed**, and **changed** frames, and can optionally write per-segment CSV reports.

7.6.1.1 R

```
report <- dw_compare(
  current   = dw_use(name = "dw_ed_edu.csv", sector = "ed", kind = "wrk"),
  reference = dw_use(path = file.path(teamsWrkDataCanonical, "ed/dw_ed_edu.csv")),
  by        = c("DATAFLOW", "REF_AREA", "INDICATOR", "SEX",
                "WEALTH_QUINTILE", "TIME_PERIOD"),
  value_cols = c("OBS_VALUE", "DATA_SOURCE"),
  numeric_value_cols = "OBS_VALUE",
  tol        = 1e-5,
  label      = "ed_dw_edu",
  write_report_to = file.path(tempdir(), "ed_compare")
)
report$summary
```

7.6.1.2 Python

```
report = dw_compare(  
    current=dw_use(name="dw_ed_edu.csv", sector="ed", kind="wrk"),  
    reference=dw_use(path=ref_path),  
    by=["DATAFLOW", "REF_AREA", "INDICATOR", "SEX",  
        "WEALTH_QUINTILE", "TIME_PERIOD"],  
    value_cols=["OBS_VALUE", "DATA_SOURCE"],  
    numeric_value_cols=["OBS_VALUE"],  
    tol=1e-5,  
    label="ed_dw_edu",  
)  
report["summary"]
```

7.6.1.3 Stata

```
* idvars / valuevars / tol / report / label; compares two .dta files  
dw_compare , current("dw_ed_edu.dta") reference("ref_dw_ed_edu.dta") ///  
    idvars(DATAFLOW REF_AREA INDICATOR SEX WEALTH_QUINTILE TIME_PERIOD) ///  
    valuevars(OBS_VALUE) tol(0.00001) label("ed_dw_edu")
```

`dw_compare` tolerates a degenerate first deposit gracefully: in R and Python a missing or unreadable reference is treated as empty (with a warning), so every current row reports as added rather than crashing the run. Note the divergences. R and Python name the arguments `value_cols` / `numeric_value_cols`; Stata uses `valuevars` and infers numeric-versus-string comparison from each variable's storage type. The numeric tolerance defaults differ too — `1e-5` in R and Python, `1e-8` in Stata — so set `tol` explicitly when it matters. The Stata `dw_compare` prints a console summary and, via `report()`, an optional Markdown file of row counts and per-column change counts; it is sized for a publication-gate diff, not the rich row-level Markdown of its upstream `comparefiles` ancestor.

7.6.2 `dw_merge` — Stata-style join with cardinality assertion

`dw_merge` is a join that *asserts its cardinality*. You declare `how` as one of `m:1`, `1:1`, `1:m`, or `m:m`, and the helper warns when the actual duplication of the keys on either side disagrees with what you declared — catching the silent fan-out that turns a clean join into a row explosion. The `using` side can be a data frame already in memory or a path string, which is auto-read through `dw_use`.

7.6.2.1 R

```
enriched <- dw_merge(
  edu_sdg_uis,
  using = file.path(teamsRawData, "ed/metadata/regions.csv"),
  by    = "ISO3",
  how   = "m:1"
)
```

7.6.2.2 Python

```
enriched = dw_merge(
  edu_sdg_uis,
  using=os.path.join(teamsRawData, "ed/metadata/regions.csv"),
  by="ISO3",
  how="m:1",
)
```

7.6.2.3 Stata

```
* Stata has the native `merge`, which already takes the cardinality;
* dw_merge is an R / Python helper.
merge m:1 ISO3 using "regions.dta"
```

Under the hood `dw_merge` is a left join; the value it adds over a bare join is the cardinality warning. It is an R and Python helper — Stata’s built-in `merge m:1 / 1:1 / ...` already carries the cardinality declaration natively, so no wrapper is shipped.

7.7 Putting it together

A typical producer step reads its inputs, prepares the data, asserts uniqueness, and deposits the result with provenance — all through the contract, so the same script runs unmodified in reviewer mode (reading network-first, writing only to the sandbox) and in producer mode (reading local-first, writing redundantly to Teams and Z:):

7.7.0.1 R

```
raw <- dw_use(name = "uis_sdg.parquet", sector = "ed", kind = "raw")
edu <- aggregate_indicators(raw)           # ... your prep ...

dw_save(edu, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
```

```

isid = c("REF_AREA", "INDICATOR", "TIME_PERIOD"),
metadata = list(title = "Education indicators",
                producer = "ed/02_aggregate.R",
                sources = c("UIS bulk SDG_092025"),
                vintage = "2026-05"))

```

7.7.0.2 Python

```

raw = dw_use(name="uis_sdg.parquet", sector="ed", kind="raw")
edu = aggregate_indicators(raw)          # ... your prep ...

dw_save(edu, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        isid=["REF_AREA", "INDICATOR", "TIME_PERIOD"],
        metadata={"title": "Education indicators",
                  "producer": "ed/02_aggregate.py",
                  "sources": ["UIS bulk SDG_092025"],
                  "vintage": "2026-05"})

```

7.7.0.3 Stata

```

* uis_sdg is .parquet upstream; the producer hands Stata a .dta to consume.
dw_use , filename("uis_sdg.dta") path("$teamsRawData/ed")
* ... your prep ...
dw_save , filename("dw_ed_edu") path("$teamsWrkData/ed") ///
    idvars(REF_AREA INDICATOR TIME_PERIOD)                ///
    title("Education indicators")                          ///
    producer("ed/02_aggregate.do")                        ///
    sources("UIS bulk SDG_092025")                        ///
    vintage("2026-05")

```

The roles and mode-contract chapter explains how the producer/reviewer switch is set and what each role may do; the external-API chapter covers the remote-fetch side of IO (which, on the Stata side, must route through R or Python — Stata has no API wrapper); and the provenance-sidecars chapter and the appendices give the sidecar schema, the error taxonomy, the full function reference, and the cross-language coverage matrix that this chapter has summarised in prose.

8 The external-API contract

A reproducible analysis must be able to re-run today and produce the same numbers it produced last month — even if a remote statistical agency has since revised its figures, renamed an indicator, or simply gone down for maintenance. Yet most analytics code reaches for live data with bare calls to `fromJSON()`, `rsdmx::readSDMX()`, `wbstats::wb_data()`, `httr::GET`, `requests.get()`, or `wbgapi`. Each such call is a silent dependency on the network and on whatever the upstream server happens to be serving at that instant. The external-API contract removes that fragility. It gives every consumer of `cso-toolkit` a single, mode-aware wrapper that fetches once, freezes the result to a provenance-stamped cache, and thereafter reads from the frozen copy.

Three roles govern who may touch the network. A *producer* (a session running in producer mode) hits the live API and populates the cache; a *reviewer* (a session running in reviewer mode) reads the frozen cache and is forbidden from touching the network at all; and the *Database Manager* — the team member who owns the deposit — is the producer who decides when each cache is refreshed. These are session modes and a stewardship role, not three separate job titles: the same person is a producer one day and a reviewer the next, depending on how their session is configured. The roles-and-mode contract chapter defines all three in full; this chapter shows how the API wrapper enforces them.

This chapter documents the three public entry points of the contract — `dw_api_fetch`, `dw_api_cached`, and `dw_api_inventory` — the eleven external sources they cover (ten shared with Python; `csv` is R-only so far), the cache layout and provenance sidecars they emit, and the producer/reviewer behaviour that makes the whole thing reproducible. It builds on the roles-and-mode contract (the chapter on roles and the mode contract), on the IO contract (the wrapper writes and reads its cache through `dw_save` and `dw_use`, covered in the IO contract chapter), and on the provenance-sidecar and error-envelope conventions (the provenance-sidecars chapter and the error-taxonomy chapter).

8.1 Why a wrapper at all

Calling a remote API directly couples three concerns that ought to be separate: *what* you want (an indicator, a codelist, a flow), *how* you reach it (the package, the URL shape, the auth), and *when* it was last retrieved. The wrapper separates them. You name the source with a short `api = string`, you name the result with a stable `cache_key`, and the wrapper dispatches to the right package, builds the URL, retries or fails with a readable error, writes the result to disk, and records the provenance of the fetch. A sector script stops being a tangle of bespoke HTTP and becomes a manifest of named fetches.

The reference material records one migration in which fourteen hand-coded SDMX codelist GETs — about 290 lines of repeated `httr::GET / stop_for_status / JSON-unwrap` boilerplate — collapsed into roughly fifty lines: a small manifest tibble and a loop over `dw_api_fetch(api = "sdmx_codelist", ...)`.

8.2 The three entry points

The contract exposes exactly three public functions. They share names across R and Python. Stata, as noted below, ships only the enforcement half of the contract and has no fetch functions of its own.

`dw_api_fetch(api, cache_key, ...)` is the workhorse. It checks the cache first; on a miss it either fetches live (producer) or stops (reviewer). Its signature also accepts `refresh`, `ext`, and `metadata` keywords, described in the sections that follow, plus per-source arguments that flow through `...` to the internal fetcher. `dw_api_cached(api, cache_key)` is the explicit cache-only reader — it never touches the network under any mode, and raises if the cache is absent. `dw_api_inventory(api = NULL)` lists what has already been cached, reading the provenance sidecars so the Database Manager can audit freshness.

8.2.0.1 R

```
# Fetch (or read cache): one named source, one stable cache key.
entrance_age_primary <- dw_api_fetch(
  api      = "uis",
  endpoint = "indicators",
  params   = list(indicator = "299905"),
  cache_key = "uis_entrance_age_primary"
)

# Cache-only read; never hits the network, stops if absent.
lp <- dw_api_cached(api = "wb", cache_key = "wb_learning_poverty_primary")

# What is already cached?
dw_api_inventory("wb")
```

8.2.0.2 Python

```
from cso_toolkit import dw_api_fetch, dw_api_cached, dw_api_inventory

entrance_age_primary = dw_api_fetch(
    api="uis",
    endpoint="indicators",
```



```

    params={"indicator": "299905"},
    cache_key="uis_entrance_age_primary",
)

# Raises FileNotFoundError if no cache exists.
lp = dw_api_cached(api="wb", cache_key="wb_learning_poverty_primary")

dw_api_inventory("wb")

```

8.2.0.3 Stata

```

* Stata has no API fetcher. There is no dw_api_fetch / dw_api_cached /
* dw_api_inventory in the Stata package. A Stata sector script that needs
* external data either reads a frozen cache that an R or Python producer
* already deposited (via dw_use), or asserts the no-API contract before any
* live network call it might make of its own:
dw_require_no_api , context("ed/06_pull_uis")

```

i Stata coverage is partial by design

The Stata package ships `dw_require_no_api` — the reviewer-mode gate that aborts a live network call with Stata error 459 (a generic assertion failure) — but it ships **no** `dw_api_fetch`, `dw_api_cached`, or `dw_api_inventory`. There is no Stata API wrapper. The intended pattern is to let an R or Python producer populate the `_apis/` cache and have Stata read the frozen result through `dw_use`. Treat the Stata tab in this chapter as “route via R/Python,” not as parity. The cross-language coverage matrix appendix records the full picture.

8.3 The mode contract, applied to the network

The *session mode* — not an argument — controls how `dw_api_fetch` behaves. There is deliberately no `mode =` parameter on the fetch function. Mode is a session property: it is set in `user_config.yml` as `dw_mode` and read by the repo’s profile; in Python it lives on the shared `_state` module. The profile derives a second switch from it, `dw_apis_allowed`, which is set to `True` exactly when `dw_mode == "producer"`. That boolean is the operative gate the wrapper actually tests before it reaches the network, so the two switches are not independent: `dw_apis_allowed` is the wrapper-facing spelling of producer mode. The Python examples in this chapter configure `dw_apis_allowed=True` directly because they stand alone without a profile; in a real consumer, the profile sets it for you when the mode is **producer**.

The full matrix:

Session	Cache state	Action
Producer	cache present, <code>refresh = FALSE</code>	reads the cache
Producer	cache missing or <code>refresh = TRUE</code>	hits the live API, writes the cache (via <code>dw_save</code>), returns the result
Reviewer	cache present (canonical)	reads the cache
Reviewer	cache missing	STOPS — <code>dw_require_no_api()</code> in R, <code>PermissionError</code> in Python

Two refinements matter. First, the cache lookup is two-tiered: on a non-refresh call the wrapper looks in the producer sandbox path (`teamsRawData`) first and then falls back to the canonical read-only path (`teamsRawDataCanonical`). A reviewer therefore reads from the frozen canonical deposit even when no sandbox copy exists. (Both roots are introduced in the IO contract chapter; here they serve only as the two tiers of the cache lookup.) Second, `refresh = TRUE` is a producer-only escape hatch: it forces a live fetch even when a cache is present, and is how the Database Manager deliberately re-pulls a source on a schedule the team owns.

The reviewer lockout is loud and instructive rather than a bare error. In Python, a missing cache under reviewer mode raises `PermissionError` naming the offending call, the path it expected, and a two-step fix (verify the Database Manager populated the canonical cache, or switch to producer mode via `_state.configure(...)`). In R the equivalent stop comes from `dw_require_no_api()`, the profile-defined guard that aborts with `[X] External API access is forbidden in reviewer mode`. The Stata sibling, for any live call a Stata script makes directly, aborts with the same envelope shape and Stata error 459.

8.4 The supported sources

The wrapper dispatches on the `api = string`. The R implementation supports eleven values; the Python port supports the same ten with the exception of `csv`, which was added to R in v0.6.0. Each maps to a thin internal fetcher that knows the URL shape for that source and imports its package lazily — that is, the package is loaded only when that source is actually fetched, so a consumer that never touches SDMX need not have `rsdmx` or `sdmx1` installed.

api = value	Source / what it replaces	R package	Python package	Key ar- guments
"uis"	UNESCO UIS REST (fromJSON("https://api.uis.unesco.org/..."))	jsonlite	requests	endpoint, params
"sdmx"	Generic SDMX data, any provider (rsdmx::readSDMX)	rsdmx	sdmx1	providerId, flowRef, key, version, start, end
"sdmx_codelist"	UNICEF SDMX codelist → (code, name, description) (httr::GET + JSON)	httr, tibble	requests	agency, codelist, version
"wb"	World Bank WDI (wbstats::wb_data / wbgapi.data)	wbstats	wbgapi	indicator, start_date, end_date
"wb_indicators"	World Bank indicator catalogue	wbstats	wbgapi	(catalogue query)
"ilo"	ILO SDMX (rsdmx::readSDMX(providerId = "ILO"))	rsdmx	sdmx1	flowRef, key, start, end
"unsd_sdg"	UNSD SDG API (httr::POST .../Series/DataCSV, form-encoded)	httr, readr	requests	series_codes, endpoint
"github_raw"	Pinned-commit raw.githubusercontent.com fetch	readr	requests	owner_repo, ref, path
"http"	Generic HTTP GET returning text (httr::GET)	httr	requests	url, headers
"json_get"	Generic JSON GET → parsed object (jsonlite::fromJSON)	jsonlite	requests	url
"csv" (R-only, v0.6.0)	Flat CSV-at-a-URL: SDMX REST-CSV, WB Data360 files, ILOSTAT rplumber format=.csv (readr::read_csv)	readr	— (not yet ported)	url

One serialisation default is worth calling out. Several sources return shapes that do not flatten cleanly into CSV — a nested JSON tree, plain text, the World Bank catalogue’s list-columns, or arbitrary content fetched by `github_raw`. Both languages therefore default those four sources — `wb_indicators`, `json_get`, `http`, and `github_raw` — to a non-tabular serialisation: RDS in R, pickle (`pk1`) in Python. The remaining flat-tabular sources default to CSV — the six shared ones plus R’s `csv` fetcher. The `ext =` argument lets a caller override the per-API default when needed, for example to force a UIS response into a non-tabular cache:

8.4.0.1 R

```
# Force a non-tabular cache when the response isn't flat-rectangular.
raw_tree <- dw_api_fetch(
  api      = "uis",
  endpoint = "indicators",
  params   = list(indicator = "299905"),
  cache_key = "uis_entrance_age_primary_raw",
  ext      = "rds"
)
```

8.4.0.2 Python

```
raw_tree = dw_api_fetch(
    api="uis",
    endpoint="indicators",
    params={"indicator": "299905"},
    cache_key="uis_entrance_age_primary_raw",
    ext="pkl",
)
```

8.4.0.3 Stata

```
* n/a - no Stata fetcher; the producer chooses ext on the R/Python side.
```

A few worked calls, side by side:

8.4.0.4 R

```
# UNICEF SDMX codelist (one of the 14 collapsed into a loop)
codebook_residence <- dw_api_fetch(
  api      = "sdmx_codelist",
  agency   = "UNICEF",
  codelist = "CL_RESIDENCE",
  version  = "1.0",
  cache_key = "sdmx_codelist_UNICEF_CL_RESIDENCE_1_0"
)

# World Bank Learning Poverty
lp_data <- dw_api_fetch(
  api      = "wb",
```

```

    indicator = c("SE.LPV.PRIM", "SE.LPV.PRIM.FE", "SE.LPV.PRIM.MA"),
    start_date = 2000,
    end_date   = 2025,
    cache_key  = "wb_learning_poverty_primary"
)

# Pinned-commit GitHub metadata (ref pins reproducibility)
regions <- dw_api_fetch(
  api      = "github_raw",
  owner_repo = "unicef-drp/Country-and-Region-Metadata",
  ref      = "main",
  path     = "UNICEF_REP_REG_GLOBAL.csv",
  cache_key = "github_unicef_regions_global"
)

```

8.4.0.5 Python

```

from cso_toolkit import _state, dw_api_fetch

# Standalone configuration (a real consumer's profile does this for you,
# deriving dw_apis_allowed=True from dw_mode == "producer").
_state.configure(
    teamsRawData="/data/raw",
    teamsRawDataCanonical="/data/raw-canonical",
    dw_mode="producer",
    dw_apis_allowed=True,
)

codebook_residence = dw_api_fetch(
    api="sdmx_codelist",
    agency="UNICEF",
    codelist="CL_RESIDENCE",
    version="1.0",
    cache_key="sdmx_codelist_UNICEF_CL_RESIDENCE_1_0",
)

lp_data = dw_api_fetch(
    api="wb",
    indicator=["SE.LPV.PRIM"],
    start_date=2000,
    end_date=2025,
    cache_key="wb_learning_poverty_primary",
)

```

8.4.0.6 Stata

```
* No Stata fetcher. Read the frozen cache an R/Python producer deposited:  
dw_use, filename("wb_learning_poverty_primary.csv") path("$teamsRawData/_apis/wb")
```

8.5 The cache and its provenance

Every fetch writes two files into the deposit, under a fixed layout keyed by `api` and `cache_key`:

```
<rawdata>/_apis/<api>/<cache_key>.<ext>  
<rawdata>/_apis/<api>/<cache_key>.<ext>.provenance.json
```

The `cache_key` is the filename basename, so it should be human-readable snake_case carrying the distinguishing parameters — prefer `wb_learning_poverty_primary` or `sdmx_codelist_UNICEF_CL_RESIDENCE_1_0` over a hash. The wrapper does not invent the cache write: it delegates to `dw_save`. Three things follow from that delegation. The cache inherits the mode-aware write path; it inherits the Z: mirror when the drive is mapped; and it inherits the standard `.provenance.json` sidecar described in the provenance-sidecars chapter. Reads go back through `dw_use`.

The sidecar records both *what* the cache contains and *how* the wrapper obtained it. Beyond the IO contract's base fields (`path`, `format`, `written_at`, `user`, `dw_mode`, `sha256`, `schema`), `dw_api_fetch` merges an API-specific metadata block:

```
{  
  "path": ".../sdmx_codelist_UNICEF_CL_RESIDENCE_1_0.csv",  
  "format": "csv",  
  "written_at": "2026-05-25T14:23:11+0000",  
  "user": "jpazvd",  
  "dw_mode": "producer",  
  "sha256": "...",  
  "schema": {"rows": 4, "cols": 3, "columns": ["code", "name", "description"]},  
  "metadata": {  
    "api": "sdmx_codelist",  
    "cache_key": "sdmx_codelist_UNICEF_CL_RESIDENCE_1_0",  
    "fetched_at": "2026-05-25T14:23:11+0000",  
    "elapsed_secs": 0.412,  
    "refresh_flag": false,  
    "fetch_args": {"agency": "UNICEF", "codelist": "CL_RESIDENCE", "version": "1.0"}  
  }  
}
```

The `fetch_args` capture is what lets a reviewer (or a future producer) see exactly which parameters produced the frozen result, and `fetch_at` plus `elapsed_secs` give the freshness and cost of the fetch. The `refresh_flag` records whether the fetch was a routine populate or an explicit re-pull. A caller's own `metadata =` argument is merged on top of these auto-recorded fields, which is how a load-bearing fetch carries a reason, a ticket, or a vintage into the sidecar alongside the parameters.

i Sensitive keys are redacted before they reach disk — in Python

The Python wrapper redacts the values of `fetch_args` whose key names look sensitive — anything containing `token`, `auth`, `password`, `secret`, `api_key`, `headers`, `cookie`, `bearer`, and similar substrings — replacing them with "<redacted>" in the persisted sidecar, so credentials passed by the caller never land in `.provenance.json`. The R wrapper does **not** redact: it stores `fetch_args` verbatim, so an R producer who passes a credential through ... will see it in the sidecar. This redaction parity is currently a Python-only feature; the cross-language coverage matrix appendix records it.

Caches **never expire automatically**. Refresh is always an explicit `refresh = TRUE`, and the Database Manager owns the refresh cadence per cache.

8.6 Auditing what is cached

`dw_api_inventory` walks `teamsRawDataCanonical/_apis/`, skips the `.provenance.json` sidecars, and returns one row per cache file. When a sidecar is present it pulls `metadata$fetch_at` (falling back to `written_at`) into the result, so the Database Manager can spot stale caches at a glance. The result frame carries `api`, `cache_key`, `ext`, `size_bytes`, `mtime`, and `fetch_at`; it returns an empty frame when nothing is cached.

8.6.0.1 R

```
dw_api_inventory()
#   api  cache_key                ext  size_bytes  mtime                fetched_at
# 1 wb    wb_learning_poverty_primary csv    142136    2026-05-23 16:42  2026-05-23T16:42
```

8.6.0.2 Python

```
inv = dw_api_inventory()
# DataFrame columns: api / cache_key / ext / size_bytes / mtime / fetched_at
```

8.6.0.3 Stata

```
* No Stata inventory function. Inspect the deposit directly, e.g.:  
dir "_apis/"
```

8.7 The HTTP error envelope

Every Python fetcher routes its network call through a single helper, `_http_call`, which wraps `requests.{get,post}` and translates the `requests.RequestException` family into the toolkit’s uniform “what / why / how” envelope. The three cases are distinct and each ends in a concrete fix:

- **Timeout** → `TimeoutError`, with a hint to extend `timeout=` or `retry`, and a specific note that a corporate (UNICEF-laptop) proxy can silently block long-lived calls — try a different network.
- **Connection failure** → `ConnectionError`, suggesting a hotspot or VPN when the corporate proxy may be blocking the host.
- **HTTP status error** → `RuntimeError`, carrying the status code, a 300-character body snippet, and a decode of common causes (400 bad params, 401/403 auth, 404 missing, 429 rate-limited).

This is the same “what / why / how” envelope used elsewhere in the toolkit; the error-taxonomy chapter covers the full classification. The R fetchers raise the underlying package errors (for example, `httr::stop_for_status`) rather than routing through a single shared envelope, so the error-envelope parity is currently a Python feature. This is a coverage gap; the cross-language coverage matrix appendix records it.

 API failures are about the network, not your `cache_key`

A timeout or 403 surfaces only on a live fetch — that is, in producer mode on a cache miss or an explicit `refresh = TRUE`. A reviewer never sees these because a reviewer never reaches the network. If you hit a recurring 403 burst against the UNICEF SDMX endpoint, that is an upstream rate-limit pattern, not a wrapper bug; the cure is to retry later or fetch from a different network, then commit the resulting frozen cache.

8.8 Migrating a sector script

The contract is adopted one call at a time. The checklist is the same in both languages:

1. Find each raw API call — `fromJSON`, `readSDMX`, `wbstats::wb_data`, `httr::GET/POST` in R; `requests.get`, `httpx.get`, `sdmx.Client`, `wbgapi.data.fetch` in Python.
2. Pick the matching `api =` value from the table above.

3. Choose a stable, descriptive snake_case `cache_key` that embeds the distinguishing parameters — it becomes the cache filename.
4. Replace the call with `dw_api_fetch(api = ..., cache_key = ..., ...)`, and stamp load-bearing fetches with the `metadata =` argument (a reason, a ticket, a vintage) so that context flows into the sidecar.
5. Run once in producer mode to populate the cache.
6. Open a PR. The smoke test passes when reviewer mode can run the script end-to-end from the cached result with **no** network access.

Step 6 is where the scaffolding-and-contract-auditor tooling earns its keep: in Python, `cso_toolkit.test_scripts(<sector_folder>, error_on_violation=True)` confirms that no raw network calls remain. The contract auditor and its CI integration are covered in the scaffolding-and-contract-auditor chapter and the auditor-in-CI chapter; the function reference appendix lists every signature in one place, and the cross-language coverage matrix appendix summarises exactly where R, Python, and Stata diverge — including the Stata gaps this chapter has flagged.

9 Aggregation

Aggregation is the step where many country-level observations collapse into a few regional or global numbers. It is also where most reproducibility bugs hide: an unweighted mean where a population-weighted one was intended, a “global” figure built from countries covering a third of the world’s children, a regional rate that silently dropped every country missing the denominator. The aggregation *contract* — the fixed set of functions and behaviours every language implementation must honour — exists to make these decisions explicit, auditable, and identical across the languages a sector team happens to use.

The contract has two layers. At the bottom sits `aggregate_data_v2()`, a general-purpose grouped aggregator that computes a value together with the coverage metadata needed to judge whether that value is trustworthy. On top sit a set of convenience helpers, each pairing one recurring problem with one function: `apply_time_window()` (picking the latest observation in a window), `dw_nestweight()` (redistributing weights over missing rows), `dw_pop()` (fetching population denominators), and `dw_regions()` (producing UNICEF regional aggregates). Each feeds its output into the aggregator. A companion helper, `generate_agg_footnote()`, turns the aggregator’s coverage numbers into the caption that travels with the result.

Warning

Cross-language parity is partial here, and this chapter flags every gap. `aggregate_data_v2`, `apply_time_window`, `generate_agg_footnote`, `dw_nestweight`, and `dw_pop` ship in **R and Python**. `dw_regions` is **R-only**: its source file carries a TODO marker for the missing Python and Stata siblings (tracked at issue #18); `dw_pop`’s remaining gap is its Stata sibling (tracked at issue #17). **Stata has none of the aggregation helpers** — its `src/` directory stops at file IO (`dw_save`, `dw_use`, and friends). And even where R and Python both ship a function, one method differs: the R aggregator supports a `"total"` method that the Python aggregator does not. See the cross-language coverage matrix appendix for the consolidated picture.

9.1 The core aggregator: `aggregate_data_v2`

`aggregate_data_v2()` takes a long-format data frame — one observation per row — of country (or unit) observations and returns one row per group, where a *group* is defined by the `by` columns. Each output row carries the aggregated value plus a configurable set of coverage columns.

The signature is the same in both languages, apart from two mechanical differences: R uses `.-separated` logical flags (`pop.coverage`, `country.coverage`) and a positional/named mix, while Python makes everything after `by` keyword-only and renames the handful of arguments whose R spelling collides with Python syntax (`global` becomes `global_`, the dotted flags become underscored).

9.1.0.1 R

```
agg <- aggregate_data_v2(
  data,
  value = "OBS_VALUE",
  weight = "population",
  by = c("INDICATOR", "TIME_PERIOD"),
  method = "weighted_mean",
  coverage_threshold = 0.5
)
```

9.1.0.2 Python

```
agg = aggregate_data_v2(
    data,
    value="OBS_VALUE",
    weight="population",
    by=["INDICATOR", "TIME_PERIOD"],
    method="weighted_mean",
    coverage_threshold=0.5,
)
```

9.1.0.3 Stata

```
* No Stata sibling. Aggregate with native Stata commands
* (collapse / egen) and treat this contract as R/Python-only.
```

9.1.1 Arguments

The full argument set, with R and Python spellings side by side:

Purpose	R	Python	Default
Input frame	<code>data</code>	<code>data</code>	—
Column to aggregate	<code>value</code>	<code>value</code>	—

Purpose	R	Python	Default
Weight column	<code>weight</code>	<code>weight</code>	—
Grouping columns	<code>by</code>	<code>by</code>	—
Country identifier (for coverage)	<code>country_id</code>	<code>country_id</code>	"REF_AREA"
Append a global row	<code>global</code>	<code>global_</code>	TRUE / True
Aggregation method	<code>method</code>	<code>method</code>	"weighted_mean"
Min population coverage to report	<code>coverage_threshold</code>	<code>coverage_threshold</code>	NULL / None
Return population coverage	<code>pop.coverage</code>	<code>pop_coverage</code>	TRUE / True
Return country count	<code>country.coverage</code>	<code>country_coverage</code>	TRUE / True
Return total population columns	<code>total.population</code>	<code>total_population</code>	FALSE / False
Return number affected	<code>number.affected</code>	<code>number_affected</code>	FALSE / False
Label for the global row	<code>global_label</code>	<code>global_label</code>	"WORLD"
Run input validation	<code>validate</code>	<code>validate</code>	TRUE / True

The R function additionally accepts `verbose` and `debug`, which inherit from `getOption("dw.verbose")` / `getOption("dw.debug")` when left NULL — the standard toolkit verbosity contract described in the roles-and-mode chapter. The Python aggregator does not expose those two arguments.

9.1.2 Methods

Pick the method to match the *kind* of indicator, because each method encodes a different assumption about what the numbers mean:

Method	Formula within group	Use for
<code>weighted_mean</code>	<code>weighted.mean(value, weight)</code>	rates / percentages weighted by population
<code>mean</code>	<code>mean(value)</code>	simple unweighted average
<code>sum</code>	<code>sum(value) / sum(weight) * 100</code> — not an additive total; see note below	a population-weighted <i>rate</i> expressed as a percent
<code>proportion</code>	<code>sum((value/100) * weight) / sum(weight_non_na) * 100</code>	indicators given as percentages, reweighted to a population proportion

Method	Formula within group	Use for
<code>total</code> (R only)	<code>sum(value)</code>	additive person-count stocks (refugees, displaced persons, births) where the regional value is the sum of country values

Two method names mislead. `sum` does **not** return a plain additive total; it returns a population-weighted rate, dividing the summed values by the summed weights. If you want a genuine additive count — the regional figure being the literal sum of the country figures — you want `total`, and `total` exists only in the R implementation. The Python aggregator’s valid set is (`"weighted_mean"`, `"mean"`, `"sum"`, `"proportion"`); passing `"total"` to it raises a `ValueError`. Until the Python sibling gains the method, person-count totals must be aggregated in R (or computed by hand).

`proportion` interprets the value column as a percentage (it divides by 100 internally) and reconstructs the number affected before re-expressing the result as a proportion of the covered population. It is the right choice when each country reports, say, “12.4 % of children stunted” and you want a population-correct regional percentage rather than an average of the country percentages.

9.1.3 Coverage and the threshold

The reason to prefer this aggregator over a hand-written group-and-summarise (R’s `group_by |> summarise`, or pandas’ `groupby().agg()`) is the coverage accounting. For every group, the function computes:

- **Pop_Covered** — the fraction of the group’s total weight that sits on rows with a non-missing value. A region whose reporting countries hold 90 % of its population has `Pop_Covered = 0.9`; the 10 % living in non-reporting countries show up in `Pop_Covered`, rather than vanishing into a falsely confident average.
- **Country_Coverage** and **Total_Countries** — the count of distinct countries contributing a non-missing value, and the count of distinct countries in the group. These come from `country_id` (default `REF_AREA`). If that column is absent, the function falls back to a row count and warns that the count may overstate coverage when a country has multiple rows.
- **Total_Population** and **Data_Population** — the summed weight over all rows and over non-missing rows respectively (returned only when `total.population / total_population` is on).
- **Number_Affected** — the reconstructed count, returned only when `number.affected` is on *and* `method == "proportion"`.

`coverage_threshold` turns `Pop_Covered` into a gate: any group below the threshold has its value suppressed. Set it to a fraction in `[0, 1]`, and the aggregator blanks `Aggregate` to NA for any group that falls below the threshold — but it still returns the coverage columns,

so the reader can see *why* the value was suppressed. The common UNICEF convention, and the value the error messages suggest, is 0.5 (report a regional figure only when reporting countries cover at least half the population). Passing a value outside `[0, 1]` is a validation error.

The metadata columns are opt-out for the cheap ones (`pop.coverage`, `country.coverage` default on) and opt-in for the rest. The output always begins with the `by` columns and `Aggregate`, then appends whichever metadata columns you requested:

```
# Output column order, given default flags:
#   <by...>, Aggregate, Pop_Covered, Country_Coverage, Total_Countries
```

9.1.4 The global row

With `global = TRUE` (the default), the same calculation is run once more over the entire input, ignoring the grouping, and the result is appended as one extra row whose `by` columns are all set to `global_label` (default "WORLD"). This is convenient for charts that want a “world” reference line without a second call. Set `global = FALSE` when you are stacking aggregates yourself — which is exactly what `dw_regions()` does internally, so it never asks the aggregator for a global row.

9.1.5 Validation and the error envelope

When `validate` is on (the default), the function checks that `value`, `weight`, and every `by` column exist, and on failure emits the toolkit’s standard error envelope — a structured message carrying the offending columns and a one-line fix hint (`[cso_toolkit.aggregate_data_v2]` ...). This is the same envelope shape described in the provenance-and-error-envelope chapter, and it is identical across R and Python down to the wording. Set `validate = FALSE` to skip the checks on hot paths where you have already verified the schema (again, `dw_regions()` does this and re-implements the checks itself so callers still get a clean envelope).

9.2 Building the standard footnote: `generate_agg_footnote`

Coverage numbers are only useful if they reach the reader. `generate_agg_footnote()` turns the coverage outputs into the canonical one-line caption that sits under aggregated tables and charts: "N/M countries (P % population coverage)", with an optional "Based on latest estimates from YYYY to YYYY." prefix and optional `Exemptions:` / `Exclusions:` suffixes. It ships in both R and Python.

9.2.0.1 R

```
note <- generate_agg_footnote(  
  country_coverage = 41,  
  total_countries  = 48,  
  pop_coverage     = 0.87,  
  start_year = 2015, end_year = 2023  
)  
# "Based on latest estimates from 2015 to 2023. 41/48 countries (87 % population coverage)"
```

9.2.0.2 Python

```
note = generate_agg_footnote(  
    country_coverage=41,  
    total_countries=48,  
    pop_coverage=0.87,  
    start_year=2015, end_year=2023,  
)
```

9.2.0.3 Stata

```
* No Stata sibling. Compose the footnote string by hand.
```

`pop_coverage` is a fraction in `[0, 1]` and is rendered as an integer percent. The `exemptions` and `exclusions` arguments take vectors of country codes and append human-readable notes — the same codes you would pass to `apply_time_window()`, described next, so a window-filtered aggregate and its footnote stay consistent.

9.3 Picking the latest observation: `apply_time_window`

Most regional aggregates are “latest available estimate per country within a window.” `apply_time_window()` performs that selection. Within each country it keeps the single row with the most recent `time_col`, but only if that latest year falls inside the inclusive `[start_year, end_year]` window. The helper ships in both R and Python.

9.3.0.1 R

```
latest <- apply_time_window(
  data,
  country_col = "REF_AREA",
  time_col    = "TIME_PERIOD",
  start_year  = 2015,
  end_year    = 2023,
  exemptions  = c("SOM", "YEM") # fragile states: keep latest regardless
)
```

9.3.0.2 Python

```
latest = apply_time_window(
    data,
    country_col="REF_AREA",
    time_col="TIME_PERIOD",
    start_year=2015,
    end_year=2023,
    exemptions=["SOM", "YEM"],
)
```

9.3.0.3 Stata

```
* No Stata sibling. Replicate with bysort + keep on the max year.
```

Two override lists tune the selection. Countries listed in `exemptions` keep their latest observation *even when it predates the window* — the standard treatment for fragile states whose most recent survey is old but still the best available. Countries listed in `exclusions` are dropped entirely before ranking, so they never reach the aggregate. The result is one row per surviving country (or none, for a country that satisfied neither the window nor an exemption), with the internal ranking columns stripped from the output. Feeding the same `exemptions` / `exclusions` into `generate_agg_footnote()` keeps the caption honest about which countries were treated specially.

i Note

The Python signature makes every argument after `data` keyword-only, so `start_year` and `end_year` must be passed by name. They have no defaults in either language — omitting them is an error.

9.4 Redistributing weights over missing rows: `dw_nestweight`

When a survey value is missing on some rows and that missingness is not completely at random within a stratum, a naive weighted mean over the non-missing rows under-counts. `dw_nestweight()` corrects this by reweighting: within each stratum (a group defined by the `by` argument) it scales the weights on the *observed* rows up so their sum equals the sum of weights over *all* rows in the stratum. The stratum total is preserved; the mass is concentrated on the rows that actually contribute to the estimate. It ships in **R** and **Python**.

The scale factor for stratum l is the ratio of total stratum weight to eligible stratum weight, and the new weight is the original weight times that scale on observed rows, zero elsewhere:

$$\text{scale}_l = \frac{\sum_{i \in l} w_i}{\sum_{i \in l, v_i \text{ observed}} w_i}, \quad w'_i = w_i \cdot \text{scale}_l \text{ if } v_i \text{ observed, else } 0.$$

(This ports the World Bank EduAnalyticsToolkit's `edukit_nestweight`, by Diana Goldemberg.)

9.4.0.1 R

```
adj <- dw_nestweight(  
  data,  
  value = "stunting",  
  by     = "stratum_id",  
  weight = "hh_weight",  
  new_weight = "hh_weight_adj"  
)  
# adj now has hh_weight_adj; sum(hh_weight_adj) == sum(hh_weight) per stratum
```

9.4.0.2 Python

```
adj = dw_nestweight(  
    data,  
    value="stunting",  
    by="stratum_id",  
    weight="hh_weight",  
    new_weight="hh_weight_adj",  
)
```

9.4.0.3 Stata

```
* No Stata sibling. The original lives in the World Bank
* EduAnalyticsToolkit as `nestweight` / `edukit_nestweight`.
```

The function returns the input data frame with one new column, `new_weight` (default `"weight_adj"`), appended; it does not aggregate. That redistributed weight then feeds `aggregate_data_v2(weight = "hh_weight_adj", method = "weighted_mean")`, which produces the actual estimate.

A few behavioral details, identical across the two implementations:

- `weight = NULL` (R) / `weight=None` (Python) uses an implicit unit weight, so the helper degenerates to per-stratum non-missing counts.
- `only` takes a logical mask of length `nrow(data)` restricting which rows are eligible for the denominator — useful for “only redistribute over respondents who were actually asked the question.”
- Rows with an NA stratum get weight 0 and trigger a warning.
- `verbose` reports the pooled mean of `value` under the original versus redistributed weights, but only when `value` is numeric; for a character or factor value it prints a note and skips the diagnostic rather than erroring.

Note one small interface difference: the R signature carries both `verbose` and `debug` (inheriting from the global options), while the Python signature exposes `verbose` only.

9.5 Demographics helpers: `dw_pop` and `dw_regions`

The remaining two helpers wrap the demographics-and-regions workflow. `dw_pop` now ships in **R** and **Python** (`python/src/dw_pop.py`); `dw_regions` is still **R-only**. Each file carries a TODO marking the siblings that remain: a Stata port for `dw_pop` (`stata/src/dw_pop.ado`, issue #17) and both a Python and a Stata port for `dw_regions` (`python/src/dw_regions.py`, `stata/src/dw_regions.ado`, issue #18). There is no Stata equivalent for either today, and Stata has no API wrapper to build on — see the external-API contract chapter for why the Stata side stops at file IO.

9.5.1 `dw_pop` — population denominators

`dw_pop()` is a thin convenience wrapper over `dw_api_fetch(api = "wb")` for the World Bank total-population indicator `SP.POP.TOTL`. Every sector that wants a population-weighted regional mean needs a denominator; `dw_pop()` hides the indicator code, the cache key, and the work of normalising column names, returning a clean three-column table: `REF_AREA`, `TIME_PERIOD`, `OBS_VALUE`.

9.5.1.1 R

```
# All countries, latest year per country
pop <- dw_pop()

# All countries, a specific year
pop_2023 <- dw_pop(year = 2023)

# A specific country list
pop_sahel <- dw_pop(countries = c("BFA", "MLI", "NER", "TCD"))
```

9.5.1.2 Python

```
from cso_toolkit import dw_pop

# All countries, latest year per country
pop = dw_pop()

# All countries, a specific year
pop_2023 = dw_pop(year=2023)

# A specific country list
pop_sahel = dw_pop(countries=["BFA", "MLI", "NER", "TCD"])
```

Arguments: `year` (a year or list of years; `None` returns the latest year per country), `indicator` (default "SP.POP.TOTL"), `countries` (ISO3/M49 filter; `None` keeps all), `refresh` (force a live fetch and overwrite the cache, producer mode only), and `cache_key` (the cache basename forwarded to `dw_api_fetch`, default "wb_population_sp_pop_totl"). The function tolerates the fact that the World Bank client packages have changed their response columns across versions — it probes for `iso3c/iso2c`, `date/TIME_PERIOD`, and `value/OBS_VALUE` columns — and drops rows missing any of the key triplet so sparse regional-aggregate rows from the World Bank do not leak through. The R and Python siblings share the same argument names, defaults, and cache key, so a producer can populate the cache in one language and a reviewer read it back in the other.

Because it routes through `dw_api_fetch`, `dw_pop` inherits the mode contract exactly: a producer session may hit the network and cache the result, while a reviewer session reads the cached deposit only and never fetches live. The mode mechanics live in the roles-and-mode chapter; the cache layer in the external-API contract chapter.

9.5.2 `dw_regions` — UNICEF regional aggregates

`dw_regions()` is the highest-level helper in this chapter: hand it a country-level tibble and it returns that tibble with UNICEF regional rows appended — one summary row per

UNICEF reporting region (for example ESA, WCA, SA), each carrying the population-weighted aggregate and its coverage metadata. It ties together everything above.

```
# Country-level tibble with INDICATOR + TIME_PERIOD + OBS_VALUE
national <- dw_use(name = "dw_ed_edu.csv", sector = "ed", kind = "wrk")

# Add population-weighted UNICEF region rows
enriched <- dw_regions(national, value = "OBS_VALUE")

# Or weight by a sector-specific column already in the data
alt <- dw_regions(national, value = "OBS_VALUE", weight = "births_under5")
```

It runs a five-step pipeline:

1. **Resolve the country-to-region map** by fetching "<taxonomy>.csv" from the public `unicef-drp/Country-and-Region-Metadata` repo via `dw_api_fetch(api = "github_raw")`. The default taxonomy is `UNICEF_REP_REG_GLOBAL`; pass `refresh_metadata = TRUE` to force a re-fetch. The map columns are probed against common spellings (`REF_AREA/ISO3/iso3c/country_code` for the country, `REGION/region/REGION_CODE/UNICEF_REGION` for the region), and rows with a blank or missing region are dropped.
2. **Attach the weight.** When `weight = "population"` (the default), `dw_pop()` supplies World Bank denominators. It merges them on `REF_AREA + TIME_PERIOD` when the data has a `TIME_PERIOD` column, and on `REF_AREA` alone otherwise. (When the World Bank has not yet published a given year, the merge falls back to each country's latest available population.) Any other string is treated as the name of a weight column that must already exist in data.
3. **Join data to the region map**, warning about — and excluding — any country code absent from the taxonomy rather than silently dropping it.
4. **Aggregate per region** by calling `aggregate_data_v2(by = c("REGION", by), global = FALSE, validate = FALSE, method = method)`. It passes `global = FALSE` because it stacks the rows itself, and `validate = FALSE` because it has already validated the schema up front.
5. **Concatenate.** The regional rows are reshaped to be schema-compatible with the input (`REGION` renamed to `REF_AREA`, the `Aggregate` column renamed to the caller's `value`, the merged `.pop` weight dropped) and `bind_rows`-ed onto the original country rows. Country rows keep their original columns; regional rows additionally carry the coverage metadata (`Pop_Covered`, `Country_Coverage`, ...), NA-filled on the country rows.

Arguments: `data`, `value`, `weight` (default `"population"`), `method` (forwarded to the aggregator, default `"weighted_mean"`), `taxonomy` (CSV basename, default `"UNICEF_REP_REG_GLOBAL"`), `by` (grouping columns surviving inside each region, default `c("INDICATOR", "TIME_PERIOD")`), and `refresh_metadata`.

`dw_regions` adds two guards beyond what the aggregator does. It validates up front that `REF_AREA`, `value`, and every `by` column are present (re-emitting the toolkit envelope,

since it disables the aggregator's own validation for speed), and it refuses a `value` argument that collides with one of the aggregator's reserved output column names (`Aggregate`, `Pop_Covered`, `Country_Coverage`, `Total_Countries`, `Total_Population`, `Data_Population`, `Number_Affected`) — a collision that would otherwise surface as an obscure aggregator error. If you hit it, rename the `value` column on your tibble before calling.

9.6 Putting it together

The helpers are designed to chain. A typical sector regional-aggregate pipeline, in R, reads:

```
latest <- apply_time_window(national, start_year = 2015, end_year = 2023,
                             exemptions = c("SOM", "YEM"))
adj     <- dw_nestweight(latest, value = "OBS_VALUE", by = "stratum_id",
                         weight = "hh_weight")
out     <- dw_regions(adj, value = "OBS_VALUE", weight = "hh_weight_adj")
note    <- generate_agg_footnote(country_coverage = 41, total_countries = 48,
                                pop_coverage = 0.87,
                                start_year = 2015, end_year = 2023,
                                exemptions = c("SOM", "YEM"))
```

A Python pipeline can run the first two steps and the footnote identically (`apply_time_window`, `dw_nestweight`, `generate_agg_footnote` all ship in Python), fetch population denominators with `dw_pop`, then call `aggregate_data_v2` directly for the grouped aggregate — but it cannot yet call `dw_regions`, and it cannot use `method = "total"`. A Stata pipeline has none of these helpers and must perform aggregation with native Stata commands, treating this contract as R/Python-only for now. The function reference and coverage matrix appendices give the authoritative per-language inventory.

10 Vintage sync

Every consuming repository carries its own frozen copy of the `cso-toolkit`'s helper code. That is the central design decision documented in the vendoring chapter: helpers are *copied* into a repo's `00_functions/` directory and tracked in that repo's git history, never `source()`'d or `pip install`'d live from the network. The benefit is reproducibility — a reviewer with no network access re-runs the exact code that was in effect at any commit. The cost is drift: the moment the upstream toolkit cuts a new release, every vendored copy is silently behind, and nothing forces a refresh.

Vintage sync is the machinery that makes that drift visible and controllable. It answers three questions, in order: **Am I behind?** (`cso_toolkit_check`), **What changed?** (`cso_toolkit_diff`), and **Refresh me to a known tag** (`cso_toolkit_pull`). The pin that anchors all three is `.toolkit_manifest.yml`, a small YAML file vendored alongside the helpers it tracks. Some parts of this story are implemented today; others are deliberate stubs, flagged as such below.

i Note

As of v0.5.0, only `cso_toolkit_check` runs end to end. `cso_toolkit_diff` and `cso_toolkit_pull` are working stubs: they validate preconditions, respect the mode contract, and log what they will do, but no one has yet implemented the step that fetches files and overwrites the vendored copies. The pin and the check loop are real and in daily use; the automated refresh is the planned completion of the same design.

10.1 The pin: `.toolkit_manifest.yml`

The manifest is the single source of truth for which vintage a repo carries. It lives next to the vendored helpers — by convention in the consumer's `00_functions/` — and is committed to that repo's git history right next to the code it describes. To create one, a consumer copies the template that ships in the toolkit at `templates/.toolkit_manifest.yml`, fills in the version, and updates it on every re-pull.

```
source: "unicef-drp/cso-toolkit"
pulled_version: "v0.4.0"
pulled_date: "2026-05-26"
pulled_by: "<your-name>"
```

```
# Files vendored from cso-toolkit/r/R/ into consumer's 00_functions/
```

```

vendored:
- dw_io.R
- dw_api.R
- cso_toolkit_sync.R
# Optional - copy only the ones your pipeline uses:
# - aggregate_data.R
# - dw_nestweight.R
# - create_sector_script.R

# Local edits applied on top of vendored files. List paths here so that
# cso_toolkit_pull() skips them on update. (Empty by default.)
local_edits: []

```

The fields:

Field	Meaning
source	GitHub owner/repo slug of the upstream toolkit. The check helper queries this repo's releases.
pulled_version	The tag in effect at the last pull, e.g. "v0.4.0". This is the pinned vintage that cso_toolkit_check compares against upstream.
pulled_date	ISO-8601 date of the pull. Free-form — the helper does not parse it; it is a human breadcrumb only.
pulled_by	Who ran the pull. A breadcrumb, not used by any helper.
vendored	The list of files copied from the toolkit into this repo. The planned cso_toolkit_pull iterates this list.
local_edits	Files in vendored that have intentional local divergence. cso_toolkit_pull will skip these on update so a refresh does not clobber a deliberate patch.

Two values of **pulled_version** carry special meaning:

- A tag like "v0.4.0" is a normal pin. The check compares it against the upstream latest tag and flags a newer release.
- The substring **inrepo** (e.g. "0.0.0-inrepo-2026-05-24") marks helpers that were edited locally and should never trigger a sync warning. **cso_toolkit_check** treats any **pulled_version** containing **inrepo** as “do not nag me” and reports no updates available regardless of upstream state. This is the escape hatch for the case where

the toolkit’s own scaffolding lived inside the consuming repo before the upstream repo existed.

Warning

The helpers compute manifest hashes against working-tree content, so line endings matter. The toolkit pins LF endings via `r/.gitattributes` for `*.R`, `*.yaml`, `*.md`, and friends precisely so that a vendored file’s sha256 is identical on Windows, Linux, and macOS. Without this, a Windows checkout’s CRLF line endings would make every file appear “changed” to a future hash-based diff — the bug fixed in issue #52.

10.2 `cso_toolkit_check` — am I behind?

`cso_toolkit_check` is the only one of the three helpers that runs end to end today. It is designed to be called quietly at session start: by default it returns a result structure and prints nothing, so a profile can wire it into startup without spamming the console. Pass `quiet = FALSE` (R) / `quiet=False` (Python) to log a human-readable verdict.

10.2.0.1 R

```
# Silent: returns a list, prints nothing
res <- cso_toolkit_check()

# Loud: logs the verdict to the console
res <- cso_toolkit_check(quiet = FALSE)
if (!is.null(res) && isTRUE(res$updates_available)) {
  cso_toolkit_diff()
}
```

10.2.0.2 Python

```
from cso_toolkit import cso_toolkit_check, cso_toolkit_diff

# Silent: returns a dict, logs nothing
res = cso_toolkit_check()

# Loud: logs the verdict
res = cso_toolkit_check(quiet=False)
if res is not None and res["updates_available"]:
    cso_toolkit_diff()
```


10.2.0.3 Stata

```
* Not available. Stata has no cso_toolkit_check / diff / pull helpers.  
* Pin the vintage manually in .toolkit_manifest.yml and refresh by  
* copying stata/src/ from a tagged checkout. See the gap note below.
```

The R and Python implementations are deliberate ports of one another. Both return the same structure with the same keys:

Key	Meaning
source	The upstream slug from the manifest.
pinned_version	The manifest's pulled_version.
upstream_version	The latest release tag found upstream.
updates_available	TRUE/True when pinned and upstream differ and the pin is not an inrepo pin.
updated_files	Always NULL/None here — reserved to be populated by <code>cso_toolkit_diff</code> .

10.2.1 What the check guards against

The order of these guards matters. The function returns NULL/None — silently, unless `quiet` is off — in any of the following cases:

1. **Reviewer mode.** This is the very first check. Reviewer mode — the offline, no-network role defined in the roles and mode-contract chapter — forbids external network calls, and a GitHub release lookup is exactly such a call. If `dw_apis_allowed` is not TRUE, the function returns immediately. Reviewers therefore never see sync chatter; they run against whatever vintage the producer pinned.
2. **Missing or unreadable manifest.** If the `yaml/PyYAML` package is absent, or no `.toolkit_manifest.yml` exists next to the helpers, the helper warns and bails. Helpers that are not vendored from any upstream simply have nothing to check.
3. **Manifest with no source.** A manifest that does not name an upstream repo cannot be checked.
4. **Upstream unreachable or has no releases.** The release lookup is wrapped so that a blocked proxy, an absent repo, or a repo with zero releases all resolve to “no result” rather than an error.

Only when all four guards pass does the function compare versions. The comparison is intentionally crude: both the pinned and upstream tags are stripped of a leading `v` and compared as strings for inequality. There is no semantic-version ordering — a difference is a difference. This is enough for the signal the helper exists to give: you are not on the latest tag. It does not distinguish newer from older, and is not meant to.

When an update *is* available and `quiet` is off, the producer sees a yellow alert (an ANSI escape in R; a log line in Python) naming the pinned and upstream versions and suggesting the next two commands:

```
[cso-toolkit] unicef-drp/cso-toolkit: pinned v0.4.0 -> upstream v0.5.0 (newer tag available)
               Run cso_toolkit_diff() to see what changed.
               Run cso_toolkit_pull('v0.5.0') to refresh.
```

One rendering quirk distinguishes the two ports. The format string is `pinned v%s` in both, but the R port interpolates the raw `pulled_version` while the Python port first strips a leading `v` (`pinned_norm`, a Copilot fix on PR #7). Because the shipped manifest pins `pulled_version: "v0.4.0"`, the R alert actually reads `pinned vv0.4.0` whereas Python reads `pinned v0.4.0`. The text above shows the Python form; expect a doubled `v` from R when the pin carries the `v` prefix. The comparison logic is unaffected — both strip the `v` before testing for difference.

10.2.2 How the upstream tag is resolved

Both ports resolve the latest tag through the same two-tier strategy, in an internal helper named `.cso_upstream_latest_tag` (R) / `_cso_upstream_latest_tag` (Python):

1. **Prefer the `gh` CLI** when it is on `PATH`: `gh api repos/<source>/releases/latest --jq .tag_name`. Using `gh` avoids an auth flicker when `gh auth status` is already good.
2. **Fall back to a raw GitHub API call** — `http::GET` in R, `requests.get` in Python — against `https://api.github.com/repos/<source>/releases/latest`, reading `tag_name` from the JSON body on an HTTP 200.

If neither path yields a tag (no `gh`, no HTTP client, blocked network, or a repo with no releases), the resolver returns `NULL/None`, and the check degrades gracefully to “cannot determine — stay silent”.

10.3 `cso_toolkit_diff` — what changed?

`cso_toolkit_diff` is the bridge step: once the check tells you a newer tag exists, the diff is meant to show you which vendored files actually differ before you overwrite anything. Its signature is implemented and it respects the manifest, but the comparison body is a stub.

10.3.0.1 R

```
# Diff against the latest upstream tag
cso_toolkit_diff()

# Diff against a specific tag
cso_toolkit_diff(target_version = "v0.5.0")
```

10.3.0.2 Python

```
from cso_toolkit import cso_toolkit_diff

cso_toolkit_diff()
cso_toolkit_diff(target_version="v0.5.0")
```

10.3.0.3 Stata

```
* Not available - no Stata diff helper.
```

Today, both ports load the manifest, return early if it is missing, and then log:

```
cso_toolkit_diff: not yet implemented.
  Upstream repo (unicef-drp/cso-toolkit) needs to exist before diff is meaningful.
  Once it does, this will fetch each file at the target tag
  and compare sha256 against the vendored copy.
```

The planned behavior is to fetch each upstream file at `target_version` (defaulting to the latest tag) and compare it by sha256 against the vendored copy, with an optional textual diff. That comparison is the mechanism the manifest's LF-pinning was built to support: a stable working-tree hash on every platform.

10.4 cso_toolkit_pull — refresh to a tag

`cso_toolkit_pull` is the action step. It is the most heavily guarded of the three because it is the one that would *write* — overwriting vendored code and rewriting the manifest. The guards are real and enforced today; only the final step — fetching files and overwriting the vendored copies — is stubbed.

10.4.0.1 R

```
# Refresh to a specific tag, prompting per file
cso_toolkit_pull("v0.5.0")

# Preview without writing
cso_toolkit_pull("v0.5.0", dry_run = TRUE)

# Refresh without per-file prompts
cso_toolkit_pull("v0.5.0", confirm = FALSE)
```

10.4.0.2 Python

```
from cso_toolkit import cso_toolkit_pull

cso_toolkit_pull("v0.5.0")
cso_toolkit_pull("v0.5.0", dry_run=True)
cso_toolkit_pull("v0.5.0", confirm=False)
```

10.4.0.3 Stata

```
* Not available - no Stata pull helper. Refresh by copying the tagged
* stata/src/ contents into the consumer's adopath and editing the
* manifest by hand.
```

The arguments are parallel across the two implemented ports:

Argument	Default	Meaning
<code>target_version</code>	(required)	The tag to pull, e.g. "v0.5.0".
<code>confirm</code>	TRUE / True	Prompt per file before overwriting.
<code>dry_run</code>	FALSE / False	Show what would change without writing.

The guard sequence, in order, is the spine of the function:

1. **Reviewer mode is a hard stop, not a silent skip.** Unlike `cso_toolkit_check`, which quietly returns NULL for reviewers, `cso_toolkit_pull` raises. A pull requires hitting GitHub, which the reviewer-mode contract forbids, and the error explains the fix — switch the profile to producer mode (`dw_mode: producer` in `user_config.yml` for R; `_state.configure(dw_mode="producer", dw_apis_allowed=True)` in Python). A reviewer who wants a different vintage must do it in a separate producer-mode session, never by silently mutating the pin.

2. **No manifest -> error.** The function names the path it looked under and points at the template schema.
3. **Manifest with no source -> error.** It cannot pull from an unnamed upstream.
4. **Upstream unreachable -> error.** Resolved via the same `.cso_upstream_latest_tag` helper as the check. The error enumerates the likely causes — repo does not exist yet, a UNICEF corporate proxy is blocking `api.github.com`, or no `gh/HTTP` client is installed — and the fix.

These four errors all follow the toolkit’s error-envelope convention with a `[cso_toolkit.cso_toolkit_pull]` prefix, a reason, and a fix (see the error-taxonomy appendix and the provenance-and-errors chapter). Only after all four pass does the stub log its “implementation TBD” message naming the target version and `dry_run` setting.

The planned implementation, documented in the stub’s own roxygen and docstring, runs as follows:

1. Read the manifest.
2. For each file in `vendored`, fetch it from `source` at `target_version`, then compute its sha256 against the current vendored copy.
3. On a difference, prompt the user (overwrite / skip / show-diff), honoring `confirm` and `dry_run`, and skip anything listed under `local_edits`.
4. Rewrite the manifest with the new `pulled_version` and per-file hashes.
5. Log a summary.

10.5 The refresh flow

The three helpers compose into one producer-driven loop. The toolkit strategy document lays it out as: the producer runs `cso_toolkit_check` at session start; if a newer tag is detected, a yellow alert names the pinned and upstream versions, points at `NEWS.md`, and suggests a pull; the producer reads the release notes, decides whether the new vintage is appropriate, and either runs `cso_toolkit_pull("<tag>")` or does nothing and keeps the pin.

```
cso_toolkit_check()      -> "pinned v0.4.0 -> upstream v0.5.0 (newer tag available)"
    |                    (silent if up to date, in reviewer mode, or inrepo-pinned)
    v
cso_toolkit_diff()       -> per-file sha256 comparison (planned) - what actually changed
    |
    v
cso_toolkit_pull("v0.5.0") -> fetch, prompt per file, rewrite manifest (planned)
```

Three properties make this loop safe by design:

- **It is producer-only.** Every step is gated on `dw_apis_allowed`. Reviewers never poll GitHub and never see the alert; they run the pinned vintage. This is what keeps reviewer reproducibility offline-capable.

- **Updates are never automatic.** Nothing pulls without an explicit `cso_toolkit_pull` call. The producer reviews `NEWS.md`, judges fitness, and chooses. A repo can sit two releases behind indefinitely, and that is a valid, deliberate state.
- **The pin is the audit trail.** Because `.toolkit_manifest.yml` is committed alongside the vendored code, the consuming repo's git history shows exactly which toolkit vintage was in effect at every commit. There is no gap between what the pin records and what the code actually is.

10.5.1 Asserting a minimum vintage in a script

A complementary, always-available check is `dw_toolkit_version` (defined in `dw_io.R` / `dw_io.py`, new in v0.4.0). It returns the tag the *currently vendored* helper triplet was lifted from, so a sector script can assert a floor before relying on a feature that only exists in a recent contract. Unlike the sync helpers, this touches no network and works in every mode.

Warning

The R and Python stamps are meant to track each other but can drift between releases — in earlier releases the R `dw_toolkit_version()` returned "0.5.0" while the Python port returned "0.4.0"; they are aligned at "0.6.0" in the version this handbook documents. The stamp is a hard-coded constant in each port (`_TOOLKIT_VERSION` in Python), so a release that bumps only one side can still leave the other behind. Treat the value as a floor for the language you are calling it from, not as a cross-language equality check.

10.5.1.1 R

```
if (utils::compareVersion(dw_toolkit_version(), "0.4.0") < 0) {
  stop("This script requires cso-toolkit >= 0.4.0; ",
       "found ", dw_toolkit_version(), ". Run cso_toolkit_pull('v0.4.0').")
}
```

10.5.1.2 Python

```
from cso_toolkit import dw_toolkit_version

# dw_toolkit_version() returns a semver string (e.g. "0.4.0").
# No compareVersion built-in; split and compare tuples, or assert equality.
if tuple(map(int, dw_toolkit_version().split("."))) < (0, 4, 0):
    raise RuntimeError(
        f"This script requires cso-toolkit >= 0.4.0; "
```

```
f"found {dw_toolkit_version()})."
)
```

10.5.1.3 Stata

```
* No dw_toolkit_version equivalent in Stata. Each .ado carries its own
* per-file `version` header (e.g. dw_save.ado: "!! version 1.1 26MAY2026").
* The header stamps a per-file version + date, not the toolkit tag.
* Record the pinned toolkit tag by hand in .toolkit_manifest.yml.
```

10.6 Cross-language coverage, honestly

Cross-language parity is weakest here, and the page would mislead if it implied otherwise:

Capability	R	Python	Stata
<code>.toolkit_manifest.yml</code> pin	yes (read)	yes (read)	referenced in README, hand-edited only
<code>cso_toolkit_check</code>	yes (full)	yes (full, ported)	none
<code>cso_toolkit_diff</code>	yes (stub)	yes (stub)	none
<code>cso_toolkit_pull</code>	yes (stub)	yes (stub)	none
<code>dw_toolkit_version</code>	yes ("0.6.0")	yes ("0.6.0")	none (per- <code>.ado</code> version headers)

The R and Python ports of the sync helpers are genuine mirrors of each other: same function names, same arguments, same return structure, same guard order, same two-tier `gh`-then-HTTP tag resolver, even the same `inrepo` escape hatch. Both also expose `dw_toolkit_version`, though the two stamps can fall out of step across a release, as the callout above warns.

Stata has **no sync helpers at all**. Its `stata/src/` directory ships the IO and config helpers (`dw_save`, `dw_use`, `dw_compare`, `dw_load_config`, `dw_mkdir`, `dw_require_no_api`) but nothing named `cso_toolkit_check` / `_diff` / `_pull`, and no `dw_toolkit_version` — each `.ado` instead carries its own per-file `version` header. The Stata README acknowledges the manifest only as a place to *record* the pinned version by hand; refreshing a Stata consumer means copying `stata/src/` from a tagged checkout into the project's adopath and editing `.toolkit_manifest.yml` manually. There is no automated drift detection on the Stata side. This is consistent with the broader Stata coverage gaps noted in the cross-language coverage appendix (no API wrapper, no `parquet`). When you adopt the toolkit in a Stata-only repo, treat vintage management as a manual, documented step rather than a toolled one.

11 Scaffolding and the contract auditor

A contract is only as good as a team’s willingness to honour it, and two costs erode that willingness over time. The first is the cost of *adoption*: if wiring a new repository into the toolkit means hand-copying a profile, remembering the sentinel convention, and recreating the producer/reviewer plumbing from scratch, people will cut corners and the contract will fray at the edges. The second is the cost of *enforcement*: once a repository is wired, nothing stops a contributor from quietly writing `read_csv()` or `http::GET()` in a new script, bypassing the IO and external-API contracts the rest of the codebase observes.

This chapter covers four helpers that attack those two costs. Three are *scaffolders* — `create_profile`, `create_sector_script`, and `review_profile` — that generate and verify the boilerplate a contract-compliant repository needs. The fourth is the *contract auditor*, `test_scripts`, which reads a directory of source files and flags every raw IO or HTTP call that should have gone through a toolkit wrapper. Together they make the right thing the easy thing on the way in, and make the wrong thing visible on the way out.

One coverage caveat. The scaffolders and the auditor are implemented in R and Python only. Stata has no `create_profile`, no `create_sector_script`, no `review_profile`, and no `test_scripts`. The Stata side of the toolkit ships the runtime helpers (`dw_save`, `dw_use`, `dw_compare`, `dw_require_no_api`, `dw_load_config`) but leaves profile setup and contract auditing to convention rather than code. This is an honest gap, not an oversight to gloss over; where a Stata equivalent is missing, the tabsets below say so plainly. Throughout, the worked examples lean on DW-Production (the UNICEF Data Warehouse production repository, the toolkit’s reference deployment), because that is the codebase these helpers were lifted from and generalised away from.

11.1 The scaffolding model

The three scaffolders form a small dependency chain that mirrors how a repository is actually built:

1. `create_profile()` writes a `profile_<repo>.R` (or `.py`) — the script every session sources first. It establishes the user’s identity, loads configuration, resolves the producer/reviewer mode, loads packages, and finally sets a **sentinel**: a single object whose existence proves the whole profile ran to completion.
2. `create_sector_script()` writes a `00_run_<sector>.R` (or `.py`) — a per-sector pipeline driver that *refuses to run* unless that sentinel is present, then wraps the sector’s work in logging and error handling.

3. `review_profile()` audits a profile after the fact, reporting `pass` / `warn` / `fail` for each block the contract depends on — useful for profiles that predate the scaffolder, or that someone hand-edited.

The sentinel is the linchpin. `create_profile()` ends by setting `profile_<repo> <- TRUE`, and `create_sector_script()` generates code that checks for the profile being loaded. The two helpers therefore share one convention: if the profile loaded, the whole profile (config, mode, packages) ran to completion. The roles, the mode contract, and the sentinel are described in the roles-and-mode-contract chapter; here we look at how the scaffolders produce them.

11.2 Generating a profile: `create_profile`

`create_profile()` is a code generator, not a runtime helper — it writes a template to disk and returns the path. The generated profile is patterned on DW-Production's `profile_DW-Production.R`: it reads the username under whichever convention the OS uses (Windows, Mac, or Linux), loads a YAML user config, resolves `dw_mode`, loads packages, and sets the sentinel.

11.2.0.1 R

```
# Generic project:
create_profile("My-Sector-Project", project_title = "My Sector Project")

# DW-Production, with the Z: drive advisory and overwrite:
create_profile("DW-Production",
               project_title      = "UNICEF DW Production",
               include_z_drive_check = TRUE,
               overwrite          = TRUE)
```

11.2.0.2 Python

```
# Generic project:
create_profile("My-Sector-Project", project_title="My Sector Project")

# DW-Production, with the Z: drive advisory and overwrite:
create_profile("DW-Production",
               project_title="UNICEF DW Production",
               include_z_drive_check=True,
               overwrite=True)
```

11.2.0.3 Stata

```
* No equivalent. Stata has no create_profile scaffolder.  
* Author the profile do-file by hand and call dw_load_config inside it  
* to read ~/.config/user_config.yml and set $dw_mode + team globals.
```

The arguments are the same across R and Python:

Argument	Default	Purpose
repo_name	(required)	Repository folder name. Drives the sentinel name (profile_<repo>, with - and . mapped to _) and the filename (profile_<repo>.R / .py).
project_title	repo_name	Human-readable title for the header block.
output_path	"."	Directory the file is written to.
include_dw_mode	TRUE	Emit the producer/reviewer dw_mode block and the dw_require_no_api guard.
include_z_drive_check	FALSE	Emit the Z: drive availability advisory. The Z: drive is the legacy Azure file-share mirror of canonical deposits, so this block is DW-Production-specific.
author	system user	Author name in the header.
overwrite	FALSE	Stop if the file already exists, unless TRUE.

The generated profile opens with a **header** — title, filename, author, timestamp, and a pointer to the toolkit repository, followed by a reproducibility seed (`set.seed(12345)` / `random.seed(12345)`). After it come eight numbered blocks, assembled in order:

1. **User identification** — reads USERNAME / USER and USERPROFILE, falling back across Windows / Mac / Linux conventions.
2. **Flags and controls** — e.g. `create_missing_folders <- FALSE`.
3. **Repository name** — records the repo as a literal, e.g. `repo_name <- "DW-Production"`.
4. **Z: drive advisory** (only when `include_z_drive_check = TRUE`) — a *non-blocking* check; the absence of the legacy mirror prints a warning rather than stopping execution.

5. **Config load** — reads `~/.config/user_config.yml` via `yaml::read_yaml()` / `yaml.safe_load()`, hard-failing if the file is missing.
6. **Producer/reviewer mode** (only when `include_dw_mode = TRUE`) — reads `dw_mode` from the config, refuses any value other than "producer" or "reviewer", and defines the `dw_require_no_api()` guard that reviewer-mode analysis code calls to forbid network access.
7. **Packages / imports** — a placeholder block listing the pipeline's dependencies.
8. **Profile sentinel** — sets `profile_<repo> <- TRUE` (R) or `profile_<repo> = True` (Python) and prints a confirmation.

Block 6 hides a meaningful cross-language difference. The Python profile wires the resolved mode into the toolkit's shared state, so `dw_save` / `dw_use` / `dw_api_fetch` route by mode automatically — it imports `cso_toolkit._state` and calls `_cso_state.configure(dw_mode=..., dw_apis_allowed=(dw_mode == "producer"))`. The R profile does no such wiring; its generated `dw_require_no_api()` simply inspects `dw_mode` and `stop()`s in reviewer mode. There is a second asymmetry in block 8: the Python profile also defines a `log_message()` helper there, so that sector scripts have a logger to import. These differences reflect each language's idioms; the contract they implement is identical.

i Note

The YAML config load in block 5 is the one place a profile *must* read a file directly — it bootstraps the very configuration the toolkit needs before any toolkit helper is available. The auditor knows this: the generated Python profile annotates the line `yaml.safe_load(f)` # `cso-allow: io-yaml`, exempting it from the auditor's IO rule. See “The escape hatch” below.

11.3 Generating a sector driver: `create_sector_script`

Where `create_profile()` scaffolds the once-per-repository profile, `create_sector_script()` scaffolds a once-per-sector run script: `<base_dir>/<sector_code>/00_run_<sector_code>.R` (or `.py`). The template it writes is deliberately runnable out of the box — it verifies the profile, sets up logging and runtime tracking, builds input/output paths, and wraps the (placeholder) work in error handling.

11.3.0.1 R

```
# Generic - writes ./ws/00_run_ws.R relative to cwd:
create_sector_script("WASH", "ws")

# Into a chosen subtree:
create_sector_script("Nutrition", "nt", base_dir = "pipelines/sectors")
```

```
# DW-Production conventions wrapper (writes under 01_dw_prep/012_codes/):
create_dw_sector_script("WASH", "ws")
```

11.3.0.2 Python

```
# Generic - writes ./ws/00_run_ws.py relative to cwd:
create_sector_script("WASH", "ws")

# Into a chosen subtree:
create_sector_script("Nutrition", "nt", base_dir="pipelines/sectors")

# DW-Production conventions wrapper (writes under 01_dw_prep/012_codes/):
create_dw_sector_script("WASH", "ws")
```

11.3.0.3 Stata

```
* No equivalent. Stata has no create_sector_script scaffolder.
* Author 00_run_<sector>.do by hand; guard it on $dw_mode being set
* (dw_load_config must have run) before doing the sector's work.
```

The key arguments (R and Python share names and meaning, with one default that differs — flagged below):

Argument	Default (R)	Purpose
sector_name	(required)	Full name, e.g. "Nutrition". Used in headers and log lines.
sector_code	(required)	Short code, e.g. "nt". Drives the folder and filename.
base_dir	"."	Parent directory under which <sector_code>/00_run_<sector_code>.* is created.
profile_name	"projectFolder"	The variable the sector script asserts must exist before it will run. (Python defaults this to the sentinel "profile_DW_Production" instead — see below.)

Argument	Default (R)	Purpose
<code>profile_file</code>	<code>"profile_DW-Production.R"</code>	Profile the script tells the user to source on failure.
<code>input_subpath /</code> <code>output_subpath</code>	<code>c("01_dw_prep",</code> <code>"011_rawdata") /</code> <code>c("01_dw_prep",</code> <code>"013_wrkdata")</code>	Path components, relative to <code>projectFolder</code> , for the sector's input and output folders.
<code>overwrite</code>	<code>FALSE</code>	Stop if the target exists, unless <code>TRUE</code> .

The generated R template contains five sections:

- **0. Profile Verification** — `if (!exists(profile_name) || is.null(profile_name)) stop(...)`. The check is satisfied by any non-null value, so the sentinel can be a character, numeric, or logical. Note this default differs from the profile sentinel `profile_<repo>` introduced in the scaffolding-model section: the R template defaults `profile_name` to `"projectFolder"` rather than the sentinel, because `projectFolder` is the variable the template *also* uses to build paths — so the guard and the path construction share one source of truth. (The Python template makes the opposite choice; see the cross-language note below.)
- A **log_message fallback** — if the profile did not define one, the template installs a minimal timestamped logger so the script runs out of the box.
- An **errorOccurred flag** — initialised to `FALSE` so the `tryCatch` handler can set it with `<<-` without the caller pre-declaring the global. The top-level runner consumes this flag.
- **1. Logging** — emits a “Starting module” line and records `start_time`.
- **2. Sector Execution Block** — a `tryCatch({...}, error = ...)` wrapper that defines `input_folder / output_folder` from `projectFolder`, creates the output folder, leaves commented placeholders for load/process/export, and finishes by logging the duration. On error it sets `errorOccurred <<- TRUE` and logs the message.

The Python template covers the same five sections with idiomatic equivalents — a `try/except` block instead of `tryCatch`, `pathlib.Path` for path building. Verifying the profile takes more work in Python. A profile filename like `profile_DW-Production.py` contains a hyphen, so it is not a legal Python module name. The template therefore loads it via `importlib.util.spec_from_file_location` — walking up the ancestor directories to find it — then pulls the sentinel, `log_message`, and `projectFolder` from the loaded module. This is the Python answer to R's `source()`, and it is worth knowing about when a sector script cannot find its profile. Because the Python template reads the sentinel out of the loaded module, it defaults `profile_name` to the sentinel `"profile_DW_Production"` (note the underscores) rather than to `"projectFolder"`.

Warning

The default `input_subpath` / `output_subpath` differ between languages. R defaults to `011_rawdata` / `013_wrkdata`; the Python port defaults to `011_input` / `013_output`. If you scaffold the same sector in both languages, set the paths explicitly so the two drivers agree.

Both languages provide a `create_dw_sector_script()` — a thin wrapper that fills in the DW-Production layout: scripts land at `01_dw_prep/012_codes/<sector>/`, with inputs and outputs under the canonical sibling folders. Reach for the wrapper inside DW-Production; reach for the generic `create_sector_script()` everywhere else. (The R package additionally exports `dw_create_sector_script` as a `dw_`-prefixed alias of the generic helper — see “A note on naming”; the Python package does not.)

11.4 Auditing a profile: `review_profile`

`create_profile()` produces a profile that passes every check by construction. `review_profile()` is for the other case: a profile that someone wrote by hand, hand-edited, or inherited from before the scaffolder existed. It reads the file and runs eight presence checks, reporting `pass`, `warn`, or `fail` for each.

The audit is deliberately **pattern-based, not parse-based** — it greps for the load-bearing lines rather than parsing the script, so it tolerates the stylistic variation found across older profiles. It does not execute the profile.

11.4.0.1 R

```
review_profile("profile_DW-Production.R")

# Profile that does not touch dw_io/dw_api - downgrade dw_mode to a warning:
review_profile("profile_my-project.R", require_dw_mode = FALSE)
```

11.4.0.2 Python

```
review_profile("profile_DW-Production.py")

# Profile that does not touch dw_io/dw_api - downgrade dw_mode to a warning:
review_profile("profile_my-project.py", require_dw_mode=False)
```

11.4.0.3 Stata

* No equivalent. Stata has no `review_profile` auditor.

The eight checks, and the level each fails at:

#	Check	Fails at	What it looks for
1	Profile sentinel object	fail	<code>profile_<repo> <- TRUE / = True</code> — without it, sector scripts cannot verify the profile ran.
2	Cross-platform user identification	fail	<code>Sys.getenv("USERNAME"/"USER")</code> / <code>os.environ.get(...)</code> .
3	Reproducibility seed	warn	<code>set.seed(R);</code> <code>random.seed(/</code> <code>np.random.seed(</code> (Python).
4	<code>user_config.yml</code> load	fail	A reference to <code>user_config.yml</code> and a <code>yaml::read_yaml /</code> <code>yaml.safe_load</code> call.
5	<code>dw_mode</code> resolution	fail*	The strings <code>dw_mode</code> , <code>producer</code> , and <code>reviewer</code> all present.
6	<code>dw_require_no_api</code> guard	fail*	<code>dw_require_no_api</code> defined or invoked.
7	Z: drive advisory	warn†	<code>dw_z_available</code> or <code>network_root <- "Z: / network_root = "Z:.</code>
8	Packages / imports block	warn	<code>requireNamespace /</code> <code>install.packages /</code> <code>library(R);</code> an <code>import / from</code> statement (Python).

* Checks 5 and 6 drop from `fail` to `warn` when `require_dw_mode = FALSE` — appropriate for a profile that never touches the IO or API wrappers. † Check 7 is a `warn` by default and rises to `fail` only when `require_z_drive_check = TRUE`.

Both functions return a data frame (`check`, `status`, `detail`) — invisibly in R, directly in Python — and print a formatted summary with [OK] / [!] / [X] markers and a tally. Use the returned frame in CI to gate on `status == "fail"`.

11.5 The contract auditor: `test_scripts`

`test_scripts()` is the enforcement half of the chapter — the answer to the *second* cost named at the start. It recursively scans a directory of source files and flags every line that calls a raw file-IO or external-API function the toolkit is meant to wrap. The contract it enforces is exactly the one described in the IO-contract and external-API-contract chapters:

- **File IO** must go through `dw_save` / `dw_use` / `dw_compare` / `dw_merge` — never `read_csv()`, `write_xlsx()`, `saveRDS()`, `pd.read_csv`, `df.to_excel`, `pickle.dump`, and so on.
- **External APIs** must go through `dw_api_fetch` / `dw_api_cached` — never `httr::GET()`, `rsdmx::readSDMX()`, `wbstats::wb_data()`, `requests.get`, `sdmx.Client`, `wbgapi`, and so on.

11.5.0.1 R

```
# Audit a sector codebase:
test_scripts("01_dw_prep/012_codes/nt")

# CI mode - stop with a non-zero exit on any io/api violation:
test_scripts("01_dw_prep/012_codes", error_on_violation = TRUE)

# dw_-prefixed alias (identical behaviour):
dw_test_scripts("01_dw_prep/012_codes")
```

11.5.0.2 Python

```
# Audit a sector codebase:
test_scripts("01_dw_prep/012_codes/nt")

# CI mode - raise RuntimeError on any io/api violation:
test_scripts("01_dw_prep/012_codes", error_on_violation=True)
```

11.5.0.3 Stata


```
* No equivalent. Stata has no contract auditor.
* The Stata wrappers (dw_save / dw_use / dw_require_no_api) enforce the
* contract at the call site, but nothing scans .do files for raw imports.
```

11.5.1 How it works

For each matching file (default filename pattern `\.R$ / \.py$`), the auditor reads it line by line. Before matching, it scrubs string literals (so a URL inside a quoted string does not trigger a rule) and strips trailing comments (so commented-out code is not flagged) — while preserving any `# cso-allow:` directive on the line. It then tests each line against a registry of rules. A rule carries an `id`, a `family` (`"io"` or `"api"`), a `pattern`, a `message`, and a `suggestion`. The R patterns are written to catch both unqualified and namespaced calls — `read_csv()` and `readr::read_csv()` both match — using a negative lookbehind so that `my_read_csv()` does not.

Each violation becomes a row with `file`, `line`, `rule`, `family`, `message`, `suggest`, and `snippet`. The function returns the full data frame (invisibly in R) and, when `verbose`, prints a per-violation report:

```
[api-sdmx] 01_dw_prep/012_codes/nt/01_fetch.R:42
  Direct rsdmx::readSDMX call.
  > dat <- rsdmx::readSDMX(providerId = "UNICEF", ...)
  -> dw_api_fetch(api = 'sdmx', providerId = ..., flowRef = ..., key = ..., cache_key = ..
```

The auditor ignores its own implementation files by default (`dw_io`, `dw_api`, `cso_toolkit_sync`, `profile_helpers`, `test_scripts`, plus `_state.py` and `__init__.py` on the Python side) — those *must* call the wrapped commands — and skips noise directories (`.git`, `renv`, `packrat`, `.venv`, `__pycache__`, and so on). Both lists are overridable via `ignore_files / ignore_dirs`, and you can extend the registry with `custom_rules`.

11.5.2 What it flags

The R registry (in `test_scripts.R`) covers ten IO rules and six API rules:

family	rule id	Flags
io	io-read-csv / io-write-csv	read.csv/read_csv/read_tsv/read_delim and the write counterparts.
io	io-fread	data.table::fread / fwrite.
io	io-rds	saveRDS / readRDS.
io	io-load-save	save() / load() for .RData.

family	rule id	Flags
io	io-dta	read_dta / write_dta / read_stata / write_stata (haven).
io	io-xlsx	read_excel/read_xlsx/write_xlsx/read.x
io	io-parquet	read_parquet / write_parquet (arrow).
io	io-json-file	read_json / write_json (jsonlite).
io	io-yaml	read_yaml / write_yaml / yaml.load_file.
api	api-http	GET/POST/PUT/PATCH/DELETE/request (http / http2).
api	api-fromjson-url	fromJSON() called on a http(s):// / ftp:// URL string.
api	api-sdmx	rsdmx::readSDMX.
api	api-wbstats	wb_data / wb_indicators / wb_countries.
api	api-ilostat	get_ilostat (Rilostat).
api	api-download	download.file / curl::curl_download / curl::curl_fetch_* — bypass the cache.

The Python registry (in `test_scripts.py`) is the language-appropriate mirror: twelve IO rules covering `pd.read_csv / .to_csv`, `read_excel / .to_excel`, `read_stata / .to_stata`, `read_parquet / .to_parquet`, `pickle.dump/load`, `json.dump/load`, `yaml read/write`, and a heuristic `io-open-write` rule that flags `open(..., "w")`-style calls; plus seven API rules covering `requests`, `httpx`, `urllib.request.urlopen`, `sdmx.Client`, `wbgapi`, `wbdata`, and `urllib.request.urlretrieve`. The rule *ids* are intentionally close across languages (`io-yaml` exists in both, for instance) so a `# cso-allow:` directive reads the same regardless of language.

i Note

The auditor and the wrappers are two layers of the same defence. The wrappers enforce the contract *at the call site* — once you call `dw_save`, mode-routing and provenance happen whether you like it or not. The auditor enforces it *at review time* — it catches the calls that never reached a wrapper at all. The wrappers cannot see a `read_csv()` that bypasses them; the auditor exists precisely to surface it.

11.5.3 The escape hatch: # cso-allow:

The contract has principled exceptions — the YAML config bootstrap in a profile being the canonical one. For those, append a `# cso-allow: <rule-id>` comment to the offending line and the auditor will skip that one rule on that one line:

11.5.3.1 R

```
config <- yaml::read_yaml(config_path) # cso-allow: io-yaml
```

11.5.3.2 Python

```
with open(config_path, encoding="utf-8") as _f:
    user_config = yaml.safe_load(_f) # cso-allow: io-yaml
```

11.5.3.3 Stata

```
* No auditor, so no escape hatch. Document intentional raw IO in a comment.
```

Multiple rules can be silenced on one line as a comma-separated list — `# cso-allow: io-yaml, io-json`. The directive is read from the *raw* line (not the string-scrubbed copy), so it survives the comment-stripping the matcher applies. Use it sparingly: every allow comment is a documented, reviewable decision that this particular raw call is genuinely intended.

11.5.4 Running it in CI

`test_scripts()` returns clean by default — it reports and moves on. To make it gate a build, pass `error_on_violation = TRUE` (R) / `error_on_violation=True` (Python). When set, the function `stop()`s (R) or raises `RuntimeError` (Python) after printing the report, but only if at least one violation is in the `io` or `api` family. The error message lists up to five offending files (with an “and N more” suffix) and restates the fix: replace the raw call with its toolkit equivalent, or annotate it with `# cso-allow:.` This is the form to wire into a pre-merge check; the contract-auditor-in-CI chapter covers the GitHub Actions plumbing in detail.

11.6 A note on naming

The auditor predates a naming cleanup. Its canonical name is `test_scripts`, but the `test_` prefix collides cosmetically with `testthat::test_*` unit-test naming — and the function is *not* a unit-test runner; it audits call sites against the contract. The R package therefore also exports `dw_test_scripts` as a `dw_`-prefixed alias, and applies the same treatment to the other R scaffolders: `dw_create_profile`, `dw_create_sector_script`, and `dw_review_profile`. (These aliases are R-only; the Python package exports the un-prefixed names alone.) A future release may rename `test_scripts` to something like `dw_audit_scripts` to make the intent unmistakable; until then the alias keeps the change additive and back-compatible. Prefer the `dw_`-prefixed names in new R code. See the function-reference appendix for the full alias map.

11.7 Summary

The scaffolders lower the cost of *adopting* the contract; the auditor raises the cost of *violating* it. `create_profile` and `create_sector_script` generate boilerplate that is correct by construction — sentinel, mode block, logging, error handling all wired the same way every time. `review_profile` verifies that wiring after the fact. `test_scripts` reads the codebase and names every raw IO or HTTP call that slipped past a wrapper, with a per-line escape hatch for the rare legitimate exception and a CI mode that fails the build. All four live in R and Python with shared argument names and shared rule ids; none exist in Stata, where the contract is enforced at the call site by the wrappers alone. For the vendoring mechanics that put these helpers into a consumer repository, see the vendoring-model and onboarding chapters.

Part IV

Part IV — Adoption & Operations

12 The vendoring model

The most consequential design decision in `cso-toolkit` — the shared helper library behind DW-Production and its sibling analytics pipelines — is not in any of its functions. It is in how those functions reach the repositories that use them. The toolkit is **vendored, not installed**: a consuming repository copies the helper files into its own tree, commits them to its own git history, and pins the version it copied in a small manifest file. There is no live `source()`, no `pip install` at session start, no `net install` from a network location. Why the project chose vendoring, where the line falls between toolkit and consumer, and how the refresh flow keeps a vendored copy current without surprising a reviewer — those are the subjects of this chapter.

12.1 What “vendoring” means here

Vendoring is the practice of copying a dependency’s source code directly into your project rather than referencing it from an external package registry. When DW-Production (a consuming pipeline repository) adopts `cso-toolkit`, it does not declare a dependency that some installer resolves later. Instead, the canonical helper files — `dw_io.R`, `dw_api.R`, `cso_toolkit_sync.R`, and any siblings the pipeline actually uses — are copied from the toolkit’s `r/R/` directory into DW-Production’s own `00_functions/` directory and tracked in DW-Production’s git. The copies freeze the vintage: every commit in the consuming repo records exactly which helper code was in effect at that point in history.

A single small file records what was copied and from where. The `.toolkit_manifest.yml` (its schema lives in the toolkit’s `templates/.toolkit_manifest.yml`) sits next to the vendored helpers and pins the version:

```
source: "unicef-drp/cso-toolkit"
pulled_version: "v0.4.0"
pulled_date: "2026-05-26"
pulled_by: "<your-name>"

vendored:
  - dw_io.R
  - dw_api.R
  - cso_toolkit_sync.R

local_edits: []
```

The `source` field names the upstream repository; `pulled_version` is the tag that was in effect at the most recent copy; `vendored` lists the files that were copied; and `local_edits` lists any vendored files the consumer has since modified, so the refresh workflow knows to leave them alone. The manifest is committed alongside the helpers and updated every time the toolkit is re-pulled. (See the provenance sidecars chapter for the related sidecar manifests that travel with data deposits, and the appendix on the sidecar schema for field-level detail.)

i Note

The same manifest schema applies to every language. R and Python consumers vendor into `00_functions/`; Stata consumers copy `.ado` and `.sthlp` files into the project's `ado/` directory (or `PERSONAL` adopath) and pin the same `pulled_version`. The contract is identical; only the destination directory differs by language idiom.

12.2 Why vendor instead of install

Four reasons drive the choice, and they reinforce one another. The toolkit's strategy document and README make the same case; the four points below restate what the project says explicitly.

12.2.1 1. Reproducibility — vintage permanence

A publication that cites a `dw_<sector>.csv` deposit in 2026 must, two years later, produce identical numbers when someone re-runs the pipeline. That guarantee fails the moment the helper code can drift out from under the analysis. With vendoring, every consuming repo's git history shows exactly which helper code was in effect at any commit — there is no situation where “the repo's pinned version was X but the running R session installed Y.” A 2026-05 release that needs re-running uses the helper code as it stood in 2026-05, because that code is physically in the repo at that commit. Network sourcing, by contrast, resolves to *whatever is current at run time*, which is precisely the drift the toolkit exists to prevent.

12.2.2 2. Producer-controlled refresh

Toolkit updates are not pulled automatically. The producer reviews the release notes, decides whether the new vintage is appropriate for the current work, and explicitly refreshes. This deliberately reverses the usual package-manager default, where dependencies creep forward on their own. Here the producer holds the upgrade decision, and a vendored copy never changes until someone chooses to change it and commits the result. (See the versioning and releases chapter for how the toolkit signals what changed between versions.)

12.2.3 3. Offline reviewer reproducibility

A reviewer with no network access — on a plane, behind a corporate proxy, sitting in customs — can still re-run the full pipeline, because the vendored helpers are right there in the repo. This dovetails with the mode contract (see the roles and mode contract chapter): reviewer mode physically forbids external network calls, so a reviewer could not fetch helpers over the network even if the design allowed it. Vendoring is what makes the offline reviewer path coherent: the frozen canonical deposit, the frozen API cache, and the frozen helper code all travel with the repo.

12.2.4 4. Cross-language symmetry

A package model fits R and Python tolerably but fits Stata poorly. Stata has no native package-install path that matters for this workflow. Vendoring works uniformly across all three languages: copy the files in, pin the vintage, done. The R helpers copy into `00_functions/`, the Python modules into the same place, and the Stata `.ado/.sthlp` files into `ado/`. One model, three languages, no special case.

A practical fifth consideration reinforces the rest: **AppLocker reality**. UNICEF laptops block many script-installable paths outright. Copying files into a project directory always works where a network installer might be silently denied.

12.3 Vendoring is the production model — not the only path

For day-to-day local development, each language *also* supports a native install path. This is a convenience for working on the toolkit itself or experimenting, not the model that consuming repositories ship with.

12.3.0.1 R

```
# Development convenience - install the package locally
devtools::install_local("r/")

# Production: vendored copies are sourced from 00_functions/
source("00_functions/dw_io.R")
source("00_functions/dw_api.R")
```

12.3.0.2 Python


```
# Development convenience - editable install
# pip install -e python/

# Production: import the vendored package after configuring the session
from cso_toolkit import _state, dw_save, dw_use
```

12.3.0.3 Stata

```
* Development convenience - extend the adopath to the toolkit source
adopath ++ "C:/Github/myados/cso-toolkit/stata/src"

* Production: copy src/ into the project's ado/ (or PERSONAL adopath);
* Stata picks them up automatically once they are on the path.
```

In every language, the development install path is the exception and vendoring is the rule. The reproducibility, offline, and producer-control guarantees above all depend on the production path being vendored.

12.4 What lives in cso-toolkit vs. the consumer

The boundary between the toolkit and its consumers is sharp, and keeping it sharp is what lets many repositories share one contract without leaking sector-specific detail upstream. The operative rule is simple: **if a helper would need to mention a specific sector to do its job, it does not belong in the toolkit.** The toolkit supplies the reproducibility floor — uniform IO, mode-aware API caching, provenance sidecars, the aggregation and scaffolding families — and the consumer supplies the domain logic that sits on top of it. The two tables below are the evidence for that rule.

Lives in cso-toolkit — anything repo-neutral and reusable:

Subtree	Content
r/R/	Canonical R helper source (dw_io.R, dw_api.R, cso_toolkit_sync.R, aggregation, scaffolding, demographics)
python/src/	Python siblings of the same helpers, plus _state.py for session configuration
stata/src/	Stata .ado + .sthlp equivalents for the subset of helpers that fit Stata idioms
templates/	Repo-neutral templates: .toolkit_manifest.yml, dbm_submission_template.md

Subtree	Content
<code>docs/</code>	Canonical operational docs: roles and workflow, mode-contract integration, per-function references
<code>tests/</code>	Unit and regression tests for the helpers
<code>NEWS.md</code> + git tags	Per-version release notes and semantic-version tags

Lives in the consumer, never in the toolkit — anything sector- or repo-specific:

- A consumer's profile script and its sector code
- A consumer's `user_config.yml` (the producer/reviewer mode setting, paths, and the like)
- Sector-specific data, indicator lists, and pipeline conductors
- Anything that references a particular sector's domain

This division is why new sectors and repositories — UNPD-Population, datalib, future deposit-bearing repos — inherit the floor for free instead of reinventing it. It is also why a drift problem the project hit early — four regional-metadata CSVs read independently across five sectors, with no shared vintage record — does not recur once everyone vendors the same helpers and pins them through one manifest.

12.5 The refresh decision flow

A vendored copy goes stale the moment the toolkit tags a newer version. The sync helpers exist to surface that gap and to drive the upgrade — but only on the producer's terms. Three functions implement the flow, and they carry the same names across the languages that support them.

Warning

Two of the three sync helpers are stubs today. `cso_toolkit_check` is fully implemented: it queries the upstream tag and reports whether you are behind. `cso_toolkit_diff` and `cso_toolkit_pull` are deliberate no-op stubs that validate state and explain themselves, but do not yet fetch or overwrite files; they will be wired once the upstream refresh path is finalized. The design intent below is fixed, so read Steps 3 and 4 as the contract those stubs will honor.

12.5.0.1 R

```
# At session start (producer mode), quietly check for a newer upstream tag
res <- cso_toolkit_check(quiet = FALSE)

# Inspect what would change before overwriting anything (stub today)
cso_toolkit_diff()

# Refresh the vendored copies to a target tag, prompting per file (stub today)
cso_toolkit_pull("v0.5.0")
```

12.5.0.2 Python

```
from cso_toolkit import cso_toolkit_check, cso_toolkit_diff, cso_toolkit_pull

res = cso_toolkit_check(quiet=False)
cso_toolkit_diff()          # stub today
cso_toolkit_pull("v0.5.0")  # stub today
```

12.5.0.3 Stata

```
* No Stata sibling of the sync helpers exists.
* Stata consumers refresh manually: re-copy stata/src/*.ado into ado/
* and bump pulled_version in .toolkit_manifest.yml by hand.
```

Warning

Coverage gap — Stata has no sync helper. `cso_toolkit_check`, `cso_toolkit_diff`, and `cso_toolkit_pull` exist in R and Python only. The `stata/src/` directory ships `dw_save`, `dw_use`, `dw_compare`, `dw_mkdir`, `dw_require_no_api`, and `dw_load_config` — but no `cso_toolkit_sync` equivalent. A Stata consumer keeps its vendored vintage current by hand: re-copy the `.ado/.sthlp` files and edit `pulled_version` in the manifest. (See the cross-language coverage matrix appendix for the full parity picture.)

The decision flow runs as follows.

Step 1 — Check (producer only, quiet by default). At session start the producer runs `cso_toolkit_check()`. It reads the local `.toolkit_manifest.yml`, queries the upstream repository for its latest release tag, and compares that tag against the pinned `pulled_version`. The check is gated by the mode contract: if the session's `dw_apis_allowed` flag is not `TRUE` — that is, in reviewer mode — it returns immediately and silently. Reviewers never see this branch, because the contract forbids the GitHub lookup. The lookup itself prefers the `gh` CLI (`gh api repos/<source>/releases/latest`

--jq .tag_name) and falls back to a raw `api.github.com` request — via `httr` in R, `requests` in Python — when `gh` is absent. If neither path reaches the network, the check returns `NULL` (R) or `None` (Python) and stays quiet.

Step 2 — Decide. If a newer tag is detected, `cso_toolkit_check(quiet = FALSE)` logs a yellow alert: it names the pinned and upstream versions, points at the release notes, and suggests the exact `cso_toolkit_pull()` call to run. If the pinned version is current — or if `pulled_version` is set to `inrepo`, the convention for a copy whose helpers have been edited locally and should never trigger sync warnings — the check reports no update and stays quiet. Note how this differs from the `local_edits` list: `inrepo` is a manifest-wide switch on `pulled_version` that silences the whole comparison (the check skips any tag whose pinned value contains the string `inrepo`), whereas `local_edits` is a per-file skip list that lets the check still run but tells a future `cso_toolkit_pull()` which individual files to leave untouched. The producer now chooses: refresh, or keep the pinned vintage. Nothing changes unless the producer acts.

Step 3 — Diff (look before you leap). `cso_toolkit_diff()` is the inspection step: by design it fetches each upstream file at the target tag and compares it against the vendored copy by `sha256`, so the producer can see exactly what would change before committing to an overwrite. Today it is a stub that explains itself and no-ops; the comparison logic lands once the upstream refresh path is wired.

Step 4 — Pull. `cso_toolkit_pull("v0.5.0")` performs the refresh: for each file in the manifest it fetches the upstream version at the target tag, computes a `sha256` against the current vendored copy, and — when they differ — prompts per file (overwrite, skip, or show diff). Files listed under `local_edits` are skipped so a consumer's deliberate local divergence is never clobbered. It then updates the manifest's `pulled_version`, `pulled_date`, and hashes, and logs a summary. Like `check`, `pull` is producer-only: in reviewer mode it raises a mode-lock error rather than touching the network, with the standard three-part error message — what failed, why, and how to fix it — directing the user to switch to producer mode (see the error taxonomy appendix). This too is a stub today; before it does anything, it validates the manifest, the `source` key, and upstream reachability, and refuses loudly if any are missing.

The decision flow, in outline:

```
cso-toolkit (upstream)
v0.1.0  v0.2.0  v0.3.0 ...
      |
      |
(producer runs cso_toolkit_check() at session start)
      |
      v
Newer tag detected?
-----
YES: log a yellow alert with
      pinned version, upstream
      version, pointer to NEWS.md,
```

```

                                and a suggested pull command
                                NO: stay silent
                                |
                                (producer decides; reviewer never reaches this branch
                                because the check is gated by dw_apis_allowed)
                                |
                                +-----+
                                v         v
cso_toolkit_pull("v0.2.0")    do nothing (keep pinned)
* fetch files at the tag      * vendored vintage is
* sha256-diff vs vendored     frozen until the next
* prompt per file             explicit pull
* update the manifest
* log a summary

```

Reviewers always run against whatever vintage the producer has pinned. If a reviewer wants to validate against a *different* vintage, they cannot do it from a reviewer-mode session — they do it through a separate producer-mode session, refresh the pin there, and commit. The mode contract and the vendoring model are two halves of the same guarantee: the reviewer’s environment is frozen, and freezing the helper code is what makes the rest of the freeze meaningful.

12.6 Why not a package, in one sentence

A package model would let a consuming repo’s helper code drift independently of its git history, would fail offline, would surrender the upgrade decision to an installer, and would not fit Stata at all. Vendoring trades the small cost of an explicit refresh step for vintage permanence across three languages and any number of consumers — which is exactly the bargain a reproducibility-first toolkit should make.

The vendoring model is the connective tissue of everything else in this book. The IO and API contracts are only reproducible because the code that enforces them is frozen in the consumer’s history; the mode contract’s offline reviewer path is only coherent because the helpers travel with the repo. The next chapter walks through onboarding a repository end to end — copying the helpers, writing the manifest, wiring the profile — as a concrete worked example of this model in practice.

13 Onboarding a repo: a worked example

This chapter walks the producer/reviewer deposit contract into a real repository, step by step, with the exact commands you would run. (The preceding chapters set out the pieces in the abstract: the producer/reviewer mode contract in the roles chapter, the IO and external-API contracts in Part III, and the vendoring model in Part IV.) The running example is a hypothetical new deposit-bearing repo — call it `unicef-drp/UNPD-Population`, one of the DW-Production-style repos that publish curated data to a shared Teams/Z: warehouse — but every step is repo-neutral; substitute your own owner, repo, and sector codes.

The walkthrough has six steps:

1. Vendor the helpers into `00_functions/`.
2. Pin the vintage in `.toolkit_manifest.yml`.
3. Wire the mode contract into the profile.
4. Make a first producer run: cache an API source, deposit an output.
5. Re-run as a reviewer and compare against the deposit.
6. Add the contract auditor to CI.

By the end you will have a repo whose pipeline writes to the canonical deposit only in producer mode, isolates reviewer runs into a sandbox, and proves on every push that the deposit stays write-protected in reviewer mode and that no script bypasses the API contract.

Note

The toolkit ships three language implementations that share command names and function signatures: R (`r/R/*.R`), Python (`python/src/*.py`), and Stata (`stata/src/*.ado`). Where a step involves code, all three are shown together in a tabset. The three languages do not cover the same features, and we flag each gap where it bites — see also the cross-language coverage matrix in the appendices.

13.1 Step 0: Decide what you are adopting

Before copying any files, settle two questions.

Which helpers does your pipeline actually need? The toolkit is modular. A repo that only reads and writes deposit files needs the IO contract (`dw_save` / `dw_use` / `dw_compare`) and the sync helper. A repo that also fetches from upstream APIs needs the external-API contract (`dw_api_fetch` / `dw_api_cached` / `dw_api_inventory`). The vendoring manifest's

vendored: list is meant to be trimmed to exactly what you use — do not copy files your pipeline never calls.

Which languages? R and Python are at near-parity. Stata covers the producer/reviewer IO and config contract (`dw_save`, `dw_use`, `dw_compare`, `dw_load_config`, `dw_mkdir`, `dw_require_no_api`) but has **no API wrapper** (`dw_api_fetch` and friends are R+Python only) and **no sync helper** (`cso_toolkit_check` / `_pull` are R+Python only), and Parquet output is R+Python-only (Stata writes `.dta`; R adds `.rds`, Python adds `.pkl`). If your Stata code needs a remote source, it cannot route through `dw_api_fetch`; you must keep API fetching in R or Python and have Stata read the cached deposit with `dw_use`. Plan around that gap now rather than discovering it mid-onboarding.

13.2 Step 1: Vendor the helpers

The toolkit is **vendored, not installed** (see the vendoring chapter for the rationale). You copy the source files into your repo and commit them. There is no live `source()` or `install_github` from the network at session start; the committed copies freeze the vintage in your git history, so a reviewer with no network access can still re-run the pipeline.

Create the conventional `00_functions/` directory and copy in the helpers you settled on in Step 0.

13.2.0.1 R

```
# From the root of your consumer repo
mkdir -p 00_functions
cp /path/to/cso-toolkit/r/R/dw_io.R          00_functions/
cp /path/to/cso-toolkit/r/R/dw_api.R        00_functions/
cp /path/to/cso-toolkit/r/R/cso_toolkit_sync.R 00_functions/
```

Source them at the top of your profile (Step 3 wires this in):

```
for (f in list.files("00_functions", pattern = "\\..R$", full.names = TRUE)) {
  source(f)
}
```

13.2.0.2 Python

The Python implementation lives under `python/src/`. For the vendoring model, copy the modules alongside your pipeline code:

```
mkdir -p 00_functions
cp /path/to/cso-toolkit/python/src/dw_io.py 00_functions/
cp /path/to/cso-toolkit/python/src/dw_api.py 00_functions/
cp /path/to/cso-toolkit/python/src/cso_toolkit_sync.py 00_functions/
```

```
import sys
sys.path.insert(0, "00_functions")
from dw_io import dw_save, dw_use, dw_compare
from dw_api import dw_api_fetch, dw_api_inventory
```

13.2.0.3 Stata

Stata has no native package install path that matters here; vendoring is the production model. Either extend the adopath during development or copy the `.ado` / `.sthlp` files into the consumer's `ado/` (or `PERSONAL` adopath) for production.

```
* Option A -- extend the adopath (development)
adopath ++ "C:/path/to/cso-toolkit/stata/src"
which dw_save

* Option B -- copy into the project ado/ (production)
* (shell: cp cso-toolkit/stata/src/*.ado ado/ ; cp cso-toolkit/stata/src/*.sthlp ado/)
```

There is no `dw_api.ado` and no `cso_toolkit_sync.ado` to copy — those contracts are R+Python only.

13.3 Step 2: Pin the vintage in `.toolkit_manifest.yml`

The manifest records which upstream vintage you vendored, so the sync helper can tell you later when you have fallen behind. It lives **inside** `00_functions/`, **next to the helpers it tracks** — that is exactly where `cso_toolkit_check()` looks for it (it resolves the directory from the `dwFunct` global the profile sets, falling back to `<getwd()>/00_functions`). Vendor the template into `00_functions/.toolkit_manifest.yml` and edit it to match what you actually copied.

```
source: "unicef-drp/cso-toolkit"
pulled_version: "v0.5.0"
pulled_date: "2026-06-28"
pulled_by: "<your-name>"

# Files vendored from cso-toolkit/r/R/ into consumer's 00_functions/
vendored:
```



```

- dw_io.R
- dw_api.R
- cso_toolkit_sync.R

# Local edits applied on top of vendored files. List paths here so that
# cso_toolkit_pull() skips them on update. (Empty by default.)
local_edits: []

```

The fields, from the template’s own header:

Field	Meaning
<code>source</code>	GitHub owner/repo of the upstream toolkit.
<code>pulled_version</code>	The tag at the time of the most recent pull (e.g. <code>v0.5.0</code>). Use inrepo if you edited the helpers locally and do not want sync warnings.
<code>pulled_date</code>	ISO-8601 date of the pull. Free-form; the helper does not parse it.
<code>pulled_by</code>	Human breadcrumb for who ran the pull.
<code>vendored</code>	List of files copied from the toolkit into your <code>00_functions/</code> . Trim to what you use.
<code>local_edits</code>	Files in vendored: that you have diverged locally; <code>cso_toolkit_pull()</code> skips these on update. Empty by default.

Commit the manifest in version control alongside the vendored helpers, and update it every time you re-pull. Keep **vendored:** honest: list a file only if it is actually present in `00_functions/`.

Warning

`pulled_version` must be a real tag once the upstream repo has releases. The Database Manager (DBM) pre-deposit checklist (the submission template referenced in the roles chapter) gates a deposit on the manifest being on a tagged version — no **inrepo** or **unreleased** suffixes. Use **inrepo** only during local development of the helpers themselves, never for a deposit run. Note that `cso_toolkit_check()` still queries upstream when `pulled_version` contains **inrepo** but forces **updates_available** to `FALSE`, so a stray **inrepo** silently suppresses drift warnings.

You can confirm at any point that the helper sees your manifest by running the sync check. In producer mode it queries the GitHub API for the latest upstream tag and warns if you are behind; in reviewer mode it quietly returns `NULL` because the mode contract forbids network calls.

13.3.0.1 R

```
res <- cso_toolkit_check(quiet = FALSE)
# res$pinned_version / res$upstream_version / res$updates_available
```

13.3.0.2 Python

```
res = cso_toolkit_check(quiet=False)
# res["pinned_version"] / res["upstream_version"] / res["updates_available"]
```

13.3.0.3 Stata

```
* No Stata sync helper. Inspect .toolkit_manifest.yml by hand,
* or run cso_toolkit_check() from the R/Python side of the same repo.
```

13.4 Step 3: Wire the mode contract into the profile

This is the load-bearing step. Everything downstream — which directory a `dw_save` lands in, whether an API fetch is allowed — flows from the `dw_mode` setting and the globals the profile derives from it.

13.4.1 3a. The `user_config.yml` contract

Each user keeps a personal block in their config file. In R and Python the profile reads it from `file.path(USERPROFILE, ".config", "user_config.yml")` — that is `~/.config/user_config.yml` on macOS/Linux and `%USERPROFILE%\config\user_config.yml` on Windows. The Stata helper resolves the same location via `$HOME` then `%USERPROFILE%`.

```
jpazevedo:
  githubFolder: "C:/GitHub/mytasks"
  teamsRoot: "C:/Users/jpazevedo/UNICEF/Chief Statistician Office - Documents"
  dw_mode: "reviewer" # producer | reviewer
  sandboxRoot: "C:/Users/jpazevedo/dw-repro" # only used when dw_mode = reviewer
```

`dw_mode` is **required**, and it must be exactly `producer` or `reviewer`. The profile loader must hard-fail if `dw_mode` is absent or holds any other value — defaulting to either side is unsafe. Defaulting to `producer` corrupts the deposit; defaulting to `reviewer` surprises a DBM mid-deposit.

13.4.2 3b. Load the config and derive the path globals

There is no single `dw_load_config()` helper in R or Python: the profile loads the YAML itself with `yaml::read_yaml()` (R) and validates `dw_mode` directly, exactly as the toolkit's `create_profile()` scaffold generates. Stata is the exception — it ships `dw_load_config`, a hand-rolled `key: value` parser, because Stata cannot link an external YAML package on AppLocker-locked corporate Windows.

After loading the config, the profile derives the routing globals from `dw_mode`. The `*Canonical` aliases let reviewer-mode code *read* the deposit without ever *writing* to it (you will use exactly these aliases in Step 5's compare); producer mode collapses them to the same path.

Global	Producer mode	Reviewer mode
<code>teamsFolder</code>	user's actual Teams sync root	<code>sandboxRoot</code>
<code>teamsRawData</code>	<code>teamsFolder/01_dw_prep/011_rawdata</code>	<code>sandboxRoot/01_dw_prep/011_rawdata</code>
<code>teamsWrkData</code>	<code>teamsFolder/01_dw_prep/013_wrkdata</code>	<code>sandboxRoot/01_dw_prep/013_wrkdata</code>
<code>teamsFolderCanonical</code>	identical to <code>teamsFolder</code>	user's actual Teams sync root (read-only)
<code>teamsRawDataCanonical</code>	identical to <code>teamsRawData</code>	<code>teamsFolderCanonical/01_dw_prep/011_rawdata</code>
<code>teamsWrkDataCanonical</code>	identical to <code>teamsWrkData</code>	<code>teamsFolderCanonical/01_dw_prep/013_wrkdata</code>
<code>dw_apis_allowed</code>	TRUE	FALSE

The toolkit's IO and API helpers read these globals through a safe accessor: `dw_save` resolves its destination through `teamsWrkData` / `teamsRawData`, `dw_api_fetch` checks `dw_apis_allowed` before any network call, and `dw_is_canonical()` tests a path against the `*Canonical` roots (plus the OneDrive 060.DW-MASTER pattern and the Z: drive) to gate reviewer writes. Set the globals correctly here and the rest of the contract follows for free.

Load the config and emit a prominent mode banner so the operator can never mistake which mode the session is in.

13.4.2.1 R

```

USERPROFILE <- Sys.getenv("USERPROFILE", unset = "~")
config_path <- file.path(USERPROFILE, ".config", "user_config.yml")
user_config <- yaml::read_yaml(config_path)           # stops if file missing

dw_mode <- user_config[[USERNAME]]$dw_mode
if (is.null(dw_mode) || !dw_mode %in% c("producer", "reviewer")) {
  stop("user_config.yml must set dw_mode to 'producer' or 'reviewer'.")
}

```

```
# ... derive teamsFolder, teamsRawData, teamsWrkData, the *Canonical aliases,
#      and dw_apis_allowed from dw_mode per the table above ...

if (isTRUE(dw_apis_allowed)) {
  message("PRODUCER mode -- writes will land in the Teams deposit.")
} else {
  message("REVIEWER mode -- writes isolated to ", sandboxRoot,
          "; canonical deposit read but never written.")
}
```

13.4.2.2 Python

```
import os, yaml
config_path = os.path.join(os.environ.get("USERPROFILE", os.path.expanduser("~")),
                           ".config", "user_config.yml")
with open(config_path) as fh:
    user_config = yaml.safe_load(fh)

dw_mode = user_config[username]["dw_mode"]
if dw_mode not in ("producer", "reviewer"):
    raise SystemExit("user_config.yml must set dw_mode to 'producer' or 'reviewer'.")
# derive the same globals from dw_mode; set dw_apis_allowed accordingly
```

13.4.2.3 Stata

```
dw_load_config          // reads ~/.config/user_config.yml (HOME/USERPROFILE)
* Sets $dw_mode plus exactly the path globals the YAML provides
* ($teamsWrkData, $teamsRawData, the *Canonical keys, $dwZDrive, $sandboxRoot).
* dw_load_config does NOT derive mode-aware globals; the profile then swaps
* $teamsWrkData / $teamsRawData to canonical (producer) or sandbox (reviewer)
* itself after the call, exactly as the R profile does.
```

`dw_load_config` (Stata) errors clearly if neither `HOME` nor `USERPROFILE` is set, errors if the file is missing, and hard-stops with error 459 when `dw_mode` is missing or invalid. The R and Python profiles raise equivalently — because the toolkit needs `dw_mode` and the path globals before any other helper call.

i Note

A real divergence to plan around: Stata's `dw_load_config` sets only the keys present in the YAML and leaves the producer/reviewer path swap to the profile, whereas the R/Python profiles do the parse and the swap in one place. The contract is the same —

`dw_mode` plus the derived path globals — but the seam between “parse” and “derive” sits in different files across languages.

13.4.3 3c. Shadow the globals at the conductor

The profile sets *defaults*; the conductor *redirects* them for this run. The conductor here means each sector’s `0_execute_conductor.R` — the top-level script that drives a run end to end. At the top of it, after sourcing the profile, shadow the globals so every downstream script gets mode-correct paths without per-script edits.

```
# After computing outputdir for this run (with branch-slug isolation):
finaldir <- file.path(outputdir, "final")

# Conductor-level shadow: redirect downstream reads/writes without per-script edits
teamsRawData <- teamsRawDataCanonical    # reviewer-mode: read canonical inputs
teamsWrkData  <- finaldir                  # reviewer-mode: writes go to sandbox final
dwWrkData     <- finaldir                  # same
```

Why shadow at the conductor and not per-script: a sector codebase has 10–40 scripts that reference the path globals (`teamsRawData`, `teamsWrkData`, and the rest). Editing each one breaks the diff audit. Shadowing once lets every downstream script keep its existing code and still get mode-correct paths. The Stata side uses the same trick — once the profile mode-routes `$teamsFolder`, existing `save` calls that reference it automatically land in the sandbox in reviewer mode, with no per-call edits.

Optionally compose this with branch-slug isolation so feature-branch outputs land under `.../sector/output/branches/<slug>/` rather than the canonical deposit, even when a reviewer is on a feature branch:

```
branch <- tryCatch(
  system("git rev-parse --abbrev-ref HEAD", intern = TRUE),
  error = function(e) "main"
)
branch_slug <- gsub("[^A-Za-z0-9._-]", "-", branch)

canonical_outputdir <- file.path(teamsRawData, sector, "output")
outputdir <- if (branch %in% c("main", "dev", "master")) {
  canonical_outputdir
} else {
  file.path(canonical_outputdir, "branches", branch_slug)
}
```

13.5 Step 4: First producer run — cache an API source and deposit an output

Switch your `user_config.yml` to `dw_mode: "producer"` and confirm the load banner reads PRODUCER. Now run the pipeline for the first time. Two things happen on a first producer run that never happen again — and both are one-time precisely because the run populates state that later runs reuse: an API source is fetched live and cached (later runs hit the cache), and an output file is written to the deposit where no prior vintage exists (later runs find one already there).

13.5.1 4a. Fetch and cache an upstream source

`dw_api_fetch` tries the cache first (sandbox, then canonical fallback) and only fetches live when there is no hit and the session is in producer mode. On a first run the cache is empty, so it fetches, writes the cache via `dw_save` (mode-aware path plus Z: mirror), and emits a provenance sidecar recording endpoint, params, `fetch_at`, user, mode, and sha256 (see the provenance-sidecars chapter for the sidecar's shape). Caches never expire automatically; refresh is explicit (`refresh = TRUE`), and the DBM owns refresh cadence.

13.5.1.1 R

```
wb <- dw_api_fetch(  
  api      = "wb",  
  cache_key = "wb_lp_primary_2000_2024",  
  indicator = c("SE.LPV.PRIM"),  
  start_date = 2000,  
  end_date   = 2024  
)
```

13.5.1.2 Python

```
wb = dw_api_fetch(  
    api="wb",  
    cache_key="wb_lp_primary_2000_2024",  
    indicator=["SE.LPV.PRIM"],  
    start_date=2000,  
    end_date=2024,  
)
```

13.5.1.3 Stata

```
* No dw_api_fetch in Stata. Fetch + cache the source from the R or
* Python side of the repo (producer mode), then have Stata read the
* cached deposit file with dw_use.
```

The supported `api` values are `uis`, `sdmx`, `sdmx_codelist`, `wb`, `wb_indicators`, `ilo`, `unsd_sdg`, `github_raw`, `http`, and `json_get`; the API-specific arguments (`indicator`, `flowRef`, `series_codes`, and so on) pass straight through to the per-API fetcher.

If you run this in reviewer mode and the cache is missing, `dw_api_fetch` does not fall back to the network — it raises through the `dw_require_no_api` guard, telling you the input must be pre-deposited and to escalate to the sector DBM rather than calling the API. That is the contract working as designed.

13.5.2 4b. Write a deposit output

Write your processed output with `dw_save`. The R and Python helpers take a `name` + `sector` + `kind` shorthand that resolves the canonical path for you; pass `isid` to enforce row uniqueness on the key columns. In producer mode the write lands in the deposit, and is carbon-copied automatically to both the Teams canonical equivalent and the Z: drive when the profile makes those roots available. (The per-call `mirror_to_z` flag is deprecated — pass it and `dw_save` warns and ignores it; mirroring is now automatic and paired, controlled by which remote roots the profile makes available.) A provenance sidecar is written alongside the file.

13.5.2.1 R

```
dw_save(
  df,
  name      = "dw_pop_tot.csv",
  sector    = "pop",
  kind      = "wrk",
  isid      = c("REF_AREA", "TIME_PERIOD"),
  metadata  = list(title = "Total population, by area and year")
)
```

13.5.2.2 Python

```
dw_save(
  df,
  name="dw_pop_tot.csv",
  sector="pop",
  kind="wrk",
  isid=["REF_AREA", "TIME_PERIOD"],
  metadata={"title": "Total population, by area and year"},
)
```

13.5.2.3 Stata

```
dw_save , filename("dw_pop_tot.dta")    ///
        path("$teamsWrkData/pop")      ///
        idvars("REF_AREA TIME_PERIOD") ///
        title("Total population, by area and year")
```

i Note

Two cross-language gaps in `dw_save`. First, the Stata helper takes an explicit `path()` and `filename()` rather than the `name / sector / kind` shorthand, and writes `.dta`. Second, the overwrite gate differs: R uses an `overwrite = NULL` sentinel (resolving to `TRUE` in reviewer mode, `FALSE` in producer mode) plus a `force` confirmation prompt on existing producer deposits; Python's `overwrite` defaults to a plain `False` and has no `force` parameter. In all three languages the provenance sidecar has the same shape, so reviewer-mode integrity checks work across languages. There is no Parquet output on the Stata side.

After the run, inventory what you cached so you have a record for the deposit checklist:

13.5.2.4 R

```
dw_api_inventory()      # all caches, with size, mtime, and fetched_at
```

13.5.2.5 Python

```
dw_api_inventory()
```

13.5.2.6 Stata


```
* No dw_api_inventory in Stata; inspect the _apis/ cache dir directly
* or run the inventory from R/Python.
```

13.6 Step 5: Reviewer re-run and compare

Now prove the deposit is reproducible. Switch a second user (or your own config) to `dw_mode: "reviewer"`, set a `sandboxRoot`, and confirm the load banner reads `REVIEWER`. Run the same conductor. Every artifact lands in the sandbox, and the canonical deposit's modification time is unchanged before and after the run — because the profile routes `teamsRawData` / `teamsWrkData` under `sandboxRoot` and the conductor shadow sends writes to `finaldir`. Any stray API call raises through the `dw_require_no_api` guard instead of hitting the network.

Then compare the reviewer's freshly produced file against the canonical deposit with `dw_compare`. The `reference` (the deposited “old” side) is read through the canonical alias, which reviewer-mode code may read but never write. `dw_compare` is tolerant of a missing reference — on a genuine first deposit it warns rather than stopping — but here the reference exists from Step 4.

13.6.0.1 R

```
report <- dw_compare(
  current      = file.path(finaldir, "pop", "dw_pop_tot.csv"),
  reference    = file.path(teamsWrkDataCanonical, "pop", "dw_pop_tot.csv"),
  by           = c("REF_AREA", "TIME_PERIOD"),
  value_cols   = c("OBS_VALUE", "DATA_SOURCE"),
  numeric_value_cols = c("OBS_VALUE"),
  tol          = 1e-5,
  label        = "dw_pop_tot",
  write_report_to = file.path(tempdir(), "pop_compare_warehouse")
)
report$summary
```

13.6.0.2 Python

```
import os, tempfile
report = dw_compare(
    current=os.path.join(finaldir, "pop", "dw_pop_tot.csv"),
    reference=os.path.join(teamsWrkDataCanonical, "pop", "dw_pop_tot.csv"),
    by=["REF_AREA", "TIME_PERIOD"],
    value_cols=["OBS_VALUE", "DATA_SOURCE"],
```

```

numeric_value_cols=["OBS_VALUE"],
tol=1e-5,
label="dw_pop_tot",
write_report_to=os.path.join(tempfile.gettempdir(), "pop_compare_warehouse"),
)
report["summary"]

```

13.6.0.3 Stata

```

dw_compare , current("`finaldir'/pop/dw_pop_tot.dta")          ///
              reference("$teamsWrkDataCanonical/pop/dw_pop_tot.dta") ///
              idvars("REF_AREA TIME_PERIOD")                  ///
              valuevars("OBS_VALUE DATA_SOURCE")              ///
              tol(1e-5) label("dw_pop_tot")                    ///
              report("`c(tmpdir)'/pop_compare.md")

```

Warning

The Stata `dw_compare` option names differ from R/Python: the join key is `idvars()` (not `by`), the columns to compare are `valuevars()` (not `value_cols` / `numeric_value_cols`), and the Markdown summary path is `report()` (not `write_report_to`). The diff logic and the added/removed/changed classification are the same; only the keyword spellings diverge. For a full editorial diff with per-row tables, the Stata port points you at the upstream `comparefiles` command instead.

The summary reports **added**, **removed**, and **changed** row counts (per the comparison reporting in the aggregation chapter). On a clean reproduction those are all zero. When they are not, the DBM submission template's triage taxonomy classifies each non-zero result — **stale-cache**, **retired-upstream-indicator**, **methodology-change**, **rounding**, **bug**, or **uninvestigated** — and a deposit cannot be signed off while any row remains **uninvestigated**.

This reviewer re-run plus the compare report is what it means, operationally, for a repo to have adopted the mode contract: a reviewer-mode end-to-end run produces every artifact under `sandboxRoot`, the canonical deposit is byte-unchanged, and the compare report exists.

13.7 Step 6: Add the contract auditor to CI

The final step makes the guarantees enforceable on every push rather than dependent on the operator remembering to run them. The toolkit's own CI is a useful reference: `r-check.yml` runs R CMD `check` across a Linux/macOS/Windows matrix, and because the helper tests

live in `tests/testthat`, the check runs them automatically — no separate test step. The workflow deliberately does not set `dw_mode` at the job level; the mode is a session-level concept that each test sets explicitly, so a test never leaks producer-mode behaviour into another.

For your consumer repo, add a workflow that (a) runs your pipeline's unit tests under both modes and (b) runs the contract auditor — the scaffolding check that verifies no script bypasses the contract (see the scaffolding and contract-auditor chapter for what the auditor inspects). The auditor's job mirrors the verification checklist from the mode-contract integration doc:

- Sourcing the profile with `dw_mode: "reviewer"` produces the green banner.
- The conductor writes `finaldir` and shadows the path globals before any other script is sourced.
- No script inside the sector calls a remote without going through `dw_api_fetch` (grep for `httr::GET`, `httr::POST`, `RETRY`, `read.csv("http`, and the like).
- A reviewer-mode run leaves the canonical deposit's `mtime` unchanged.

A minimal addition to `.github/workflows/`:

```
name: contract-audit

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  audit:
    runs-on: ubuntu-latest
    env:
      # Mode is set per-test inside the suite; do not set it job-wide.
      GITHUB_PAT: ${ secrets.GITHUB_TOKEN }
    steps:
      - uses: actions/checkout@v4
      - uses: r-lib/actions/setup-r@v2
        with:
          use-public-rspm: true
      - uses: r-lib/actions/setup-r-dependencies@v2
        with:
          working-directory: ./00_functions
          extra-packages: any::testthat
      - name: Run pipeline tests + contract auditor
        run: Rscript -e 'testthat::test_dir("tests/testthat")'
```

⚠ Warning

CI cannot fetch from the corporate-net-only upstream APIs, and it must never run in producer mode against a real deposit. Audit the *contract* — banner, shadow, no-bypass, deposit-mtime-unchanged — using the cached fixtures and reviewer mode. Live API fetching belongs in a producer-mode session on a machine with network access, not in CI. This is the same reason `cso_toolkit_check` is a producer-mode-only operation: reviewers, and CI by extension, run against whatever vintage the producer has pinned.

13.8 Recap: the adoption checklist

A repo has fully adopted the contract when all of the following hold. These map directly onto the verification checklist in the mode-contract integration doc and the sign-off section of the DBM submission template.

#	Check	How to confirm
1	Helpers vendored into <code>00_functions/</code>	files present and committed
2	Manifest pins a real tag	<code>.toolkit_manifest.yml</code> <code>pulled_version</code> is <code>vX.Y.Z</code> , not <code>inrepo</code>
3	Profile hard-fails without <code>dw_mode</code>	load with the key absent errors out
4	Mode banner is unmistakable	reviewer load prints the sandbox-isolation banner
5	Conductor shadows the path globals	<code>finaldir</code> set and globals reassigned before other scripts source
6	First producer run caches + deposits	<code>dw_api_inventory()</code> shows the cache; deposit file + sidecar exist
7	Reviewer run is deposit-safe	every artifact under <code>sandboxRoot</code> ; canonical <code>mtime</code> unchanged
8	Compare report exists	<code>dw_compare</code> summary written under the sandbox temp dir
9	No script bypasses the API contract	grep for raw <code>http</code> / <code>read.csv("http</code> returns nothing
10	CI audits the contract on every push	<code>contract-audit</code> workflow green

Steps 7, 8, and the underlying reviewer run are the operational test that the contract is real and not merely declared. Once they pass and CI enforces them, the repo joins DW-Production as a deposit-bearing consumer of the same contract. To keep it in sync as the toolkit releases new vintages, see the vendoring and versioning chapters in Part IV.

14 The contract auditor in continuous integration (CI)

Earlier chapters describe a contract — the rule that all file and network access goes through a small set of toolkit wrappers rather than raw library calls. File IO flows through `dw_save` / `dw_use` / `dw_compare` / `dw_merge`, and external-API access flows through `dw_api_fetch` / `dw_api_cached`. But a contract that is only documented decays. New scripts reach for `read_csv()` out of habit; a quick `httr::GET()` slips into a one-off fetch; a reviewer approves a pull request without noticing either. The contract auditor closes that gap. It is a single function — `test_scripts()` — that recursively scans a directory of analysis scripts and flags every line that calls a raw IO or HTTP command the toolkit is meant to wrap. Run interactively it is a code-review aid; run in continuous integration with `error_on_violation = TRUE` it becomes a build gate that fails the moment a bypass lands.

What follows shows the auditor working: what it checks, how it reports, how to silence the rare legitimate exception with `# cso-allow`, and how to wire it into a GitHub Actions job. The cross-language coverage matrix (Appendix A) and the scaffolding chapter give the wider picture; here the focus is narrow and operational.

14.1 What the auditor does

`test_scripts()` walks a directory tree, reads each source file line by line, and matches every line against a registry of regular-expression rules. Each rule describes one *raw* call that the toolkit wraps — a direct CSV read, a direct HTTP request, a direct SDMX fetch (SDMX is the statistical-data-exchange format used by agencies like the World Bank and ILO) — together with the toolkit function the author should have used instead. A match is a *violation*. The function returns one row per violation as a data frame and, optionally, prints a human-readable summary and stops the process.

The auditor is a *lexical* check, not a parser. It strips string literals and trailing comments from each line and then applies its patterns, so it is fast, dependency-light, and would work the same way for any language, since it only matches text — but it sees text, not an abstract syntax tree. Because it sees only text, the auditor sometimes misfires on a call that is genuinely correct; the `# cso-allow` escape hatch (below) handles those cases.

The signatures are deliberately parallel across the two languages that ship an auditor:

14.1.0.1 R

```
test_scripts(  
  path,  
  pattern          = "\\..R$",  
  recursive        = TRUE,  
  ignore_files     = c("dw_io.R", "dw_api.R", "cso_toolkit_sync.R",  
                       "profile_helpers.R", "test_scripts.R"),  
  ignore_dirs      = c(".git", "renv", "packrat",  
                       "node_modules", ".Rproj.user"),  
  custom_rules     = NULL,  
  error_on_violation = FALSE,  
  verbose          = TRUE,  
  debug            = NULL  
)
```

14.1.0.2 Python

```
test_scripts(  
  path,  
  pattern="\\.py$",  
  *,  
  recursive=True,  
  ignore_files=("dw_io.py", "dw_api.py", "cso_toolkit_sync.py",  
               "profile_helpers.py", "test_scripts.py", "_state.py",  
               "__init__.py"),  
  ignore_dirs=(".git", ".venv", "venv", "env", "__pycache__",  
               ".tox", ".pytest_cache", "node_modules"),  
  custom_rules=None,  
  error_on_violation=False,  
  verbose=True,  
)
```

14.1.0.3 Stata

```
* No Stata auditor.  
* cso-toolkit ships test_scripts() in R and Python only.  
* Stata sector code is audited by code review against the same  
* contract; see the coverage note below.
```

The two signatures match except for one R-only argument. R's `debug` (default `NULL`) is an internal diagnostics switch: when `TRUE` it prints the file count, rule count, and per-family

violation tallies, and otherwise it inherits `options(dw.debug)`. It tunes the toolkit's own verbosity machinery and does not change what the auditor flags. Python has no equivalent; the asymmetry is cosmetic, not a coverage gap.

⚠ Coverage gap: Stata has no auditor

The contract auditor exists in R (`r/R/test_scripts.R`) and Python (`python/src/test_scripts.py`) only. There is no Stata implementation — the Stata source tree (`stata/src/`) ships `dw_save.ado`, `dw_use.ado`, `dw_compare.ado`, `dw_load_config.ado`, `dw_mkdir.ado`, and `dw_require_no_api.ado`, but nothing named `test_scripts.ado`. Stata consumers must enforce the contract by reading each other's code, since there is no automated scan to wire into a Stata build. Treat the R/Python parity below as the limit of automated coverage, not a claim about Stata.

In both languages the return value is a data frame (an R `data.frame`, a pandas `DataFrame`) with the same seven columns: `file`, `line`, `rule`, `family`, `message`, `suggest`, and `snippet`. An empty frame means a clean scan. The columns are stable, which matters for CI: a downstream step can read them to post annotations or build a report.

14.2 The rule set

Every rule has five fields — `id`, `family`, `pattern`, `message`, and `suggest`. The `family` is either `"io"` or `"api"`, and only those two families can fail a build (see `error_on_violation` below). The `pattern` is a PCRE regular expression matched against the line after string literals and the trailing comment are scrubbed. The patterns deliberately match both the unqualified and the namespaced form of a call, so `read_csv()` and `readr::read_csv()` are both caught, as are `pd.read_csv()` and `pandas.read_csv()`.

The two registries are not identical, because the raw calls differ by language ecosystem. The R registry targets the tidyverse / `data.table` / `haven` / `arrow` / `rsdmx` / `wbstats` / `Rilostat` surface; the Python registry targets the `pandas` / `requests` / `httpx` / `sdmx` / `wbgapi` / `wbdata` surface. The table below pairs them by intent.

Intent	R rule id	R catches (examples)	Python rule id	Python catches (examples)
CSV read	<code>io-read-csv</code>	<code>read.csv</code> , <code>read_csv</code> , <code>read_tsv</code> , <code>read_delim</code>	<code>io-read-csv</code>	<code>pd.read_csv</code> , <code>read_csv</code>
CSV write	<code>io-write-csv</code>	<code>write.csv</code> , <code>write_csv</code> , <code>write_tsv</code> , <code>write_delim</code>	<code>io-to-csv</code>	<code>.to_csv()</code>

Intent	R rule id	R catches (examples)	Python rule id	Python catches (examples)
data.table	io-fread	fread, fwrite	—	(covered by pandas rules)
RDS / pickle	io-rds	saveRDS, readRDS	io-pickle	pickle.dump, pickle.load
.RData	io-load-save	save, load	—	—
Stata .dta	io-dta	read_dta, write_dta, read_stata, write_stata	io-read-stata / io-to-stata	pd.read_stata, .to_stata(
Excel	io-xlsx	read_excel, read_xlsx, write_xlsx, read.xlsx, write.xlsx, getSheetNames, excel_sheets, loadWorkbook	io-read-excel / io-to-excel	pd.read_excel, .to_excel(
Parquet	io-parquet	read_parquet, write_parquet	io-read-parquet / io-to-parquet	pd.read_parquet, .to_parquet(
JSON file	io-json-file	read_json, write_json	io-json	json.dump, json.load
YAML	io-yaml	read_yaml, write_yaml, yaml.load_file	io-yaml	yaml.safe_load, yaml.load, yaml.safe_dump, yaml.dump
Raw file handle	—	—	io-open-write	open(..., "w") and other write/append/binary modes
HTTP	api-hhttp	http::GET/POST/PUT/PATCH/DELETE, http2::request	api-http / api-urllib	requests.get, httpx.get, urllib.request.urlopen
JSON-from- URL	api-fromjson-url	fromJSON("https://...")	—	(covered by HTTP rules)
SDMX	api-sdmx	readSDMX	api-sdmx	sdmx.Client(
World Bank	api-wbstats	wb_data, wb_indicators, wb_countries	api-wbgapi / api-wbdata	wbgapi.data...., wbgapi.series...., wbdata.get_data, wbdata.get_dataframe
ILO	api-ilostat	get_ilostat	—	—

Intent	R rule id	R catches (examples)	Python rule id	Python catches (examples)
File download	api-download	download.file, curl::curl_download, curl::curl_fetch_memory, curl::curl_fetch_disk	api-urlretrieve	urllib.request.urlretrieve

The `message` and `suggest` fields are what make a violation actionable. A caught `read_csv` does not just say “rule io-read-csv matched”; it says *Direct CSV read* and points to `dw_use(name = ..., sector = ..., kind = ...)`. The auditor’s job is not only to catch the bypass but to name the wrapper that should replace it.

14.2.1 Files and directories the auditor skips

The toolkit’s own implementation files *must* call the raw commands — that is their whole purpose — so they are excluded by default through `ignore_files` (`dw_io`, `dw_api`, the sync helper, the profile helpers, and `test_scripts` itself; the Python list also skips `_state.py` and `__init__.py`). Common machinery directories are excluded through `ignore_dirs`: `.git`, the R dependency caches (`renv`, `packrat`), and the Python equivalents (`.venv`, `venv`, `__pycache__`, `.tox`, `.pytest_cache`), among others. When you point the auditor at a sector folder you are scanning *analysis* code, not library code or vendored dependencies.

14.2.2 Extending the rule set

Both implementations accept `custom_rules`, merged with the built-in registry. A consumer repo that wraps an additional data source can add a rule for it without editing the toolkit. Each custom rule must supply the same five fields.

14.2.2.1 R

```
my_rules <- list(
  list(id      = "api-mysource",
       family  = "api",
       pattern = "(?

```

14.2.2.2 Python

```
from cso_toolkit.test_scripts import _Rule, test_scripts

my_rules = [
    _Rule(
        id="api-mysource", family="api",
        pattern=r"\bmysource_fetch\s*\(",
        message="Direct mysource_fetch call bypasses the cache.",
        suggest="dw_api_fetch(api='mysource', cache_key=...)",
    )
]
test_scripts("01_dw_prep", custom_rules=my_rules)
```

14.2.2.3 Stata

```
* No auditor in Stata; custom rules are an R / Python concept.
```

14.3 Reporting and the build gate

With the default `verbose = TRUE`, the auditor prints a fixed-format block: a header rule, the file count, the violation count, then one stanza per violation showing the rule id, the file and line, the message, the offending snippet, and the suggested replacement. (The printed block shows every column except `family`, which only governs the build gate.) A clean run prints `[OK] Clean..` The format is identical in both languages, by design, so a reader who has seen one has seen both.

```
cso-toolkit contract audit
-----
Files scanned : 14
Violations    : 1

[io-read-csv] 01_dw_prep/012_codes/nt/build.R:42
  Direct CSV read.
  > df <- read_csv("raw/nt.csv")
  -> dw_use(name = ..., sector = ..., kind = ...)
-----
```

By default the function only *reports* — it returns the violations frame and lets you decide. The CI gate is `error_on_violation`. When it is `TRUE` **and** at least one violation belongs to the `io` or `api` family, the function raises after printing, with the toolkit's standard three-part error envelope (the `[cso_toolkit.test_scripts]` prefix, the count of violations and

offending files, and a `Fix:` line). In R this is a `stop(call. = FALSE)`; in Python it is a `RuntimeError`. Either way the process exits non-zero, which is exactly what a CI runner needs to mark the step failed.

14.3.0.1 R

```
# Interactive: report only, inspect the frame.
v <- test_scripts("01_dw_prep/012_codes")

# CI gate: stop (non-zero exit) on any io/api violation.
test_scripts("01_dw_prep/012_codes", error_on_violation = TRUE)
```

14.3.0.2 Python

```
# Interactive: report only, inspect the frame.
v = test_scripts("01_dw_prep/012_codes")

# CI gate: raise RuntimeError (non-zero exit) on any io/api violation.
test_scripts("01_dw_prep/012_codes", error_on_violation=True)
```

14.3.0.3 Stata

```
* No auditor in Stata.
```

The family restriction is deliberate. Only `io` and `api` violations break the build, because those are the two families the contract actually governs. If you add a custom rule under some other family for advisory purposes, it will be reported but will not fail CI.

i In R, `test_scripts` returns invisibly

The R function returns its violations frame *invisibly*, so assign it (`v <- test_scripts(...)`) when you want to inspect it; a bare call at the console prints the summary but not the frame. The R alias `dw_test_scripts()` is identical — it exists so the name carries the `dw_` family prefix, and a future release may rename the canonical function to `dw_audit_scripts` to make clear that this audits call sites and is *not* a unit-test runner despite the `test_` prefix. The Python function returns its frame normally.

14.4 Managing false positives with `# cso-allow`

Because the auditor is lexical, it occasionally flags a call that is genuinely correct. The canonical example is the profile loader: every CSO profile reads its own YAML config with a raw `read_yaml` / `yaml.safe_load`, and that read is *supposed* to be raw — it bootstraps the configuration before any toolkit path resolution exists. Forcing it through `dw_use` would be circular. The toolkit’s own `profile_helpers.py` carries exactly such an allow on its `yaml.safe_load` config read.

The escape hatch is a trailing comment, `# cso-allow: <rule-id>`, on the offending line. The auditor reads the comment, parses the rule ids out of it, and skips exactly those rules for that one line. Every other rule still applies to the line, and every other line is still scanned. You can silence several rules at once by listing their ids comma-separated.

14.4.0.1 R

```
# Profile bootstrap - the one legitimate raw YAML read:
config <- yaml::read_yaml(config_path) # cso-allow: io-yaml

# Silence two rules on one line:
x <- read_csv(p) # cso-allow: io-read-csv, io-write-csv
```

14.4.0.2 Python

```
# Profile bootstrap - the one legitimate raw YAML read:
config = yaml.safe_load(open(config_path)) # cso-allow: io-yaml

# Silence two rules on one line:
x = pd.read_csv(p) # cso-allow: io-read-csv, io-to-csv
```

14.4.0.3 Stata

```
* No auditor, hence no escape-hatch comment in Stata.
```

Two discipline points. First, `# cso-allow` is line-scoped and rule-scoped: it never silences a whole file or a whole rule globally, so a stray allow comment cannot quietly disable the contract for a directory. Second, an allow is a documented exception, not a convenience — it should mark a call that genuinely cannot go through a wrapper (the profile bootstrap is essentially the only routine case), and it asks for the reviewer’s attention rather than dodging it. The toolkit’s own `generate_markdown_report.py` uses one such allow, on its descriptive-stats CSV read.

14.5 Wiring the auditor into a GitHub Actions job

Today the toolkit's own repository ships one R workflow, `.github/workflows/r-check.yml`, which runs R CMD `check` across a four-way matrix (Ubuntu release and devel, macOS release, Windows release). That R CMD `check` run exercises the toolkit's test suite, and the suite includes `test-test_scripts.R`, which asserts that the auditor returns a clean frame on conforming code, catches a direct `read_csv`, catches a direct `httr::GET`, honors the `# cso-allow` escape hatch, raises the proper error envelope under `error_on_violation = TRUE`, and raises the envelope on a missing path. CI today proves the *auditor itself* works; it does not yet run the auditor against a consumer's analysis scripts, because the toolkit repository has no consumer scripts to scan.

A consumer repo — DW-Production, UNPD-Population, a datalib analysis project — is where the auditor becomes a contract gate. The pattern is a small dedicated job that installs the toolkit, runs `test_scripts(..., error_on_violation = TRUE)` over the analysis tree, and lets the non-zero exit fail the build. The two language variants:

14.5.0.1 R

```
name: contract-audit
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: r-lib/actions/setup-r@v2
        with:
          use-public-rspm: true
      - name: Install cso-toolkit
        run: Rscript -e 'install.packages("remotes"); remotes::install_github("unicef-drp/')"
      - name: Run contract auditor
        run: |
          Rscript -e 'csotoolkit::test_scripts("01_dw_prep", error_on_violation = TRUE)'
```

14.5.0.2 Python

```

name: contract-audit
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install cso-toolkit
        run: pip install "cso-toolkit @ git+https://github.com/unicef-drp/cso-toolkit#subd
      - name: Run contract auditor
        run: |
          python -c "from cso_toolkit import test_scripts; test_scripts('01_dw_prep', erro

```

14.5.0.3 Stata

```

# No auditor in Stata; there is no contract-audit job to add for
# a Stata-only consumer. Enforce the contract by code review.

```

The R package installs under the name `csotoolkit` (note: no hyphen, no underscore), so the call is `csotoolkit::test_scripts(...)`. The Python distribution is named `cso-toolkit` but imports as `cso_toolkit`, so the install line uses the hyphen and the call uses the underscore.

Version skew between the two packages

As of writing, `python/pyproject.toml` and `r/DESCRIPTION` both declare 0.6.0. A matching version is not a promise of rule-set parity, though: if it matters to you, pin the version you install and check that the Python registry carries the rules your contract relies on, rather than assuming the two languages catch exactly the same calls. The [versioning-and-releases](#) chapter explains how the two version lines are reconciled.

A few practical notes, each worth keeping distinct:

- **Keep the audit job separate** from the test or build job, so a contract violation produces a clearly labelled red check rather than hiding inside a larger failure.

- **No credentials needed.** The job needs neither network access nor a database connection: the auditor only reads source text, so it runs fast and is safe to trigger on every pull request.
- **Scope the path tightly.** Point it at the directory that actually holds analysis scripts (often the `sector` or `prep` folder), not the repository root, so vendored code and generated files stay out of scope on top of the built-in `ignore_dirs`.
- **The gate fires narrowly.** Because `error_on_violation` fires only on the `io` and `api` families, the build fails on exactly the bypasses the contract forbids and nothing else.

For richer feedback than a pass/fail, capture the returned frame before the gate: run the auditor once with `error_on_violation = FALSE` to write the violations frame to an artifact (a CSV, or a Markdown table built the same way `generate_markdown_report` builds its tables), then run the gate. The first run gives a reviewer the full list with file, line, message, and suggested fix; the second turns the build red.

14.6 How this fits the rest of the contract

The auditor is the enforcement arm of the contract the IO and external-API chapters define and the vendoring chapter distributes. It does not replace human review — a lexical scan cannot judge whether a `dw_save` call uses the right `sector` or `isid` — but it removes the most common and most mechanical failure: a raw call that should never have been written by hand. Pair it with `review_profile()` from the scaffolding chapter, which audits a repo’s *profile* for the standard building blocks, and the two together cover both halves of conformance: `review_profile` checks that the repo is wired to the contract, and `test_scripts` checks that the analysis code honors it. The versioning-and-releases chapter explains how the rule set itself evolves across toolkit versions — adding a rule can newly fail a previously green build, so rule-set changes follow the same semantic-versioning discipline as the rest of the API.

15 Versioning and releases

The toolkit makes a promise: a 2026 analytics product that cites `dw_<sector>.csv` must, two years on, re-run to identical numbers from the canonical deposit (the immutable data file the product cites, deposited to the Teams folder and its Z: mirror) and the vendored helper code (the helper files copied into the consumer’s own project) as it stood at release time. That promise holds only if every consumer can point at one exact, immutable version of the toolkit. Versions must therefore be meaningful, immutable, and documented.

Two roles run through this chapter, and the whole reproducibility argument rests on the distinction between them. A **producer** (the Database Manager) runs a sector pipeline, pulls live APIs, and deposits canonical artefacts; a **reviewer** re-runs the pipeline from those frozen artefacts to reproduce the numbers and is physically prevented from touching the network. Mode is a session property, not a per-call argument (see the roles and mode-contract chapter). The producer chooses when to refresh the toolkit; the reviewer always runs against whatever vintage the producer has pinned. A **consumer** is a downstream analytics project that vendors the toolkit; the three **siblings** are the R, Python, and Stata implementations of the same contract.

A handful of public verbs recur below. `dw_use()` reads a deposit, `dw_save()` writes one, and `dw_publish()` submits an artefact to the Helix registry (the `dw_save` / `dw_publish` split — filesystem versus API — is the subject of the IO and external-API contract chapters). The `cso_toolkit_*` family is different in kind: it manages versions of the toolkit *itself* — checking, diffing, and pulling the vendored copy.

15.1 Semantic versioning

The toolkit follows Semantic Versioning — MAJOR.MINOR.PATCH. The three components carry their conventional meanings, interpreted against the toolkit’s public surface: the exported function names, their signatures, the `.provenance.json` sidecar schema (see the provenance sidecars chapter), and the error-envelope shape — the structure of the object a failing call returns (see the error taxonomy appendix).

Component	Bump when	Example from history
MAJOR	A change breaks source compatibility for an existing consumer — a removed function, a renamed or reordered required argument, a default that flips in a way that changes results, a sidecar-schema change a reviewer would notice. While the toolkit is pre-1.0, breaking changes are still permitted within the 0.x line (see “The road to v1.0.0”).	The 0.x line has shipped breaking changes inside MINOR bumps. v0.4.0 flipped <code>dw_save(overwrite = ...)</code> from TRUE to FALSE, explicitly tagged BREAKING — a change a stable 1.x line would have made a MAJOR.
MINOR	New capability, backward-compatible: a new exported function, a new optional (trailing) argument, a new <code>api = value</code> , or behaviour that broadens what the function accepts without breaking existing calls.	v0.4.10 added the <code>"total"</code> aggregation method and the toolkit-wide <code>verbose / debug</code> convention — all new, trailing, optional arguments. v0.5.0 added <code>dw_use(sheet = NULL)</code> to read every sheet of a workbook while leaving the <code>sheet = 1</code> default untouched.
PATCH	A bug fix or correctness change with no public-API change. The signature is unchanged; the fix makes the function do what it already promised.	v0.4.2 fixed <code>dw_save(dialect = "base")</code> silently writing comma-separated content to a <code>.tsv</code> file. The signature did not move; callers passing <code>.tsv / .txt</code> with <code>dialect = "base"</code> got correct tab output instead of the silent comma output that was the bug.

The toolkit has also used a **four-segment** patch (v0.4.3.1) to fix a freshly cut release the same day. v0.4.3 had bumped `r/DESCRIPTION`'s `Version` and `NEWS.md` but missed three hard-coded version stamps inside `r/R/dw_io.R`: the header banner, the `dw_toolkit_version()` docstring, and its return value. Because `dw_toolkit_version()` is stamped into provenance sidecars, the stale "0.4.2" return would have polluted `dw_publish()` provenance with the wrong toolkit version. The .1 segment corrected the

stamps without claiming a new behavioural release.

i Note

The pre-1.0 0.x line is deliberately permissive about breaking changes: under SemVer, anything goes below 1.0.0, and the toolkit has used that latitude to tighten the mode contract (v0.4.0) while the consumer set was small and well-known. The compatibility promise hardens at v1.0.0. Until then, **migration notes in NEWS.md** — not the version number alone — are how a consumer learns a release will require code changes.

15.1.1 What “public API” means across the three languages

A change is breaking only if it breaks a *documented, exported* entry point. The cross-language coverage matrix appendix is the authoritative list, but the rule of thumb is symmetric across siblings: the exported `dw_*` functions, their required arguments, the sidecar schema, and the envelope shape. Internal helpers (the `.dw_*` family in R) are not part of the contract and can change in a PATCH.

The siblings are versioned **in lock-step under a single tag** — there is one v0.5.0, not an `r-v0.5.0` and a `py-v0.5.0`. A release may, however, advance one sibling’s *coverage* while leaving another untouched. v0.4.0 shipped `dw_pop()` and `dw_regions()` **R-only**, with Python and Stata parity tracked at the same GitHub issues for a future minor. That is a MINOR bump — new capability, nothing broken — even though parity is incomplete. The honest reading of the version number is “the contract gained a function in at least one language,” not “every language gained it.” Consumers must consult the coverage matrix, not the tag, to learn whether *their* language has a given helper.

15.2 The NEWS.md discipline

The toolkit keeps a single changelog, `NEWS.md`, following the R-ecosystem convention. Per-release notes — including breaking-change migration notes and per-PR provenance — live there; there is no separate `CHANGELOG.md`. There is exactly one changelog, and a second must not be added.

15.2.1 The Unreleased → released lifecycle

`NEWS.md` opens with an `## Unreleased` section whose own header documents the discipline:

```
## Unreleased
```

```
_Entries land here as PRs merge into `develop`. When the next release  
is cut, this header is renamed `## vX.Y.Z (YYYY-MM-DD)` and a fresh
```

`## Unreleased` section is added back._

The flow follows three steps:

1. A PR merges into `develop` and adds its entry under `## Unreleased`.
2. Entries accumulate as more PRs land in the cycle.
3. At release time, `## Unreleased` is renamed to `## vX.Y.Z (YYYY-MM-DD)` and a fresh empty `## Unreleased` is opened above it.

This makes the changelog **per-PR** rather than reconstructed per-release as an afterthought. Each entry is written at the moment the change is understood, by the person who made it, not pieced together from `git log` weeks later.

15.2.2 What a good entry contains

The historical entries set a high bar. A strong `NEWS.md` entry carries, in order:

- **A bold headline** naming the user-visible change — for example, “**`dw_save()` confirms before overwriting a producer deposit.**”
- **Per-PR / per-issue provenance** — the issue and PR numbers, linked, so a reader can trace the change to its discussion. Most entries cite both (for example, issue #25 and PR #75).
- **The empirical origin**, where there is one. Many fixes were surfaced by a specific audit run, and the entry says so: the `v0.4.3` parquet fix records that it was “surfaced empirically by DW-Production Run #6 (NT pipeline): 13+ stages downstream ... failed with ‘object COLUMN not found’.” This is provenance for *why the bug mattered*, not merely that it existed.
- **A Migration: note** whenever a consumer must change code. These are explicit and actionable.

The migration notes are the load-bearing part of the discipline. The `v0.5.0` overwrite-confirmation change states the action a consumer must take:

Migration: non-interactive producer pipelines that re-deposit must add `force = TRUE` (or run interactively).

The `v0.4.3` `cols_lenient` change spelled out the before/after for each affected call form:

Migration:

```
- dw_use(cols = dplyr::any_of(c(...))) -> dw_use(cols = c(...), cols_lenient = TRUE)
- dw_use(cols = dplyr::all_of(c(...))) -> dw_use(cols = c(...)) (strict)
```

And the headline `v0.4.0` breaking change carried a full **Migration guide** offering two routes — pass `overwrite = TRUE` for daily re-runs of the same vintage, or sequence writes under a fresh vintage subfolder for archival work — plus an explicit “Reviewer-mode callers

do not need changes” reassurance, so a consumer can tell at a glance whether the change touches them.

Warning

A breaking change with no migration note is a contract violation, not a tidiness lapse. The version number tells a consumer *that* something broke; the migration note is the only thing that tells them *how to fix it*. Every entry flagged **BREAKING** in the history is paired with a migration note — keep it that way.

15.2.3 Entries are language-agnostic but coverage-honest

Because the siblings share one tag, one NEWS.md covers all three. Entries name which languages a change touches when coverage is uneven — “(R only)”, “R + Python siblings ship in lock-step”, “Stata-side `dw_api_fetch` and Parquet / RDS read support remain out of scope by design.” This keeps the changelog from implying parity the code does not have — the same honesty the coverage matrix appendix enforces.

15.3 Tags and the release cut

Releases are Git tags of the form `vX.Y.Z` (and `vX.Y.Z-rcN` for release candidates). The first cut, `v0.1.0-rc1`, followed the workspace convention of tagging a release candidate before the real release so the untested release flow could be exercised reversibly; `v0.1.0` was tagged only once the candidate’s `cso_toolkit_pull()` flow smoke-tested cleanly. The README’s version table is the human-readable index of tags, each row dated and summarised.

15.3.1 The version is stamped in more than one place

A version stamp that disagrees with the tag silently mislabels the provenance of every artefact written under that release, because `dw_toolkit_version()` is recorded into `.provenance.json` sidecars and into `dw_publish()` payloads. A release cut must therefore keep several stamps in agreement; they are read by different consumers:

<code>r/DESCRIPTION</code>	<code>Version: 0.5.0</code>	<code># R package metadata, R CMD check</code>
<code>r/R/dw_io.R</code>	<code># Toolkit version: 0.5.0</code>	<code># header banner</code>
<code>r/R/dw_io.R</code>	<code>dw_toolkit_version() -> "0.5.0"</code>	<code># docstring + return value</code>
<code>NEWS.md</code>	<code>## v0.5.0 (2026-06-21)</code>	<code># changelog header</code>
<code>README.md</code>	<code>version table row</code>	
<code>python/pyproject.toml</code>	<code>Python package version</code>	

The `v0.4.3.1` incident — three `dw_io.R` stamps left at `"0.4.2"` while `DESCRIPTION` said `v0.4.3` — produced a concrete release-cut checklist item that should run before every tag:

The release-cut checklist for the next minor (v0.5.0) should include a `grep -rn '0\.[0-9]\.[0-9]' r/R/` step to catch any future stamp drift before tagging.

15.3.2 The tag is immutable; consumers pin to it

A published tag is never moved. Consumers vendor the helper *files* into their own `00_functions/` and record the tag in `00_functions/.toolkit_manifest.yml` (see the vendoring model chapter). The manifest's `pulled_version` field is the consumer's pin:

```
source: "unicef-drp/cso-toolkit"
pulled_version: "v0.4.0"
pulled_date: "2026-05-26"
pulled_by: "<your-name>"
```

`cso_toolkit_check()` reads this file, queries the GitHub API for the latest tag on `source:`, and warns when the consumer is behind. Refreshing to a newer tag is a **producer-controlled, explicit** act: the producer reads the release notes, decides the new vintage is appropriate, and runs `cso_toolkit_pull(target_version = "vX.Y.Z")`. Nothing is pulled automatically.

This is what makes a tag's immutability load-bearing. Reviewer mode forbids the network lookup entirely (`cso_toolkit_check()` and `cso_toolkit_pull()` both refuse in reviewer mode), so a reviewer always runs against whatever vintage the producer has pinned, never a silently newer one. The whole reproducibility argument rests on a pinned tag meaning the same bytes forever.

The check / pull surface is itself part of the contract, but it is **R-first**: the vintage-sync helpers (`cso_toolkit_check` / `cso_toolkit_diff` / `cso_toolkit_pull`) are exercised through the R sibling, and `cso_toolkit_pull()`'s file-refresh body is still being finished. A Stata consumer has no automated check or pull at all and edits `.toolkit_manifest.yml` by hand.

15.3.2.1 R

```
# Producer-mode: is the pinned vintage behind upstream?
cso_toolkit_check()

# Refresh the vendored files to a chosen tag (explicit, producer-only)
cso_toolkit_pull(target_version = "v0.5.0")
```

15.3.2.2 Python

```
# Coverage gap: the cso_toolkit_* vintage-sync helpers are R-first.
# A Python consumer pins .toolkit_manifest.yml and refreshes the
# vendored python/src/ files by hand, then records the new tag in
# pulled_version. See the vintage sync chapter and the coverage
# matrix appendix.
```

15.3.2.3 Stata

```
* Coverage gap: no Stata sibling of the vintage-sync helpers ships.
* Stata consumers pin .toolkit_manifest.yml by hand and refresh the
* vendored .ado files manually. See the vintage sync chapter and the
* coverage matrix appendix.
```

15.4 The road to v1.0.0

The toolkit is deliberately still in its 0.x line. The README’s version table records the gate for v1.0.0 precisely:

v1.0.0 — committed API — After the **ed** sector pilot lands and a second sector vendors the helpers without modification.

Two conditions must both hold.

1. **The education (ed) sector pilot lands.** The education sector is the first-party proving ground for the full contract.
2. **A second sector vendors the helpers without modification.** This is the real test. A second, independent consumer must be able to adopt the toolkit *as published* — with an empty `local_edits: []` in its manifest — rather than carrying local patches.

The second condition is exacting on purpose. Much of the 0.4.x history is fixes that were *first applied as local edits inside DW-Production* and only later upstreamed: the v0.4.3 `dw_use()` fixes “landed first on the DW-Production vendored copy as `local_edits`; this release lets the next `cso_toolkit_pull(target_version = "v0.4.3")` drop those local edits.” The manifest’s `local_edits:` field exists precisely to record such divergence so that a later pull skips those files — but every entry in it is a small breach of the vendoring model, because it means the published toolkit was not, on its own, sufficient. A clean second adoption — no `local_edits` — is the empirical signal that the contract has stabilised enough to commit to. The candidate downstream consumers named in the strategy docs are other repositories in the same programme: `unicef-drp/UNPD-Population` and `unicef-drp/datalib-dev`.

One marquee prerequisite is still outstanding. v0.4.0 shipped `dw_publish()` as a deliberate dry-run-only STUB so sector scripts could wire the canonical call site early

and have the live branch “light up automatically when v0.5.0 lands.” That live branch has **not** landed yet: at the current v0.5.0 HEAD, `r/R/dw_publish.R` still raises `[cso_toolkit.dw_publish] Live submission not yet implemented` on any `dry_run = FALSE` call, and its own banner still describes the live Helix submission as “planned v0.5.0.” The v0.5.0 release instead shipped read- and write-side refinements — `dw_use(sheet = NULL)`, `dw_is_canonical()` Z: recognition, a `dw_compare()` degenerate-sides fix, and the `dw_save()` overwrite-confirmation change above. So what remains before 1.0.0 is both the two-part adoption gate — pilot in, second consumer clean — and the live `dw_publish()` submission branch that the dry-run STUB is still standing in for.

Once v1.0.0 is cut, the SemVer contract hardens: breaking changes can no longer ride inside MINOR bumps as they have through the 0.4.x line; they require a MAJOR bump and a migration note. Until then, read the migration notes, not just the digits.

Part V

Part V — The Microdata Archive (datalib)

16 The microdata archive: archive, catalogue, tool

A survey costs millions of dollars and several years to field. Re-analysing the microdata it produces costs an afternoon. That asymmetry is the whole argument for this part of the book. The household- and person-level records that every published indicator ultimately rests on are the most expensive and least reproducible asset the office holds — and, in practice, they are often managed with *less* rigour than the code that reads them. A pipeline gets a git history, a manifest, and a provenance sidecar; the raw survey it consumes gets a folder on a shared drive whose name means something only to the person who created it. Part V documents **datalib**, the tool that extends the reproducibility floor — the guarantee, set out in the why-a-toolkit chapter, that a 2026 analysis must re-run identically in 2028 — from the aggregate outputs down to the survey microdata those outputs are built from.

This opening chapter introduces the whole of Part V: the boundary between the microdata archive and the indicator warehouse, the three-layer picture (archive, catalogue, tool) that frames the initiative, the stock-and-flow approach to deposits, the deposit-gate-promote governance, and the access and licensing floor. The IHSN-grammar chapter that follows takes up the folder convention in detail; the datalib API and conformance chapter takes up the cross-language contract and its oracle test suite.

Scope today is deliberately narrow. The archive covers **household surveys for global child monitoring** (MICS, DHS, LSMS, including the MICS Harmonized Database) *now*; other survey types *next*; anonymized public-use microdata *later*. Nothing in Part V assumes more than that.

i Version and source of authority

This part documents datalib **contract v1** as shipped in **datalib v0.7.0**, and the governance model at **draft v0.3**; `unicef-drp/datalib-unicef` is authoritative — `config/grammar.md` for the contract, `docs/GOVERNANCE_microdata_deposit.md` for governance.

i Note

Access: internal — request via the CSO team (repository visibility is tracked in datalib issue #20).

16.1 Why a microdata archive

The reproducibility floor is a guarantee you inherit rather than a feature you remember to switch on. For the aggregate warehouse, the toolkit builds that floor from provenance sidecars, a replayable frozen deposit, and uniqueness checks. For survey microdata the failure modes are different, but the promise is the same: a number computed from a 2019 survey today must be computable from the *same* records, read from the *same* place, by a different analyst in two years.

Three things routinely break that promise for microdata specifically.

- **Silent divergence of “the same” survey.** Two teams each hold a copy of the same MICS round. One recoded a variable; the other dropped a module. Neither copy records what was done, and there is no shared name that would even reveal the two are meant to be identical. When their indicators disagree, there is nothing to point at.
- **Folder names that carry no contract.** A directory called `Tajikistan final v2 USE THIS` tells a script nothing. Which country code, which year, which survey, which vintage? The information a loader needs to resolve a file deterministically is exactly the information a hand-typed folder name tends to omit.
- **Transformations with no reverse.** The deposited `.dta` is three recodes downstream of the original, and the recodes live in a script no one can find. The output cannot be reproduced because its inputs cannot be reconstructed.

datalib’s answer is to make the *shelving convention itself* executable. A survey’s country, year, survey instrument, and vintage are encoded in a machine-parseable folder name (the IHSN grammar), the untouched source is preserved alongside every derived form, and one library — in R, Python, and Stata — resolves, loads, and merges by reading that convention rather than trusting a human to remember it. Provenance is recorded *structurally*: the vintage is in the folder name and `Data/Original/` is never mutated. That is the same intent as the toolkit’s `.provenance.json` sidecar, reached by a different mechanism; the provenance-schema appendix carries only a short pointer to this difference, not a second sidecar specification.

16.2 Two data layers, one office

The single most important thing to hold clear before reading the rest of Part V is that datalib is **not** part of the data warehouse. The CSO office runs two data layers under one roof. They share a team and an operator config file; they do not share a store, a contract, or a grain.

Dimension	Aggregate / indicator warehouse	Survey microdata archive
Contract prefix	<code>dw_*</code>	<code>datalib_*</code>

Dimension	Aggregate / indicator warehouse	Survey microdata archive
Canonical store	Teams 060.DW-MASTER (+ its Z: mirror)	Z:/datalib* trees + a staging area
What lives there	Deposited indicator files (aggregates)	Survey vintages (household / person records)
Grain	one row per indicator $\times$ unit $\times$ time	one row per household or per person
Documented in	Parts I–IV of this book	Part V (this part)
Team	CSO — Data & Analytics (OSE)	the same team
Operator config	~/.config/user_config.yml	the same file

State it plainly: **microdata deposits NEVER enter 060.DW-MASTER, and indicator deposits never land in the Z:/datalib* trees.** The warehouse holds the aggregates a producer blesses and a publisher pushes downstream to Helix and the public pages; the archive holds the survey records those aggregates were computed from. Same office, two contracts, two stores, different data layer. Everything in Part V lives on the microdata side of that line, and the boundary is referenced again in both the IHSN-grammar chapter and the datalib API and conformance chapter.

The one genuinely shared thing is the operator config file. Both contracts are designed to key off the single `~/.config/user_config.yml`, datalib reading a `datalib:` block from its user section for the library root. As the configuration-across-tiers discussion in the architecture chapter notes, the *toolkit* side of that shared-file coexistence is still gated on a parser guard, so the `datalib:` block is not yet added to the toolkit's own `user_config.yml`; treat the two contracts as separate namespaces that happen to share a file.

16.3 Archive, catalogue, tool

The governance draft frames the initiative as three distinct layers, and confusing them is the most common way to misunderstand the project. datalib the software is only one of the three.

Layer	What it is	What it is not	Status
Microdata archive	the governed store of the data files themselves — the Z:/datalib* trees, the staging area, and the deposit process that fills them	not a website; not a metadata index	this initiative (all of Part V)

Layer	What it is	What it is not	Status
Microdata catalogue	the metadata and discovery layer (IHSN / NADA tooling over a DDI model)	not the files themselves	a Phase-2+ product, seeded by the tool's auto-generated inventories
Tool (datalib)	software in R, Python, and Stata that encodes the shelving convention as executable code	neither the store nor the catalogue	shipped: contract v1 in datalib v0.7.0

The one-liner is worth memorising: **an archive without a catalogue is unfindable; a catalogue without an archive points at silos; both without a tool are manual drudgery.** The three depend on each other, and datalib is the leg that exists today — the executable convention that also produces the inventories the future catalogue will index.

16.4 Stock and flow

The archive would never fill if conformance were a precondition for deposit — teams hold years of surveys under their own naming, and demanding they rename everything first guarantees they deposit nothing. So the governance draws a sharp line between what already exists and what is created from now on, and treats the two differently.

	Stock — existing holdings	Flow — new work
Rule of thumb	“copy it as it is, don’t touch it”	“name it right from day one”
Naming	preserved <i>exactly</i> as-is, folder structure intact so existing scripts keep running	follows the datalib IHSN grammar from the start
Destination	Z:/staging-area/<domain>/<name of data as-is>	same folder as the id main’s canonical tree as masters / adaptations
Conformance	never a precondition; progressive, survey by survey	required at deposit

Depositing stock as-is is not a compromise; it is the point. Preserving the original folder unlocks a four-phase payoff, each phase enabled by the one before:

1. **Preserve and make portable** — get the survey off a personal drive and into a governed, backed-up store without breaking the scripts that already read it.

2. **Compare inputs across teams** — once two teams’ copies of the same survey sit side by side, their differences become visible for the first time.
3. **Reverse-engineer transformations** — with the original preserved next to each derived form, the recodes between them can be reconstructed.
4. **Consolidate** — converge on one authoritative file per survey.

Conformance, in other words, is something the archive *arrives at* survey by survey, not a gate that keeps surveys out.

16.5 Deposit, gate, promote

A deposit is not simply a file copy. It passes a seven-item review gate before anything is promoted from staging into a canonical tree. The gate is summarised here; the deposit quickstart and the governance draft in the datalib repository (`docs/GOVERNANCE_microdata_deposit.md`) carry the operative detail, and this chapter points to them rather than reproducing them.

1. **Inventory logged** — the deposit is recorded.
2. **Naming conforms** — the IHSN grammar is checked (flow only; stock is exempt).
3. **Reproducibility** — a non-author can re-run the deposit’s script from staging.
4. **No breakage outside its staging folder** — the deposit touches nothing else.
5. **Licence recorded** — MICS as country-owned, DHS per-project, IPUMS per its conditions.
6. **Access tier assigned** — one of the tiers below.
7. **De-identification confirmed** — de-identified microdata only.

The division of labour mirrors the toolkit’s producer → reviewer → publisher relay, mapped onto an organisational standard rather than a session-level mode: the **Core Group decides** the gate outcome, and the **Data Steward operates** the checks and the move. A pass lets the Data Steward promote the conformed vintages into the domain’s canonical tree — **never into 060.DW-MASTER**.

Microdata role	Verb	Responsibility	Toolkit analogue
Deposit focal (one per unit)	deposits	assembles and lodges the unit’s deposit into staging	PRODUCER
Microdata Core Group (deposit focals + Data Steward as chair)	decides	rules on each gate outcome; converges transformations	REVIEWER (the compare-and-bless step)
Data Steward (+ deputy)	operates / promotes	runs gate checks, executes the assisted move, promotes conformed vintages, keeps logs	REVIEWER’s operational half + PUBLISHER

Microdata role	Verb	Responsibility	Toolkit analogue
Standards Board (all Unit Chiefs)	ratifies	ratify the standard, the domain registry, promotion-to- production	— (governance above the three runners)
Process Lead (a Unit Chief; currently Yves Jaques)	chairs / unblocks	chairs the Standards Board, sets cadence, unblocks	—
Chief Statistician	accountable	ratifies the standard and resourcing; final arbiter; delegates process guidance	—
Unit Chiefs	own	own their unit's deposits and access; name the focal	—

The toolkit's three session-level runners (see the roles-and-mode-contract chapter) need no ratifying layer above them because the toolkit's governance lives elsewhere; the microdata standard, being an office-wide convention, carries that extra scaffolding of Board, Process Lead, and accountable owner.

i Note

Promotion targets a **thematic domain** tree, not one global folder: health → `Z:/datalib-hlt` (HLT), education → `Z:/datalib-edu` (EDU), nutrition → `Z:/datalib-nut` (NUT), child-protection → `Z:/datalib-chp` (CHP), cross-cutting → `Z:/datalib`. A *domain* is the organisational grouping of holdings; a *collection* is the harmonization code in the folder grammar (the IHSN-grammar chapter covers collections). The two are aligned but not identical — and as of v0.7.0 / contract v1 only HLT (and IPUMS) is registered in datalib's merge registry; EDU, NUT, and CHP become first-class as their module lists and keys are verified in later phases.

16.6 Access and licensing

Deposit does **not** re-license anything: the producer's terms travel with the files, and the archive's job is to record and honour them, not to widen them. Every survey is assigned one of three access tiers at the gate, on a least-privilege default (a staging folder is readable by the depositing unit, the Data Steward, and the Core Group; wider access is granted per survey by the depositing Unit Chief).

Tier	Meaning
Open-internal	shareable within UNICEF

Tier	Meaning
Licensed	use bounded by the producer's licence terms
Restricted	tightest access; named users only

A fourth, openly-shareable tier arrives only with anonymized public-use files, which are a later-scope concern. Two floors hold across all tiers. First, licensing is producer-defined: MICS microdata is **country-owned**, DHS is licensed **per project**, IPUMS travels under **its own conditions** — the archive records the tier but does not grant beyond it. Second, **only de-identified microdata is admitted**: no direct identifiers, and no geography below the approved level. Finally, remember that **staging is a waypoint, not an archive** — a deposit is reviewed within a quarter and retired after it is either promoted or turned down. Nothing is meant to live in `Z:/staging-area/` indefinitely.

 Governance is a draft, not a settled standard

The governance summarised in this part is **draft v0.3 and pre-ratification**. Appointments are partly TBD — the Data Steward, in particular, is not yet named. Four decisions are open before the Standards Board and should not be presented as resolved:

1. **Canonical topology** — one library root versus per-domain roots (`Z:/datalib`, `Z:/datalib-hlt`, `Z:/datalib-edu`, ...); to be decided before the first promotion.
2. **Domain registry** — confirmation of the thematic-domain list.
3. **Access-tier defaults** — the default tier and its per-survey overrides.
4. **MICS Harmonized Database deposit route** — reference-in-place versus copy into staging.

Read Part V as documentation of a working tool and a proposed governance model, not as a ratified office standard. Where the governance draft and this chapter differ, the draft in `unicef-drp/datalib-unicef (docs/GOVERNANCE_microdata_deposit.md)` is authoritative.

17 The IHSN convention and the folder grammar

This is the second chapter of Part V, and it takes up the one idea the whole microdata archive rests on: a survey’s provenance lives in the *path*. The reproducibility mission the rest of this book serves — a 2026 number must be recomputable from the same records, read from the same place, by a different analyst in 2028 — has an aggregate half kept by the toolkit’s `.provenance.json` sidecar (see the provenance sidecars chapter) and a microdata half kept here, by a different mechanism entirely. On the aggregate side, a receipt is written *beside* each output. On the microdata side, the receipt *is the shelving*: the country, the year, the survey instrument, and the vintage are all encoded in a machine-parseable folder name that follows the IHSN survey-cataloguing convention, and the untouched producer files are preserved under `Data/Original/`. Nothing is stamped onto the data; where a file sits, and what its folder is called, is the record. This chapter documents that grammar — the token positions, the vintage rules, the collection registry, the seven shared semantics every language implementation honours, and the deployment topologies the estate runs today.

The boundary the microdata-archive chapter draws governs this chapter as it governs the rest of Part V. The toolkit’s `dw_*` contract serves the **aggregate / indicator warehouse** — the canonical `060.DW-MASTER` Teams deposit. `datalib`’s `datalib_*` contract serves survey **microdata** — the `Z:/datalib*` trees plus a staging area. Same office, same `~/.config/user_config.yml` file, a **different data layer**; a microdata deposit never enters `060.DW-MASTER`. Everything below is about how the microdata side names things so that a loader can find them without trusting a human to remember.

i Version and source of authority

This part documents `datalib contract v1` as shipped in **`datalib v0.7.0`**, and the governance model at **`draft v0.3`**; `unicef-drp/datalib-unicef` is authoritative — `config/grammar.md` for the contract, `docs/GOVERNANCE_microdata_deposit.md` for governance.

i Note

Access: internal — request via the CSO team (repository visibility is tracked in `datalib` issue #20).

17.1 Naming as structural provenance

The toolkit and datalib solve the same problem in opposite directions. When the toolkit writes an aggregate through `dw_save`, it leaves the data file's bytes alone and writes a *separate* JSON receipt — `<path>.provenance.json` — that records who wrote the file, when, in which session mode, and with what content hash (see the provenance sidecars chapter for the full schema). The provenance is a companion artefact; strip the sidecar and the data file no longer knows where it came from.

datalib inverts that. There is no sidecar, because there is nothing to add: the provenance is already carried by the folder name and by the discipline of never mutating the original. Read `TJK_2009_TLSS/TJK_2009_TLSS_v01_M/` and you already know the country (Tajikistan), the year (2009), the survey (the Tajikistan Living Standards Survey), and the master vintage (the first) — without opening a single file. Descend into `Data/Original/` and you find the producer's files exactly as they were deposited, byte-for-byte, so that the derived forms under `Data/Stata/`, `Data/R/`, and `Data/Other/` can always be checked back against the source they were built from. The vintage is in the path; the source is preserved; the two together *are* the receipt. This is **structural provenance** — the microdata counterpart of the sidecar, reached by encoding rather than annotation.

The consequence for the appendices is deliberate: the provenance-schema appendix, which specifies the toolkit's sidecar field-by-field, carries only a short pointer to this difference, not a second datalib-flavoured sidecar section. There is no datalib sidecar to specify. The mechanism differs; the intent — a frozen artefact that can be trusted and traced years later — is identical.

Because the provenance is the naming, the naming has to be exact. A folder called `Tajikistan final v2 USE THIS` carries the same information to a human and *no* information to a loader; the grammar below is what lets a machine resolve a file deterministically from a country, a year, a survey, and an optional vintage.

17.2 The folder grammar

The grammar is vendored verbatim from datalib's `config/grammar.md` at the pinned contract version, so that this chapter and the loader can never drift apart:

```
<root>/<CCC>/                                1 token
<root>/<CCC>/<CCC>_<YYYY>_<SSSS>/              3 tokens ("_"-separated)
  master vintage:    <CCC>_<YYYY>_<SSSS>_v<MM>_M    5 tokens
  adaptation vintage: <CCC>_<YYYY>_<SSSS>_v<MM>_M_v<AA>_A_<HHHH> 8 tokens
inside a vintage:    Data/{Original,Stata,R,Other} Doc/ Programs/
data files:          <vintage-folder-name>_<module>.dta  under Data/Stata/
```

Token positions when splitting on `_`: 1 country, 2 year, 3 survey, 4 master vintage (v01), 6 adaptation vintage, 8 collection.

Read the tree from the root down. The **first token** is a country: a directory named by its three-letter code (TJK, ZWE), one token, holding everything for that country. The **survey folder** one level down is three tokens joined by `_` — country, year, survey — so `TJK_2009_TLSS` is Tajikistan’s 2009 Tajikistan Living Standards Survey, and `ZWE_2019_MICS` would be Zimbabwe’s 2019 MICS. Inside a survey folder, each *vintage* is its own directory whose name extends the three-token stem: a **master vintage** appends `_v<MM>_M` for five tokens (`TJK_2009_TLSS_v01_M`), and an **adaptation vintage** appends a further `_v<AA>_A_<HHHH>` for eight (`TJK_2009_TLSS_v01_M_v01_A_HLT`). Every vintage folder holds the same three subtrees — `Data/{Original,Stata,R,Other}`, `Doc/`, and `Programs/` — and each data file under `Data/Stata/` is named for its own vintage folder plus a module suffix, e.g. `TJK_2009_TLSS_v01_M_household.dta`.

The token positions are fixed, and this is what makes the name parseable rather than merely descriptive. Splitting a fully-qualified adaptation name on `_`, position **1** is the country, position **2** the year, position **3** the survey, position **4** the master vintage (`v01`), position **6** the adaptation vintage, and position **8** the collection code. Those positions are the contract: an implementation reads a country from position 1 and a collection from position 8, never by pattern-guessing from the surrounding text.

Mapping the grammar onto a call makes the correspondence concrete. To resolve Zimbabwe’s 2019 MICS in the HLT collection — country in position 1, year in 2, survey in 3, collection in 8 — each language spells the same request its own way:

17.2.0.1 R

```
datalib_resolve("ZWE", 2019, "MICS", collection = "HLT", root = "Z:/datalib")
#> resolves to the vintage folder for ZWE_2019_MICS in the HLT collection
```

17.2.0.2 Python

```
from datalib import datalib_resolve
datalib_resolve("ZWE", 2019, "MICS", collection="HLT", root="Z:/datalib")
```

17.2.0.3 Stata

```
datalib, country(ZWE) year(2019) survey(MICS) collection(HLT)
* with no arguments, `datalib` opens interactive folder navigation instead;
* `datalib help` documents the option set.
```

The arguments are the token positions by another name. What the loader does with them — how it uppercases, how it matches, how it fills the vintage when you omit it — is the subject of the two sections that follow.

17.3 Vintages: masters and adaptations

A survey folder accumulates two kinds of vintage, and the distinction is load-bearing. A **master vintage**, `<stem>_v<MM>_M`, is a cleaned-but-general version of the survey — the office’s canonical rendering of the raw instrument, not yet specialised to any analytical purpose. An **adaptation vintage**, `<stem>_v<MM>_M_v<AA>_A_<HHHH>`, is a harmonisation built *on top of* a specific master for a specific collection: the `v<AA>_A` counts the adaptation, and the trailing `<HHHH>` names the collection it harmonises to (position 8). So `TJK_2009_TLSS_v01_M` is the first master, and `TJK_2009_TLSS_v01_M_v01_A_HLT` is the first HLT-collection adaptation of that first master. Masters descend from the original; adaptations descend from a master.

Vintages are **integers**, and how they are compared is a contract, not an implementation detail. Inputs are liberal — `1`, `01`, `v01`, and `V01` all denote the same vintage — but the internal form is a zero-padded `v%02d`. “Latest” means the **numeric maximum**, never the alphabetical or directory-listing maximum. This is the difference that bites: listed as strings, `v10` sorts *before* `v09`, so a loader that trusted the filesystem’s ordering would silently pick the wrong, older vintage once a survey passed nine revisions. `datalib` compares the integers, so `v10 > v09` and the tenth master wins. The rule sounds pedantic until the tenth vintage exists; then it is the difference between reproducible and wrong.

Because vintages are ordered, they can be defaulted, and the defaulting rules are what let a caller name only what they care about. Omit the **year** and `datalib` resolves the latest year for that country and survey; omit the **master version** and it takes the latest master; omit the **adaptation version** and it takes the latest adaptation *of the requested collection*. Each default is the numeric maximum of its level, and the resolved vintage folder must actually exist on disk — resolution verifies the folder before any load is attempted, so a default can never silently point at nothing.

Enumeration and defaulting are two views of the same ordering. You can ask for the list explicitly, or lean on the default and let the loader take the maximum:

17.3.0.1 R

```
# Enumerate - the returned integers are compared numerically, not as strings
datalib_vintages("ZWE", 2019, "MICS", root = "Z:/datalib")
#> [1] 1 2 10          # latest master is v10, NOT v02
datalib_adaptations("ZWE", 2019, "MICS", collection = "HLT", root = "Z:/datalib")

# Or omit the versions and let resolve default to the numeric-latest of each
datalib_resolve("ZWE", 2019, "MICS", collection = "HLT", root = "Z:/datalib")
```

17.3.0.2 Python

```
from datalib import datalib_vintages, datalib_adaptations, datalib_resolve
datalib_vintages("ZWE", 2019, "MICS", root="Z:/datalib")          # -> [1, 2, 10]
datalib_adaptations("ZWE", 2019, "MICS", collection="HLT", root="Z:/datalib")
datalib_resolve("ZWE", 2019, "MICS", collection="HLT", root="Z:/datalib")
```

17.3.0.3 Stata

```
* Omit year() and the version options; datalib resolves the numeric-latest
* at each level - latest year, then latest master, then latest HLT adaptation:
datalib, country(ZWE) survey(MICS) collection(HLT)
```

17.4 Collections and modules

A **collection** is the harmonisation target named in position 8 of an adaptation folder, and it is more than a label: it fixes *which modules a survey ships*, *what level each module sits at* (one row per household, or one row per person), and *which keys merge them*. That mapping is not hard-coded in each language; it lives in one registry, `config/collections.yml`, vendored here at the pinned contract version:

i The vendored collection registry (`config/collections.yml`, `contract_version 1`)

|

|

**18 vendored from datalib-unicef
config/collections.yml @ v0.7.0 — do
not hand-edit; refresh via
_manifest.yml**

|

|

19 datalib collection registry — THE single source of truth for module lists,

|

|

20 module levels, and merge keys. R and Python consume synced byte-identical

|

|

21 copies (each package's test suite asserts equality). The Stata leg mirrors

|

|

**22 this registry by hand in
stata/src/_/_dlw.ado (no YAML
parser there); edits to**

|

|

**23 this file MUST be mirrored in _dlw.ado
— the shared conformance cases in**

|

|

24 tests/ are the drift guard.

|

|

|

|

26 contract_version: increment when semantics change; every implementation pins it.

contract_version: 1



27 Keys:



**28 modules : the modules a collection
ships (file suffix after the vintage stem)**



**29 level : hh (one row per household) or
person (one row per person)**

|

|

**30 linevar : the module's person-line
variable when it is NOT already**

|

|

**31 called line_number; implementations
normalize it on load**



32 keys_hh : merge keys for hh-level modules



**33 keys_person : merge keys for
person-level modules (after linevar
normalization)**

|

|



**35 Merge semantics (contract):
person-level modules chain 1:1 on
keys_person,**

- 36 then hh-level modules attach m:1 on keys_hh. Keys are validated per module**
- 37 (isid-equivalent); modules whose identifiers are missing/duplicated stop the**
- 38 merge with an error. Loaders never mutate values.**

```
collections: HLT: keys_hh: [svy_id, cluster_id, household_id] keys_person: [svy_id, cluster_id, household_id, line_number] modules: household: level: hh hhmembers: level: person linevar: hh_line_number adult: level: person children: level: person IPUMS: # NOTE: keys unverified against real IPUMS deposits (no IPUMS tree available # at contract time); the per-module key validation guards against error. keys_hh: [svy_id, household_id] keys_person: [svy_id, household_id, line_number] modules: hh: level: hh bh: level: person ch: level: person fs: level: person hl: level: person mn: level: person wm: level: person
```

Rendered as a table, the two registered collections — HLT and IPUMS — and their modules read as follows. This table is authored from the registry above; it mirrors the vendored YAML rather than adding to it.

Collection	Module	Level	line_number source	Merge keys
HLT	household	hh	—	svy_id, cluster_id, household_id
HLT	hhmembers	person	hh_line_number → line_number	svy_id, cluster_id, household_id, line_number
HLT	adult	person	line_number	svy_id, cluster_id, household_id, line_number
HLT	children	person	line_number	svy_id, cluster_id, household_id, line_number
IPUMS	hh	hh	—	svy_id, household_id
IPUMS	bh	person	line_number	svy_id, household_id, line_number
IPUMS	ch	person	line_number	svy_id, household_id, line_number
IPUMS	fs	person	line_number	svy_id, household_id, line_number
IPUMS	hl	person	line_number	svy_id, household_id, line_number
IPUMS	mn	person	line_number	svy_id, household_id, line_number
IPUMS	wm	person	line_number	svy_id, household_id, line_number

The registry is a single source of truth with a hand-mirrored leg. R and Python consume **byte-identical synced copies** of `collections.yml`, and each package’s test suite asserts that its copy equals the source. Stata has no YAML parser in the relevant path, so it mirrors the same registry **by hand in `_dlw.ado`**; a change to the registry that is not reflected in the Stata leg is a bug, and the shared conformance cases (see the `datalib` API and conformance chapter) are the drift guard that catches it. The `contract_version: 1` field is the pin every implementation carries, incremented only when the semantics change.

Two caveats belong on this table. First, the **IPUMS keys are unverified**: at contract time there was no real IPUMS tree to validate against, so the `keys_hh/keys_person` shown for IPUMS are the registry’s best statement rather than a checked fact. What keeps that honest at run time is the per-module key validation described in the semantics below — every module’s keys are checked for uniqueness on load, so an incorrect key stops the merge with a typed error rather than producing a silently wrong join. Second, **collection is not the same thing as domain**. A *domain* is the organisational grouping of holdings that governance uses to route deposits into thematic trees — health → `Z:/datalib-hlt`, education → `Z:/datalib-edu`, nutrition → `Z:/datalib-nut`, child-protection → `Z:/datalib-chp`, cross-cutting → `Z:/datalib` (see the microdata-archive chapter). A *collection* is the harmonisation code in position 8 of the folder grammar. The two are **aligned but not identical**: the HLT domain and the HLT collection point at the same subject matter, but the domain is where a survey is *shelved* and the collection is how a vintage is *harmonised and merged*. As of v0.7.0 / contract v1, only HLT (and IPUMS) is registered in the merge registry; EDU, NUT, and CHP become first-class collections as their module lists and keys are verified in later phases, even though their domains already exist as deposit destinations.

38.1 The seven shared semantics

The grammar is only as trustworthy as the rules every language applies to it, and datalib pins seven. They are the shared semantics of `config/grammar.md`, and each one is a place where an implementation could plausibly differ and is contractually forbidden to.

1 — Uppercase once, match exactly. Case is normalised a single time, at the input boundary, and thereafter matching is exact: a country is matched by its whole code, a survey folder by its exact `*_<SURVEY>` suffix. Matching is never a substring test — MICS does not match MICS6, and ZWE does not match a folder that merely contains those letters. Normalising once and matching exactly is what stops a loose match from resolving the wrong survey.

2 — Vintages are integers, latest is the numeric maximum. As the vintages section sets out: inputs accept `1/01/v01/V01`, the internal form is `v%02d`, and “latest” is the numeric maximum, never the alphabetical one, so `v10 > v09`. This is the single rule most likely to be got wrong by a naïve directory listing and the single rule most consequential when it is.

3 — Defaults resolve to the latest, and the target must exist. Omit the year and get the latest year; omit the master version and get the latest master; omit the adaptation version and get the latest adaptation of the requested collection. Every default is the numeric maximum of its level, and resolution verifies that the chosen vintage folder exists before any load proceeds — a default never resolves to a folder that is not there.

4 — Loaders never mutate. A loader reads; it does not recode, drop, or rename values. There are exactly **two documented schema additions**, and both are structural, not analytical. First, a person module whose registry `linevar` is not already `line_number` — HLT’s `hhmembers`, whose line variable is `hh_line_number` — has that column *renamed* to

`line_number` on load, purely to normalise the merge key. Second, provenance columns `ctrycode` and `year` are added when they are absent, never overwritten when they are present, on every load. Both additions are of a piece with structural provenance: the loader records where a record came from without ever altering what the record says.

5 — Merge is keyed, per-module-validated, and outer. Person modules chain **1:1** on `keys_person`; household modules attach **m:1** on `keys_hh` (or **1:1** when only a household module is present). Before any join, each module’s declared keys are validated for uniqueness — an isid-equivalent check — and a module whose keys are missing or duplicated stops the merge with a **typed error that names the offending module**, rather than letting a bad key produce a silently wrong result. Unmatched rows are kept (the join is outer), and the merge indicator is not retained in the output. A toolkit reader will recognise the instinct: this is the same cardinality-first discipline as the toolkit’s `dw_merge`, and the per-module uniqueness check is the microdata sibling of the toolkit’s `dw_isid` guard — declare the grain, prove it, then join.

6 — Errors map to a shared taxonomy. The failure classes — invalid input, not found, config file missing, user block missing, library root unset — map onto Stata exit codes, R condition classes (`datalib_error_*`), and Python exception types, so the same contract violation is recognisable across the three languages. The mapping table is vendored into the error taxonomy appendix, whose `datalib` section carries it alongside the toolkit’s own envelope.

7 — Library-root resolution follows a fixed precedence. Where a language reads the library root from is ordered, and the order differs by language idiom. In **R** and **Python** the precedence is: an explicit `root` argument, then the `DATALIB_ROOT` environment variable, then the language option (`option(datalib.root)` in R), then an optional `datalib:` key in the user block of `~/.config/user_config.yml`, then an error if none is set. In **Stata** it is: the `root()` option, then the `${datalib}` session global — which `getuserconfig/datalib_config` fills from the `config` key when it is unset, so a config-derived global outranks the environment variable *once it is loaded* — then `DATALIB_ROOT`, then an error. The precedence orderings are deliberately not identical; they reflect each language’s native configuration conventions, and the `datalib` API and conformance chapter documents the practical consequence.

38.2 Deployment topologies

The grammar is written for a **single library root**: countries sit directly under `<root>`, and harmonisations live as adaptation folders beneath them. That is the model the contract describes, and it is a complete and valid deployment on its own.

The UNICEF estate today also runs a **multi-root** topology — one tree per thematic domain: `Z:/datalib` for cross-cutting holdings, and `Z:/datalib-hlt`, `Z:/datalib-edu`, `Z:/datalib-nut`, and `Z:/datalib-chp` for health, education, nutrition, and child-protection respectively. Both topologies are valid, and the grammar inside any one root is identical either way — a country folder under `Z:/datalib-hlt` obeys exactly the same

token positions as one under `Z:/datalib`. The operational fact that follows is worth stating plainly: **`datalib_catalog` inventories one root per call**. A catalogue of the whole estate under a multi-root layout is the union of one call per domain root, not a single traversal; a caller that forgets this inventories only the domain it happened to point at.

Which topology becomes canonical is not settled. Whether UNICEF consolidates to a single library root or keeps per-domain roots is one of the four **open Standards-Board decisions** flagged in the microdata-archive chapter, and the governance draft asks for it to be decided before the first promotion into a canonical tree. Until then, treat both topologies as supported, write code that names its root explicitly, and do not assume a single-root world when catalog-level work may be spanning several domain trees. The grammar is stable across that decision; only the number of roots it is applied under is in question.

39 The `datalib_*` API and its conformance suite

This is the third chapter of Part V, and it is where the microdata archive stops being a folder convention and becomes executable code. The microdata-archive chapter draws the boundary and sets out the governance; the IHSN grammar chapter documents the shelving convention — the country/survey/vintage folder tree that encodes provenance in a name. This chapter documents the software that reads and writes that tree: the thirteen shared functions of the `datalib_*` surface, and the conformance suite that keeps the R, Python, and Stata siblings honest against each other and against a committed specification. The reproducibility mission is the same one the whole book serves — a survey deposited once must load the same way, in the same shape, from any of the three languages, years after the analyst who deposited it has moved on. A shelving convention is only as trustworthy as the loader that honours it, so the loader is where the contract has to be pinned down and tested.

Recall the boundary the microdata-archive chapter draws, because it governs everything below. The toolkit's `dw_*` contract serves the **aggregate / indicator warehouse** — the canonical 060.DW-MASTER Teams deposit. `datalib`'s `datalib_*` contract serves survey **microdata** — the `Z:/datalib*` trees plus a staging area. Same office, same `~/.config/user_config.yml` file, a **different data layer**. A microdata deposit never enters 060.DW-MASTER. This chapter is entirely about the microdata side; where a `dw_*` function shares a name-shape or a design idea with a `datalib_*` one, the resemblance is a family likeness, not a shared code path.

i Note

This part documents `datalib contract v1` as shipped in **`datalib v0.7.0`**, and the governance model at **`draft v0.3`**; `unicef-drp/datalib-unicef` is authoritative — `config/grammar.md` for the contract, `docs/GOVERNANCE_microdata_deposit.md` for governance.

39.1 The thirteen-function shared surface

The public surface is small and deliberately identical across the three languages. The same thirteen verbs exist in each, so a script's *intent* reads the same whether it is written in R, Python, or Stata; only the calling syntax differs.

`datalib_config · datalib_root · datalib_countries · datalib_surveys · datalib_vintages · datalib_adaptations · datalib_resolve · datalib_files` (sections `data · data-original · data-r · data-other · doc · programs`, identical in all three languages) · `datalib_catalog · datalib_load · datalib_browse · datalib_create · datalib_map_drive`.

All 13 exist in Stata and Python; Python additionally exports the verbatim aliases `getuserconfig / mapzdrive` (Stata keeps those as the original commands; R exports the 13 canonical names only).

The surface is **thirteen functions as of datalib v0.7.0 / contract v1**. They fall into six groups, each a stage in the same pipeline — from *where is the library* to *give me the data*:

Group	Functions	What they do
Config & root	<code>datalib_config · datalib_root</code>	Read and inspect the per-operator configuration (the <code>datalib:</code> block), and resolve the active library root through the precedence chain below.
Enumerate	<code>datalib_countries · datalib_surveys · datalib_vintages · datalib_adaptations</code>	Walk the tree one level at a time: which countries live under a root, which survey folders a country holds (matched by exact <code>*_<SURVEY></code> suffix, never substring), which master vintages exist (as integers, so “latest” is the numeric maximum — <code>v10</code> beats <code>v09</code> , never an alphabetical listing), and which adaptation vintages a collection has.

Group	Functions	What they do
Resolve, files, catalog	<code>datalib_resolve</code> · <code>datalib_files</code> · <code>datalib_catalog</code>	Turn a country/year/survey/collection request plus optional versions into a single verified vintage folder (the resolved folder must exist — it is checked before any load); list the files inside it by section (<code>data</code> , <code>data-original</code> , <code>data-r</code> , <code>data-other</code> , <code>doc</code> , <code>programs</code>); and inventory a whole root.
Load & browse	<code>datalib_load</code> · <code>datalib_browse</code>	Load and merge the modules of a resolved vintage into memory, and navigate the tree interactively.
Create	<code>datalib_create</code>	Scaffold a new, grammar-conformant vintage folder — the entry point for <i>flow</i> deposits that follow the convention from day one.
Map drive	<code>datalib_map_drive</code>	Map the Z: drive so the library root is reachable on a fresh machine.

The defaults thread through the whole surface: omit the year and you get the latest year; omit the master version and you get the latest master; omit the adaptation version and you get the latest adaptation of the requested collection. Every default resolves to a concrete, existing folder before a byte is read.

39.2 Worked examples

The most important groups are *resolve* — turning a human request into a verified folder — and *load* — turning that folder into a data frame. Both use the real syntax shipped in the `datalib` README, worked here against the running example of Zimbabwe’s 2019 MICS, health collection (HLT).

Resolve. `datalib_resolve` is pure name-arithmetic: it applies the defaults, resolves the vintage, confirms the folder exists, and returns the resolved path. It reads nothing.

39.2.0.1 R

```
datalib_resolve("ZWE", 2019, "MICS", collection = "HLT", root = "Z:/datalib")
```

39.2.0.2 Python

```
from datalib import datalib_resolve
datalib_resolve("ZWE", 2019, "MICS", collection="HLT", root="Z:/datalib")
```

39.2.0.3 Stata

```
datalib_resolve, country(ZWE) year(2019) survey(MICS) collection(HLT) root("Z:/datalib")
* The umbrella command `datalib, country(ZWE) year(2019) survey(MICS)
* collection(HLT)` resolves *and then* loads in one step; `datalib_resolve`
* is the resolve-only leg of that same path.
```

Load. `datalib_load` resolves, reads the modules the collection registry lists, and merges them: person-level modules chain 1:1 on `keys_person`, then household-level modules attach m:1 on `keys_hh`, with keys validated per module. It returns a single data frame when `merge` is on (the default) and a named list (R) or dict (Python) of one frame per module when `merge` is off. Unmatched rows are kept — the merge is an outer join and the merge indicator is not retained.

39.2.0.4 R

```
# Merged data frame (default):
df <- datalib_load("ZWE", 2019, "MICS", collection = "HLT", root = "Z:/datalib")

# Unmerged: a named list, one data frame per module:
mods <- datalib_load("ZWE", 2019, "MICS", collection = "HLT",
                    root = "Z:/datalib", merge = FALSE)
```

39.2.0.5 Python

```
from datalib import datalib_load

# Merged DataFrame (default):
df = datalib_load("ZWE", 2019, "MICS", collection="HLT", root="Z:/datalib")
```

```
# Unmerged: a dict of DataFrames, one per module:
mods = datalib_load("ZWE", 2019, "MICS", collection="HLT",
                    root="Z:/datalib", merge=False)
```

39.2.0.6 Stata

```
* The umbrella command resolves, loads, and merges into memory:
datalib, country(ZWE) year(2019) survey(MICS) collection(HLT)

* `datalib` with no arguments opens interactive folder navigation
* (the datalib_browse group); `datalib help` prints the command reference.
* Stata holds one dataset in memory, so the merge=FALSE "named list" shape
* has no direct Stata analogue - load modules individually to inspect them.
```

i Loaders never mutate values

`datalib_load` performs **no recodes and no drops**. It makes exactly two documented *schema* additions, and never touches the values that were already there: (a) in a person module whose registry `linevar` is not already `line_number`, that column is renamed to `line_number` so the merge keys line up; and (b) provenance columns `ctrycode` and `year` are added when absent (and never overwritten). Everything else — every value in `Data/Original/` — arrives exactly as it was deposited. This is `datalib`'s provenance mechanism: not a sidecar file, but the untouched original plus a name that encodes the vintage. The provenance-schema appendix contrasts it with the toolkit's `.provenance.json` sidecar.

Enumerate and catalogue. Before you resolve, you often need to know what is on the shelf. The `enumerate` group answers one level at a time; `datalib_catalog` inventories a whole root at once. Python additionally ships a command-line front end.

39.2.0.7 R

```
datalib_countries(root = "Z:/datalib")
datalib_surveys("ZWE", root = "Z:/datalib")
datalib_catalog(root = "Z:/datalib")
```

39.2.0.8 Python

```
from datalib import datalib_countries, datalib_catalog
datalib_countries(root="Z:/datalib")
datalib_catalog(root="Z:/datalib")
```

```
# Or from a shell, without importing:
# python -m datalib ls --root Z:/datalib
```

39.2.0.9 Stata

```
datalib_countries, root("Z:/datalib")
datalib_catalog, root("Z:/datalib")
* No `python -m datalib ls` equivalent in Stata; use interactive `datalib`.
```

`datalib_catalog` inventories **one root per call**. That matters under the multi-root deployment described below: a full picture of a per-domain estate is one `datalib_catalog` call per root, not a single sweep.

39.3 Root and config resolution

Every function above needs to know *which* library root it is operating on, and the surface never guesses. It resolves the root through a fixed precedence chain, and the chain differs between the two script-language siblings and Stata:

Language	Root precedence (first match wins)
R / Python	explicit <code>root =</code> argument → <code>DATALIB_ROOT</code> environment variable → language option (<code>options(datalib.root =)</code> in R) → the <code>datalib:</code> key in the user block of <code>~/.config/user_config.yml</code> → error
Stata	<code>root()</code> option → <code>`\${datalib}`</code> session global → <code>DATALIB_ROOT</code> environment variable → error

The two chains look different because Stata folds the config file in one step earlier. The ``${datalib}`` global is not an independent source: when it is unset, `getuserconfig / datalib_config` fill it *from the config key*. So once the config has been loaded into the session, a config-derived ``${datalib}`` outranks `DATALIB_ROOT` — the opposite of the R/Python ordering, where the environment variable outranks the config key. The precedence is identical in spirit (explicit argument beats session state beats environment beats file), but a Stata operator who sets `DATALIB_ROOT` expecting it to win should know the loaded config global sits ahead of it.

This is where the boundary from the microdata-archive chapter becomes concrete at the file level. `datalib` reads its root from a **`datalib:` block** in `~/.config/user_config.yml` — the *same file* the toolkit reads its own flat `dw_*` keys from, but a **distinct namespace**

within it. The intent is one shared operator-config file across the office; the reality today is two separate namespaces in one file, because the toolkit's own loader does not yet tolerate a `datalib:` block (a parser guard in `dw_load_config` is pending). The architecture chapter's *configuration across the tiers* note is the authority on that caveat; treat the `datalib:` and `dw_*` keys as separate namespaces until it lifts.

Two onboarding utilities put the pieces in place on a fresh machine, and each is one of the thirteen with a verbatim alias for muscle memory: `getuserconfig` / `datalib_config` writes and reads the per-operator config, including the `datalib:` root key, and `mapzdrive` / `datalib_map_drive` maps `Z:`. These, together with the loader itself, are the **narrow slice** slated to migrate into the CSO Toolkit under the `dw_*` prefix — the onboarding-plus-loader functions, not all thirteen. The architecture chapter frames `datalib` as a provisional tier-(ii) ecosystem package for exactly this reason: the handbook points at `datalib`'s own repository for the full surface and documents here only the positioning and the cross-cutting contracts. The migration does not change the boundary — a migrated `dw_`-prefixed loader would still read microdata from `Z:/datalib*`, never from `060.DW-MASTER`.

39.4 Errors

`datalib` raises a small, closed set of contract conditions, mapped consistently across the three languages: invalid input, not found (country / survey / vintage / file), config file missing, user block missing, and library root unset each map to a Stata exit code, an R condition class in the `datalib_error_*` family, and a Python exception; separately, per-module key validation failure during a merge stops with a typed error that **names the offending module**, so a duplicated or missing identifier is diagnosable without opening the data. The full mapping — Stata codes, R classes, Python exceptions, side by side — has a single home in the `datalib` section of the error-taxonomy appendix, and is not reproduced here.

39.5 The conformance suite

`datalib`'s conformance kit is an **oracle** harness: each implementation is run against a *committed specification of expected outputs*, not against the other implementations. As of **`datalib v0.7.0` / `contract v1`**, the suite ships two golden-case files — `cases_resolve.csv` and `cases_load.csv` — exercised against a single committed synthetic fixture library and run per language: R with `testthat`, Python with `pytest`, and a Stata do-file harness. The precise case count lives in the repository and is deliberately left unstated in prose, because it grows survey by survey as collections are verified; what is fixed is the shape — resolve cases and load cases, one fixture tree, three runners, all asserting the same expected answers. Deliberate departures from the pre-contract Stata behaviour are recorded, with rationale, in `tests/DIVERGENCES.md`, so a divergence is a documented decision rather than a silent drift.

It is worth setting this against the toolkit's **own** conformance harness — the **`conformance/`** machinery shipped in [#134](#) — because the two are complementary designs, not the same

tool applied twice. The toolkit’s harness is a **parity** harness: it runs the R, Python, and Stata siblings against *each other* and asserts they agree to a 1e-9 numeric tolerance. It answers *do the siblings agree?* datalib’s oracle harness answers a different question — *does each sibling match the specified answer?* — by checking every implementation against committed expected outputs. Parity guards against the siblings drifting apart; an oracle guards against all of them drifting together away from the specification. The collection registry (reproduced in the IHSN grammar chapter, and carrying the note that the IPUMS keys are unverified against real deposits, guarded per-module until one lands) is the shared source both languages consume, and the golden cases are the drift guard that keeps the hand-mirrored Stata copy byte-aligned with the R and Python copies. Read together, datalib’s oracle suite is the microdata counterpart of the toolkit’s contract discipline: the same conviction that a cross-language contract is only real if a test can fail when it is broken.

39.6 Access

Access: internal — request via the CSO team (repository visibility is tracked in datalib issue #20).

Part VI

Part VI — The Ecosystem

40 The UNICEF analytics ecosystem: three tiers

The architecture chapter already introduced this ecosystem in miniature. Its “wider ecosystem” section gave the orientation: a compact three-tier table, a one-sentence membership rule, the parity pledge, and a note about why configuration diverges across packages today. That section did its job — it told a reader building a pipeline where the surrounding code comes from — and then it pointed here. This chapter is the full map. It is the authoritative treatment: the same three tiers, but with the columns, the membership gate, and the governance policy set out in full, so that the orientation upstream can stay short.

The organizing idea is simple and it governs everything that follows. Shared code around a CSO pipeline is not one undifferentiated pile. It falls into three tiers, and the tier a piece of code belongs to decides two things at once: **who maintains it**, and **what this book is willing to assert about it**. Those two questions travel together. The toolkit’s own functions carry the strongest maintenance commitment and the strongest claim — the book *is* their specification. An external package used in one production pipeline carries neither a maintenance promise nor a conformance claim; the book asserts only that a specific version did a specific thing on a specific day. Everything between those poles is the middle tier, and most of the governance in this chapter exists to keep that middle tier honest.

Because the tiers are a reproducibility instrument, not an org chart, this chapter is deliberately conservative about claims. Where the book cannot stand behind something, it says so — and it says so in a *structured* way, so that a reader can tell an institutional commitment from a named individual’s good work, and a live conformance guarantee from a dated observation.

40.1 The three tiers

The table below is the authoritative version. It carries more columns than the orientation table in the architecture chapter — in particular a **visibility** column and a **support owner** column — because those two columns are exactly where an over-strong claim would otherwise hide.

Tier	What it is	Members	Canonical home	Visibility	Support owner	What the book asserts
(i) Toolkit internals	The <code>dw_*</code> contract and its three siblings — the toolkit itself	The <code>dw_*</code> function surface	<code>unicef-drp/PSB/d61</code>	CSO — the book is the docs	maintainer (toolkit)	Normative. The book is the specification; the coverage-matrix appendix is the honesty device that records where languages have not yet reached parity.

Tier	What it is	Members	Canonical home	Visibility	Support owner	What the book asserts
(ii) UNICEF- supported public packages	Full packages with their own release lines, maintained by CSO staff in their own repos	unicefData (FULL); datalib (PROVISIONAL); a mapping / boundaries capability (CANDIDATE)	Each package's own <code>unicef-drp</code> repo	PUBLIC (unicef-Data) internal, pending #20 (datalib) private candidate (mapping)	A named CSO maintainer, per package	Relationships and cross-cutting contracts. The book documents how the packages fit together (config, conformance conventions) and points at each package's own per-function docs. It never re-documents a package's API.

Tier	What it is	Members	Canonical home	Visibility	Support owner	What the book asserts
(iii) Curated third-party functions	External packages used in production pipelines	A <i>derived</i> list (see the curated-third-party appendix)	Upstream (external)	External	None — upstream maintainer; no CSO promise	Behaved as described, on a date. Only that a named version behaved as described on a stated date in a stated pipeline. No maintenance promise, no conformance claim.

Tier (i) is normative. For the toolkit’s own functions there is no separate reference manual that the book could fall out of sync with — the book is the reference. That is a strong claim, and it is earned by a strong honesty device: the coverage-matrix appendix records, function by function and language by language, where the three implementations actually reach parity and where they do not. The steady-state pledge for tier (i) is equal R, Python, and Stata support, and that pledge is not just prose — the toolkit’s conformance harness executes it, comparing the three implementations against one another to a tolerance of $1e-9$ (shipped in PR #134).

Tier (ii) is relational. These are full packages with their own release cadences, their own changelogs, and their own maintainers. The book’s job for tier (ii) is *not* to mirror their documentation — that would drift the moment either side changed — but to document the **relationships** and the **cross-cutting contracts** that hold the ecosystem together: how configuration is (and is not) shared, what conformance conventions a member is expected to follow. For anything package-specific, the book points at that package’s own docs. The chapters that follow this one in Part VI are the worked example of that pointing.

Tier (iii) is evidentiary. A third-party function earns a row only because it was actually used in a production pipeline and a sector lead verified its behaviour. The book’s claim

is correspondingly narrow: this version, this pipeline, this date, this observed behaviour. Nothing about future releases, nothing about cross-language parity. The rules and the table live in the curated-third-party appendix.

40.2 The tier-(ii) membership gate

Tier (ii) is the only tier with an admission test, because it is the only tier where the book makes a *standing* claim (“this is a supported ecosystem member”) rather than a one-off one. The gate is six criteria, applied uniformly to every candidate:

1. **(a) A public unicef-drp home** — the package lives in the organization’s namespace, not a personal fork.
2. **(b) A tagged public release** — a versioned, publicly installable artifact, not just a branch.
3. **(c) CI-executed tests** — tests that run in continuous integration, not tests that merely exist.
4. **(d) Trilingual, or explicitly scoped with a roadmap** — R, Python, and Stata support, or an honest statement of which languages are in scope and a plan for the rest.
5. **(e) Changelog and versioning discipline** — a maintained changelog and a versioning scheme a consumer can pin against.
6. **(f) Governance wording reconciled with any author disclaimer** — where a package’s own README carries an author disclaimer, the book’s governance language is reconciled with it rather than contradicting it.

A **PROVISIONAL member** is a package that meets **(a)**, **(c)**, **(d)**, **(e)**, and **(f)** but has **(b) pending** — it is a real, maintained, tested, trilingual ecosystem member that has not yet cut its public release.

Applying the gate today:

- **unicefData — FULL member.** It meets all six criteria: a public `unicef-drp/unicefData` home, tagged public releases on CRAN / PyPI / SSC, CI-executed tests, trilingual coverage, a maintained changelog, and governance wording reconciled with its author disclaimer. The `unicefData` chapter that follows is its per-package pointer.
- **datalib — PROVISIONAL member.** It meets (a), (c), (d), (e), and (f), but criterion (b) — a *public* tagged release — is pending. `datalib` is maintained in `unicef-drp/datalib-unicef`, which is internal today; the move to a public release is tracked by `datalib` issue #20. Until that closes, `datalib` is a full ecosystem member in every respect except public visibility, and the book labels it exactly that. The `microdata-archive` chapters in Part V document `datalib` itself.
- **A mapping / boundaries capability — CANDIDATE only.** This capability is still under evaluation. There are two internal candidates, and **neither is named in this book** — not because they are secret, but because naming a winner before the fork resolves and before the winner has actually passed the gate would be a claim the

book cannot yet stand behind. When one candidate clears the gate, it will be named here and given its pointer; until then it is a candidate, not a member.

i What “UNICEF-supported” does — and does not — mean

Tier (ii) is labelled “UNICEF-supported public packages,” and that phrase is easy to over-read. It does **not** mean the organization has signed a service-level agreement for these packages. Support for each package is a **named CSO maintainer** — a person, recorded in the support-owner column — who keeps the package alive. Institutional adoption, the step that would turn a named maintainer into an organizational commitment, is a **Phase-4 decision** that has not been taken. The book deliberately does not write an SLA the organization has not signed. The support-owner and visibility columns exist precisely so that “supported” reads as “a named person maintains this,” never as “the institution guarantees this.”

40.3 Curated third-party functions

Tier (iii) has no admission committee. Its list is **derived, not curated** — a subtle but load-bearing distinction. A row does not appear because someone decided a package was good; it appears because two facts became true: the package was **used in a production pipeline**, and a **sector lead attached a verification stamp** recording that it behaved as described. The list is a byproduct of production evidence, which is why the book can make a claim about it at all.

The claim stays narrow on purpose. A tier-(iii) row asserts that a *named version* behaved as described on a *stated date* in a *stated pipeline* — and nothing more. There is no maintenance promise and no conformance claim, because the book does not maintain these packages and does not run them through the parity harness.

One consequence is worth stating plainly: **stale rows display stale; they never block a release**. If a verification stamp ages, the appendix shows it aging rather than silently dropping the row or holding up a book build. A visible stale stamp is information; a suppressed one is a lie. The full rules and the derived table live in the curated-third-party appendix, which today holds the inclusion rules and a deliberately-empty derived table awaiting its first production-verified row.

40.4 Configuration across the tiers

Configuration is the clearest place where the three tiers are genuinely, deliberately divergent — and the architecture chapter’s config-divergence note pointed here for the reason. Today there are **three separate configuration contracts**, and they should be treated as **three separate namespaces**. They are not yet meant to interoperate.

Contract	Config source	Mechanism	Reads the toolkit's <code>user_config.yml</code> ?
Toolkit (tier i)	<code>~/.config/user_config.yml</code>	Flat keys , written as generated profile blocks and read by <code>dw_load_config</code>	Yes — this <i>is</i> its file
datalib (tier ii)	Its own package-specific config file	A <code>datalib:</code> block in datalib's own file	No — separate file today
unicefData (tier ii)	Environment variables <code>UNICEF_DATA_HOME</code> / <code>UNICEF_CONFIG_PATH</code>	Read from the environment; no user config file is read at all	No — reads no user file

The toolkit reads **flat keys** from `~/.config/user_config.yml`, populated as generated profile blocks and consumed by `dw_load_config`. datalib reads a `datalib:` block from its **own** package-specific config file. A shared-file arrangement — a single `user_config.yml` carrying a `datalib:` block alongside the toolkit's flat keys — is attractive, but it waits on a **parser guard** in `dw_load_config` that does not exist yet.

 Do not add a `datalib:` block to the toolkit's `user_config.yml` yet

The shared-file arrangement is not shipped. Until `dw_load_config` grows the parser guard that lets it ignore a nested `datalib:` block cleanly, adding one to the toolkit's `user_config.yml` risks the toolkit's flat-key reader tripping over it. Keep datalib's configuration in datalib's own file for now.

`unicefData` sits apart from both. It is configured entirely by **environment variables** — `UNICEF_DATA_HOME` and `UNICEF_CONFIG_PATH` — and reads **no user config file at all**. This is not an oversight: `unicefData` is CRAN-track, and CRAN policy dislikes packages that read user files by surprise. The environment-variable contract keeps it CRAN-clean.

Convergence of these three contracts onto a single shared configuration surface is a **Phase-4 decision**, not something the ecosystem has shipped. Until that decision is taken, the safe mental model is three namespaces that happen to sit near one another, not one namespace with three dialects.

40.5 What the handbook asserts, tier by tier

Pulling the policy back into one place, so a reader can carry it without the table:

- **Tier (i) — normative.** For the toolkit's own `dw_*` functions, the book is the specification. The coverage-matrix appendix records where the three languages have

and have not reached parity, and the conformance harness (#134) executes the parity pledge to 1e-9.

- **Tier (ii) — relationships and cross-cutting contracts, never the API.** For UNICEF-supported packages, the book documents how they fit together — configuration namespaces, conformance conventions, membership status — and points at each package’s own per-function docs. It does not re-document a package’s API, and each package’s support is a named CSO maintainer, not an institutional SLA.
- **Tier (iii) — behaved as described, on a date.** For curated third-party functions, the book asserts only that a named version behaved as described on a stated date in a stated pipeline. The list is derived from production use plus a sector lead’s verification stamp; stale stamps display stale and never block a release.

That is the whole governance model, and it is the lens for the rest of Part VI. The next chapter applies it to the first full tier-(ii) member, `unicefData` — pointing at its docs, its conformance exemplar, and the decision of when to reach for it rather than the toolkit’s own fetch path, without ever re-documenting its API.

41 unicefData: public child statistics in three languages

The reproducibility mission this book serves does not stop at the toolkit’s own boundary. A CSO analyst reaches for published child-statistics indicators constantly — to explore a series before it enters a pipeline, to teach a method, to sanity-check an aggregate against the public figure — and those reaches should be as consistent across R, Python, and Stata as the `dw_*` contract is. `unicefData` is the package that makes them so: trilingual, tidy access to the **public** UNICEF SDMX data warehouse (`sdmx.data.unicef.org`, surfaced to the world as `data.unicef.org`). The ecosystem chapter set out the three tiers of shared code, the membership gate, and the three configuration contracts in full; the architecture chapter’s orientation pointed here for the worked treatment. This chapter is that treatment for the **first full tier-(ii) member**. It positions `unicefData` and draws its boundaries against the rest of the ecosystem — it does **not** re-document the package’s API, which lives, canonically, in `unicef-drp/unicefData`.

`unicefData` is the first tier-(ii) package to clear the whole membership gate rather than most of it: a public `unicef-drp` home, a tagged public release, CI-executed tests, a trilingual surface, changelog and versioning discipline, and a governance wording reconciled with its author disclaimer. That last item matters, and it is where honesty has to be explicit. By contrast, `datalib` — documented across Part V — meets every criterion **except** a public release and is a *provisional* member pending its visibility decision, and the mapping/boundaries capability is a *candidate* under evaluation, not yet named in this book. `unicefData` is the one that is fully in.

Official data, unofficial client

The `data.unicefData` returns is UNICEF’s official public statistics — the same figures published at `data.unicef.org`. The **package** is not an official UNICEF product. Its own README states plainly that it is “not an official UNICEF product ... the views of the author.” The honest reconciliation, and the one this book adopts, is that `unicefData` is **CSO-staff-maintained, MIT-licensed, and supported by a named maintainer** — useful and backed, but not an institutional product with an SLA the organisation has signed. This is exactly why the tier-(ii) status table carries a *named support owner* column rather than a bare “UNICEF-supported” label: support is a person on the CSO team, and institutional adoption is a Phase-4 decision, not a claim this chapter makes.

i Version and source of authority

This chapter is pinned to **unicefData v2.4.2**, bundled-metadata vintage **2026-04-20**, at which point the client exposes **733 indicators**. Per-language versions differ under the *minimum bump rule*: a release tag is cut across all three siblings at once, but each registry re-publishes only when that language’s own surface changed, so the versions lag independently — **R 2.4.1** (CRAN), **Python 2.4.2** (PyPI, package **unicefdata**), **Stata 2.4.1** (SSC), all under tag **v2.4.2**. Counts and versions below are stamped because they drift; **unicef-drp/unicefData** is the canonical, authoritative documentation for the surface itself.

41.1 When to use what

This is the decision the audience actually needs, because three tools in the CSO estate can all put a UNICEF number in front of you and they are **not** interchangeable. The question is never only “where does the number come from” but “what contract is wrapped around the read.”

Tool	Data layer	Mode contract?	Lands in a deposit?	Use it when
<code>dw_api_fetch</code> (toolkit)	Aggregate indicators, fetched to build the warehouse	Yes — producer / reviewer mode-gated	Yes — cached into the <code>060.DW-MASTER</code> deposit, vintage-frozen	You are building or refreshing the aggregate warehouse and need the fetched bytes to replay identically years later
<code>unicefData()</code> (this chapter)	Aggregate indicators, published public series	No	No	You want tidy public indicators for analysis, exploration, or teaching against <code>data.unicef.org</code>
<code>datalib_load</code> (microdata archive)	Survey microdata — household / person records	No (its own governance)	No — reads the <code>Z:/datalib*</code> archive	You need the underlying survey records, not indicators

The boundaries are one sentence each. `dw_api_fetch` is the warehouse-builder’s tool: it

is mode-gated, and every response it fetches is cached **into** the DW-Production deposit so the exact bytes behind a published aggregate can be reproduced vintage-permanent long after the fetch. `unicefData()` is the analyst's tool: it returns the same class of published public indicators as a tidy frame with **no** mode contract and **no** deposit, for exploration, teaching, and one-off analysis. `datalib_load` is not on the indicator layer at all: it loads survey microdata — the household and person records that indicators are computed *from* — and belongs to the microdata-archive chapter's contract, not this one. The subtlety worth stating explicitly is that `dw_api_fetch` and `unicefData()` may read the very same public SDMX warehouse; what separates them is the mode gate and the frozen deposit on one side versus a bare tidy read on the other.

41.2 Quick start

Because `unicefData` is a **public** package, install instructions belong here (unlike the internal packages in Part V). All three lines are pinned to v2.4.2, and each honours the minimum bump rule — you get the latest published version for your language, which may lag the tag.

41.2.0.1 R

```
# Install (public) - R 2.4.1 on CRAN:
install.packages("unicefData")
# or the development head:
# devtools::install_github("unicef-drp/unicefData")

library(unicefData)

# Under-five mortality (CME_MRYOT4), three countries, 2015–2023:
u5mr <- unicefData(
  indicator = "CME_MRYOT4",
  countries = c("ALB", "USA", "BRA"),
  year      = "2015:2023"
)

search_indicators("mortality") # find an indicator code
list_categories()              # list dataflows / categories
```

41.2.0.2 Python

```
# Install (public) - Python 2.4.2 on PyPI, package name `unicefdata`:
# pip install unicefdata
```

```

from unicefdata import unicefData, search_indicators, list_categories

# Under-five mortality (CME_MRYOT4), three countries, 2015–2023:
u5mr = unicefData(
    indicator="CME_MRYOT4",
    countries=["ALB", "USA", "BRA"],
    year="2015:2023",
)

search_indicators("mortality") # find an indicator code
list_categories()             # list dataflows / categories

```

41.2.0.3 Stata

```

* Install (public) – Stata 2.4.1 on SSC:
*   ssc install unicefdata
* or the GitHub route:
*   net install unicefdata, from("https://raw.githubusercontent.com/unicef-drp/unicefData/

* Under-five mortality (CME_MRYOT4), three countries, 2015–2023:
unicefdata, indicator(CME_MRYOT4) countries(ALB USA BRA) year(2015:2023) clear

unicefdata, search(mortality) // find an indicator code
unicefdata, flows             // list dataflows / categories

```

i Caching and offline use

The indicator metadata `unicefData` needs to resolve codes and dataflows **auto-downloads on first use and refreshes every 30 days**; the cache is cleared with `clear_unicef_cache()` (R), `clear_cache()` (Python), or `unicefdata, clearcache` (Stata). Because the package also **bundles** a metadata snapshot (`metadata/current/*.yaml`, vintage **2026-04-20** at v2.4.2), search and category listing work offline against that snapshot even before the first network refresh — a fetch of actual observations still needs the warehouse.

41.3 The surface

The whole point of a trilingual package is that the same verbs and the same output schema appear in every language, so a script’s *intent* reads identically whether it is R, Python, or Stata. The core surface is a handful of functions — fetch a series, search the catalogue, list categories, manage the cache — and every fetch returns one **normalised tidy schema**

(`dataflow`, `iso3`, `indicator`, `sex`, `age`, `period`, `value`, `unit`, `obs_status`, `data_source`), mapped from the underlying SDMX columns the same way in all three. The vendored table below is pinned to v2.4.2 and is transcluded rather than retyped; the package’s own documentation remains canonical for anything the pin does not cover.

Surface as of `unicefData` v2.4.2 — R 2.4.1 (CRAN) · Python 2.4.2 (PyPI) · Stata 2.4.1 (SSC); bundled-metadata vintage 2026-04-20. Counts and columns drift with releases; this table is pinned, and the package’s own documentation is canonical.

Core public surface — the same functions in all three languages.

Purpose	R / Python	Stata
Fetch an indicator series (dataflow auto-detected)	<code>unicefData(indicator=, countries=, year=)</code>	<code>unicefdata, indicator() countries() year() clear</code>
Search the indicator catalogue	<code>search_indicators("mortality")</code>	<code>unicefdata, search(mortality)</code>
List categories / dataflows	<code>list_categories()</code>	<code>unicefdata, flows</code>
Indicators by category / SDG	<code>get_indicators_by_category()</code> . <code>get_indicators_by_sdg()</code>	<code>(via search)</code>
Clear the metadata cache	<code>clear_unicef_cache()</code> (R) · <code>clear_cache()</code> (Python)	<code>unicefdata, clearcache</code>

Normalized output schema — every fetch returns the same tidy columns regardless of language (SDMX column → output column):

SDMX column	Output column	Type	Required
DATAFLOW	<code>dataflow</code>	string	yes
REF_AREA	<code>iso3</code>	string	yes
INDICATOR	<code>indicator</code>	string	yes
SEX	<code>sex</code>	string	no
AGE	<code>age</code>	string	no
TIME_PERIOD	<code>period</code>	numeric	yes
OBS_VALUE	<code>value</code>	numeric	yes
UNIT_MEASURE	<code>unit</code>	string	no
OBS_STATUS	<code>obs_status</code>	string	no
DATA_SOURCE	<code>data_source</code>	string	no

41.4 Configuration

`unicefData` sits in a **third** configuration namespace, deliberately distinct from the other two tiers. It is configured entirely through **environment variables** — `UNICEF_DATA_HOME`

for its data and cache root, `UNICEF_CONFIG_PATH` for an optional config location — and it reads **no** `~/.config/user_config.yml` at all. That is a design constraint, not an oversight: `unicefData` is CRAN-track, and CRAN policy dislikes packages that read user files by surprise, so the package takes only what the environment hands it. The contrast with the rest of the ecosystem is clean. The toolkit reads **flat keys** from `~/.config/user_config.yml` through generated profile blocks and `dw_load_config`; `datalib` reads a `datalib:` block from its **own** package-specific config file (the shared-file arrangement still waits on a parser guard in `dw_load_config`); `unicefData` reads environment variables and nothing on disk. Do not expect a toolkit profile block to configure `unicefData`, and treat all three as separate namespaces. The ecosystem chapter's *configuration across the tiers* section is the authority on why the three diverge today and why convergence onto a shared contract is a Phase-4 decision rather than something already shipped.

41.5 A conformance exemplar

`unicefData` did the parity thing before the toolkit's own harness shipped, and it is worth reading the two together. The package carries a cross-language output test — `tests/test_cross_language_output.{R,py}` — that runs the **same query** through the R and Python siblings and **compares the outputs**, backed by an expected-columns schema fixture that pins the tidy shape every language must produce. That is the same conviction the toolkit's conformance harness (shipped in [#134](#)) executes: a cross-language contract is only real if a test can fail when it is broken. The difference is one of reach, and it is the honest one to state — `unicefData`'s automated comparison today pairs **R against Python**, with the schema fixture guarding the columns all three (including Stata) must return, whereas the toolkit's harness generalises the same instinct to a three-way parity check that asserts the R, Python, and Stata siblings agree to a **1e-9** numeric tolerance. Read in sequence, `unicefData`'s cross-language test is the exemplar and the toolkit's harness is the exemplar carried to its full form: the parity pledge the coverage-matrix appendix tracks is not an aspiration invented in a vacuum, but a practice a shipping tier-(ii) member already modelled. The precise number of test cases is deliberately left unstated — it drifts with releases and differs on the development fork — so this description is pinned to the **v2.4.2** approach, not to a case count.

A Cross-language coverage matrix

The promise of `cso-toolkit` is parity: the same command names, the same arguments, and the same enforced behaviour across R, Python, and Stata. Parity is the goal, but it is not yet uniform. R is the reference implementation and the most complete; Python is a near-complete port; Stata covers a deliberately scoped subset of the contract that fits its idioms. This appendix sets out exactly where the three siblings agree and where they diverge — first by function, then by file format, then by the supporting machinery (documentation, tests, CI) that surrounds each language.

Read this page as a map of expectations. If you are vendoring the helpers into a Stata-only pipeline, the gaps here tell you which capabilities you must route through R or Python instead. But if you are an author wiring a multi-language sector, the matrix tells you which call sites will behave identically on both sides of the language boundary.

On authority: **this appendix is the contract view** — the canonical statement of what exists in which language; the function-reference appendix is the *finding aid* that points to each language’s per-function documentation. Where the two ever disagree, this matrix wins.

Four terms recur throughout, and the whole matrix rests on them. Glossed once, here:

- **Mode contract** — the rule governing what a session may do. A *reviewer session* forbids live network calls and writes to canonical paths; a *producer session* permits both. The toolkit reads the mode from a session global (`dw_mode`).
- **Canonical path** — the published, authoritative location for a dataset (the Teams deposit, mirrored to the Z: drive). Writing there is a producer-only act.
- **Provenance sidecar** — the `.provenance.json` file the toolkit emits beside each saved output, recording who wrote it, when, in what mode, and a content hash.
- **Error envelope** — the toolkit’s standard error shape: every raised message starts with a `[cso_toolkit.<func>]` prefix and carries a **Fix:** remediation line.

The roles-and-mode-contract chapter develops all four in full; the glosses above are enough to read this page linearly.

A.1 Legend

Symbol	Meaning
Yes	Supported in this language.

Symbol	Meaning
Planned	On the roadmap; tracked by an issue. Not callable today.
n/a	Not applicable to this language by design — a deliberate scope decision, not an oversight.
Route via R/Python	The capability does not exist in this language and is not planned; the documented workaround: do the operation in a sibling language and hand the result back.

The tables below render as text, not colour; read the cell contents, not cell shading.

A.2 How parity is defined

Two siblings are “at parity” for a function when they share three things: the function **name**, the **argument set** (allowing for language-idiomatic spelling — R’s `isid = "REF_AREA"` versus Python’s `isid=["REF_AREA"]` versus Stata’s `idvars(REF_AREA)`), and the **enforced behaviour** (the mode contract, the provenance sidecar, the error envelope). Where a function exists in two languages but enforces the contract differently, the matrix notes the divergence rather than marking a clean “Yes”.

The mode contract itself is at parity across all three languages: a reviewer-session write to a canonical path is blocked in every sibling — R raises `stop()`, Python raises `PermissionError`, and Stata raises error 459 (its generic “this is forbidden” code) — and all three honour the `allow_canonical_write` escape hatch reserved for Database-Manager bootstraps.

i A version skew worth knowing

The repository headline version is **v0.7.0**, carried by the R package (`csotoolkit 0.7.0`), the Python package (`__version__ = "0.7.0"`), and the shared `NEWS.md`. The R and Python version stamps are now level; the Stata package versions per-command via its `.ado` files. A matching version number is a floor, not a guarantee of feature parity, though: where a cell below says “Yes” for a recent feature, R is the sibling guaranteed to have it first.

A.3 Function coverage matrix

The table below is grouped by the five capability families. Each row is one public function; each cell is the state of that function in that language. (The R package additionally exports

`dw_`-prefixed aliases of most public names — `dw_save/dw_use` already carry the prefix, while `aggregate_data` is also reachable as `dw_aggregate_data`. The aliases are conveniences, not separate functions, and are omitted from the rows below.)

A.3.1 IO contract

Function	R	Python	Stata
<code>dw_save</code>	Yes	Yes	Yes
<code>dw_use</code>	Yes	Yes	Yes
<code>dw_compare</code>	Yes	Yes	Yes
<code>dw_merge</code>	Yes	Yes	n/a
<code>dw_isid</code>	Yes	Yes	Embedded in <code>dw_save</code> (no standalone command)
<code>dw_verify_z</code>	Yes	Yes	n/a (integrity via <code>datasignature</code> inside <code>dw_use</code>)
<code>dw_stage</code>	Yes	n/a	n/a
<code>dw_root</code>	Yes	Yes	n/a
<code>dw_resolve_path</code>	Yes	Yes	Internal to <code>dw_use</code> / <code>dw_save</code>
<code>dw_is_canonical</code>	Yes	Yes	Internal to the mode guard
<code>dw_mkdir</code>	via base R <code>dir.create</code>	via <code>os.makedirs</code>	Yes (<code>dw_mkdir.ado</code> — Stata’s built-in cannot nest)
<code>dw_verbosity</code>	Yes (session verbose/debug switch)	Planned	n/a
<code>dw_default_unicef_allowed_urls</code>	Yes (no remote-URL freeze allowlist)	Yes	n/a (no remote-URL reads in Stata)
<code>dw_map_drive</code>	n/a	n/a	Yes (<code>dw_map_drive.ado</code> — Stata-only onboarding: maps the team network drive from <code>dwZDrive/dwZDriveUNC</code> ; the toolkit port of datalib’s <code>mapzdrive</code>)

A few caveats sit behind the “Yes” cells:

- **dw_use reached Stata at v0.4.0.** The Stata package README and source catalogue confirm it shipped, scoped to `.dta` / `.csv` / `.xlsx`. Treat it as available.
- **dw_merge and dw_verify_z are R-and-Python only.** There is no Stata sibling and none is planned. `dw_merge` is a join-with-duplicate-key guard (it warns when left/right duplicate structure does not match the declared `how`); `dw_verify_z` compares a file against its Z:-drive mirror, by file size (default, fast) or by deep SHA-256. A Stata pipeline needing either routes through R or Python.
- **dw_mkdir is the mirror image.** It is a first-class *Stata* helper because Stata's built-in `mkdir` does not accept nested paths; R and Python have no need for it because their standard libraries already create directories recursively.
- **dw_root is R + Python; dw_stage is R-only.** `dw_root` resolves the mode-aware repo/canonical root for a vendored `kind` (`"wrk"` / `"raw"` / `"meta"`) so a sector script can build paths without knowing whether the session is producer or reviewer; `dw_stage` copies a reviewer-mode sandbox file into place with a `.staged.json` integrity sidecar. `dw_stage` has no Python or Stata sibling yet.

i Note

`dw_isid` is not a missing Stata feature. The uniqueness check it performs is built directly into `dw_save.ado` via Stata's native `isid`, so the guarantee runs at every write — there is simply no separately callable `dw_isid` command. Declare your key with `idvars(...)` and the check fires.

The `dw_save` call is the clearest illustration of name-and-argument parity with idiomatic spelling:

A.3.1.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        isid = "REF_AREA",
        metadata = list(title = "Education indicators", vintage = "2026-05"))
```

A.3.1.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        isid=["REF_AREA"],
        metadata={"title": "Education indicators", "vintage": "2026-05"})
```

A.3.1.3 Stata

```
dw_save, filename("dw_ed_edu.dta")    ///
    path("$teamsWrkData/ed")          ///
    idvars(REF_AREA)                  ///
    title("Education indicators")      ///
    vintage("2026-05")
```

Note the surface difference in how metadata is passed: R and Python accept a single structured `metadata` object; Stata flattens the common fields into named options (`title()`, `vintage()`, `producer()`, `sources()`). The resulting `.provenance.json` sidecar is the same shape on all three sides — see the provenance-sidecar appendix for the schema.

A.3.2 External-API contract

Function	R	Python	Stata
<code>dw_api_fetch</code>	Yes	Yes	Route via R/Python
<code>dw_api_cached</code>	Yes	Yes	Route via R/Python
<code>dw_api_inventory</code>	Yes	Yes	Route via R/Python
<code>dw_require_no_api</code> (reviewer gate)	via profile helpers	via profile helpers	Yes (<code>dw_require_no_api.ado</code>)
<code>dw_publish</code> (Helix submission)	Skeleton — exported but errors with guidance; the Helix endpoint integration is tracked in issue #15	Planned	n/a (API-side capability)

The largest gap in the toolkit: **Stata has no external-API wrapper.** There is no `dw_api_fetch` in Stata and none is planned. External data must be pulled in R or Python, which writes the result into the deposit cache (`teamsRawData/_apis/<api>/<cache_key>.<ext>`, the canonical landing place for fetched upstream data, mirrored to the Z: drive); the Stata pipeline then reads that cache with `dw_use`.

What Stata *does* provide is the reviewer-side guard. `dw_require_no_api` is a preflight gate you place at the top of any `.do` script that would otherwise reach a live API; in a reviewer session it aborts with error 459 before any network code runs, naming the failing call site via its `context()` option. So Stata enforces the no-network half of the mode contract even though it cannot perform the network half.

Warning

A Stata-only pipeline cannot acquire fresh upstream data on its own. If your sector runs entirely in Stata, the producer step that hits UIS, SDMX, the World Bank, ILO, UNSD-SDG, or GitHub-raw must be written in R or Python. Plan the language split

at design time, not after the fact.

R and Python share **ten** `api=` dispatch keys — `uis`, `sdmx`, `sdmx_codelist`, `wb`, `wb_indicators`, `ilo`, `unsd_sdg`, `github_raw`, `http`, and `json_get` — with complete parity. Since v0.6.0, R adds an eleventh, `csv` (flat CSV-at-a-URL: SDMX REST-CSV, World Bank Data360 file URLs, ILOSTAT `rplumber format=.csv`); the Python sibling has not yet picked it up, so the two are level at ten shared keys with `csv` R-only.

A.3.2.1 R

```
df <- dw_api_fetch(api = "wb", cache_key = "wb_SI.POV.DDAY",
                   indicator = "SI.POV.DDAY", country = "AGO")
```

A.3.2.2 Python

```
df = dw_api_fetch(api="wb", cache_key="wb_SI.POV.DDAY",
                  indicator="SI.POV.DDAY", country="AGO")
```

A.3.2.3 Stata

```
* No dw_api_fetch in Stata. Pull in R/Python, then:
dw_use , filename("wb_SI.POV.DDAY.dta") path("$teamsRawData/_apis/wb")
```

A.3.3 Aggregation and survey weights

Function	R	Python	Stata
<code>aggregate_data</code> (v1, back-compatible)	Yes	Yes	n/a
<code>aggregate_data_v2</code>	Yes	Yes	n/a
<code>apply_time_window</code>	Yes	Yes	n/a
<code>generate_agg_footnote</code>	Yes	Yes	n/a
<code>dw_nestweight</code>	Yes	Yes	n/a
<code>dw_pop</code>	Yes	Yes	n/a
<code>dw_regions</code>	Yes	n/a	n/a

Most of the aggregation family is R-and-Python; of the two demographics convenience wrappers added at v0.4.0, `dw_pop` now ships in R and Python while `dw_regions` remains R-only. None of these functions has a Stata sibling, and none is on the roadmap. This is a scope decision: Stata sector pipelines aggregate with native commands

(collapse, egen). `dw_nestweight` ports the World Bank EduAnalyticsToolkit's Stata command `edukit_nestweight` to R and Python; it was deliberately not ported back to Stata, since a Stata pipeline can call the upstream `edukit` command directly. `dw_pop` wraps `dw_api_fetch()` for the World Bank total-population indicator (SP.POP.TOTL); `dw_regions` fetches the UNICEF region taxonomy and appends regional aggregates computed via `aggregate_data_v2()`. `dw_regions` ships in the R package only; both still lack a Stata sibling.

A.3.3.1 R

```
out <- aggregate_data_v2(df, value = "OBS_VALUE", by = "REGION",
                        method = "weighted_mean", weight = "POP",
                        coverage_threshold = 0.5)
```

A.3.3.2 Python

```
out = aggregate_data_v2(df, value="OBS_VALUE", by="REGION",
                        method="weighted_mean", weight="POP",
                        coverage_threshold=0.5)
```

A.3.3.3 Stata

```
* No aggregate_data_v2 in Stata. Aggregate natively (collapse / egen)
* or perform the aggregation in R/Python and dw_save the result for Stata.
```

A.3.4 Vintage sync

Function	R	Python	Stata
<code>cso_toolkit_check</code>	Yes	Yes	n/a
<code>cso_toolkit_diff</code>	Yes	Yes	n/a
<code>cso_toolkit_pull</code>	Yes	Yes	n/a

The version-drift helpers are R-and-Python only. They read the consumer's `.toolkit_manifest.yml`, query the upstream GitHub API for the latest tag, and warn or refresh when the pin is behind. Stata vendoring pins the same manifest, but the drift check has no Stata sibling — a Stata-only consumer either runs the check from R/Python or compares its `pulled_version` against the upstream tag list by hand. See the vintage-sync chapter for the manifest schema and upgrade flow.

A.3.5 Scaffolding and contract auditor

Function	R	Python	Stata
<code>create_profile</code>	Yes	Yes	n/a
<code>review_profile</code>	Yes	Yes	n/a
<code>create_sector_script</code>	Yes	Yes	n/a
<code>create_dw_sector_script</code>	Yes	Yes	n/a
<code>generate_markdown_report</code>	Yes	Yes	n/a
<code>process_all_csv_files</code>	Yes (folder mode of the report)	Yes	n/a
<code>test_scripts</code> (contract auditor)	Yes (.R scripts)	Yes (.py scripts)	n/a (no Stata auditor)
<code>dw_load_config</code> (profile config loader)	inside <code>create_profile</code> output	inside <code>create_profile</code> output	Yes (<code>dw_load_config.ado</code>)

Two points here:

- **The contract auditor does not cover Stata.** `test_scripts` scans .R scripts in R and .py scripts in Python, flagging raw IO/API calls the toolkit is meant to wrap (its `io-xlsx` rule, for instance, catches `getSheetNames/excel_sheets/loadWorkbook` and points the author at `dw_use(sheet = NULL)`). There is no equivalent scanner for .do / .ado files. A Stata sector's discipline is enforced at runtime by `dw_require_no_api` and the `dw_save` mode guard, not by a static auditor in CI. See the contract-auditor chapter.
- **`dw_load_config` is Stata's standalone analogue of the profile config-load step.** In R and Python, `create_profile` emits the YAML config load and `dw_mode` resolution into the scaffolded profile. Stata exposes a dedicated `dw_load_config.ado` that reads `~/.config/user_config.yml` with a hand-rolled key:value parser (no external dependency, AppLocker-safe) and populates the session globals. Same effect, different packaging — note that `dw_load_config` sets exactly the keys the YAML provides; it does not derive mode-aware globals on its own.

A.4 File-format support matrix

Auto-dispatch by file extension is the heart of the IO contract: you call `dw_save/dw_use` with a path, and the toolkit selects the reader/writer from the extension. The set of recognised extensions differs by language.

Format	Extension	R	Python	Stata
CSV	.csv	Yes	Yes	Yes

Format	Extension	R	Python	Stata
Gzipped CSV/TSV	<code>.csv.gz</code> , <code>.tsv.gz</code>	Yes	Yes	n/a
TSV / tab-delimited	<code>.tsv</code> , <code>.txt</code>	Yes	Yes	n/a
Excel	<code>.xlsx</code>	Yes	Yes (needs <code>openpyxl</code>)	Yes
Stata	<code>.dta</code>	Yes	Yes	Yes (native)
Parquet	<code>.parquet</code>	Yes	Yes (needs <code>pyarrow</code>)	Route via R/Python
JSON	<code>.json</code>	Yes	Yes	n/a
YAML	<code>.yaml</code> , <code>.yml</code>	Yes	Yes (needs <code>PyYAML</code>)	Read-only config via <code>dw_load_config</code>
R serialized	<code>.rds</code>	Yes	n/a	n/a
R image	<code>.RData</code> , <code>.rda</code>	Yes (sidecar skipped)	n/a	n/a
Python pickle	<code>.pkl</code> , <code>.pickle</code>	n/a	Yes	n/a

The cross-language reading of this table:

- **Stata’s IO is scoped to `.dta` / `.csv` / `.xlsx`.** This is an explicit design boundary. Parquet and the R/Python binary serialization formats (`.rds`, `.pkl`) are out of scope for Stata; a Stata pipeline needing a binary interchange format routes through R or Python, which `dw_saves` to `.dta` for Stata consumption.
- **`.rds` (R) and `.pkl`/`.pickle` (Python) are language-native mirrors.** Each language has exactly one native fast-serialization format, and the toolkit recognises only that language’s format. They are analogues of one another, not a shared capability.
- **`.RData` / `.rda` writes skip the provenance sidecar.** This is the one IO path where a write does *not* emit `.provenance.json`, because the format can hold multiple objects and the sidecar’s single-object schema does not apply. Every other write path emits the sidecar by default (opt out with `provenance = FALSE`).
- **Optional Python dependencies are lazy-imported.** Excel, Parquet, and YAML support in Python load `openpyxl`, `pyarrow`, and `PyYAML` only when the corresponding reader/writer is actually called. If you vendor the Python helpers without those extras, the core CSV / `.dta` / JSON paths still work; the format-specific paths raise only when invoked.

A.5 Content-integrity mechanism

The provenance sidecar records a content hash, but the hash is not computed the same way in every language — a subtle parity gap worth stating plainly, because it affects cross-tool integrity checks.

Language	Hash mechanism	Sidecar field
R	File-level SHA-256	<code>sha256</code>
Python	File-level SHA-256	<code>sha256</code>
Stata	<code>datasignature</code> (Stata-native SHA-1 of variables and values)	<code>datasignature</code>

Stata deliberately uses `datasignature` rather than a file-level SHA-256 so the helper never has to shell out — which keeps it AppLocker-safe on UNICEF laptops. The mechanisms differ in two ways: Stata’s signature is SHA-1 over the dataset’s *content* (variables and values), whereas R and Python hash the *bytes of the file on disk*. The consequence is twofold — an R-written file and a Stata-written file of the same data share neither the byte-for-byte file hash nor the same field name in the sidecar. Cross-tool integrity comparisons should therefore compare like with like, and a comparison that spans the Stata/R-or-Python boundary cannot rely on a single hash field. See the provenance-sidecar appendix for the full schema and the integrity-check semantics.

A.6 Supporting machinery: documentation, tests, CI

Parity is about more than the functions. The scaffolding around each language — reference docs, test surface, and continuous integration — is uneven, and that unevenness is part of an accurate picture of how production-ready each sibling is.

Capability	R	Python	Stata
Native package install	<code>devtools::install_local("src/...")</code>	<code>pip install -e python/</code>	n/a (vendoring only; no native install)
Generated API reference	Yes (Roxygen → 34 <code>.Rd</code> files + pkgdown site)	No autodoc; prose reference in <code>docs/</code> only	<code>.sthlp</code> help file per command
Type/interface metadata	NAMESPACE exports (~44, incl. aliases)	PEP 561 <code>py.typed</code> , full type hints	n/a
Unit/regression test suite	Yes (<code>testthat</code> : 32 test files, 370+ <code>expect_*</code> assertions)	Manual smoke test + error-envelope test	No automated suite
Wired into CI	Yes (<code>r-check.yml</code> : Ubuntu release+devel, macOS, Windows)	Conformance driver only (<code>conformance.yml</code>); unit tests manual-only	No (Stata conformance driver pending license provisioning, issue #133)

Capability	R	Python	Stata
Cross-language conformance gate	Yes (driver + comparator in <code>conformance.yml</code>)	Yes (driver + comparator in <code>conformance.yml</code>)	Pending (issue #133)
Error-envelope verification	<code>expect_envelope()</code> helper used across the suite	dedicated <code>error_envelope_test.py</code>	Enforced at call site; not separately tested

The headline gaps:

- **R is the only sibling whose *unit tests* run in CI.** GitHub Actions runs R CMD check (which invokes the `testthat` suite) on every push and pull request to `main/develop` across four configurations: Ubuntu on R-release and R-devel, macOS, and Windows. The Python smoke and error-envelope tests are real and runnable, but they are **manual-only** — bootstrapped by copying `python/src/` into a temp dir and run by hand, not gated in CI. The Stata side has no automated test suite at all.
- **Cross-language parity is now executed, not just asserted.** The Conformance workflow (`conformance.yml`, on every push and PR to `main/develop`) runs the R and Python drivers over the shared fixture — a `dw_use` → `dw_save` → `dw_use` round-trip — and a comparator asserts the outputs are value-equal within $1e-9$. The Stata leg of the same harness waits on a STATA_LIC secret or self-hosted runner (issue #133); until provisioned, Stata parity remains a local release gate.
- **Python lacks a generated reference site.** R ships a pkgdown-style autodoc built from Roxygen comments (34 `.Rd` files); Python has no equivalent pkgdown/Sphinx-style autodoc. Its public surface is documented in prose under `docs/` and in the per-module catalogue, and its interfaces are machine-readable via full type hints and a PEP 561 `py.typed` marker, but there is no rendered API site.
- **Stata documents per-command via `.sthlp`.** Each Stata `.ado` ships a companion `.sthlp` help file, accessible with `help dw_save` and friends. There is no aggregated reference site.

A.7 Summary: how complete is each sibling?

- **R** is the reference implementation: every capability family, the full format set including `.rds` and gzipped CSV/TSV, the only generated reference site, and the only CI-gated test suite. Its version stamp is v0.8.0, level with Python. If a behaviour is ambiguous, R defines it.
- **Python** is a near-complete functional port — core IO (including `.pkl`), all ten external-API dispatch keys, aggregation, vintage sync, and scaffolding — with full type hints, but no generated reference site, CI-manual-only testing, a couple of R-only helpers it has not yet picked up (`dw_stage`, `dw_regions`; `dw_root` now ships in Python too). Its version stamp matches R at 0.8.0, though those helper gaps remain.

- **Stata** is a deliberately scoped subset: full IO and mode contract for `.dta` / `.csv` / `.xlsx`, the reviewer no-API gate, recursive `mkdir`, and config loading — but **no external-API wrapper, no aggregation, no vintage-sync helpers, no contract auditor, no Parquet/binary formats, and no automated tests**. It enforces every half of the mode contract it can reach, and routes everything else through R or Python.

When you vendor, choose the language that fits the pipeline — and where a Stata pipeline needs a capability this matrix marks “Route via R/Python,” plan that crossing explicitly. The vendoring-model and onboarding chapters walk through how to wire a multi-language sector so the crossings are clean.

A.8 Related contracts outside this matrix

i Note

The survey-microdata archive tool **datalib** maintains its own trilingual contract — thirteen `datalib_*` functions implemented identically in Stata, R, and Python, pinned by an oracle-style golden-case conformance suite — tracked in its own repository (`unicef-drp/datalib-unicef`, internal at the time of writing). It is a tier-(ii) package in the ecosystem sense of the architecture chapter, not part of this matrix: its coverage is asserted and tested in its own repository, not here.

B Provenance sidecar schema

Every successful write through `dw_save()` deposits a companion file next to the primary artifact: `<path>.provenance.json`. Write `ed/dw_ed_edu.csv` and you also get `ed/dw_ed_edu.csv.provenance.json` in the same directory. The sidecar is the toolkit's record of who made the file and a way to check whether the bytes on disk have since been altered — it answers *who wrote this, when, in which session mode, from what shape of data, and does the content still match what was recorded*. Reviewer-mode integrity checks (re-hashing a file to confirm it has not changed since it was written) depend on it, which is why a producer-mode write copies the sidecar alongside the primary file to the two backup locations the toolkit mirrors canonical files to (a Teams folder and the shared Z: drive). The IO contract chapter and the vintage-sync chapter both lean on these checks.

The three languages diverge in one field that matters; this appendix is explicit about where.

B.1 Where the sidecar comes from

An internal helper writes the sidecar at the end of every primary write, unless you turn it off:

B.1.0.1 R

```
# r/R/dw_io.R - .write_provenance(path, x, fmt, vintage, metadata, isid)
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        provenance = TRUE) # TRUE is the default; pass FALSE to skip
```

B.1.0.2 Python

```
# python/src/dw_io.py - _write_provenance(path, x, fmt, vintage, metadata, isid)
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        provenance=True) # True is the default; pass False to skip
```

B.1.0.3 Stata

```
* stata/src/dw_save.ado - sidecar emitted unless `nosidecar` is passed
dw_save, filename("dw_ed_edu.dta") path("$teamsWrkData/ed") ///
    idvars(REF_AREA INDICATOR TIME_PERIOD)
* pass the `nosidecar` option to suppress
```

The R and Python helpers skip the sidecar for `.RData` / `.rda` writes, because those formats can hold arbitrary objects rather than a single tabular schema. A failed sidecar write never aborts the primary write: if a metadata value cannot be serialised to JSON, the helper warns you and leaves the primary file intact. The sidecar is metadata about the asset; the asset is what matters.

B.2 The core schema

The following fields appear in every sidecar. The table gives the JSON key, its type, the field's meaning, and which languages emit it.

Key	Type	Meaning	Languages
<code>path</code>	string	Absolute path of the primary file the sidecar describes.	R / Py / Stata
<code>format</code>	string	In R/Python, the dispatch format of the primary file (" <code>csv</code> ", " <code>parquet</code> ", " <code>dta</code> ", ...). In Stata, a literal schema-version tag " <code>dw_save.provenance/1</code> ".	R / Py / Stata (different meaning — see below)
<code>written_at</code>	string	Write timestamp. R/Python: ISO-8601 UTC, YYYY-MM-DDTHH:MM:SS+0000. Stata: its local <code>c(current_date)</code> <code>c(current_time)</code> , e.g. 28 Jun 2026 14:03:51.	R / Py / Stata (different format)

Key	Type	Meaning	Languages
<code>user</code>	string	The writing user. R/Python read <code>USERNAME</code> (Python falls back to <code>getpass.getuser()</code>); Stata reads <code>c(username)</code> .	R / Py / Stata
<code>dw_mode</code>	string	The session mode at write time (<code>"producer"</code> , <code>"reviewer"</code> , or unset). A session property, never a per-call argument — see the roles and mode-contract chapter.	R / Py / Stata
<code>sha256</code>	string null	Hex SHA-256 of the primary file's bytes, computed after the file is written. R/Python only. In R it is <code>null</code> when the optional <code>digest</code> package is not installed.	R / Py
<code>datasignature</code>	string	Stata's native <code>datasignature</code> content hash (a checksum of variable names and values), used in place of <code>sha256</code> . Stata only.	Stata

Key	Type	Meaning	Languages
<code>vintage</code>	string null	Optional vintage tag (e.g. "2026-05"), echoed from the <code>vintage</code> argument; null when not supplied. R/Python only (Stata records vintage inside <code>metadata</code> , see below).	R / Py
<code>isid</code>	array<string> null	The key columns asserted unique before the write (the <code>isid</code> quality contract). null when no uniqueness check was requested. R/Python key is isid.	R / Py
<code>idvars</code>	string	Stata's equivalent of <code>isid</code> — the space-separated <code>idvars()</code> columns passed to the uniqueness check. Stata key is idvars.	Stata
<code>schema</code>	object	The shape of the written data: <code>rows</code> , <code>cols</code> , and (R/Python only) <code>columns</code> . Empty object {} when the object was not a data frame.	R / Py / Stata
<code>metadata</code>	object	The caller-supplied descriptive metadata. Present only when the caller passed a <code>metadata</code> argument.	R / Py / Stata

i When the session mode is unset

`dw_mode` reflects whatever the profile placed in the session state. If no profile set a mode, the key is still emitted but carries a falsy value: in R it serialises as the empty string `""` because `Sys.getenv` / the unset global resolve to empty; in Python it serialises as `null` (the unset state returns `None`). The key is therefore always present — a sidecar with a blank or `null dw_mode` was written by a session that never declared producer or reviewer mode. (The API-fetch sidecar below substitutes the literal `"unknown"` in this case; the write sidecar does not.)

B.2.1 The schema object

`schema` captures the dimensions of what was written so a reader can sanity-check a file before trusting it.

Key	Type	Meaning	Languages
<code>schema.rows</code>	integer	Row count of the written data frame.	R / Py / Stata
<code>schema.cols</code>	integer	Column (variable) count.	R / Py / Stata
<code>schema.columns</code>	array<string>	Ordered list of column names. R/Python only — Stata records counts but not the name list.	R / Py

B.2.2 The metadata object

`metadata` is free-form descriptive lineage supplied by the caller. R and Python merge an arbitrary named list / mapping verbatim into the sidecar, so any key is permitted. Stata has no native nested-dictionary type. It therefore exposes a fixed set of named options plus a semicolon-separated pass-through, and closes the block with a sentinel `"_end": "true"` trailer — an artifact of writing JSON by hand, one line at a time, to avoid a dangling comma. The conventional keys, shared across languages, are:

Key	Type	Meaning
<code>title</code>	string	Human-readable title of the dataset.
<code>abstract</code>	string	Longer description (R/Python free-form).

Key	Type	Meaning
producer	string	The script that produced the file (e.g. 01_dw_prep/.../02_aggregate_uis_sd
sources	array<string> (R/Py) / string (Stata)	Upstream data sources (e.g. "UIS bulk SDG_092025", "WPP 2024").
contact	string	Owner handle (e.g. @karavan88).
vintage	string	Data vintage; in Stata this lives inside <code>metadata</code> , in R/Python it is a top-level field as well.

B.3 A real example (R / Python)

A write such as:

```
dw_save(edu_sdg_uis,
  name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
  isid = c("DATAFLOW", "REF_AREA", "INDICATOR", "TIME_PERIOD"),
  vintage = "2026-05",
  metadata = list(
    title = "Education indicators - UNICEF DW format",
    producer = "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.R",
    sources = c("UIS bulk SDG_092025", "WPP 2024"),
    contact = "@karavan88"
  ))
```

produces a sidecar of this shape:

```
{
  "path": "C:/Users/.../teamsWrkData/ed/dw_ed_edu.csv",
  "format": "csv",
  "written_at": "2026-06-28T14:03:51+0000",
  "user": "jpazevedo",
  "dw_mode": "producer",
  "vintage": "2026-05",
  "sha256": "9f2c1a7b4e0d8c5f3a6b2e1d9c0f4a7b8e3d2c1f0a9b8c7d6e5f4a3b2c1d0e9f",
  "isid": ["DATAFLOW", "REF_AREA", "INDICATOR", "TIME_PERIOD"],
  "schema": {
    "rows": 48213,
    "cols": 9,
```



```

    "columns": ["DATAFLOW", "REF_AREA", "INDICATOR", "SEX",
                "WEALTH_QUINTILE", "RESIDENCE", "TIME_PERIOD",
                "OBS_VALUE", "DATA_SOURCE"]
  },
  "metadata": {
    "title": "Education indicators - UNICEF DW format",
    "producer": "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.R",
    "sources": ["UIS bulk SDG_092025", "WPP 2024"],
    "contact": "@karavan88"
  }
}

```

The Python sidecar is the same shape: the helper builds the identical key set, serialises with `json.dump(..., indent=2)`, and computes `sha256` by streaming the file in 64-KiB blocks. R serialises with `jsonlite::write_json(..., auto_unbox = TRUE, pretty = TRUE)`. The two pretty-printers differ in whitespace, and `auto_unbox` means a single-key `isid` in R collapses to a bare string rather than a one-element array — but the keys, types, and values otherwise match.

B.4 The Stata sidecar: same shape, one difference that matters

Stata emits a sidecar built to interoperate with the R and Python siblings — the comment in `dw_save.ado` states the intent plainly: “Write a sibling `.provenance.json` sidecar with the same shape the R helpers emit, so reviewer-mode integrity checks work across both languages.” Two fields nonetheless cannot be identical, for reasons rooted in the Stata environment:

```

{
  "format": "dw_save.provenance/1",
  "path": "C:\\Users\\...\\teamsWrkData\\ed\\dw_ed_edu.dta",
  "written_at": "28 Jun 2026 14:03:51",
  "user": "jpazevedo",
  "dw_mode": "producer",
  "datasignature": "9:9(12345):1234567890:0987654321",
  "schema": { "rows": 48213, "cols": 9 },
  "idvars": "DATAFLOW REF_AREA INDICATOR TIME_PERIOD",
  "metadata": {
    "title": "Education indicators - UNICEF DW format",
    "producer": "01_dw_prep/012_codes/ed/02_aggregate_uis_sdg.R",
    "vintage": "2026-05",
    "_end": "true"
  }
}

```

The `datasignature` value above — `"9:9(12345):..."` — is a colon-delimited string of variable count, observation count, and two checksums, opaque by design. It is what stands in for `sha256`.

 The integrity field is not portable across Stata and R/Python

R and Python record a `sha256` — a hash of the file's *bytes*. Stata records `datasignature` — a hash of the *dataset's variables and values*, computed by Stata's built-in `datasignature` command. These are not comparable: a `.dta` and a `.csv` of the same logical table have different bytes (so different `sha256`) but could share a `datasignature`; conversely the same `datasignature` says nothing about file-byte equality. The choice is intentional. `datasignature` is Stata-native, needs no external dependency, and survives the UNICEF-laptop AppLocker restrictions that would block a third-party hashing binary. The practical consequence: the byte-equality check `dw_verify_z(compare = "sha256")` (which re-hashes a canonical file against its Z: mirror) works between R and Python sidecars, but a Stata-written file's integrity is verified against *its own* `datasignature`, not against an R/Python `sha256`.

The other differences are smaller but worth cataloguing:

- **written_at format.** R/Python emit ISO-8601 UTC (2026-06-28T14:03:51+0000). Stata emits its local human-readable clock (28 Jun 2026 14:03:51). A parser that wants a single timestamp type must normalise.
- **Key name for the uniqueness columns.** R/Python use `isid` (an array). Stata uses `idvars` (a single space-separated string).
- **format field meaning.** In R/Python `format` is the file's dispatch format (`"csv"`, `"parquet"`, ...). In Stata `format` is a literal schema-version tag, `"dw_save.provenance/1"`. The same key carries different semantics — do not assume `format` tells you the file type across all three languages.
- **schema.columns.** Present in R/Python, absent in Stata (which records only `rows` and `cols`).
- **vintage placement.** A top-level field in R/Python; nested inside `metadata` in Stata.
- **_end sentinel.** Stata appends `"_end": "true"` as the final key of the `metadata` block — a harmless artifact of hand-writing JSON one line at a time to avoid a dangling comma. Consumers should ignore it.

B.5 A note on the API-fetch sidecar

`dw_save()` is not the only producer of `.provenance.json` files. The external-API contract (see that chapter) freezes a fetched payload to disk and writes its own sidecar with a *different* shape — `url`, `sha256`, `bytes`, `fetched_at`, `fetched_by`, `dw_mode` (which falls back to the literal `"unknown"` when no mode is set). That variant records the provenance of a network fetch rather than a local write. It shares the `.provenance.json` extension and the `dw_mode` / hashing conventions, but its key set is distinct; do not parse it with the same reader you use for the write sidecar. This appendix documents only the `dw_save()` write sidecar.

B.6 Consuming the sidecar

A reviewer or a CI check typically does three things with a sidecar: read it, confirm the recorded `sha256` still matches the file on disk, and inspect `dw_mode` / `metadata` for lineage. Reading is plain JSON:

B.6.0.1 R

```
prov <- jsonlite::read_json("ed/dw_ed_edu.csv.provenance.json",
                           simplifyVector = TRUE)
identical(prov$sha256, digest::digest(file = "ed/dw_ed_edu.csv",
                                       algo = "sha256"))
```

B.6.0.2 Python

```
import json, hashlib
prov = json.load(open("ed/dw_ed_edu.csv.provenance.json"))
h = hashlib.sha256(open("ed/dw_ed_edu.csv", "rb").read()).hexdigest()
prov["sha256"] == h
```

B.6.0.3 Stata

```
* Stata has no JSON reader in the toolkit; verify via the native signature
use "ed/dw_ed_edu.dta", clear
datasignature
* compare r(datasignature) against the sidecar's "datasignature" value by eye
* or with a small text-parsing do-file
```

A `FALSE` / `False` result here means the file changed since it was written — the lineage record no longer describes the bytes on disk, so the sidecar can no longer vouch for the artifact. R and Python can run this byte-integrity check directly because they recorded a byte hash. Stata verification is intra-Stata: re-open the dataset, recompute `datasignature`, and compare against the value in the sidecar. There is no toolkit-provided JSON reader on the Stata side — a real coverage gap, consistent with Stata's lack of an API wrapper and parquet support documented in the cross-language coverage matrix appendix.

B.7 Summary

Across R and Python the sidecar is effectively identical — same keys, same types, a byte-level `sha256`, ISO-8601 UTC timestamps, an `isid` array, and a full `schema.columns` list. Stata emits a parallel file with the same intent and most of the same keys, but substitutes `datasignature` for `sha256`, `idvars` for `isid`, a local timestamp for the UTC one, and a version tag for `format`, and omits `schema.columns`. A cross-language consumer must therefore key off the right integrity field per language, normalise the timestamp, and treat a Stata sidecar’s hash as non-comparable to an R or Python one.

Relevant source files (all under `C:/GitHub/myados/cso-toolkit`):

- `r/R/dw_io.R` — `.write_provenance()` (R write sidecar) and `.write_remote_provenance()` (API-fetch sidecar)
- `python/src/dw_io.py` — `_write_provenance()`, `_sha256_file()`, `_utc_now_iso()` (Python write sidecar); `_write_remote_provenance()` (API-fetch sidecar)
- `stata/src/dw_save.ado` — Stata sidecar emission (`datasignature`, `idvars`, `format: dw_save.provenance/1`)
- `docs/dw_io_reference.md` — sidecar field overview

i Microdata records provenance structurally, not in a sidecar

The survey-microdata archive — datalib’s `Z:/datalib*` trees, a different data layer from the aggregate warehouse this sidecar serves — records provenance **structurally** rather than in a `.provenance.json` companion: the vintage is encoded in the IHSN folder name (`<CCC>_<YYYY>_<SSSS>_v<MM>_M` and its adaptation extension), and `Data/Original/` is kept untouched so the raw deposit is its own lineage record. Different mechanism, same intent as the sidecar — there is no datalib sidecar to document here. See the IHSN convention chapter.

C Error taxonomy

Every error the toolkit raises shares one shape. Wherever a `dw_*` helper calls `stop()` (R), `raise` (Python), or aborts a Stata command, the message follows a three-part **error envelope**: a WHAT line, a Why line, and a numbered Fix: block.

```
[cso_toolkit.<func>] WHAT went wrong
  Why this is a problem (the contract being protected)
  Fix:
    1. Do this, OR
    2. Do that.
```

The leading `[cso_toolkit.<func>]` prefix is the load-bearing convention. It separates library messages from your own code’s errors, and — because it is a stable literal — it lets you grep a whole consumer project for every site that hits a given error class. The R `testthat` suite holds raises to this shape through a custom `expect_envelope()` matcher (it asserts the `^[cso_toolkit.<func>]` prefix on the raised condition), and the Python suite runs a parallel `error_envelope_test.py` that exercises 30 raise-paths. The WHAT / Why / Fix structure is therefore a tested invariant, not a stylistic suggestion.

This appendix is a debugging quick-reference. It groups the toolkit’s errors into five categories, gives the actual envelope text for each, explains the contract the envelope defends, and tells you how to fix it. For the full per-function reference see the function reference appendix and the IO and external-API contract chapters; for the sidecar fields named below, see the provenance sidecar schema appendix.

Two terms anchor everything that follows. An **error envelope** is the three-part message above. A session’s **role** is one of three: a *producer* may write canonical data and reach the network; a *reviewer* is read-only and offline; a *publisher* submits to the upstream warehouse. The roles and mode-contract chapter is the full treatment; this appendix leans only on the producer-versus-reviewer split.

i Note

Most toolkit errors are *deliberate stops that protect reproducibility*, not bugs. A reviewer-mode write refusal, an `isid` failure, or a “cache missing” stop is the contract doing its job. The fix is almost never “make the toolkit stop complaining” — it is “satisfy the contract.”

C.1 The five categories at a glance

Category	Typical prefix	Root cause	Where it is raised
Mode-contract violation	[cso_toolkit.dw_save] [cso_toolkit.dw_use] [cso_toolkit.dw_api]	A session in the <i>reviewer</i> mode tried a forbidden action (reviewer writing canonical, reviewer reaching the network).	IO contract, external-API contract
Unsupported / mismatched format	[cso_toolkit.dw_save] [cso_toolkit.dw_use]	A file extension the toolkit does not dispatch, or a format unavailable in the language sibling you are using.	IO contract
API failure / blocked	[cso_toolkit.dw_api]	Reviewer-mode lockout with no cache, or a live HTTP failure (timeout, connection, non-2xx status).	External-API contract
Integrity / isid failure	[cso_toolkit.dw_isid] [cso_toolkit.dw_save] [cso_toolkit.dw_stage] [cso_toolkit.dw_verify]	Declared keys are not unique, or two copies of a “frozen” key disagree by hash.	IO contract, staging
Vendor drift	[cso-toolkit] (sync log), [cso_toolkit.cso_toolkit_tag]	The vendored helper version trails the <i>isid</i> toolkit tag pinned in the manifest.	Vintage-sync contract

C.2 1. Mode-contract violation

The toolkit enforces the producer / reviewer / publisher mode contract at every wrapped call site, not by convention (see the roles and mode-contract chapter). A reviewer session is *physically* prevented from two things: writing under canonical paths, and reaching the network. When a reviewer-mode session attempts either, the toolkit stops before any side effect occurs.

C.2.1 Reviewer writing under canonical

This is the canonical example of the whole envelope convention. A `dw_save()` whose resolved destination lies under `teamsWrkDataCanonical`, `teamsRawDataCanonical`, or the configured Z: drive root (`dwZDrive`) stops in reviewer mode:

```
[cso_toolkit.dw_save] Reviewer mode forbids writes to <root>: <path>
Reviewer sessions must keep canonical / Z: deposits read-only to preserve
vintage permanence; writes go to the local repo sandbox.
Fix:
  1. Resolve a sandbox path instead (the profile's `teamsWrkData` should
     point there in reviewer mode), OR
  2. If this is a deliberate Database Manager bootstrap, pass
     `allow_canonical_write = TRUE`.
```

The Z: branch of this check arrived in v0.4.0 (issue #14); v0.3.0 refused only Teams-canonical writes. The fix is to point the write at your sandbox — in reviewer mode the profile's `teamsWrkData` should already resolve there, so the artefact lands outside the deposit. The escape hatch exists for one case only: you genuinely are the Database Manager seeding a one-time cache into the canonical store. Then pass the explicit flag:

C.2.1.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        allow_canonical_write = TRUE)
```

C.2.1.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        allow_canonical_write=True)
```

C.2.1.3 Stata

```
* Stata's dw_save guards canonical + Z: writes via $dw_mode and accepts
* the same escape hatch as a flag (DBM bootstraps only):
dw_save, filename("dw_ed_edu.dta") path("$teamsWrkDataCanonical/ed") ///
    idvars(REF_AREA INDICATOR) allow_canonical_write
```

C.2.2 Reviewer fetching from the network (frozen remote)

`dw_use()` accepts HTTPS URLs since v0.4.0: a producer downloads once and freezes the response under a frozen root with a sidecar, and the reviewer reads only the frozen copy. If the frozen copy is missing, the reviewer cannot fetch it live, so the call stops. Since v0.4.x the envelope also names the resolution tier it chose — so you can tell whether the *path* is wrong or the *file* is genuinely absent:

```
[cso_toolkit.dw_use:remote] Reviewer mode forbids fetching from the network.
Missing frozen copy: <path>
URL: <url>
Frozen-root resolution: <chosen-root> (<tier-name>)
Fix:
  1. If the path above is wrong, set `dw_frozen_root <- '<your-canonical-frozen-path>`
  2. Otherwise, a producer must call dw_use('<url>') once and commit the frozen file + s
```

The tier line was added after a costly miss: a reviewer-mode audit spent three runs (75+ minutes on a slow network) tracing a fallback that resolved to `<githubFolder>/_frozen` instead of the project's expected `_frozen` path. The envelope now surfaces that mismatch at the point of failure. The frozen root resolves in tiers — an explicit `dw_frozen_root` global first, a `githubFolder` fallback next — and the toolkit warns once per session whenever it falls back past the first tier.

C.2.3 Reviewer reaching a live API

The external-API wrapper carries the same lock. When `dw_apis_allowed` is false and the requested cache is absent, `dw_api_fetch()` stops. In Python it raises a `PermissionError` with the full envelope:

```
[cso_toolkit.dw_api] Reviewer mode forbids live API calls.
Call:   dw_api_fetch('sdmx_codelist', cache_key='sdmx_codelist_UNICEF_CL_RESIDENCE_1_0')
Reason: cache missing at /data/raw-canonical/_apis/sdmx_codelist/sdmx_codelist_UNICEF_CL
Fix:
  1. The expected workflow is that a Database Manager has already populated
     the cache under teamsRawDataCanonical/_apis/. Verify the cache file exists.
  2. If you ARE the producer, switch modes:
     from cso_toolkit import _state
     _state.configure(dw_mode="producer", dw_apis_allowed=True)
```

The fix for a reviewer is to confirm the producer has deposited the cache — most often, nobody populated it. The cross-language way to assert the mode you expect appears in the external-API contract chapter; do it once at profile-load time, never per call.

Warning

The **Fix**: block here is genuinely the *expected* remedy for a reviewer: **verify the cache exists**, not “switch to producer.” Switching modes is the producer’s path. A reviewer who flips `dw_apis_allowed = TRUE` to make this stop go away has silently left the reproducibility floor.

The R and Stata siblings enforce the same lock through a `dw_require_no_api()` guard rather than a per-call envelope. In Stata, `dw_require_no_api` aborts with Stata error 459 and its own prefix:

```
[cso_toolkit.dw_require_no_api] Reviewer mode forbids live API calls.  
Why: reviewer sessions must read frozen producer caches to preserve  
vintage permanence; any live API call breaks provenance.
```

In R the guard is installed by the repo profile; its exact text depends on that profile, so match on the `[cso_toolkit.dw_api]` or `dw_require_no_api` token rather than on a fixed string.

Note

Coverage note: Stata ships the *guard* (`dw_require_no_api.ado`) but not the *fetcher*. `dw_api_fetch` / `dw_api_cached` / `dw_api_inventory` are R + Python only. A Stata pipeline that needs live data routes the fetch through R or Python on the producer side, then reads the cached `.dta`. See the coverage matrix appendix.

C.3 2. Unsupported / mismatched format

The R and Python IO helpers auto-dispatch on the file extension, covering CSV / TSV / TXT, XLSX (single and multi-sheet), RDS, RData, DTA, Parquet, JSON, and YAML, with optional `.csv.gz` compression. An extension outside that set raises from `dw_save()` / `dw_use()` in the same envelope shape:

```
[cso_toolkit.dw_save] Unsupported file extension '<ext>' (path: <path>).  
Supported extensions: csv, tsv, txt, xlsx, rds, RData, rda, dta, parquet, json, yml, yaml  
Fix: rename the output so it has one of the supported extensions.
```

The real gap, though, is **coverage asymmetry across siblings** (see the coverage matrix appendix). The format you can write in R is not always the format you can write in Stata or Python:

- **Stata `dw_save` writes `.dta` only.** It does not dispatch on extension — it always saves a Stata dataset, so there is no Parquet, RDS, or CSV write path on the Stata

side. Pipelines that need those formats produce them in R or Python and let Stata consume the `.dta`.

- **Stata `dw_use` reads `.dta`, `.csv`, and `.xlsx` only.** Parquet, RDS, and RData reads are out of scope on the Stata side; route them through R or Python.
- **`dw_merge`, `dw_verify_z`, `dw_stage`, and the standalone `dw_isid` are R + Python only.** Calling a name that exists in R but not in Stata fails as an *unknown command*, not as a toolkit envelope — so a “command not found” from Stata is itself a coverage signal, not a bug.
- **`.Rdata` / `.rda` writes skip the provenance sidecar** — not an error, but expect it: the write succeeds silently, with no `.provenance.json` companion.

The fix is to choose an extension the target sibling supports, or move the step to the language whose sibling implements the format. The contract is identical *where the format is implemented*; the gaps are in coverage, not in semantics. When in doubt, the coverage matrix appendix is the authoritative source for what each sibling can read and write.

C.4 3. API failure / blocked

Two distinct failure modes wear the `[cso_toolkit.dw_api]` prefix; separate them when debugging.

Blocked (policy): the reviewer-mode lockout from category 1. The network was never attempted; the toolkit refused on principle. The **Reason** line will say `cache missing`.

Failed (transport): a live call was attempted (you are in producer mode, the cache absent or `refresh = TRUE`) and the upstream HTTP call itself failed. In Python, every fetcher routes through a single internal HTTP layer (`_http_call`) that translates timeouts, connection errors, and non-2xx statuses into the same three-part envelope, annotated with the method, URL, status code, a truncated body snippet, and a fix hint:

```
[cso_toolkit.dw_api] HTTP GET <url> returned status 403.  
Body (truncated): '...'  
Fix: check the API's docs for that status code; common causes are bad  
query parameters (400), missing auth (401/403), missing cache_key (404),  
or rate limiting (429).
```

i Note

This wrapped transport envelope is a Python feature. The R fetchers call `httr::stop_for_status()`, so an R transport failure surfaces as a raw `httr` condition without the `[cso_toolkit.*]` prefix. Don't grep for the prefix on the R side of a transport error; match on the function and the HTTP status instead.

Which path you are on depends on your role and the cache state:

Session	Cache state	Action
Producer	cache present, <code>refresh = FALSE</code>	reads cache
Producer	cache missing OR <code>refresh = TRUE</code>	hits live API, writes cache, returns
Reviewer	cache present	reads cache
Reviewer	cache missing	STOPS (blocked — category 1)

So if you see a transport failure at all, you are necessarily a producer: a reviewer never reaches the transport layer. The toolkit cannot retry upstream availability for you. Re-run once the source is reachable; the producer-side cache then absorbs the result so the reviewer never depends on upstream uptime. (UNICEF SDMX, in particular, is prone to multi-hour 403 bursts — the cache is exactly the buffer that decouples reviewers from those bursts.) Use `dw_api_inventory()` to confirm what is already cached before re-fetching:

C.4.0.1 R

```
dw_api_inventory("wb")    # what's cached for the World Bank source?
```

C.4.0.2 Python

```
dw_api_inventory("wb")    # DataFrame: api / cache_key / ext / size_bytes / mtime / fetcher
```

C.4.0.3 Stata

```
* No API wrapper in the Stata sibling - dw_api_fetch / dw_api_cached /
* dw_api_inventory are R + Python only (see the coverage matrix appendix).
```

i Note

On the Python side, the toolkit redacts secrets (tokens, keys) from caller keyword arguments before writing them to the provenance sidecar — a `<redacted>` placeholder replaces the value (the `_redact_sensitive` helper, added after a Copilot finding on PR #7). If a persisted sidecar's `fetch_args` look scrubbed, that is the redaction layer working as intended. This redaction is implemented in the Python sibling; do not assume an R-side equivalent.

C.5 4. Integrity / isid failure

This category protects the *content* of an artefact, not the *permission* to write it. Three sub-cases.

C.5.1 isid uniqueness failure

`isid` is Stata's *is-identifier* check: does this set of columns uniquely identify each row? `dw_save(..., isid = ...)` runs that assertion on the declared key columns *before* writing. If duplicate rows exist on those keys, the call stops, prints the duplicate count and the first offending rows, and writes nothing:

```
[cso_toolkit.dw_isid] (<path>) <N> duplicate row(s) on key (<keys>).  
First duplicates:  
<sample rows>  
Fix: deduplicate before saving (`df <- dplyr::distinct(df, <keys>, .keep_all = TRUE)`)  
or extend the key set so the rows become unique.
```

C.5.1.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",  
        isid = c("DATAFLOW", "REF_AREA", "INDICATOR", "SEX",  
                  "WEALTH_QUINTILE", "RESIDENCE", "TIME_PERIOD"))
```

C.5.1.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",  
        isid=["DATAFLOW", "REF_AREA", "INDICATOR", "SEX",  
              "WEALTH_QUINTILE", "RESIDENCE", "TIME_PERIOD"])
```

C.5.1.3 Stata

```
dw_save, filename("dw_ed_edu.dta") path("$teamsWrkData/ed") ///  
    idvars(DATAFLOW REF_AREA INDICATOR SEX WEALTH_QUINTILE RESIDENCE TIME_PERIOD)
```

In Stata the uniqueness check is embedded in `dw_save` (the `idvars()` option), not exposed as a standalone command. Note that the Stata failure message is terser and does *not* carry the `[cso_toolkit.*]` prefix:

```
dw_save: idvars [<keys>] do not uniquely identify the dataset (or contain missing). No fil
```

In R and Python, `dw_isid(df, keys)` is also callable on its own as a pre-flight check, and that standalone path raises the full `[cso_toolkit.dw_isid]` envelope above. Either way the fix is the same: the duplicates are real — collapse, de-duplicate, or widen the key set so the declared keys truly identify a row. The printed sample tells you *which* keys collide.

C.5.2 Frozen-copy / staging integrity (`dw_stage`)

The staging lifecycle copies a canonical artefact into the reviewer's sandbox and hashes it, so a reviewer reruns against a byte-verified local copy. Several integrity stops live here, each with its own WHAT token:

- `[cso_toolkit.dw_stage]` COPY INTEGRITY — the freshly staged copy's hash does not match the source hash after a retry. The copy is corrupt; the staged file and sidecar are not trusted.
- `[cso_toolkit.dw_stage]` SANDBOX DRIFT — the sandbox file's hash no longer matches the hash archived in its `.staged.json` sidecar. Something altered the sandbox copy out of band. Fix: re-stage with `overwrite = TRUE`.
- `[cso_toolkit.dw_stage]` CANONICAL INTEGRITY — both the Z: and Teams mirrors hold the file but disagree by hash. Neither is used as a source; nothing is written. Fix: this is a genuine deposit inconsistency — escalate to the producer to reconcile the two mirrors before any reviewer run can proceed.

An upstream-changed condition (the sandbox still matches its own archive, but the Z:/Teams source has since changed) is a **warning, not a stop** (UPSTREAM CHANGED): the sandbox is never auto-refreshed, because re-staging is the reviewer's explicit decision.

C.5.3 Z: mirror integrity (`dw_verify_z`)

On a canonical read in producer mode, `dw_use()` compares the Teams and Z: copies (by file size by default; `verify_z = "sha256"` for a deep check). A mismatch emits a **warning** and the read still completes — this is intentionally non-fatal, because a size difference is often a benign line-ending artefact rather than a content divergence. Pass `verify_z = FALSE` to skip the check, or `verify_z = "sha256"` when you need certainty. (`dw_verify_z` is R + Python only; the Stata `dw_use` runs an equivalent non-blocking size/datasignature check inline.)

C.6 5. Vendor drift

The toolkit is **vendored**, not installed over the network: consumers copy the helpers into their own `00_functions/` and pin a version in `.toolkit_manifest.yml` (see the vendoring-model chapter). Drift is the gap between that pinned version and the upstream latest tag.

`cso_toolkit_check()` reads the local manifest, asks the GitHub API for the latest tag, and surfaces a log line when the consumer is behind. The message is a `message()` (a yellow console log), not a hard stop:

```
[cso-toolkit] unicef-drp/cso-toolkit: pinned v0.4.0 -> upstream 0.5.0 (newer tag available)
```

When the pin matches the upstream tag, the log reads (`up to date`) instead.

C.6.0.1 R

```
cso_toolkit_check()    # logs if the pinned version trails the upstream tag
cso_toolkit_diff()     # what changed between pinned and latest
cso_toolkit_pull()     # refresh the vendored copy (producer / online only)
```

C.6.0.2 Python

```
cso_toolkit_check()
cso_toolkit_diff()
cso_toolkit_pull()
```

C.6.0.3 Stata

```
* The vintage-sync family (cso_toolkit_check / _diff / _pull) is R + Python
* only; the Stata sibling has no manifest-sync helper (coverage matrix appendix).
```

Two limits to note:

- **Drift detection is tag-level, not file-level.** `cso_toolkit_check()` compares the pinned manifest version against the upstream latest tag; per-file hash drift (catching a hand-edited vendored copy that *claims* a clean version) is planned, not shipped. A consumer that locally edited `dw_io.R` but kept the manifest pin is **not** flagged.
- **The check itself needs the network.** `cso_toolkit_check()` calls the GitHub API, so it is unavailable in reviewer mode (it skips with a notice). Run the drift check as a producer, or in CI, not from a locked reviewer session. `cso_toolkit_pull()` goes further and *stops* in reviewer mode:

```
[cso_toolkit.cso_toolkit_pull] Forbidden in reviewer mode.
```

```
Reason: pulling a new toolkit version requires hitting GitHub, which the
reviewer-mode contract forbids.
```

```
Fix: switch the profile to producer mode (set dw_mode: producer in
user_config.yml) before re-running.
```

When behind, run `cso_toolkit_diff()` to read the changes, then `cso_toolkit_pull()` to refresh the vendored copy and re-pin the manifest. Because vendoring exists precisely to guarantee vintage permanence, treat a pull as a deliberate version bump in the consumer repo — review the diff, do not auto-pull on every session.

C.7 Reading any envelope: a debugging recipe

When you hit a `[cso_toolkit.*]` stop and are not sure which category it is:

1. **Read the `<func>` token** — it tells you the contract layer: `dw_save` / `dw_use` (IO), `dw_api` / `dw_require_no_api` (external API), `dw_isid` / `dw_stage` / `dw_verify_z` (integrity), `cso_toolkit_pull` (drift).
2. **Read the second word of the WHAT line.** Reviewer mode forbids → category 1. COPY INTEGRITY / SANDBOX DRIFT / CANONICAL INTEGRITY → category 4. duplicate row(s) → category 4 (isid). HTTP ... returned status → category 3. newer tag available → category 5.
3. **Check your `dw_mode` first.** A large share of these stops dissolve once you confirm whether the session is a producer or a reviewer and whether that role is *correct* for the step you are running.
4. **Grep the consumer project for the prefix** (for example `[cso_toolkit.dw_save]`) to find every other site that can hit the same class — useful when one fix should be applied repo-wide. Remember the R transport errors (category 3) and the Stata `isid` message are the two cases that do *not* carry the prefix.
5. **Do what the Fix: block says, in order.** The numbered options are ranked: option 1 is the expected remedy; option 2 is usually the deliberate escape hatch reserved for the Database Manager.

For the precise arguments and return values behind each helper named above, see the function reference appendix; for the sidecar fields the integrity checks compare, see the provenance sidecar schema appendix.

C.8 The datalib error taxonomy

The survey-microdata tool `datalib` uses a **typed** error taxonomy too, but with a **different envelope shape** from the toolkit's: it raises a stable class/code per failure — a Stata exit code, an R condition class, a Python exception — rather than the toolkit's

[`cso_toolkit.<func>`] **WHAT / Why / Fix:** message prefix. The two are complementary, not interchangeable; do not merge them into one table.

Contract error	Stata	R class	Python
invalid input	198	<code>datalib_error_input</code>	<code>InvalidArgumentError</code>
not found (country/survey/vintage/file)	198/601	<code>datalib_error_input</code>	<code>NotFoundError</code>
config file missing	601	<code>datalib_error_config</code>	<code>ConfigFileNotFound</code>
user block missing	459	<code>datalib_error_user_block</code>	<code>UserBlockNotFound</code>
library root unset	198	<code>datalib_error_input</code>	<code>DatalibRootNotSet</code>

Unlike the toolkit envelope, `datalib` does not contract a **Fix:** block: the class/code *is* the machine-actionable surface — the Stata exit code, the `datalib_error_*` R condition class, or the Python exception type is what a caller catches and branches on. For the functions that raise these, see the `datalib` API chapter.

D Function reference

This appendix is a *directory, not a manual*. It lists every public function the toolkit exports, groups them into their capability families, and points you to the authoritative per-function documentation for each language. Use it to find a function by name and see which family it belongs to, which languages implement it, and where to read the argument-by-argument details. The full prose reference — argument descriptions, return values, and runnable examples — lives in the generated documentation sites described below.

The families here mirror the chapters of Part III. For the *behaviour* of a family — what it does and why — read the matching chapter (the IO contract, the external-API contract, aggregation, vintage sync, scaffolding and the contract auditor). For the *signatures*, read the language-specific reference sites named at the foot of this page. The coverage-matrix appendix is the canonical source for the cross-language gaps summarised below.

A note on terms used throughout the tables. The toolkit routes paths and gates the network by *mode*: a script runs as the data **producer** (writes the canonical deposit, may hit the live network) or as the **reviewer** (reads frozen artefacts only, with the network barred). A **canonical** write is a final published deposit, as opposed to a working artefact. These distinctions are load-bearing for the IO and external-API tables; the external-API contract chapter explains them in full.

D.1 Where the per-function reference lives

The three language siblings document themselves in three different idioms, and coverage is uneven — complete in R, site-less in Python, partial in Stata.

R — pkgdown site (complete). The R package is fully documented with roxygen2 and rendered to a pkgdown site published at <https://unicef-drp.github.io/cso-toolkit>. Every exported function has a generated .Rd help page; open it in R with `?dw_save`, or browse the site's *Reference* tab. The *Reference* index is organised into the same capability families used below. That grouping is driven by `r/_pkgdown.yml`, which assigns each function to a family through roxygen `@family` tags (`io`, `api`, `sync`, `aggregate`, `demographics`, `survey-weights`, `reporting`, `scaffolding`, `audit`). roxygen2 renders each `@family` tag as a `\concept{}` entry in the .Rd, which is what the `has_concept()` selectors in `_pkgdown.yml` match. The R site is the most complete reference of the three and is the recommended starting point even for Python and Stata users, because the behaviour contract is shared.

Python — module docstrings only (no autodoc site). The Python package carries full type hints (it ships `src/py.typed` per PEP 561) and per-function docstrings,

but there is **no generated documentation site** — no Sphinx, no pkgdown equivalent. To read the Python reference, either introspect in a session (`help(dw_save)` after `from cso_toolkit import dw_save`) or read the hand-written Markdown references in the repo: `docs/dw_io_python_reference.md` and `docs/dw_api_python_reference.md`. These two files cover the IO and external-API families; the remaining Python families are documented only by their docstrings.

Stata — native .sthlp help files (partial coverage). Each shipped Stata command has a companion `.sthlp` help file next to its `.ado` in `stata/src/`, viewable with the usual `help dw_save`. Stata implements only the subset of the contract that fits its idioms, so its reference covers far fewer functions than R or Python (see the per-family coverage notes and the coverage-matrix appendix).

i Note

The pkgdown *Reference* index is generated from the roxygen `@family` tags (which roxygen2 emits as `\concept{}` entries the `has_concept()` selectors match), not hand-maintained. If you add an exported R function, tag it with the right `@family` so it lands in the correct family on the site; an untagged export is still exported but is orphaned from the reference index.

D.2 Reading the family tables

Each family below lists its members and shows which languages export the function today. A check means the function is exported and documented in that language; an em dash means it is not implemented there; a parenthetical note records a partial or deferred state. These columns are a quick orientation only — the coverage-matrix appendix is authoritative and tracks the issues behind each gap.

Two naming conventions matter when you read across the columns:

- **R dual names.** Several R functions are exported under **two names** — a short historical name and a `dw_`-prefixed alias. The `dw_` aliases were introduced incrementally across v0.4.4 and v0.4.5 (issue #42 / PR #48) and remain exported alongside the bare names. The tables list the canonical `dw_`-prefixed name first and note the bare alias.
- **Python exports both names too.** The Python package (`python/src/__init__.py`) exports the **bare** name *and* a `dw_`-prefixed alias for every dual-named function — `aggregate_data_v2` / `dw_aggregate_data_v2`, `create_profile` / `dw_create_profile`, `test_scripts` / `dw_test_scripts`, and so on — so the canonical `dw_`-prefixed spelling resolves identically in R and Python. (Earlier these aliases were R-only, and the `dw_` spelling raised `ImportError` in Python.)

Warning

Version stamps, not feature parity. The R and Python packages both report **v0.6.0**. A matching stamp is a floor, not a guarantee that every behaviour is present in both: some noted below (for example the Z:-root recognition in `dw_is_canonical`, or the read-all-sheets `dw_use(sheet = NULL)`) landed in R and may not yet be present, or may behave differently, in the Python sibling. When a feature's vintage matters, confirm it against the language you are actually running.

D.3 IO contract

Uniform read / write / compare / merge helpers with mode-aware path routing (paths resolve differently for the producer and the reviewer), the Z: drive mirror on canonical writes, and emission of a `.provenance.json` sidecar. See the IO contract chapter for behaviour.

Function	R	Python	Stata	Notes
<code>dw_save</code>				Write + uniqueness check (isid) + sidecar + Z: mirror; prompts before overwriting a producer deposit (v0.5.0), <code>force = TRUE</code> to override
<code>dw_use</code>				Read with mode-branched resolver; accepts HTTPS URLs since v0.4.0; in R, <code>sheet = NULL</code> reads all sheets of an <code>.xlsx</code> (v0.5.0)

Function	R	Python	Stata	Notes
<code>dw_compare</code>				Diff two artefacts on id + value vars; tolerates degenerate sides with a warning (v0.5.0)
<code>dw_merge</code>			—	R + Python only
<code>dw_isid</code>			embedded	In Stata the uniqueness check lives inside <code>dw_save</code>
<code>dw_verify_z</code>			—	R + Python only
<code>dw_resolve_path</code>			—	Mode-aware path resolver
<code>dw_is_canonical</code>			—	Detects a canonical deposit; recognises the Z: (<code>dwZDrive</code>) root in R as of v0.5.0
<code>dw_root</code>		—	—	Public wrapper around the root resolver (re-exported v0.4.6)
<code>dw_stage</code>		—	—	R export
<code>dw_publish</code>	(stub)	—	—	STUB / dry-run only — see callout below
<code>dw_toolkit_version</code>			—	Reports the installed toolkit version
<code>dw_verbosity</code>		—	—	Sets session verbosity / debug options (v0.4.10)

Function	R	Python	Stata	Notes
<code>dw_mkdir</code>	—	—		Stata-only: recursive mkdir (Stata's built-in is not recursive)
<code>dw_map_drive</code>	—	—		Stata-only onboarding: maps the team network drive from config (port of datalib's <code>mapzdrive</code>)

Warning

`dw_publish` is exported but remains a **stub** as of v0.5.0: only the dry-run path (`dry_run = TRUE`, the default) is supported, returning a validated payload with a sha256. A live submission (`dry_run = FALSE`) deliberately raises an error pointing at issue #15. Do not treat it as a working submission path; the live Helix branch is still pending. See the versioning and releases chapter.

D.3.0.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk", isid = "REF_AREA")
df <- dw_use(name = "dw_ed_edu.csv", sector = "ed", kind = "wrk")
```

D.3.0.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk", isid="REF_AREA")
df = dw_use(name="dw_ed_edu.csv", sector="ed", kind="wrk")
```

D.3.0.3 Stata

```
dw_save, filename("dw_ed_edu.dta") path("$teamsWrkData/ed") idvars(REF_AREA)
dw_use, filename("dw_ed_edu.dta") path("$teamsWrkData/ed")
```

D.4 External-API contract

Mode-aware wrappers around the supported external data sources (ten in all, among them UIS, the World Bank, ILO, the UNSD-SDG database, SDMX endpoints, and GitHub-raw). The producer hits the live API and caches the result; the reviewer reads from the canonical cache only and is barred from the network. See the external-API contract chapter.

Function	R	Python	Stata	Notes
<code>dw_api_fetch</code>			—	The main dispatcher (<code>api</code> = selects the source)
<code>dw_api_cached</code>			—	Cache-only read
<code>dw_api_inventory</code>			—	List what is cached
<code>dw_default_unicef_allowlist</code>		—	—	Default URL allowlist for <code>dw_use HTTPS</code> freezing (R only)

Warning

Stata has no external-API wrapper. The entire API family is R + Python only. A Stata reviewer can still re-run from frozen artefacts, but Stata cannot fetch from UIS, SDMX, the World Bank, ILO, UNSD-SDG, or GitHub-raw through the toolkit. Stata enforces its reviewer-mode network lockout separately, through `dw_require_no_api` (listed under Stata-only helpers below).

```
# Python - api= selects the source
df = dw_api_fetch(api="wb", indicator="SP.POP.TOTL", sector="ed", kind="wrk")
```

D.5 Aggregation and survey weights

Stratified aggregation, time-window filtering, footnote formatting, and survey-weight redistribution. See the aggregation chapter.

Function (R canonical)	R	Python	Stata	Notes
<code>dw_aggregate_data_v2</code>			—	Current aggregator; bare name <code>aggregate_data_v2</code> (the Python export)
<code>dw_aggregate_data</code>			—	Earlier aggregator; bare name <code>aggregate_data</code>
<code>dw_apply_time_window</code>			—	Latest obs per country in a year window; bare name <code>apply_time_window</code>
<code>dw_generate_agg_footnote</code>			—	Standardised coverage footnote; bare name <code>generate_agg_footnote</code>
<code>dw_nestweight</code>			—	Survey-weight redistribution; R port of EduAnalytic- sToolkit's <code>edukit_nestweight</code> (no bare alias — <code>dw_nestweight</code> is the only name in both R and Python)

Per the naming conventions above, R and Python both export the `dw_-`prefixed and the bare name for the first four functions.

D.5.0.1 R

```
out <- dw_aggregate_data_v2(df, stat = "weighted_mean", by = "REGION")
```

D.5.0.2 Python

```
out = aggregate_data_v2(df, stat="weighted_mean", by="REGION")
```

D.5.0.3 Stata

```
* not implemented in Stata
```

D.6 Demographics

Convenience wrappers for the population denominators and the UNICEF region taxonomy that most sector pipelines need. See the aggregation chapter for how they feed coverage.

Function	R	Python	Stata	Notes
dw_pop			—	World Bank SP.POP.TOTL denominators; added v0.4.0, Python sibling added for issue #17
dw_regions		—	—	Append UNICEF regional aggregate rows; added v0.4.0 (issue #18)

i Note

dw_pop ships in **R** and **Python**; **dw_regions** is **R-only today**. The Stata sibling of **dw_pop** and the Python and Stata siblings of **dw_regions** are not yet shipped; a **dw_regions()** redesign against the Country-and-Region-Metadata-API (issue #40) is on the roadmap.

D.7 Vintage sync

Consumer-side helpers that read the local `.toolkit_manifest.yml`, query upstream for the latest tag, and warn or refresh when the vendored copy is behind. Reviewer mode forbids the network query. See the vintage sync chapter.

Function	R	Python	Stata	Notes
<code>cso_toolkit_check</code>			—	Compare the pinned vintage against upstream
<code>cso_toolkit_diff</code>			—	Show what changed between vintages
<code>cso_toolkit_pull</code>			—	Refresh the vendored copy

These three keep the `cso_toolkit_` prefix in both R and Python (no `dw_` alias).

D.8 Scaffolding and the contract auditor

Generators for the standard CSO building blocks (profile scripts, sector run-scripts), an auditor for existing profiles, and the raw-call contract auditor. See the scaffolding and contract auditor chapter.

Function (R canonical)	R	Python	Stata	Notes
<code>dw_create_profile</code>			—	Scaffold <code>profile_<repo>.R / .py</code> ; bare name <code>create_profile</code>
<code>dw_create_sector_script</code>			—	Scaffold a sector run-script template; bare name <code>create_sector_script</code>
<code>create_dw_sector_script</code>			—	DW-Production convenience wrapper (no <code>dw_</code> -prefixed alias by design — it already carries a <code>dw</code> infix)

Function (R canonical)	R	Python	Stata	Notes
<code>dw_review_profile</code>			—	Audit a profile for required blocks (pass/warn/fail); bare name <code>review_profile</code>
<code>dw_test_scripts</code>			—	Contract auditor: flag raw IO / API calls; bare name <code>test_scripts</code>

As above, R and Python both export the `dw_`-prefixed and the bare name for the dual-named helpers.

i Note

A future v0.5.x rename of `test_scripts` / `dw_test_scripts` to `dw_audit_scripts` (to surface the audit intent and avoid the testthat name collision) is on the design horizon; the current prefix-only alias keeps that cleanup additive.

```
# R - audit a directory of scripts for raw IO / API calls
dw_test_scripts("R/", error_on_violation = TRUE)
```

D.9 Reporting and uniqueness utilities

Smaller exported utilities that do not anchor a chapter of their own.

Function (R canonical)	R	Python	Stata	Notes
<code>dw_generate_markdown_report</code>			—	Markdown report from a CSV; bare name <code>generate_markdown_report</code>
<code>dw_process_all_csv_files</code>			—	Batch the report over a directory; bare name <code>process_all_csv_files</code>

Function (R canonical)	R	Python	Stata	Notes
<code>dw_isid</code>			embedded	Uniqueness assertion (also invoked inside <code>dw_save</code>); see the IO contract table — listed here only for cross-reference

D.10 Stata-only helpers

Three Stata commands have no R or Python sibling because they fill gaps specific to Stata’s runtime. Each is documented in its `.sthlp` file in `stata/src/`.

Command	Purpose
<code>dw_mkdir</code>	Recursive directory creation (Stata’s built-in <code>mkdir</code> is not recursive)
<code>dw_require_no_api</code>	Reviewer-mode no-API gate — Stata’s equivalent of the network lockout
<code>dw_load_config</code>	AppLocker-safe YAML config loader for the profile (loads config without an external binary the corporate AppLocker policy would block)

The full set of files under `stata/src/` is: `dw_save`, `dw_use`, `dw_compare`, `dw_mkdir`, `dw_map_drive`, `dw_require_no_api`, and `dw_load_config`, each shipped as an `.ado` plus a matching `.sthlp`.

D.11 How to find a function fast

- **You know the name.** In R, `?<name>`; in Python, `help(<name>)` after importing it from `cso_toolkit` (either the bare name or the `dw_` prefix works for the dual-named functions); in Stata, `help <command>`.
- **You know the task but not the name.** Find the matching chapter in Part III, then return here for the family table.
- **You want the rendered web reference.** Open the pkgdown site at <https://unicef-drp.github.io/cso-toolkit> and use the *Reference* tab; it is the most complete of the three languages even when you work in Python or Stata.

- **You want to know whether a language has a function.** Use the tables above for a quick read and the cross-language coverage matrix appendix for the authoritative, issue-tracked answer.

D.12 Authoritative source files

The lists on this page are generated from, and should be reconciled against, these files in the repository. When the exports drift from this page, the files below win — this appendix is a finding aid, not the contract.

- `r/NAMESPACE` — the complete list of R exports (the source of truth for the R column).
- `r/_pkgdown.yml` — the family grouping rendered on the pkgdown site, driven by roxygen `@family` tags (which roxygen2 emits as `\concept{}` entries the `has_concept()` selectors match).
- `python/src/__init__.py` — the Python `__all__` export list and the per-module import map.
- `stata/src/` — one `.ado` plus one `.sthlp` per Stata command.
- `docs/dw_io_python_reference.md`, `docs/dw_api_python_reference.md` — the hand-written Python IO and API references.
- `docs/dw_io_reference.md`, `docs/dw_api_reference.md` — the R-side narrative references for the IO and API families.

D.13 Microdata archive (`datalib_*`)

Access: internal — request via the CSO team (repository visibility is tracked in datalib issue #20).

This family belongs to **datalib**, the survey-microdata archive tool — a different data layer from the toolkit’s `dw_*` surface. It shelves and loads survey **microdata** under the `Z:/datalib*` IHSN trees; those deposits never enter the aggregate warehouse 060.DW-MASTER. It is documented in full in Part V, and appears here only as a directory entry so the two contracts can be found in one place. As of **datalib v0.7.0 / contract v1** the surface is thirteen `datalib_*` functions: Stata and Python export all thirteen; R exports the thirteen canonical names.

Function	R	Python	Stata	Notes
<code>datalib_config</code>				Load the per-operator config block (the <code>datalib:</code> root key in <code>~/.config/user_config.yml</code>). Python also exports the verbatim alias <code>getuserconfig</code> ; Stata keeps <code>getuserconfig</code> as the original command
<code>datalib_root</code>				Resolve the library root (explicit arg $\rightarrow$ <code>DATALIB_ROOT</code> $\rightarrow$ language option $\rightarrow$ config key $\rightarrow$ error)
<code>datalib_countries</code>				Enumerate country folders under a root
<code>datalib_surveys</code>				Enumerate survey folders for a country (exact <code>*_<SURVEY></code> suffix match, never substring)
<code>datalib_vintages</code>				List master vintages; “latest” is the numeric maximum (<code>v10 > v09</code>)
<code>datalib_adaptations</code>				List adaptation vintages of a collection

Function	R	Python	Stata	Notes
<code>datalib_resolve</code>				Resolve country / year / survey / collection to a verified vintage folder
<code>datalib_files</code>				List files by section: data · data-original · data-r · data-other · doc · programs (identical sections in all three languages)
<code>datalib_catalog</code>				Inventory one root per call (multi-root estates call it per root)
<code>datalib_load</code>				Load, normalise, and merge without mutating data; returns a data frame, or a named list when merge is off
<code>datalib_browse</code>				Interactive folder navigation (in Stata, bare datalib with no arguments)
<code>datalib_create</code>				Scaffold a grammar- conforming vintage folder (flow deposits)

Function	R	Python	Stata	Notes
<code>datalib_map_drive</code>				Map the Z: share. Python also exports the verbatim alias <code>mapzdrive</code> ; Stata keeps <code>mapzdrive</code> as the original command

D.13.1 Where the per-function reference lives

`datalib` documents itself in the same three idioms as the toolkit. **R** — roxygen2 man pages for the thirteen canonical names; open one with `?datalib_resolve`. **Python** — per-function docstrings (`help(datalib_resolve)` after `from datalib import datalib_resolve`) plus the CLI entry point `python -m datalib`. **Stata** — `help datalib`, and a companion `.sthlp` file next to each command's `.ado`.

This family is tracked in its own repository, `unicef-drp/datalib-unicef`, not in this book's source tree; the coverage-matrix appendix's `datalib` note is the authoritative pointer to it. Because `datalib` is a tier-(ii) ecosystem package (see the architecture chapter), its cross-language coverage is asserted and tested in that repository, not in this appendix.

E Glossary

This glossary documents `cso-toolkit` as of **v0.8.0** (2026-07-13); version notes mark when a behaviour was added. It defines the load-bearing terms used throughout the handbook. The toolkit’s vocabulary is deliberately small, but several of the terms carry precise, contractual meaning — *canonical deposit*, *reviewer mode*, *provenance sidecar* — and the most common misreading is to take one of these in its everyday sense: to treat a *canonical deposit* as merely a file location, say, rather than as a status that depends on which role is acting. Definitions are listed alphabetically. Where a term names a function and the cross-language story is non-trivial, a tabset shows how it appears in R, Python, and Stata; where a sibling does not yet implement a term, the gap is stated explicitly.

For the function-by-function reference, see the function reference appendix; for the on-disk shape of the provenance sidecar, see the provenance sidecar schema appendix; for the full vocabulary of failures, see the error taxonomy appendix.

E.1 Adaptation (vintage)

A harmonized derivative of a master survey vintage, recorded as a further token block in the IHSN folder name — `..._v<MM>_M_v<AA>_A_<HHHH>`, where `v<AA>_A` is the adaptation number and `<HHHH>` the collection. Like master vintages, adaptations are integers, and “latest adaptation” resolves to the numeric maximum for the requested collection. Contrast *Master vintage*.

E.2 Aggregation contract

The capability family that turns country-level (or finer) observations into regional and global figures with explicit coverage accounting. Its centerpiece is `aggregate_data_v2()`, which computes `weighted_mean` / `mean` / `sum` / `proportion` / `total` while tracking population and country coverage against a coverage threshold (see *coverage threshold*: below it the aggregate is blanked to NA rather than published silently). Supporting members are `apply_time_window()` (keep the latest observation per country within an inclusive year window), `generate_agg_footnote()` (assemble the human-readable coverage footnote), `dw_nestweight()` (redistribute survey weights from missing nested observations so a stratified aggregate stays consistent when missingness is not completely at random within

the stratum), and two demographic convenience wrappers — `dw_pop()` (fetch World Bank population) and `dw_regions()` (enrich a national table with the standard UNICEF regional aggregates). Stata has no aggregation sibling; `aggregate_data_v2` and the three aggregation helpers are R + Python, while `dw_pop` and `dw_regions` are R-only. See the aggregation chapter.

E.3 Cache key

The stable, descriptive identifier — `snake_case`, encoding the parameters that make a response distinct — under which `dw_api_fetch()` stores a cached response. It becomes the filename `<cache_key>.<ext>` in the frozen cache. A well-chosen cache key is what lets a reviewer find the exact frozen response a producer fetched. See *frozen cache*.

E.4 Canonical deposit

Canonical (aggregates). The single authoritative copy of a sector’s deposited artefacts — the `dw_<sector>.<ext>` files and their provenance sidecars — living under the Teams folder at `060.DW-MASTER/01_dw_prep/`, with `013_wrkdata/` holding the promoted, publication-ready outputs and `011_rawdata/` holding inputs and API caches — is the *aggregate / indicator warehouse* sense of *canonical*, and the one the `dw_*` contract means by default. A second, distinct sense — **Canonical (microdata)** — belongs to the survey-microdata archive: the IHSN trees `Z:/datalib` and `Z:/datalib-<domain>` into which the Data Steward promotes conformed survey vintages. The two senses share an office and a single `~/.config/user_config.yml` file but name different data layers — a microdata deposit is never promoted into `060.DW-MASTER`. See *Microdata archive* and the IHSN convention chapter. Whether a path counts as canonical depends on the role acting on it, not just on where it sits: the **producer** writes it (with automatic mirroring to the `Z:` drive — see *Z: mirror*), the **reviewer** may only read it, and the **publisher** publishes from it. A reviewer-mode write that targets a canonical path is a hard error. The toolkit recognises a path as canonical via `dw_is_canonical()`, which (since v0.4.6) also understands OneDrive-mounted Teams Documents paths on UNICEF laptops, and (since v0.5.0) the `dwZDrive` (`Z:`) root.

E.5 Collection

The harmonization code carried in the folder grammar’s final token (for example `HLT`, `IPUMS`), naming the module-and-key schema a vintage conforms to in `datalib`’s merge registry. A collection is *aligned with, but narrower than*, a *domain*: the domain groups an office’s holdings thematically, while the collection is the specific harmonization contract the loader merges against. As of `datalib` contract v1 only `HLT` and `IPUMS` are registered — the `IPUMS` keys still unverified against a real deposit, so per-module key validation guards each merge. See *Domain* and *Module*.

E.6 Contract auditor

The static-analysis helper, `test_scripts()`, that recursively scans a directory of R or Python scripts and flags any direct call to a raw file-IO or external-API command the toolkit is meant to wrap — for example `read_csv`, `pd.read_csv`, `http::GET`, `requests.get`, `rsdmx::readSDMX`, or `sdmx.Client`. It exists to keep a vendoring repo from quietly drifting back to un-wrapped IO. A per-line escape hatch, `# cso-allow: <rule-id>`, suppresses an intentional exception; `error_on_violation = TRUE` turns it into a build gate. The auditor is R + Python only; there is no Stata sibling. See the scaffolding and contract auditor chapter and the CI chapter.

E.6.0.1 R

```
test_scripts("01_dw_prep/", error_on_violation = TRUE)
```

E.6.0.2 Python

```
test_scripts("01_dw_prep/", error_on_violation=True)
```

E.6.0.3 Stata

```
* Not implemented - the contract auditor has no Stata sibling.
```

E.7 Coverage threshold

The minimum population coverage, expressed as a fraction in $[0, 1]$, that an aggregate must meet to be reported. Passed to `aggregate_data_v2()` as `coverage_threshold` (default `NULL`, meaning no threshold; the common UNICEF default is `0.5`, i.e. 50% of population covered). When the covered population share falls below it, the helper blanks the aggregate to `NA` rather than reporting a figure built on too few countries — under-coverage is suppressed, not published silently.

E.8 Cross-language parity

The toolkit's design commitment that the same operation carries the same function name and the same behaviour across R, Python, and Stata, sharing a single provenance sidecar schema, mode-aware path routing, and one error-envelope shape. Parity is the goal, not yet a completed state: Stata has no external-API wrapper and no Parquet support; `dw_merge`,

`dw_verify_z`, and `dw_root` are R + Python only; `dw_stage` and `dw_regions` are R-only; the contract auditor and the aggregation family have no Stata siblings; and the Python reference is hand-written (no pkgdown-style autodoc). The coverage matrix appendix records the gaps function by function.

E.9 Data Steward

In the microdata-deposit governance model (draft v0.3), the role — with a deputy — that *operates* the archive: it maintains `datalib`, runs the review-gate checks and the assisted move, executes promotion of conformed vintages into the canonical IHSN trees, and keeps the logs. The Data Steward operates the gate; the Microdata Core Group *decides* its outcomes. The appointment is still to be named (the governance model is pre-ratification). See the microdata-deposit governance chapter.

E.10 DBM / DBR / DBP

The three role acronyms: **DBM** = Database Manager = **producer**; **DBR** = Database Reviewer = **reviewer**; **DBP** = Database Publisher = **publisher**. The third role was renamed INGESTOR → PUBLISHER in v0.4.6, aligning the label with the verb the code already uses (`dw_publish()`; there is no `dw_ingest()`). For each role's responsibilities, see *producer*, *reviewer*, and *publisher*.

E.11 Domain

A thematic grouping of an office's microdata holdings, each mapped to a library root: health → `Z:/datalib-hlt`, education → `Z:/datalib-edu`, nutrition → `Z:/datalib-nut`, child-protection → `Z:/datalib-chp`, and cross-cutting → `Z:/datalib`. A domain (an organisational grouping of holdings) and a *collection* (a harmonization code in the folder grammar) are aligned but not identical. Only the health domain — via the HLT collection — and IPUMS are in the merge registry today; the rest of the domain registry is a draft-v0.3 item still to be confirmed by the Standards Board. See *Collection*.

E.12 `dw_apis_allowed`

The session-level boolean that gates every external network call. It is **TRUE** in producer mode and **FALSE** in reviewer mode, derived once from `dw_mode` at profile-load time rather than set per call. `dw_require_no_api()` reads it (via the `dw.apis_allowed` option, falling back to the global) and raises a mode-lock error before any HTTP request when it is false — so the reviewer lockout holds even inside a sector's own scripts, not only at the top-level entry points.

E.13 dw_mode

The session property — "producer" or "reviewer" — that selects which side of the mode contract a session operates under. It is read once at profile-load time from the user's block in `~/.config/user_config.yml`; it is **required**, and profile loading hard-fails if it is absent (defaulting either way is unsafe). From it the profile derives the path globals (`teamsWrkData` and its ***Canonical** aliases) and `dw_apis_allowed`. It is a session property, never a per-call argument. See the roles and mode contract chapter.

E.13.0.1 R

```
dw_mode <- "producer"          # normally set by profile_<repo>.R from YAML
dw_apis_allowed <- TRUE
```

E.13.0.2 Python

```
_state.configure(dw_mode="producer", dw_apis_allowed=True)
```

E.13.0.3 Stata

```
global dw_mode "producer"
```

E.14 Error envelope

The three-part shape every `stop()` / `raise` in the toolkit follows, so a reader can both grep for the failing function and act on it:

```
[cso_toolkit.dw_save] Reviewer mode forbids writes under canonical: /path
Reviewer sessions must keep canonical deposits read-only to preserve
vintage permanence; writes go to the sandbox.
Fix:
1. Resolve a sandbox path instead, OR
2. If this is a deliberate DBM bootstrap, pass `allow_canonical_write = TRUE`.
```

The leading `[cso_toolkit.<func>]` prefix names the raising function, so a consumer project can be grepped for every site that hits a given error class; the second line says *why*; the **Fix:** block says *what to do*. The R `testthat` suite asserts the shape with `expect_envelope()`. See the error taxonomy appendix.

E.15 External-API contract

The capability family that wraps external data sources behind a single mode-aware interface: `dw_api_fetch()` (producer hits the live API and caches; reviewer reads only the cache), `dw_api_cached()`, and `dw_api_inventory()`. Sources are selected by the `api =` argument, one of ten identifiers — `"uis"`, `"sdmx"`, `"sdmx_codelist"`, `"wb"`, `"wb_indicators"`, `"ilo"`, `"unsd_sdg"`, `"github_raw"`, `"http"`, and `"json_get"`. The family is R + Python only; Stata has no API wrapper. See the external-API contract chapter.

E.16 Frozen cache

The on-disk store of upstream API responses that a reviewer re-runs against without ever touching the network. A producer call to `dw_api_fetch()` hits the live source and persists the response under `<rawdata>/_apis/<api>/<cache_key>.<ext>` alongside a `.provenance.json` sidecar; a reviewer call reads that file, or stops via `dw_require_no_api()` if it is absent. “Frozen” captures the contract guarantee: the cached vintage does not change under a reviewer’s feet, so a 2026 publication reproduces identically two years later. This is the canonical sense of *reproduces identically* used elsewhere in this glossary (under *vintage* and *vendoring*). A related `_frozen/<host>/<path>` store holds HTTPS responses that `dw_use()` froze from a URL (see *HTTPS-URL freeze*). See also *cache key*.

E.17 HTTPS-URL freeze

The behaviour, added to `dw_use()` in v0.4.0, by which a producer may pass an HTTPS URL instead of a path: the producer downloads it once and freezes the response under `_frozen/<host>/<path>` with a provenance sidecar, and the reviewer reads only from that frozen copy. The host allowlist is configured per-consumer and is empty by default, so the capability is opt-in.

E.18 IHSN convention

The folder-naming grammar datalib encodes, following the International Household Survey Network shelving pattern: `<CCC>/<CCC>_<YYYY>_<SSSS>/` down to master (`..._v<MM>_M`) and adaptation (`..._v<MM>_M_v<AA>_A_<HHHH>`) vintages, with `Data/{Original,Stata,R,Other}`, `Doc/`, and `Programs/` inside each vintage. Because the vintage is legible from the path itself, the microdata archive records provenance *structurally* — the folder name plus an untouched `Data/Original/` — rather than in a `.provenance.json` sidecar. See the IHSN convention chapter and *Microdata archive*.

E.19 IO contract

The capability family that gives every read, write, compare, and merge a single shape: `dw_save`, `dw_use`, `dw_compare`, `dw_merge`, `dw_isid`, and `dw_verify_z`, alongside two path-and-staging helpers — `dw_root()` (resolves the mode-aware `wrk / raw / meta` root so sector scripts build paths without knowing producer-vs-reviewer mode) and `dw_stage()` (a reviewer-mode helper that copies a canonical input into the sandbox once, then hash-guards the staged copy on every later run; in producer mode it is a no-op). Dispatch is automatic by file extension (CSV / TSV / XLSX / RDS-or-PKL / DTA / Parquet / JSON / YAML); every write runs the `isid` uniqueness check, emits a provenance sidecar, mirrors to Teams and Z: on canonical writes, and verifies integrity on canonical reads. Coverage is not uniform: `dw_merge`, `dw_verify_z`, and `dw_root` are R + Python only, `dw_stage` is R-only, and Stata has no Parquet or RDS support (`dw_use.ado` reads `.dta / .csv / .xlsx` only — pipelines needing other formats route through R or Python and `dw_save()` to `.dta` for Stata consumption). See the IO contract chapter.

E.19.0.1 R

```
dw_save(df, name = "dw_ed_edu.csv", sector = "ed", kind = "wrk",
        isid = "REF_AREA")
```

E.19.0.2 Python

```
dw_save(df, name="dw_ed_edu.csv", sector="ed", kind="wrk",
        isid=["REF_AREA"])
```

E.19.0.3 Stata

```
dw_save, filename("dw_ed_edu.dta") path("$teamsWrkData/ed") idvars(REF_AREA INDICATOR)
```

E.20 isid

A uniqueness check named after the Stata command of the same idea: assert that a given set of columns identifies rows uniquely (no duplicate key tuples). The toolkit exposes it both as a standalone helper, `dw_isid(df, keys)`, and as the `isid =` argument to `dw_save()`, which runs the check automatically before writing. A failed `isid` check is a write-blocking error: a non-unique key means the deposit cannot be trusted as a keyed table. In Stata the check is embedded in `dw_save` via `idvars(...)` rather than exposed as a separate command.

E.20.0.1 R

```
dw_isid(df, keys = c("REF_AREA", "INDICATOR", "SEX"))
```

E.20.0.2 Python

```
dw_isid(df, keys=["REF_AREA", "INDICATOR", "SEX"])
```

E.20.0.3 Stata

```
dw_save, filename("out.dta") path("$teamsWrkData/ed") idvars(REF_AREA INDICATOR SEX)
```

E.21 Manifest (.toolkit_manifest.yml)

The small YAML file that records which toolkit vintage a consumer repo has vendored; it lives at `00_functions/.toolkit_manifest.yml`. Its fields are **source** (the upstream owner/repo), **pulled_version** (the pinned tag — or the literal `inrepo`, which marks a helper this repo maintains itself and so silences sync warnings on it), **pulled_date**, **pulled_by**, **vendored** (the list of copied files), and **local_edits** (vendored files with local divergence, which `cso_toolkit_pull()` skips on update). It is version-controlled alongside the helpers and updated on every re-pull. The vintage-sync helpers read it. See the vendoring model chapter.

E.22 Master vintage

A numbered master release of a survey’s harmonized data — the fourth token block of the folder name, `<CCC>_<YYYY>_<SSSS>_v<MM>_M`. Vintages are integers (inputs accept `1 / 01 / v01 / V01`; the internal form is `v%02d`), and “latest master” is the numeric maximum, never the directory-listing or alphabetical order (`v10 > v09`). An *adaptation* layers a harmonization on top of a master. Contrast *Adaptation (vintage)*.

E.23 Microdata archive

The governed store of the survey **data files themselves** — datalib’s `Z:/datalib*` IHSN trees, plus a staging area and a deposit process. It is one of three governance layers, distinct from the *microdata catalogue* (the metadata/discovery layer) and from *datalib* (the software tool). Crucially it is a **different data layer** from the aggregate / indicator warehouse `060.DW-MASTER` that the `dw_*` contract serves: microdata deposits are never promoted into `060.DW-MASTER`. See the microdata-deposit governance chapter and *Canonical deposit*.

E.24 Microdata catalogue

The metadata and discovery layer over the microdata archive — an IHSN/NADA-style index built on the DDI model — as opposed to the *archive* (the files) or the *tool* (`datalib`). It is a Phase-2-plus product, seeded by the tool’s auto-generated inventories, and is not yet built. The distinction the three layers turn on: an archive without a catalogue is unfindable, a catalogue without an archive points at silos, and both without a tool are manual drudgery.

E.25 Mode contract

What each role may and may not do at every wrapped call site, decided by `dw_mode`: producers may call APIs and write canonical; reviewers may do neither — their writes route to the sandbox, and their API calls raise. The contract is enforced by the code, not by convention: every `dw_save`, `dw_use`, and `dw_api_fetch` re-checks. One operational test of whether a sector has adopted the contract is that the canonical deposit’s modification time is unchanged before and after a reviewer-mode run. See the roles and mode contract chapter and the wiring chapter.

E.26 Module

A single survey data file within a vintage — for the HLT collection, `household`, `hhmembers`, `adult`, and `children` — stored as `<vintage-folder-name>_<module>.dta` under `Data/Stata/`. Each collection’s merge registry declares which modules are person-level and which are household-level, and the keys that validate and join them: `datalib` chains person modules 1:1 on the person keys and attaches household modules `m:1` on the household keys, keeping unmatched rows. See *Collection*.

E.27 Producer

The role (Database Manager, DBM) that owns the deposit: it refreshes upstream API caches, runs the sector pipeline, writes the canonical `dw_<sector>.<ext>` artefacts (with automatic Teams + Z: mirroring — see *Z: mirror*), and completes the pre-deposit checklist. It is the only role permitted live network access (`dw_apis_allowed = TRUE`). Set with `dw_mode: "producer"`.

E.28 Provenance sidecar

The `<path>.provenance.json` file the toolkit writes next to every primary output, recording how that output came to be: `written_at`, `user`, `dw_mode`, the `sha256` of the primary file, a schema block (rows / cols / column names), and any caller-supplied `metadata` (title,

sources, vintage, contact, among other caller-chosen fields). Sidecars are on by default (opt out with `provenance = FALSE`); `.RData / .rda` writes skip them. Secrets passed as API call arguments are redacted before they reach the sidecar. The sidecar is what makes a deposit auditable and what a publisher’s provenance chain links back through to the original API fetches. See the provenance sidecar schema appendix.

E.29 Publisher

The role (Database Publisher, DBP) that takes producer-blessed deposits from `013_wrkdata/` and pushes them into downstream infrastructure — Helix, SDMX endpoints, `data.unicef.org`, and the State of the World’s Children tables. It reads canonical, writes only to infrastructure outside the deposit repo, and is gated on the producer’s checklist plus the reviewer’s compare report.

Note

The publisher role is documented, and `dw_publish()` ships as a deliberate dry-run-only stub: `dry_run = TRUE` validates inputs and assembles a payload (`sha256`, `bytes`, `built_at`, `built_by`, version stamp), while `dry_run = FALSE` raises an envelope-shaped “*Live submission not yet implemented*” error pointing at issue #15. Despite the version number reaching v0.8.0, the live Helix submission branch has not yet landed — it remains gated on the sector leads finalising the submission contract. (Formerly INGESTOR.)

E.30 Review gate

The seven-item checklist a microdata deposit must pass before promotion (governance draft v0.3): (1) inventory logged; (2) naming conforms to the grammar (flow deposits only); (3) reproducibility — a non-author can re-run the script from staging; (4) no breakage outside the deposit’s staging folder; (5) licence recorded; (6) access tier assigned; (7) de-identification confirmed. The Microdata Core Group *decides* the outcome; the *Data Steward* operates the checks and, on a pass, promotes conformed vintages into the domain’s canonical IHSN tree — never into `060.DW-MASTER`. See *Staging area* and *Data Steward*.

E.31 Reviewer

The role (Database Reviewer, DBR) that reproduces and validates: it re-runs the same sector pipeline from the canonical deposit, writing only into its own sandbox; compares its output against canonical; and files issues. It is physically prevented from writing canonical and from calling external APIs (`dw_apis_allowed = FALSE`). A reviewer can re-run a full

pipeline offline, because the vendored helpers and the frozen cache are local. Set with `dw_mode: "reviewer"` plus a `sandboxRoot`.

E.32 Sandbox / `sandboxRoot`

The per-user, writable directory tree a reviewer session writes into instead of the canonical deposit. `sandboxRoot` is set in the user's `~/.config/user_config.yml` block and is used only in reviewer mode; the profile redirects `teamsFolder`, `teamsRawData`, and `teamsWrkData` under it so that existing pipeline code lands in the sandbox without per-script edits. (That the canonical deposit's modification time is unchanged across a reviewer run is part of the operational definition of adopting the contract — see *mode contract*.)

E.33 Scaffolding

The capability family that generates the standard CSO project building blocks: `create_profile()` writes a `profile_<repo>.R` (or `.py`) with cross-platform user identification, YAML config load, producer / reviewer `dw_mode` resolution, an optional Z: drive advisory (a non-blocking block that warns when the legacy Azure mirror is unavailable but does not stop execution), a packages block, and a sentinel object; `review_profile()` audits an existing profile against the same checklist, reporting `pass` / `warn` / `fail`; `create_sector_script()` scaffolds a sector run-script template with profile verification, logging, runtime tracking, and try/catch. See the scaffolding and contract auditor chapter.

E.34 Sentinel object

The marker a generated profile leaves in the session to signal that the profile loaded successfully and the mode contract is wired. Downstream sector scripts check for it before running, so a script never executes against an un-initialised, mode-ambiguous session.

E.35 Staging area

The waypoint tree — `Z:/staging-area/<domain>/<team-folder-as-is>/` — where a deposit lands before review, readable by least privilege (the depositing unit, the Data Steward, and the Core Group). *Stock* deposits arrive here exactly as-is; the area is reviewed within a quarter and retired after promotion or a no-go decision. It is a waypoint, not an archive. See *Stock and flow* and *Review gate*.

E.36 Stock and flow

The two deposit modes in the microdata governance model. **Stock** is existing holdings, deposited *exactly as-is* — folder structure preserved so current scripts keep running — with conformance never a precondition (“copy it as it is, don’t touch it”), promoted survey by survey. **Flow** is new work, following the datalib convention from day one (“name it right from day one”). The as-is rule buys a four-phase payoff: preserve and make portable, compare inputs across teams for the same survey, reverse-engineer transformations, and eventually consolidate one file per survey.

E.37 Vendoring

The toolkit’s production adoption model: a consumer repo **copies** the helper files into its own `00_functions/` and pins the version in `.toolkit_manifest.yml`, rather than installing or `source()`-ing them over the network. Vendoring is what buys vintage permanence (a release re-run uses the helper code as it stood at release — *reproduces identically*, in the sense defined under *frozen cache*), survives UNICEF AppLocker restrictions on script-installable paths, and lets reviewers reproduce offline. Native install paths exist for local development (`devtools::install_local`, `pip install -e, adopath ++`), but vendoring stays the production model. See the vendoring model chapter.

E.38 Vintage

A dated snapshot of a data source or of a piece of code. Two senses recur: the **data vintage** carried in a deposit’s provenance metadata (for example, `vintage = "2026-05"`), and the **toolkit vintage** pinned in `.toolkit_manifest.yml`. The whole point of freezing caches and vendoring helpers is that a published vintage reproduces identically later (see *frozen cache*).

E.39 Vintage sync

The capability family of consumer-side helpers that keep a vendored copy from silently falling behind upstream: `cso_toolkit_check()` reads the local manifest, queries upstream for the latest tag, and warns when the consumer is behind; `cso_toolkit_diff()` shows what changed; `cso_toolkit_pull()` refreshes vendored files to a target tag and updates the manifest. Only producer mode can run the check — reviewer mode’s network lockout (`dw_apis_allowed = FALSE`) blocks the upstream lookup, so reviewers always run against whatever vintage the producer pinned. See the vintage sync chapter.

E.40 Z: mirror

The carbon-copy of the canonical deposit kept on the UNICEF Azure file share (the Z: drive). A producer's canonical write is mirrored to Z: automatically and non-blockingly: a failed mirror emits an envelope-shaped warning rather than rolling back the primary write. On canonical reads, the toolkit can verify that the Teams copy and the Z: copy are byte-equal; `dw_verify_z()` exposes this check, and `dw_use()` runs it on canonical reads (`verify_z = "sha256"` for a deep check, `verify_z = FALSE` to skip). The mirror is a redundancy and integrity guarantee, not a separate deposit. `dw_verify_z` is R + Python only.

E.40.0.1 R

```
dw_verify_z("013_wrkdata/dw_ed_edu.csv")
```

E.40.0.2 Python

```
dw_verify_z("013_wrkdata/dw_ed_edu.csv")
```

E.40.0.3 Stata

```
* Not implemented - dw_verify_z has no Stata sibling.  
* (dw_use does run a non-blocking Z: integrity check on canonical reads.)
```

F Curated third-party functions

CSO production pipelines use a small number of functions the toolkit does not provide and UNICEF does not maintain — community Stata commands, CRAN and PyPI utilities. This appendix is the honest register of that third tier of the ecosystem (the architecture chapter defines the tiers): what these functions are, which versions were verified, by whom, and when.

Two things this appendix is **not**. It is not an endorsement — UNICEF curates this *list*, never the code behind it. And it is not a conformance claim: nothing here follows the toolkit’s mode contract, error envelope, or provenance sidecar unless the caveat column says otherwise. The book’s assertion for every row is exactly this: *the named version behaved as described, on the stated date, in the stated pipeline*. Nothing more.

F.1 Inclusion rules

The list is **derived, not curated by taste**. A row enters this table only when all of the following hold:

1. **Production use** — the function is called in at least one production pipeline (a DW-Production sector script or an equivalent vendored consumer). No speculative or “might be useful” entries.
2. **Verified by its user** — the sector lead (or pipeline owner) who actually uses the function stamps the row: version tested, date, and any caveats. The handbook maintainer records the stamp; the user of the code owns the claim.
3. **License compatible** — the upstream license permits the production use.
4. **Environment-safe** — installable and runnable on the UNICEF laptop baseline (AppLocker: no unsigned binaries; corporate network: no unusual endpoints at runtime).

F.2 Staleness policy

Rows are stamped, never re-certified wholesale. A row whose *Verified* date has aged past the version it names simply displays that age — stale rows are visible, and staleness **never blocks a release** of the toolkit or this handbook. If an upstream release breaks a pipeline, the fix is a new verification stamp (or the row’s removal), not an emergency edit to this book.

F.3 The register

Function / package	Language	Purpose	Version tested	Verified by	Date	Caveats
<i>(no verified entries yet — rows are added as sector leads stamp them)</i>						

i Note

This register was introduced with the ecosystem taxonomy (architecture chapter) and starts deliberately empty rather than pre-populated: per inclusion rule 2, every row needs its user's verification stamp, and importing an unverified list would put the book's name on claims nobody owns. Candidate entries — functions already visible in production pipelines — are collected by scanning the pipelines' vendored manifests and dependency loads; each becomes a row when its sector lead verifies it.